OFFICE OF THE CHIEF SYSTEMS ARCHITECT

ENGINEERING CONSTITUTION — VOLUME II

Opus Publica

The Eleven Missing Chapters

Invariants · Contracts · Events · UI · Mathematics · SLOs

Database · Operations · Economics · Evolution

Version 1.0 (Supplement)

Issued 09 August 2026 · Supplement to Volume I

Document Type	Engineering Constitution Volume II (Supplement)
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	Engineering Leadership Team → all engineers, reviewers, operators
Classification	Mandatory — Supreme Engineering Authority
Effective Date	09 August 2026
Supersedes	None (first issuance of Volume II)
Relationship to Volume I	Supplements; does not replace. Conflicts resolved in favour of the stricter provision.
Review Cadence	Quarterly, or upon RC milestone transition
Document ID	OP-CONST-RC1-002

Volume II closes the eleven gaps identified in the post-Volume-I review. It is the constitutional answer to: "what is still missing?"

Preamble — The Eleven Gaps

Volume I of this Constitution established the constitutional foundation: twenty-five chapters covering mission, philosophy, architecture, data, workflow, publication, rendering, security, observability, testing, certification, and evolution. Volume I is a complete governing standard. A post-Volume-I review, however, identified eleven gaps — eleven domains in which a descriptive constitution is insufficient and a governing constitution is required. Volume II closes those eleven gaps. It does not replace Volume I; it supplements it. Where Volume I and Volume II conflict, the stricter provision prevails.

The eleven gaps are not minor omissions. Each represents a domain in which the absence of a constitutional provision permits the implementation loop — the loop in which implementation outruns specification, bugs are patched, and the loop repeats — to re-emerge. Volume II is written to close each gap with formal contracts, measurable thresholds, runtime verification, and governance rules. The most important addition is Chapter 26 (Engineering Invariants): the mathematical truths that can NEVER become false, whose violation constitutes corruption of the scholarly record, and that the Chief Systems Architect SHALL NOT suspend under any operational pressure.

The Eleven Gaps and Their Closure

#	Gap Identified	Closing Chapter	Closing Mechanism
G1	Missing Engineering Invariants (the most important).	Chapter 26	38 invariants across 7 classes, each with predicate, monitor, and violation response.
G2	Missing System Contracts (per-subsystem).	Chapter 27	20 subsystem contracts, each with 9 mandatory fields (Inputs/Outputs/Authority/Failure/Dependencies/Performance/Certification/Acceptance/Runtime Evidence).
G3	Missing Canonical Object Registry (200+ fields).	Chapter 28	276 canonical fields across 14 domains, 25 with full 11-attribute contracts.
G4	Missing Event Constitution.	Chapter 29	34 events with 9-attribute specs, event backbone (transactional outbox), schema evolution.

G5	Missing UI Constitution.	Chapter 30	Design tokens, 11 component specs, page templates, WCAG 2.2 AA, Core Web Vitals budgets.
G6	Missing Engineering Mathematics (constitutional numbers).	Chapter 31	103 thresholds across 10 budget classes; hard limits gate release.
G7	Missing Production SLO Book.	Chapter 32	10 SLIs, 10 SLOs, 3 SLAs, error-budget policy, multi-window burn-rate alerting, 10 runbooks.
G8	Missing Database Constitution (distinct from Data Constitution).	Chapter 33	Naming, indexes, constraints, FK rules, migrations, transactions, locking, partitioning, archival, vacuum, backup (RPO 15min/RTO 1hr), replication, connection management, query performance.
G9	Missing Operational Governance (who does what).	Chapter 34	9 operational roles, 14-action authority matrix, key/cert rotation, migration/deployment/rollback approval, break-glass, DR, incident command, legal-removal process.
G10	Missing Engineering Economics.	Chapter 35	Cost model (13 categories), cost-per-publication, storage/bandwidth/compute/scaling/caching economics, FinOps budget governance, cloud strategy.
G11	Missing Evolution Governance (how the Constitution evolves).	Chapter 36	10-state amendment lifecycle, proposal-to-removal process, precedent register, emergency amendments,

			invariant amendments (80% supermajority).

The Constitutional Status of Volume II

Volume II is Constitution

Volume II is not a draft, not a proposal, and not guidance. It is Constitution. Every provision herein has the same authority as the corresponding provision in Volume I. The Engineering Leadership Team SHALL operationalize each chapter through the Engineering Playbook. The Production Acceptance Gate applies to every feature under both volumes. The Chief Systems Architect SHALL NOT suspend any provision of Volume II except through the Evolution Governance process (Chapter 36), and SHALL NOT suspend any Engineering Invariant (Chapter 26) under any circumstance.

Relationship to Volume I

Volume II presupposes Volume I. The chapter numbering continues from Volume I (Volume II chapters are 26 through 36). Cross-references to Volume I chapters (for example, “Chapter 11 (Publication State Machine)”) refer to the Volume I chapter of that number. Cross-references to Volume II chapters use the full number (for example, “Chapter 26 (Engineering Invariants)”). Where a Volume II chapter expands a Volume I chapter's coverage, the Volume II chapter SHALL be treated as the more specific provision and SHALL prevail; the Volume I chapter remains in force for matters the Volume II chapter does not address.

Completeness Criterion for Volume II

Volume II is considered complete for RC1 when every chapter (26 through 36) is authored, reviewed, signed off by its named owner, and operationalized — meaning that the invariants are monitored, the system contracts are referenced from implementation, the canonical registry is the authoritative source for canonical-field lookup, the event catalogue is the authoritative source for event schema, the UI constitution governs every user-visible surface, the engineering mathematics gate every release, the SLO book is the authoritative source for reliability commitments, the database constitution governs every schema change, the operational governance governs every production action, the engineering economics govern every budget decision, and the evolution governance governs every amendment to this Constitution.

How to Read Volume II

Volume II uses the same RFC 2119 vocabulary as Volume I. Every chapter contains the seventeen standard subsections mandated by the Global Expansion Rule. The chapters are designed to be consulted independently: an engineer investigating an invariant violation consults Chapter 26; an engineer

designing a new subsystem consults Chapter 27; an engineer adding a canonical field consults Chapter 28; and so on. The Knowledge Graph Constitution (Volume I, Chapter 23) is extended by Volume II to trace every invariant to its monitor, every system contract to its subsystem, every canonical field to its registry entry, and every event to its schema.

TABLE OF CONTENTS

Right-click the table below and select "Update Field" to refresh page numbers.

Chapter 26 — Engineering Invariants

This chapter is the most important chapter in the Constitution. It defines the engineering invariants of Opus Publica: the mathematical truths that MUST hold at every instant of the platform's operation, that can NEVER become false, and whose violation is, by definition, a corruption of the scholarly record. An invariant is not a principle (which guides judgement), not a rule (which permits exceptions under governance), and not a policy (which may be amended). An invariant is a property of the system that, if violated, indicates that the system is no longer the system — that some assumption on which every other chapter rests has been broken, and that all engineering activity SHALL halt until the invariant is restored.

The invariants in this chapter are formal. Each is stated as a predicate over the platform's state, each has a formal verification condition, each has a runtime monitor that detects violations, and each has a defined response when violated. The invariants are organized into seven classes: identity invariants (one-ness and uniqueness), immutability invariants (published-artifact permanence), resolvability invariants (every identifier resolves), completeness invariants (nothing orphaned, nothing duplicated), legality invariants (every transition legal), auditability invariants (every transition recorded), and conservation invariants (nothing lost, nothing created from nothing). Together, these seven classes constitute the mathematical contract of the platform.

The Constitutional Status of Invariants

Invariants are the only provisions of this Constitution that the Chief Systems Architect SHALL NOT suspend, even temporarily, even under catastrophic operational pressure. Where an invariant and a release deadline conflict, the deadline yields. Where an invariant and a feature conflict, the feature is re-specified. Where an invariant and a cost conflict, the cost is accepted. The invariants are the platform's reason to exist; suspending them suspends the platform.

26.1 Class I — Identity Invariants

Identity invariants assert the one-ness and uniqueness of canonical objects. They are the foundation of the platform's data model: if an article has two canonical representations, or a DOI has two minted records, or a published PDF has two versions sharing the same content address, the platform's notion of identity has broken down and every downstream system (citation, indexing, preservation) will produce inconsistent results. Identity invariants are enforced by the data model (unique constraints), by the storage layer (content addressing), and by the runtime (invariant monitors).

INV-I-01 — Exactly One Canonical Article

For every published work, there exists exactly one canonical article record. The canonical record is the master from which all derived representations (HTML, PDF, JATS, citations) are produced. There is no

second canonical record; there is no shadow record; there is no parallel record maintained for performance or convenience. A violation of this invariant indicates either a duplicate publication (two records for one work) or a fragmented publication (one record split across two). Either case corrupts the scholarly record and SHALL be detected and remediated immediately.

Attribute	Value
Predicate	∃! article : Published(article) ∧ Canonical(article)
Verification Condition	COUNT(canonical_articles WHERE work_id = W) = 1 for every published W.
Runtime Monitor	Continuous scan; hourly reconciliation; alert on COUNT ≠ 1.
Violation Response	Halt publication; page on-call; remediate by merging or fragmenting.
Owner	Head of Data Engineering.

INV-I-02 — Exactly One DOI Per Published Article Version

For every published article version, there exists exactly one DOI, registered with exactly one registration agency (Crossref or DataCite), with exactly one deposit record. A DOI is permanently bound to its article version; it is never reused, never reassigned, never duplicated. A violation indicates either a duplicate DOI minting (two DOIs for one version) or a DOI leak (one DOI for two versions) — either of which corrupts the citation graph.

Attribute	Value
Predicate	∃! doi : PublishedVersion(v) → HasDOI(v, doi) ∧ Unique(doi).
Verification Condition	COUNT(distinct dois WHERE article_version_id = V) = 1 for every published V; COUNT(article_versions WHERE doi = D) = 1 for every registered D.
Runtime Monitor	Crossref reconciliation job; daily; alert on mismatch.
Violation Response	Halt publication; retract erroneous DOI; page on-call; record in audit.
Owner	Head of Platform Engineering.

INV-I-03 — Exactly One Published PDF Per Article Version

For every published article version, there exists exactly one published PDF, content-addressed by SHA-256, immutable once written. There is no second PDF, no alternate rendering, no cached copy

treated as authoritative. A violation indicates either a rendering non-determinism (two PDFs from the same canonical input) or a silent overwrite (the PDF was modified after publication).

INV-I-04 — Exactly One Canonical HTML Per Article Version

For every published article version, there exists exactly one canonical HTML representation, content-addressed by SHA-256, immutable once written. The canonical HTML is the source of truth for the article's web rendering; all variants (mobile, print, accessible) are derived from it, never authored independently.

INV-I-05 — Exactly One Canonical JATS XML Per Article Version

For every published article version, there exists exactly one canonical JATS XML document, validated against the official NLM schema, content-addressed by SHA-256. The JATS XML is the interchange representation used by Crossref, CLOCKSS, LOCKSS, Portico, OAI-PMH harvesters, and OpenAIRE. There is no second JATS for the same version; downstream consumers either consume the canonical or diverge.

INV-I-06 — Exactly One Canonical Author Record Per Author

For every author who has contributed to a published work, there exists exactly one canonical author record, identified by a platform UUID and optionally linked to an ORCID iD. An author is not duplicated across journals, across submissions, or across affiliations. A violation indicates identity fragmentation (the same person appears as two authors) or identity collision (two different people share a record).

INV-I-07 — Exactly One Journal Record Per Journal

For every journal hosted by the platform, there exists exactly one canonical journal record, identified by its ISSN (or platform UUID pending ISSN assignment). A journal is not duplicated across tenants, across regions, or across configurations.

26.2 Class II — Immutability Invariants

Immutability invariants assert the permanence of published artifacts. Once a scholarly work is published, the published artifact is fixed: a reader who cites the work in 2026 MUST be able to retrieve the byte-identical artifact in 2036 and verify the citation. Immutability is the foundation of citability; violating it violates the scholarly record's reason to exist. Immutability is enforced by content addressing (SHA-256), by append-only storage, and by the Constitution's prohibition on in-place modification of published artifacts.

INV-M-01 — Published Article Immutable

Once an article version reaches the Published state, its canonical record, its JATS XML, its published PDF, its canonical HTML, and all its supplements are immutable. The SHA-256 of each artifact is recorded at publication and SHALL match on every subsequent retrieval. A mismatch indicates either storage corruption, silent overwrite, or an attack — each of which is a P1 incident.

INV-M-02 — Audit Log Append-Only

The audit log is append-only. No record MAY be modified, no record MAY be deleted, no record MAY be reordered. Each record is cryptographically chained to its predecessor (record N contains hash(record N-1)), so any tampering is detectable by re-computing the chain. A violation indicates either a database bug, an insider attack, or a compromised credential — each of which is a SEV-1 incident.

INV-M-03 — DOI Immutable Once Minted

Once a DOI is minted for an article version, the binding between the DOI and the article version is permanent. The DOI's metadata MAY be updated (for example, to record a retraction), but the DOI itself is never reassigned to a different article, never deleted, and never re-minted. A violation indicates a critical Crossref integration failure.

INV-M-04 — Preservation Deposit Immutable

Once an artifact is deposited to a preservation service (CLOCKSS, LOCKSS, Portico), the deposited artifact is immutable. The deposit receipt is recorded permanently. Re-deposit of the same artifact is permitted (for redundancy) but SHALL produce a byte-identical deposit; a non-identical re-deposit is a violation indicating either non-deterministic rendering or upstream corruption.

INV-M-05 — Version Relationships Immutable

Once a version relationship is recorded (version-of, supersedes, is-superseded-by, retraction-of), the relationship is immutable. A correction cannot silently become a retraction; a retraction cannot silently become a correction. Relationship changes require a new audited transition with a recorded reason.

26.3 Class III — Resolvability Invariants

Resolvability invariants assert that every identifier the platform has minted or asserted SHALL resolve. A DOI that returns 404, an ORCID iD that links to a deleted record, an artifact SHA-256 that cannot be retrieved — each is a broken promise to the scholarly community. Resolvability is verified by continuous probing and by external reconciliation (Crossref REST API, ORCID public API, storage HEAD checks).

INV-R-01 — Every DOI Resolvable

Every DOI minted by the platform SHALL resolve, via the global Handle resolution service, to the article's canonical landing page on opuspublica.com. Resolution SHALL succeed within the SLO (p95 ≤ 500 ms). A DOI that fails to resolve for more than 5 minutes is a SEV-1 incident; a DOI that fails for more than 1 hour is a scholarly-record corruption event requiring Chief Systems Architect notification.

INV-R-02 — Every Artifact Retrievable

Every published artifact (PDF, HTML, JATS, supplement) SHALL be retrievable by its content address (SHA-256) indefinitely. A HEAD request to the artifact's content address SHALL return 200 with a matching content hash. An artifact that cannot be retrieved is a scholarly-record loss event and a SEV-1 incident.

INV-R-03 — Every ORCID iD Resolvable (Where Linked)

Every ORCID iD linked to a platform author record SHALL resolve via the ORCID public API. A link to a deleted ORCID record, or to an ORCID iD that does not exist, is a violation. The platform SHALL NOT auto-delete ORCID links on resolution failure; it SHALL flag the link for editorial review.

INV-R-04 — Every OAI-PMH Identifier Resolvable

Every identifier exposed via the OAI-PMH endpoint SHALL be retrievable via the GetRecord verb. An identifier that returns noMoreRecords or an error is a violation indicating either a deleted record not properly marked or a metadata indexing gap.

INV-R-05 — Every Crossref Deposit Reconciled

Every Crossref deposit the platform has made SHALL be reconciled against Crossref's REST API within the SLO (24 hours). A deposit that cannot be reconciled is a violation indicating either a deposit failure, a Crossref delay beyond SLO, or a metadata drift between platform and Crossref.

26.4 Class IV — Completeness Invariants

Completeness invariants assert that nothing is orphaned and nothing is duplicated. An orphan storage object that no database row references is a future bug (a reader will eventually be served it, or it will be silently lost). An orphan database row that no storage object backs is a current bug (the platform claims an artifact that does not exist). A duplicate canonical package (two storage objects with the same content hash) is a non-determinism bug. Each of these is a violation.

INV-C-01 — No Orphan Storage Objects

Every object in the platform's storage buckets SHALL be referenced by at least one database row. A storage object with no referencing database row is an orphan and SHALL be detected by the nightly storage reconciliation job. Orphans are flagged for review; deletion requires governance approval (they may be artifacts in transit, recently uploaded but not yet committed).

INV-C-02 — No Orphan Database Rows

Every database row that references a storage object SHALL reference an existing storage object. A database row whose referenced storage object does not exist is an orphan and SHALL be detected by the nightly reconciliation job. Orphan rows are SEV-1 incidents indicating either storage data loss or a database/storage transaction failure.

INV-C-03 — No Orphan Metadata

Every metadata field in the platform SHALL be referenced by at least one canonical record. A metadata field that no canonical record uses is an orphan and SHALL be flagged for deprecation. Metadata fields do not accumulate indefinitely; the platform's vocabulary is bounded and curated.

INV-C-04 — No Duplicate Canonical Packages

No two storage objects SHALL have the same SHA-256 content hash unless they are explicitly registered as replicas. A duplicate canonical package indicates either non-deterministic rendering (the same input produced two different outputs that happen to collide, which is astronomically unlikely) or, more commonly, a duplicate deposit that should have been deduplicated by idempotency.

INV-C-05 — No Duplicate Crossref Deposits

No two Crossref deposits SHALL claim the same DOI for the same article version. Duplicate deposits are detected by idempotency keys and by Crossref's own deduplication; a duplicate that reaches Crossref indicates an idempotency failure.

INV-C-06 — No Duplicate Preservation Deposits

No two preservation deposits SHALL deposit the same artifact to the same backend with different content. Redundant deposits of identical content are permitted (and expected on retry); deposits of different content for the same artifact are violations indicating non-deterministic rendering.

INV-C-07 — Every State Transition Legal

Every state transition recorded in the audit log SHALL be a legal transition per the Publication State Machine (Chapter 11). An illegal transition (for example, Draft → Published without traversing Composed, Preview, Canonical Frozen, DOI Minted) is a violation indicating either a workflow bypass or a state-machine enforcement failure. The platform SHALL detect illegal transitions at the moment of attempt and SHALL refuse them; a successful illegal transition is a SEV-1 incident.

26.5 Class V — Legality Invariants

Legality invariants assert that every state, every transition, and every relationship conforms to the platform's defined contracts. A published article without a DOI is illegal. A reviewer assignment without a conflict-of-interest declaration is illegal. A version without a parent is illegal (unless it is the original). Legality is enforced by the state machine, by precondition checks on every transition, and by runtime invariant monitors that scan for illegal states.

INV-L-01 — Every Published Article Has A DOI

For every article in the Published state, the article SHALL have at least one registered DOI. A published article without a DOI is, by definition, not published; its presence in the Published state is illegal and SHALL be detected and remediated.

INV-L-02 — Every Published Article Has Valid JATS XML

For every article in the Published state, the article SHALL have a JATS XML representation validated against the official NLM schema (version pinned per article). A published article with missing or invalid JATS is illegal.

INV-L-03 — Every Reviewer Assignment Has A CoI Declaration

For every reviewer assignment in the Active or Completed state, the assignment SHALL have a recorded conflict-of-interest declaration from the reviewer. An assignment without a CoI declaration is illegal and SHALL be detected at the moment of assignment; the platform SHALL refuse to create the assignment without the declaration.

INV-L-04 — Every Publication Event Has A Preservation Deposit Queued

For every PublicationCompleted event, a preservation deposit outbox event SHALL have been committed in the same database transaction. A publication without a queued preservation deposit is illegal and indicates a transactional failure.

INV-L-05 — Every Version Has A Parent (Except Originals)

For every article version that is not the original, the version SHALL have a recorded parent version. A version without a parent is illegal and indicates either an orphan version (a serious data-integrity bug) or a missing relationship record.

INV-L-06 — Every Retraction Has A Retraction Notice

For every article in the Retracted state, a retraction notice SHALL exist as a published artifact, linked bidirectionally to the retracted article, and deposited to Crossref with the appropriate retraction metadata. A retraction without a notice is illegal and SHALL be detected.

26.6 Class VI — Auditability Invariants

Auditability invariants assert that every state transition is recorded, that the recording is complete, and that the recording is itself verifiable. An unaudited transition is, for certification purposes, a transition that did not happen — the platform cannot prove it occurred, cannot reconstruct who did it, and cannot verify its preconditions and postconditions. Auditability is enforced by the audit service, by the invariant monitors that verify log completeness, and by the cryptographic chain that detects tampering.

INV-A-01 — Every State Transition Audited

Every state transition on a published or publishable work SHALL produce exactly one audit record, committed in the same database transaction as the transition. A transition without an audit record is a violation indicating either a transactional failure or a bypass of the audit service.

INV-A-02 — Every Audit Record Complete

Every audit record SHALL contain: actor (authenticated principal or service), timestamp (UTC, monotonic within the platform), operation performed, before-state hash, after-state hash, reason (free text, required), and correlation id. A record missing any field is a violation and SHALL be detected at write time.

INV-A-03 — Audit Chain Cryptographically Verifiable

The audit log SHALL be cryptographically chained: each record contains the SHA-256 of its predecessor. A re-computation of the chain SHALL match the stored chain at every record. A mismatch indicates tampering and is a SEV-1 incident.

INV-A-04 — No Silent State Changes

No state change SHALL occur without an audit record. A database row whose state differs from the state implied by the most recent audit record is a violation indicating either a direct database mutation (forbidden by Rule OPS-005) or an audit service failure.

26.7 Class VII — Conservation Invariants

Conservation invariants assert that nothing is lost and nothing is created from nothing. Every artifact the platform has ever published is still retrievable. Every deposit the platform has ever made is still reconciled. Every credential the platform has ever rotated is still recorded. Conservation is the engineering analogue of the physical law of mass conservation: the platform's information content is monotonic non-decreasing.

INV-V-01 — Published Artifacts Never Lost

Once an artifact has been published, it SHALL remain retrievable for the lifetime of the platform. Deletion of a published artifact is forbidden; the only path to removal is a governed legal-removal process that records the removal reason, preserves a cryptographic proof of the artifact's prior existence, and updates the DOI metadata to reflect the removal.

INV-V-02 — Deposits Never Forgotten

Every deposit the platform has made to an external service (Crossref, ORCID, CLOCKSS, LOCKSS, Portico) SHALL have a permanent deposit record in the platform's database. The deposit record SHALL reference the deposit receipt and SHALL be reconciled within the SLO.

INV-V-03 — Credentials Rotated Per Policy

Every external credential the platform holds SHALL be rotated per its rotation policy. A credential past its rotation deadline is a violation indicating either a rotation-process failure or a missing rotation policy. The platform SHALL alert on approaching rotation deadlines and SHALL escalate on missed deadlines.

INV-V-04 — No Spontaneous Creation

No canonical record, no published artifact, no audit record, no deposit record SHALL be created without a triggering event recorded in the audit log. A record that exists without a triggering event is a violation indicating either a database corruption or a bypass of the platform's creation paths.

26.8 The Master Invariant Table

The following table is the authoritative catalogue of every engineering invariant of Opus Publica. Each invariant has a stable identifier (INV-X-NN), a class, a one-line statement, an owner, a monitor, and a violation severity. The table SHALL be maintained by the Office of the Chief Systems Architect and SHALL be the authoritative reference for invariant monitoring, incident response, and certification.

ID	Class	Invariant (one-line)	Severity
INV-I-01	Identity	Exactly one canonical article per published work.	SEV-1
INV-I-02	Identity	Exactly one DOI per published article version.	SEV-1
INV-I-03	Identity	Exactly one published PDF per article version.	SEV-1
INV-I-04	Identity	Exactly one canonical HTML per article version.	SEV-1
INV-I-05	Identity	Exactly one canonical JATS XML per article version.	SEV-1
INV-I-06	Identity	Exactly one canonical author record per author.	SEV-2
INV-I-07	Identity	Exactly one journal record per journal.	SEV-2
INV-M-01	Immutability	Published article artifacts immutable (SHA-256 fixed).	SEV-1
INV-M-02	Immutability	Audit log append-only and cryptographically chained.	SEV-1
INV-M-03	Immutability	DOI-to-version binding immutable once minted.	SEV-1
INV-M-04	Immutability	Preservation deposits immutable.	SEV-1
INV-M-05	Immutability	Version relationships immutable.	SEV-2
INV-R-01	Resolvability	Every DOI resolves to the article landing page.	SEV-1
INV-R-02	Resolvability	Every artifact retrievable by content hash.	SEV-1
INV-R-03	Resolvability	Every linked ORCID iD resolves via ORCID public API.	SEV-3

INV-R-04	Resolvability	Every OAI-PMH identifier retrievable via GetRecord.	SEV-2
INV-R-05	Resolvability	Every Crossref deposit reconciled within SLO.	SEV-2
INV-C-01	Completeness	No orphan storage objects.	SEV-3
INV-C-02	Completeness	No orphan database rows.	SEV-1
INV-C-03	Completeness	No orphan metadata fields.	SEV-3
INV-C-04	Completeness	No duplicate canonical packages (same SHA-256, non-replica).	SEV-2
INV-C-05	Completeness	No duplicate Crossref deposits for same DOI.	SEV-1
INV-C-06	Completeness	No duplicate preservation deposits with different content.	SEV-2
INV-C-07	Completeness	Every state transition legal per state machine.	SEV-1
INV-L-01	Legality	Every published article has a DOI.	SEV-1
INV-L-02	Legality	Every published article has valid JATS XML.	SEV-1
INV-L-03	Legality	Every reviewer assignment has a CoI declaration.	SEV-1
INV-L-04	Legality	Every publication event has a preservation deposit queued.	SEV-1
INV-L-05	Legality	Every non-original version has a parent.	SEV-2
INV-L-06	Legality	Every retraction has a retraction notice.	SEV-1
INV-A-01	Auditability	Every state transition has exactly one audit record.	SEV-1
INV-A-02	Auditability	Every audit record complete (7 mandatory fields).	SEV-1
INV-A-03	Auditability	Audit chain cryptographically verifiable.	SEV-1
INV-A-04	Auditability	No silent state changes (no	SEV-1

		audit-less transitions).	
INV-V-01	Conservation	Published artifacts never lost (lifetime retention).	SEV-1
INV-V-02	Conservation	Deposits never forgotten (permanent deposit records).	SEV-2
INV-V-03	Conservation	Credentials rotated per policy.	SEV-2
INV-V-04	Conservation	No spontaneous creation (no triggerless records).	SEV-1

26.9 Invariant Verification Framework

Each invariant is verified by a combination of static enforcement (schema constraints, type systems, precondition checks), continuous runtime monitoring (invariant monitors that scan for violations), and periodic reconciliation (jobs that compare the platform's state against external truth). The verification framework SHALL be the authoritative regime for invariant health, and the Engineering Leadership Team SHALL review the invariant dashboard daily.

26.9.1 Static Enforcement

Static enforcement prevents violations at the moment of attempt. Unique constraints prevent duplicate canonical records. Foreign-key constraints prevent orphan rows. State-machine precondition checks prevent illegal transitions. Type systems prevent type-mismatched assignments. Static enforcement is the first line of defence; an invariant that can be violated despite static enforcement indicates either a constraint gap or a constraint bypass.

26.9.2 Continuous Runtime Monitoring

Continuous runtime monitoring detects violations that escape static enforcement. Each invariant has a defined monitor that scans the platform's state on a defined cadence (continuous, hourly, daily) and raises an alert on violation. Monitors SHALL be idempotent, SHALL produce audit records of their own (so that monitoring itself is auditable), and SHALL have defined runbooks for the violations they detect.

26.9.3 Periodic Reconciliation

Periodic reconciliation detects violations that escape both static enforcement and continuous monitoring, by comparing the platform's state against external truth. Crossref deposits are reconciled against Crossref's REST API. Storage objects are reconciled against database references. Audit chains are reconciled against their cryptographic recomputation. Reconciliation jobs SHALL run on defined cadences and SHALL produce reconciliation reports that are reviewed by the owning team.

26.9.4 Violation Severity and Response

Severity	Meaning	Response Time	Escalation
SEV-1	Scholarly-record corruption or potential corruption.	Immediate (page on-call).	Chief Systems Architect notified within 1 hour.
SEV-2	Significant degradation; scholarly record at risk.	1 hour (business hours) / 4 hours (off-hours).	Engineering Manager notified.
SEV-3	Operational anomaly; no immediate record risk.	1 business day.	Owning team notified via ticket.

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

This chapter defines the engineering invariants of Opus Publica: the mathematical truths that MUST hold at every instant of the platform's operation and whose violation constitutes corruption of the scholarly record. The chapter is the ultimate authoritative reference for what the platform guarantees about itself, and every other chapter of the Constitution derives its correctness from these invariants holding.

2. Scope

In scope: the seven classes of invariants (identity, immutability, resolvability, completeness, legality, auditability, conservation) and their verification framework. Out of scope: the operational procedures for responding to violations (covered in the Failure Constitution, Chapter 21) and the certification of invariant monitors (covered in the Certification Constitution, Chapter 22).

3. Definitions

Invariant: a property of the system that MUST hold at every instant and whose violation is a corruption. Predicate: the formal statement of the invariant as a boolean expression over the platform's state. Verification Condition: the concrete checkable condition that, when true, demonstrates the invariant holds. Runtime Monitor: the component that continuously or periodically checks the verification condition and alerts on violation.

4. Responsibilities

Ownership of the invariants is distributed across the engineering organization:

- **Chief Systems Architect:** Accountable for the invariant catalogue and for refusing to suspend any invariant.
- **Head of Data Engineering:** Responsible for identity, completeness, and conservation invariants.
- **Head of Platform Engineering:** Responsible for immutability, resolvability, and legality invariants.
- **Head of Observability:** Responsible for the auditability invariants and the invariant monitoring framework.
- **Release Certification Authority:** Responsible for verifying invariant monitors are operational before release.

5. Design Principles

Invariants are formal (stated as predicates), monitorable (each has a runtime check), unsuspendable (no operational pressure justifies violation), and conservative (when in doubt, the invariant is stated strictly rather than loosely). An invariant that cannot be monitored is not an invariant; it is an aspiration.

6. Engineering Requirements

Every invariant SHALL have a stable identifier, a formal predicate, a verification condition, a runtime monitor, a defined violation response, and a named owner. The invariant dashboard SHALL display the current status of every invariant in real time. The Engineering Leadership Team SHALL review the invariant dashboard daily. Any SEV-1 invariant violation SHALL trigger an immediate incident response and SHALL be reported to the Chief Systems Architect within one hour.

7. Engineering Constraints

Invariants constrain every other chapter of the Constitution. No feature MAY be implemented that would violate an invariant. No optimization MAY be accepted that weakens an invariant's verification. No operational procedure MAY be defined that suspends an invariant. The Chief Systems Architect SHALL NOT grant exceptions to invariants; the only path to changing an invariant is to amend this chapter through the Evolution Governance process (Chapter 36), and such amendments SHALL require evidence that the new invariant set is at least as strong as the old.

8. Runtime Behaviour

At runtime, the invariant monitors continuously scan the platform's state. Each monitor runs on its defined cadence, produces an audit record of its execution, and raises an alert on violation. The invariant dashboard aggregates monitor outputs into a single real-time view. A monitor that itself fails to run is a SEV-2 incident (a blind spot) and SHALL be detected by a meta-monitor that checks monitor liveness.

9. Failure Conditions

Invariant failure conditions include:

- **Static enforcement failure:** a constraint that should have prevented a violation did not.
- **Monitor failure:** a monitor that should have detected a violation did not run or did not alert.
- **Reconciliation failure:** a reconciliation job found a discrepancy between platform and external truth.

- **Meta-monitor failure:** the meta-monitor that checks monitor liveness itself failed.
- **Cascade:** a violation of one invariant caused violations of others (e.g., orphan row → resolvability failure).

10. Recovery Procedures

Invariant-violation recovery follows the incident-response regime (Chapter 21) with the additional requirement that the recovery SHALL restore the invariant, not merely mask the symptom. Restoring an invariant may require data reconstruction (from preservation deposits, from Crossref, from backups), state rollback (to the last known-good state), or, in extreme cases, retraction of the affected article. The recovery SHALL be recorded in the audit log with a full before/after analysis.

11. Security Considerations

Invariants are the platform's ultimate security control. An attacker who can violate an invariant can corrupt the scholarly record, and the platform's auditability invariants (INV-A-03, cryptographic chain) are the defence against such corruption being undetected. The invariant monitors SHALL themselves be defended against tampering (the monitors run as separate services with separate credentials, and their outputs are replicated to a separate audit store).

12. Performance Considerations

Invariant monitoring has a performance cost: each monitor consumes resources, and each reconciliation job consumes external-API quota. The performance budget for invariant monitoring SHALL be defined in Chapter 31 (Engineering Mathematics) and SHALL not exceed 5% of platform compute capacity. Monitoring that exceeds its budget SHALL be optimized, not skipped.

13. Observability Requirements

Every invariant monitor SHALL expose its current status (pass/fail/unknown), its last execution time, its last result, and its alert history as metrics. The invariant dashboard SHALL aggregate these into a single view. A monitor in “unknown” state for more than its cadence interval is a SEV-2 incident (the platform is blind to that invariant).

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **All 38 invariants have stable identifiers, predicates, verification conditions, monitors, responses, and owners.**
- **The invariant dashboard is live and reporting all 38 invariants in real time.**
- **At least one invariant monitor has been exercised in a chaos exercise (violation injected, detected, alerted).**
- **The meta-monitor is operational and detects monitor failures.**
- **The Chief Systems Architect has signed off the invariant catalogue.**

15. Runtime Verification

Runtime verification of the invariants is the invariant monitoring framework itself: the monitors are the verification, the dashboard is the evidence, and the audit records of monitor executions are the certification artifacts. The Release Certification Authority SHALL verify that all 38 monitors are operational and reporting before RC1 sign-off.

16. Certification Requirements

RC1 certification requires: the master invariant table published; all 38 monitors operational; the invariant dashboard live; at least one chaos exercise completed and postmortemed; the meta-monitor operational; and the Chief Systems Architect's sign-off that the invariant set is complete (no known invariant is missing from the catalogue).

17. Future Evolution

The invariant set evolves through the Evolution Governance process (Chapter 36). New invariants MAY be added (typically when a new class of corruption is identified). Existing invariants MAY be strengthened (made stricter) but SHALL NOT be weakened without evidence that the weaker invariant is still sufficient to protect the scholarly record. An invariant SHALL NOT be removed; if an invariant is genuinely obsolete, it SHALL be marked deprecated with a recorded reason and a successor invariant (or a documented argument that no successor is needed).

Chapter 27 — System Contracts

This chapter establishes the constitutional contract regime for every subsystem of Opus Publica. A subsystem contract is a normative, versioned, and certification-bearing specification of what the subsystem accepts (Inputs), what it produces (Outputs), what state it is authorized to change (Authority), under what conditions it is considered failed (Failure Conditions), what other subsystems it depends on (Dependencies), what performance it MUST deliver (Performance Budget), what certification it MUST pass (Certification), what tests verify it (Acceptance Tests), and what telemetry it MUST emit to prove its correct operation at runtime (Runtime Evidence).

Every subsystem deployed to production MUST have a published contract in the form defined in section 27.1, registered in the Subsystem Catalogue (section 27.2), and elaborated as one of the per-subsystem contract tables in section 27.3. A subsystem without a published contract is, by definition, an uncontrolled component: its behaviour is unspecified, its failures are unbounded, and its certification is impossible. The Production Acceptance Gate (Foundational Principle II) SHALL refuse to release any subsystem whose contract is missing, stale, or unverified. Contracts are versioned artefacts; a change to any contract field SHALL trigger a contract-diff review by the Principal Architect and, where the change is breaking, a Constitution Amendment per Chapter 36 (Evolution Governance).

27.1 Contract Template

Every subsystem contract SHALL contain the nine fields enumerated below, in the order given. The Content Obligation column specifies what each field MUST contain for the contract to be considered complete. A field whose content obligation is unmet renders the contract incomplete and blocks the subsystem's certification. The template is normative: subsystems MAY add appendix fields (metadata, runbooks, contact information) but MAY NOT omit or reorder the nine canonical fields.

Field	Content Obligation
Inputs	Enumeration of every request type, payload schema, file format, and event the subsystem accepts. Each input SHALL reference its authoritative schema (JSON Schema, XSD, or Protobuf) and SHALL state whether the input is synchronous, asynchronous, or event-driven.
Outputs	Enumeration of every response, artifact, event, and side-effect the subsystem produces. Each output SHALL reference its schema, its persistence target (database, object store, audit log), and its downstream consumers.
Authority	A precise statement of what state the subsystem is authorized to create, transition, or delete. The

	Authority field SHALL enumerate the database tables the subsystem may write, the state transitions it may drive, and the external integrations it may invoke. Anything not enumerated is forbidden.
Failure Conditions	Enumeration of the conditions under which the subsystem is considered failed, each with a severity (SEV-1, SEV-2, SEV-3) and a reference to the recovery runbook in the Failure Constitution (Chapter 21).
Dependencies	Enumeration of every other subsystem, external service, and shared resource the subsystem depends on. Each dependency SHALL state the failure-mode (degraded, unavailable, broken) and the graceful-degradation behaviour expected when the dependency is unhealthy.
Performance Budget	Quantitative SLOs the subsystem MUST meet: latency percentiles (p50, p95, p99), throughput, error rate, and availability. Borrowed from Chapter 17 (SLI/SLO Constitution) and bound to the subsystem's error budget.
Certification	The certification regime the subsystem MUST pass before production release: contract test suite pass rate, chaos exercises, idempotency verification, invariant monitors, and the Release Certification Authority's sign-off (Chapter 22).
Acceptance Tests	The concrete test cases that verify the contract is honoured: happy-path, error-path, idempotency, concurrency, and regression tests. Each test SHALL reference its location in the test repository and its expected runtime evidence.
Runtime Evidence	The telemetry the subsystem MUST emit at runtime to prove its contract is honoured: metrics, structured logs, traces, audit records, and reconciliation reports. Each evidence stream SHALL have a named dashboard panel and a named alert.

27.2 Subsystem Catalogue

The Subsystem Catalogue is the authoritative master table of every subsystem deployed to production. Each subsystem has a stable identifier (SUB-NN), a named owner accountable for its contract, a criticality rating (Critical, High, Standard), and an SLA tier borrowed from Chapter 17. The catalogue SHALL be

maintained by the Principal Architect and SHALL be reviewed at every release. A subsystem not in this catalogue SHALL NOT be deployed; a subsystem in this catalogue SHALL NOT be deployed without a published contract in section 27.3.

ID	Subsystem	Owner	Criticality	SLA Tier
SUB-01	Submission Service	Head of Editorial Workflow	Critical	T1 (99.9%)
SUB-02	Review Service	Head of Editorial Workflow	Critical	T1 (99.9%)
SUB-03	Decision Service	Head of Editorial Workflow	Critical	T1 (99.9%)
SUB-04	Composition / Production Service	Head of Production Engineering	Critical	T1 (99.9%)
SUB-05	Canonical Freezer	Principal Architect	Critical	T1 (99.95%)
SUB-06	PDF Generator	Head of Production Engineering	High	T2 (99.5%)
SUB-07	HTML Renderer	Head of Production Engineering	High	T2 (99.5%)
SUB-08	JATS Generator	Head of Production Engineering	Critical	T1 (99.9%)
SUB-09	Artifact Verifier	Principal Architect	Critical	T1 (99.9%)
SUB-10	DOI Minter	Head of Platform Engineering	Critical	T1 (99.9%)
SUB-11	Crossref Depositor	Head of Platform Engineering	Critical	T1 (99.9%)
SUB-12	ORCID Integrator	Head of Platform Engineering	High	T2 (99.5%)
SUB-13	Preservation Depositor	Head of Platform Engineering	Critical	T1 (99.9%)
SUB-14	OAI-PMH Endpoint	Head of Platform Engineering	High	T2 (99.5%)
SUB-15	SWORD Endpoint	Head of Platform Engineering	Standard	T3 (99.0%)
SUB-16	Audit Service	Principal Architect	Critical	T1 (99.99%)
SUB-17	Notification Service	Head of Platform Engineering	High	T2 (99.5%)
SUB-18	Search / Index Service	Head of Production Engineering	High	T2 (99.5%)

SUB-19	Identity / Auth Service	Head of Platform Engineering	Critical	T1 (99.9%)
SUB-20	Storage Service	Head of Data Engineering	Critical	T1 (99.99%)

Criticality definitions. Critical (red): subsystem failure compromises the scholarly record or blocks all publication; the subsystem MUST meet a T1 SLA and MUST have a documented recovery-time objective of less than one hour. High (amber): subsystem failure degrades a major user-facing capability; the subsystem MUST meet a T2 SLA and MUST have a documented recovery-time objective of less than four hours. Standard (green): subsystem failure degrades a non-critical capability; the subsystem MUST meet a T3 SLA. Criticality is normative: it determines the on-call escalation, the alert routing, and the certification depth.

27.3 Subsystem Contracts

This section publishes the full nine-field contract for every subsystem in the catalogue. The contracts are the authoritative specification: where an implementation disagrees with its contract, the contract prevails until a governed amendment changes it. Each contract is presented as an H3 subsection followed by a nine-row Field/Value table. The order of subsystems follows the canonical publication lifecycle: intake → review → decision → production → freezing → rendering → minting → deposit → preservation → dissemination → cross-cutting services.

SUB-01 — Submission Service

The Submission Service is the platform's intake boundary. It ingests author-submitted manuscripts, validates them against the submission JSON Schema, persists submission files to the Storage Service, and creates the canonical submission record that initiates the editorial workflow. Every published work in Opus Publica begins as a submission accepted by this service.

Field	Value
Inputs	Author manuscript files (PDF, DOCX, JATS XML) up to 100 MB each; submission metadata JSON conforming to submission_schema_v1; cover letter; ethics/COI/funding/data-availability declarations; suggested and opposed reviewer lists; optional preprint URL.
Outputs	submission_id (UUID v4); submission_state transition Drafted→Submitted; SubmissionCreated event on the editorial bus; file storage receipts (SHA-256 + storage_object_id); audit record of submission creation; author receipt email via Notification Service.
Authority	Creates new submission records; transitions

	submission_state from Drafted to Submitted; creates corresponding-author linkage; writes to submission_files table. CANNOT modify article canonical records, assign reviewers, or transition submission_state beyond Submitted.
Failure Conditions	Schema validation failure (SEV-3, reject request); storage write failure (SEV-2, retry then degrade); duplicate submission detected (SEV-3, 409 Conflict); mandatory declaration missing (SEV-3, 422); concurrent modification conflict (SEV-3, 409).
Dependencies	Storage Service (file persistence, SEV-2 on outage); Identity/Auth Service (author authentication, SEV-1 on outage); Audit Service (transition logging, SEV-1 on outage); Notification Service (receipt email, SEV-3 on outage, queued retry).
Performance Budget	p95 submission save < 3 s; p95 file upload < 30 s for ≤ 100 MB; 99.9% submission-intake availability; error rate < 0.5%; idempotent retry window 24 h on submission_id.
Certification	Contract test suite (200+ cases) passing at 100%; chaos test simulating storage outage during intake; idempotency test for retried submissions; schema-validation test against the pinned JSON Schema; Release Certification Authority sign-off (Chapter 22).
Acceptance Tests	Submit a valid manuscript end-to-end; submit with missing mandatory field (rejected with 422); submit duplicate (rejected with 409); concurrent submissions to same draft (exactly one succeeds); file upload exceeding 100 MB (rejected with 413); schema-version mismatch (rejected with 400).
Runtime Evidence	submission_intake_count metric; submission_validation_failure_rate metric; submission_p95_latency histogram; SubmissionCreated event stream; submission_state_transitions audit log; daily reconciliation report comparing submission records to storage receipts.

SUB-02 — Review Service

The Review Service manages the peer-review lifecycle: reviewer assignment, invitation, acceptance, file access, review submission, and review withdrawal. It enforces double-blind review at the platform layer (Chapter 23) and SHALL refuse to create any reviewer assignment without a recorded conflict-of-interest declaration (INV-L-03).

Field	Value
Inputs	Reviewer assignment requests from editors; reviewer accept/decline responses; blinded manuscript files; review forms (recommendation, scores, comments to author/editor/confidential); CoI declarations; review withdrawal requests.
Outputs	review_id (UUID v4); review_state transitions (Invited→Accepted→InProgress→Submitted / Withdrawn); ReviewAssigned, ReviewAccepted, ReviewSubmitted events; blinded file access logs; audit records of every transition.
Authority	Creates reviewer assignments; transitions review_state; records CoI declarations; grants time-bounded access to blinded files. CANNOT make editorial decisions, modify submission_state, or reveal reviewer identities to authors under double-blind mode.
Failure Conditions	Assignment without CoI declaration (SEV-1, blocked at API); double-blind breach attempt (SEV-1, blocked, security alert); reviewer file access failure (SEV-2); review submission after deadline without editor extension (SEV-3); concurrent review modification (SEV-3, 409).
Dependencies	Decision Service (assignment requests flow from editorial decisions); Storage Service (blinded file access); Identity/Auth Service (reviewer authentication); Audit Service; Notification Service (invitation and reminder emails).
Performance Budget	p95 reviewer assignment < 1 s; p95 blinded file retrieval < 2 s; p95 review submission < 2 s; 99.9% availability; reminder emails delivered within 5 min of scheduled time.
Certification	Contract test suite (180+ cases) at 100%; double-blind enforcement test (author API MUST NOT see reviewer identity); CoI-declaration precondition test (INV-L-03); chaos test simulating audit-service outage (review service MUST refuse writes).
Acceptance Tests	Assign reviewer with CoI declaration (succeeds); assign without CoI (422); reviewer accepts invitation; reviewer withdraws with reason; author attempts to view reviewer identity (403); review submitted after deadline without extension (422); blinded file access logged with full provenance.
Runtime Evidence	review_assignment_count metric; review_withdrawal_rate; review_p95_latency;

	review_state_transitions audit stream; double-blind access-attempt alert; CoI-declaration coverage report (target 100%).

SUB-03 — Decision Service

The Decision Service records editorial decisions (Accept, Minor Revision, Major Revision, Reject, Retract) and transitions the submission accordingly. It is the human-judgement boundary: the platform enforces that decisions are made by an authorized editor, are linked to the reviews that informed them, and are recorded with comments to author and internal comments. The platform does not make decisions; it records and enforces them.

Field	Value
Inputs	Editor decision requests (decision_type, comments_to_author, comments_internal, review_round, supporting review_ids); revision deadlines; supersession requests for revised decisions.
Outputs	decision_id (UUID v4); submission_state transition Submitted→Accepted / Rejected / RevisionRequested; DecisionRecorded event; author notification via Notification Service; audit record.
Authority	Creates decision records; transitions submission_state to Accepted, Rejected, or RevisionRequested; supersedes prior decisions within the same round. CANNOT transition to Published, cannot mint DOIs, cannot assign reviewers directly (uses Review Service).
Failure Conditions	Decision by non-authorized editor (SEV-2, 403); decision without supporting reviews when required (SEV-2, 422); illegal transition attempted (SEV-1, blocked); supersession of a finalized decision (SEV-2, 422); concurrent decision recording (SEV-3, 409).
Dependencies	Review Service (supporting reviews); Identity/Auth Service (editor authorization); Audit Service; Notification Service; Submission Service (submission_state update).
Performance Budget	p95 decision recording < 1 s; 99.9% availability; author notification dispatched within 60 s of decision recording.
Certification	Contract test suite (120+ cases) at 100%; authorization matrix test (only authorized editors can decide); legal-transition test (illegal transitions blocked, INV-C-07); supersession chain test.
Acceptance Tests	Authorized editor records Accept (succeeds, submission

	transitions); non-authorized editor attempts decision (403); Accept without supporting reviews (422); illegal transition Reject→Accept without revision (422); supersession records prior decision as superseded; author receives notification.
Runtime Evidence	decision_count by type metric; decision_p95_latency; decision_state_transitions audit stream; authorization_failure alert; daily report of decisions per editor for bias review.

SUB-04 — Composition / Production Service

The Composition (or Production) Service orchestrates the conversion of an accepted submission into a canonical published article: it triggers JATS generation, PDF generation, HTML rendering, figure normalization, reference linking, and the final artifact-verification pass that precedes canonical freezing. It is the conductor of the production pipeline.

Field	Value
Inputs	Accepted submission_id; production job request from Decision Service; source manuscript files; figure files; reference list; metadata (authors, affiliations, funding).
Outputs	production_job_id; production_status transitions (Queued→InProgress→Completed / Failed); canonical artifact set (JATS XML, HTML, PDF, figures); ProductionCompleted event triggering canonical freezing.
Authority	Creates production jobs; invokes JATS Generator, PDF Generator, HTML Renderer, Artifact Verifier; transitions production_status. CANNOT modify submission_state, cannot freeze canonical artifacts (Canonical Freezer owns that).
Failure Conditions	JATS generation failure (SEV-2, retry then degrade); PDF generation failure (SEV-2); HTML rendering failure (SEV-2); artifact verification failure (SEV-1, blocks freezing); production timeout (SEV-2).
Dependencies	JATS Generator (SUB-08); PDF Generator (SUB-06); HTML Renderer (SUB-07); Artifact Verifier (SUB-09); Storage Service; Audit Service; Notification Service (production-complete alert to editors).
Performance Budget	p95 production job completion < 15 min for a typical article; 99.9% job completion within 60 min; 99% artifact-verification pass rate on first attempt.

Certification	Contract test suite (90+ cases) at 100%; end-to-end production test with a known-good manuscript; chaos test simulating PDF generator failure (production MUST fail gracefully, not freeze partial artifacts); idempotency test for retried jobs.
Acceptance Tests	Submit accepted manuscript; production completes within budget; all four artifacts produced and verified; production job retried after transient failure (idempotent, no duplicate artifacts); failure mid-production leaves no partial canonical record.
Runtime Evidence	production_job_count metric; production_p95_duration histogram; production_failure_rate by stage; ProductionCompleted event stream; production_status audit log; artifact verification pass rate.

SUB-05 — Canonical Freezer

The Canonical Freezer takes the verified artifact set produced by Composition and commits it to the canonical store as an immutable, content-addressed package. Freezing is the point of no return: after freezing, the artifacts SHALL NOT be modified by any path other than a governed version (Chapter 12) or a governed retraction (INV-M-01, INV-V-01). The Freezer records the canonical_frozen_timestamp that anchors every downstream deposit and dissemination event.

Field	Value
Inputs	Verified artifact set from Composition; canonical_freeze_request with submission_id, version_number, and parent_version_id; metadata snapshot (article, authors, journal, issue).
Outputs	canonical_package_id; content-addressed SHA-256 for each artifact; canonical_frozen_timestamp (UTC, monotonic); immutable storage write; CanonicalFrozen event; audit record.
Authority	Creates canonical_package records; writes immutable artifacts to Storage Service with read-only ACL; records canonical_frozen_timestamp. CANNOT modify previously frozen artifacts, cannot transition article_state to Published (Decision/Publish boundary owns that).
Failure Conditions	Artifact SHA-256 mismatch (SEV-1, freeze refused); missing parent version for non-original (SEV-1, INV-L-05); storage write failure (SEV-1, freeze failed); concurrent freeze attempt for same submission (SEV-2, 409); verification missing (SEV-1).

Dependencies	Artifact Verifier (SUB-09, mandatory precondition); Storage Service (immutable write); Audit Service; DOI Minter (downstream consumer of canonical_package_id).
Performance Budget	p95 freeze operation < 30 s; 99.99% freeze durability (artifact MUST survive any single-component failure); 99.9% availability.
Certification	Contract test suite (70+ cases) at 100%; immutability test (write attempt after freeze MUST fail with 403); SHA-256 determinism test (same input produces same hash across runs); chaos test simulating storage failure during freeze (freeze MUST fail, not partially commit).
Acceptance Tests	Freeze a verified artifact set (succeeds); attempt to modify frozen artifact (403); freeze non-original without parent (422, INV-L-05); freeze with SHA-256 mismatch (422); re-freeze same submission (409); canonical_frozen_timestamp recorded in audit log.
Runtime Evidence	canonical_freeze_count metric; freeze_p95_latency; freeze_failure rate; CanonicalFrozen event stream; immutable-write audit log; daily reconciliation verifying frozen artifacts are unchanged.

SUB-06 — PDF Generator

The PDF Generator renders the canonical PDF for an article from the JATS XML and figure set. It produces a PDF/A-2u compliant artefact suitable for long-term preservation and reader download. The generated PDF is content-addressed (SHA-256) and SHALL be identical across regenerations from the same input (deterministic build, INV-M-01 support).

Field	Value
Inputs	JATS XML artifact; figure files (PNG, JPEG, SVG, PDF); journal PDF template (typography, header, footer); metadata snapshot (DOI, authors, license).
Outputs	PDF/A-2u artifact; artifact_sha256; artifact_size_bytes; pdf_generation_completed_at timestamp; PDFGenerated event; audit record.
Authority	Creates PDF artifacts in the Storage Service; records pdf_generation_completed_at. CANNOT modify JATS XML, cannot freeze canonical artifacts, cannot update article_state.
Failure Conditions	Render failure (SEV-2, retry then degrade); PDF/A validation failure (SEV-2, reject); font/template missing

	(SEV-3); non-deterministic output across regenerations (SEV-2, blocks freezing).
Dependencies	Storage Service (input retrieval, output write); JATS Generator (upstream); Audit Service; Search/Index Service (PDF indexing for full-text search).
Performance Budget	p95 PDF generation < 60 s for a typical article; 99.5% availability; PDF/A-2u validation pass rate ≥ 99%.
Certification	Contract test suite (60+ cases) at 100%; PDF/A-2u conformance test (veraPDF); determinism test (regenerate from same input, compare SHA-256); chaos test simulating font-substitution failure.
Acceptance Tests	Generate PDF from valid JATS (succeeds, PDF/A-2u valid); generate from malformed JATS (fails with structured error); regenerate same input → identical SHA-256; PDF/A validation failure (rejected, no artifact written); large-article test (200+ pages) within budget.
Runtime Evidence	pdf_generation_count metric; pdf_generation_p95_latency; pdf_a_validation_failure_rate; PDFGenerated event stream; determinism-reconciliation report (regenerate sample, compare hashes).

SUB-07 — HTML Renderer

The HTML Renderer produces the canonical HTML representation of an article from the JATS XML. The HTML is the primary reader-facing rendering, is content-addressed, and SHALL be accessible at the article landing page. The renderer applies the journal's HTML template, citation style, and accessibility transformations (ARIA labels, alt text, semantic structure).

Field	Value
Inputs	JATS XML artifact; figure files; journal HTML template; citation style (CSL); metadata snapshot.
Outputs	Canonical HTML artifact (article_html_canonical); artifact_sha256; HTMLRendered event; audit record; landing-page cache invalidation signal.
Authority	Creates HTML artifacts in Storage Service; updates article_html_canonical reference; triggers landing-page cache invalidation. CANNOT modify JATS XML, cannot transition article_state.
Failure Conditions	Render failure (SEV-2); accessibility-validation failure (SEV-3, warn then degrade); citation-style failure (SEV-3);

	non-deterministic output (SEV-2).
Dependencies	Storage Service; JATS Generator; Audit Service; Search/Index Service (HTML indexing); Notification Service (cache invalidation).
Performance Budget	p95 HTML render < 20 s; p95 landing-page load < 500 ms (cached); 99.5% availability; WCAG 2.1 AA conformance.
Certification	Contract test suite (60+ cases) at 100%; WCAG 2.1 AA automated audit (axe-core); determinism test; CSL citation-rendering test against a reference set.
Acceptance Tests	Render HTML from valid JATS (succeeds); WCAG audit passes; citation rendering matches reference; regenerate → identical SHA-256; render with malformed JATS (fails with structured error).
Runtime Evidence	html_render_count metric; html_render_p95_latency; wcag_audit_failure_rate; HTMLRendered event stream; landing-page load latency.

SUB-08 — JATS Generator

The JATS Generator produces the canonical JATS XML representation of an article from the submission source. JATS XML is the platform's primary canonical format (P1) and SHALL validate against the official NLM JATS DTD (version pinned per article). The JATS artifact is the source of truth for all downstream renderings and deposits.

Field	Value
Inputs	Source manuscript (DOCX or submitted JATS); figure files; reference list with DOIs; metadata snapshot (authors, affiliations, funding, license).
Outputs	JATS XML artifact (article_jats_xml); artifact_sha256; JATSGenerated event; validation report against pinned NLM schema; audit record.
Authority	Creates JATS XML artifacts in Storage Service; updates article_jats_xml reference. CANNOT modify submission source, cannot transition article_state, cannot freeze canonical artifacts.
Failure Conditions	NLM schema validation failure (SEV-2, blocks freezing per INV-L-02); reference DOI resolution failure (SEV-3, warn); render failure (SEV-2); non-deterministic output (SEV-2).
Dependencies	Storage Service; Crossref Depositor (reference DOI verification, upstream); Audit Service; Artifact Verifier

	(downstream consumer).
Performance Budget	p95 JATS generation < 30 s; 99.9% availability; NLM schema validation pass rate ≥ 99% on first attempt.
Certification	Contract test suite (80+ cases) at 100%; NLM JATS DTD validation test (pinned version); reference-linking test (resolved DOIs recorded); determinism test.
Acceptance Tests	Generate JATS from valid DOCX (succeeds, NLM-valid); generate from malformed source (fails with structured error); NLM validation failure (rejected, INV-L-02 enforced); regenerate → identical SHA-256; large-article test with 200+ references within budget.
Runtime Evidence	jats_generation_count metric; jats_generation_p95_latency; nlm_validation_failure_rate; reference_resolution_rate; JATSGenerated event stream.

SUB-09 — Artifact Verifier

The Artifact Verifier validates every artifact produced by the production pipeline before canonical freezing. It enforces format-specific conformance (JATS NLM schema, PDF/A-2u, HTML WCAG), content-addressing determinism, cross-artifact consistency (JATS ↔ HTML ↔ PDF metadata match), and the presence of mandatory metadata (DOI, license, authors). The Verifier is the gatekeeper of INV-L-01 and INV-L-02.

Field	Value
Inputs	Artifact set from Composition (JATS XML, HTML, PDF, figures); metadata snapshot; pinned schema versions (NLM JATS DTD, PDF/A-2u, WCAG 2.1 AA).
Outputs	verification_report (per-artifact pass/fail, per-check evidence); VerificationPassed / VerificationFailed event; audit record; freeze authorization signal to Canonical Freezer.
Authority	Issues freeze authorization (positive) or freeze refusal (negative). CANNOT create artifacts, cannot freeze artifacts directly, cannot modify schemas.
Failure Conditions	Format validation failure (SEV-1, blocks freezing); cross-artifact metadata mismatch (SEV-1); missing mandatory metadata (SEV-1); determinism failure (SEV-2); verifier service outage (SEV-2, freezing blocked).
Dependencies	Storage Service (artifact retrieval); JATS Generator, PDF Generator, HTML Renderer (upstream producers);

	Canonical Freezer (downstream consumer); Audit Service.
Performance Budget	p95 verification of full artifact set < 60 s; 99.9% availability; 0 false-positive freeze authorizations tolerated.
Certification	Contract test suite (100+ cases) at 100%; negative-test suite (each known failure mode MUST be detected); NLM and PDF/A conformance tests pinned to specific schema versions; chaos test simulating verifier outage (freezing MUST block).
Acceptance Tests	Verify a known-good artifact set (passes, freeze authorized); verify set with malformed JATS (fails, freeze refused, INV-L-02); verify set with missing DOI (fails, INV-L-01); verify set with metadata mismatch across formats (fails); verifier outage → freeze refused.
Runtime Evidence	verification_count metric; verification_failure_rate by check; verification_p95_latency; VerificationPassed / VerificationFailed event streams; freeze-authorization audit log.

SUB-10 — DOI Minter

The DOI Minter allocates a Digital Object Identifier for an article version from the journal's Crossref DOI prefix. The DOI is write-once: once minted, the DOI-to-version binding is immutable (INV-M-03). The Minter does NOT register the DOI with Crossref; that is the Crossref Depositor's responsibility (SUB-11).

Field	Value
Inputs	canonical_package_id from Canonical Freezer; journal_doi_prefix; article_id; version_number; suffix-generation strategy (sequential or UUID).
Outputs	identifier_id; identifier_value (full DOI, e.g. 10.57939/OP-XXXX); identifier_minted_at timestamp; DOIMinted event; audit record.
Authority	Creates identifier records of type DOI; transitions identifier_deposit_status to Pending; binds DOI to canonical_package_id. CANNOT register the DOI with Crossref, cannot mint non-DOI identifiers, cannot re-mint a DOI for the same version.
Failure Conditions	DOI prefix not configured for journal (SEV-2); suffix collision (SEV-2, retry with new suffix); mint for already-minted version (SEV-1, INV-M-03 violation, blocked); database write failure (SEV-2).

Dependencies	Canonical Freezer (upstream, mandatory precondition); Identity/Auth Service; Audit Service; Crossref Depositor (downstream consumer).
Performance Budget	p95 mint < 500 ms; 99.9% availability; 0 duplicate DOIs tolerated (INV-I-02).
Certification	Contract test suite (50+ cases) at 100%; uniqueness test (INV-I-02); write-once test (INV-M-03); chaos test simulating database failure mid-mint (no partial state).
Acceptance Tests	Mint DOI for canonical package (succeeds); mint for same package twice (second returns existing DOI, no new mint); suffix collision (retried, no duplicate); DOI value matches journal prefix; identifier_deposit_status = Pending.
Runtime Evidence	doi_mint_count metric; doi_mint_p95_latency; duplicate_doi_attempt alert; DOIMinted event stream; identifier_state audit log; reconciliation report comparing minted DOIs to Crossref registration.

SUB-11 — Crossref Depositor

The Crossref Depositor registers minted DOIs with Crossref via the Crossref REST API, depositing the article metadata (JATS XML, references, authors, ORCID iDs, license, funding). The Depositor is idempotent: a retried deposit for the same DOI SHALL NOT create duplicate records at Crossref (INV-C-05). The Depositor records the Crossref deposit receipt and reconciles it against the Crossref REST API on a defined cadence.

Field	Value
Inputs	identifier_id (minted DOI); canonical_package_id; JATS XML artifact; metadata snapshot; Crossref API credentials.
Outputs	article_crossref_deposit_id; identifier_deposit_status transition Pending→Deposited→Reconciled; deposit receipt URL; CrossrefDeposited event; audit record.
Authority	Transitions identifier_deposit_status; records Crossref deposit receipts; invokes Crossref REST API. CANNOT mint DOIs, cannot modify canonical artifacts, cannot transition article_state.
Failure Conditions	Crossref API outage (SEV-2, retry with exponential backoff); deposit rejected by Crossref (SEV-2, alert editorial); receipt reconciliation mismatch (SEV-2); duplicate deposit detected (SEV-1, INV-C-05, blocked).
Dependencies	DOI Minter (upstream); Storage Service (JATS retrieval); Identity/Auth Service (credentials); Audit Service; ORCID

	Integrator (author ORCID iDs included in deposit).
Performance Budget	p95 deposit submission < 30 s; 99% of deposits reconciled within 24 h (INV-R-05); 99.9% service availability; idempotent retry window 7 d.
Certification	Contract test suite (70+ cases) at 100%; idempotency test (retry same deposit, no duplicate); reconciliation test against Crossref test system; chaos test simulating Crossref outage (deposits queued, retried).
Acceptance Tests	Deposit minted DOI (succeeds, receipt recorded); retry same deposit (idempotent, no duplicate); Crossref outage → deposit queued and retried; reconciliation finds mismatch → alert raised; deposit rejected by Crossref → editorial alert.
Runtime Evidence	crossref_deposit_count metric; crossref_deposit_failure_rate; crossref_reconciliation_lag; CrossrefDeposited event stream; deposit_receipt audit log; daily reconciliation report.

SUB-12 — ORCID Integrator

The ORCID Integrator manages the OAuth-2.0 three-legged flow with ORCID, retrieves authenticated author ORCID iDs, verifies them against the ORCID public API, and writes back publication records to the author's ORCID profile when consent is granted. The Integrator SHALL refuse to record an ORCID iD that does not resolve via the ORCID public API (INV-R-03).

Field	Value
Inputs	ORCID OAuth authorization code; author_id; canonical_package_id (for write-back); ORCID API credentials; author consent record.
Outputs	author_orcid_access_token; author_orcid_refresh_token; author_orcid_id (verified); author_orcid_writeback_enabled flag; ORCIDWritebackCompleted event; audit record.
Authority	Records ORCID iDs and tokens against author records; invokes ORCID public and member APIs; writes back publication records to ORCID profiles. CANNOT modify canonical article records, cannot mint ORCID iDs.
Failure Conditions	ORCID iD does not resolve via public API (SEV-2, INV-R-03, refused); OAuth flow failure (SEV-2); write-back failure (SEV-3, queued retry); token expired (SEV-3, refresh).

Dependencies	Identity/Auth Service (author authentication); Audit Service; Crossref Depositor (publication record source); Notification Service (write-back failure alerts).
Performance Budget	p95 ORCID iD verification < 2 s; p95 write-back < 10 s; 99.5% availability; token refresh proactive 7 d before expiry.
Certification	Contract test suite (60+ cases) at 100%; OAuth flow test against ORCID sandbox; iD resolution test (INV-R-03); write-back idempotency test.
Acceptance Tests	Complete OAuth flow (succeeds, token recorded); verify non-resolving ORCID iD (refused, INV-R-03); write-back with consent (succeeds); write-back without consent (refused); token expiry → proactive refresh; write-back failure → queued retry.
Runtime Evidence	orcid_verification_count metric; orcid_writeback_count; orcid_writeback_failure_rate; ORCIDWritebackCompleted event stream; token-expiry dashboard.

SUB-13 — Preservation Depositor (multi-backend)

The Preservation Depositor packages canonical artifacts and metadata into a BagIt bag and deposits them to one or more dark-archive backends (CLOCKSS, LOCKSS, Portico) per the journal's preservation policy. The Depositor is idempotent per (article, backend) pair and SHALL queue a deposit for every PublicationCompleted event (INV-L-04).

Field	Value
Inputs	canonical_package_id; PublicationCompleted event; journal preservation policy; backend credentials; BagIt manifest schema.
Outputs	preservation_receipt_id per backend; preservation_deposit_state transition Queued→Deposited→Reconciled; receipt URL; PreservationDeposited event; audit record.
Authority	Transitions preservation_deposit_state; records preservation receipts; invokes backend SWORD/HTTP deposit APIs. CANNOT modify canonical artifacts, cannot mint DOIs, cannot transition article_state.
Failure Conditions	Backend outage (SEV-2, queued retry); deposit rejected (SEV-2, alert); receipt reconciliation mismatch (SEV-2); duplicate deposit with different content (SEV-1, INV-C-06, blocked); missing preservation policy (SEV-1, INV-L-04

	violation).
Dependencies	Canonical Freezer (upstream); Storage Service (artifact packaging); Identity/Auth Service (credentials); Audit Service; Notification Service (failure alerts).
Performance Budget	p95 deposit submission < 5 min; 100% of PublicationCompleted events have a queued deposit (INV-L-04); 99% of deposits reconciled within 7 d; 99.9% availability.
Certification	Contract test suite (80+ cases) at 100%; idempotency test per (article, backend); BagIt conformance test; multi-backend failover test; chaos test simulating single-backend outage.
Acceptance Tests	Deposit to single backend (succeeds, receipt recorded); deposit to multiple backends (all succeed); retry same deposit (idempotent); backend outage → queued and retried; reconciliation mismatch → alert; missing preservation policy → INV-L-04 alert.
Runtime Evidence	preservation_deposit_count by backend; preservation_reconciliation_lag; preservation_failure_rate; PreservationDeposited event stream; deposit_receipt audit log; weekly reconciliation report.

SUB-14 — OAI-PMH Endpoint

The OAI-PMH Endpoint exposes the platform's published articles to harvesters (OpenAIRE, DOAJ, institutional repositories) via the Open Archives Initiative Protocol for Metadata Harvesting (OAI-PMH 2.0). It SHALL support the oai_dc and OpenAIRE metadata formats and SHALL expose every published article with a stable OAI-PMH identifier (INV-R-04).

Field	Value
Inputs	OAI-PMH verbs (Identify, ListRecords, GetRecord, ListIdentifiers, ListSets, ListMetadataFormats); resumption tokens; metadata format requests.
Outputs	OAI-PMH XML responses per the OAI-PMH 2.0 schema; resumption tokens; article_oai_pmh_identifier records; access logs; audit record of harvest events.
Authority	Reads canonical article metadata; serves OAI-PMH responses. CANNOT modify article records, cannot mint identifiers, cannot trigger deposits. Read-only service.
Failure Conditions	Invalid verb (SEV-3, 400); bad resumption token (SEV-3,

	400); metadata format not supported (SEV-3, 400); service outage (SEV-2, harvesters blocked); response exceeds size limit (SEV-3, resumption).
Dependencies	Search/Index Service (metadata retrieval); Storage Service (cached metadata); Audit Service; Identity/Auth Service (rate-limit policy).
Performance Budget	p95 GetRecord response < 500 ms; p95 ListRecords response < 2 s for 100 records; 99.5% availability; rate limit 100 requests/min per IP.
Certification	Contract test suite (50+ cases) at 100%; OAI-PMH 2.0 schema validation; OpenAIRE compliance test; resumption-token test; load test against realistic harvester traffic.
Acceptance Tests	Identify response valid; ListMetadataFormats includes oai_dc and OpenAIRE; ListRecords returns published articles; GetRecord retrieves by article_oai_pmh_identifier; resumption token works across pages; malformed verb rejected with 400.
Runtime Evidence	oai_pmh_request_count by verb; oai_pmh_p95_latency; oai_pmh_error_rate; harvester IP distribution; harvest-event audit log; weekly reconciliation verifying every published article has an OAI-PMH record (INV-R-04).

SUB-15 — SWORD Endpoint

The SWORD Endpoint implements the SWORD v3 server protocol, allowing external systems (institutional repositories, aggregators) to deposit and withdraw content programmatically. In RC1 the SWORD endpoint is Experimental: deployed, contract-tested, but not bound to a T1 SLA.

Field	Value
Inputs	SWORD v3 service document requests; deposit requests (BagIt or METS packages); withdrawal requests; authentication credentials.
Outputs	SWORD v3 service document; deposit receipts; withdrawal acknowledgements; SWORDDepositReceived event; audit record.
Authority	Receives deposits into an ingestion queue; grants receipts. CANNOT modify canonical records directly (deposits enter the Composition pipeline), cannot transition article_state.
Failure Conditions	Authentication failure (SEV-3, 401); malformed package

	(SEV-3, 400); package too large (SEV-3, 413); service outage (SEV-2); downstream Composition failure (SEV-3, queued).
Dependencies	Identity/Auth Service (credential verification); Composition Service (downstream consumer); Storage Service (package staging); Audit Service; Notification Service.
Performance Budget	p95 service-document response < 1 s; p95 deposit receipt < 10 s; 99.0% availability (T3); maximum package size 1 GB.
Certification	Contract test suite (50+ cases) at 100%; SWORD v3 conformance test; authentication test; package-validation test; documented as Experimental in the Capability Map.
Acceptance Tests	Retrieve service document (succeeds); deposit valid package (succeeds, receipt issued); deposit malformed package (400); deposit oversized package (413); unauthenticated deposit (401); withdrawal acknowledged.
Runtime Evidence	sword_request_count by operation; sword_p95_latency; sword_error_rate; SWORDDepositReceived event stream; deposit_receipt audit log.

SUB-16 — Audit Service

The Audit Service records every state transition in the platform as a cryptographically chained, append-only audit record. It is the platform's ultimate defence against silent corruption (INV-A-04). Every other subsystem depends on the Audit Service; an audit-service outage SHALL cause write-blocking across the platform (Chapter 18).

Field	Value
Inputs	Audit-record writes from every subsystem (actor, timestamp, operation, entity, before-state hash, after-state hash, reason, correlation id); chain-verification requests.
Outputs	audit_id; audit_chain_hash; audit_predecessor_hash; immutable append to audit log; chain-verification reports; AuditRecorded internal event.
Authority	Appends to the audit log (append-only); computes chain hashes; serves verification queries. CANNOT modify or delete audit records (INV-M-02), cannot transition any domain state, cannot issue commands to other

	subsystems.
Failure Conditions	Audit log write failure (SEV-1, all writes block); chain hash mismatch (SEV-1, INV-A-03, immediate security alert); audit record incomplete (SEV-1, INV-A-02, rejected at write); audit service unavailable (SEV-1, platform-wide write-blocking).
Dependencies	Storage Service (audit log persistence, replicated); Identity/Auth Service (actor authentication); Notification Service (SEV-1 alerts).
Performance Budget	p95 audit write < 50 ms; 99.99% availability; 0 audit records lost tolerated; chain-verification job runs hourly.
Certification	Contract test suite (90+ cases) at 100%; append-only test (INV-M-02); chain-integrity test (INV-A-03); completeness test (INV-A-02); chaos test simulating audit-service outage (all writes MUST block).
Acceptance Tests	Record a transition (succeeds, chain hash computed); attempt to modify an audit record (403, INV-M-02); record with missing mandatory field (422, INV-A-02); tamper with chain → verification detects mismatch; audit outage → platform-wide write-blocking.
Runtime Evidence	audit_record_count metric; audit_write_p95_latency; chain_verification_result; AuditRecorded event stream; hourly chain-verification report; SEV-1 alert on any chain mismatch.

SUB-17 — Notification Service

The Notification Service dispatches transactional emails and in-app notifications to platform users (authors, reviewers, editors). It abstracts the email provider, queues notifications for retry, and SHALL guarantee at-least-once delivery. Notifications are idempotent on a (recipient, template, correlation_id) key.

Field	Value
Inputs	Notification requests (template_id, recipient, context, correlation_id); in-app notification reads; dismissal events.
Outputs	notification_id; delivery state transitions (Queued→Sent→Delivered / Failed); email provider receipts; NotificationSent event; audit record.
Authority	Sends emails via provider; records delivery receipts; transitions notification delivery state. CANNOT modify

	domain records, cannot trigger workflow transitions, cannot access canonical artifacts.
Failure Conditions	Email provider outage (SEV-2, queued retry); template rendering failure (SEV-3); recipient address invalid (SEV-3); queue overflow (SEV-2); delivery timeout (SEV-3).
Dependencies	Email provider (external); Identity/Auth Service (recipient verification); Audit Service; Storage Service (template cache).
Performance Budget	p95 dispatch < 60 s; 99.5% delivery within 5 min; 99.5% availability; queue depth alert at 1000 messages.
Certification	Contract test suite (50+ cases) at 100%; idempotency test (same correlation_id → single delivery); template-render test; provider failover test.
Acceptance Tests	Send notification (succeeds, delivered); retry on provider outage (idempotent); invalid recipient (skipped, logged); template render failure (queued for human review); queue overflow → alert.
Runtime Evidence	notification_count by template; notification_delivery_p95_latency; notification_failure_rate; NotificationSent event stream; queue-depth metric.

SUB-18 — Search / Index Service

The Search/Index Service maintains a full-text and metadata index of every published article, powering the public search interface, the OAI-PMH responses, and editorial discovery. It SHALL be eventually consistent with canonical storage; reconciliation jobs SHALL detect drift and re-index affected articles.

Field	Value
Inputs	CanonicalFrozen, ArticlePublished, ArticleRetracted events; JATS XML, HTML, and metadata snapshots; search queries from public and editorial surfaces.
Outputs	Index entries (full-text + metadata); search result sets; faceted aggregations; IndexUpdated event; reconciliation reports.
Authority	Creates and updates index entries; serves search queries. CANNOT modify canonical records, cannot transition article_state, cannot issue deposits. Read-only with respect to canonical state.
Failure Conditions	Index write failure (SEV-2, retry); index drift from canonical (SEV-2, reconciliation job alerts); search latency

	breach (SEV-3); corrupt index segment (SEV-2, rebuild).
Dependencies	Canonical Freezer (event source); Storage Service (artifact retrieval); Audit Service; OAI-PMH Endpoint (consumer).
Performance Budget	p95 search query < 200 ms; p95 index update < 5 s; 99.5% availability; reconciliation job daily.
Certification	Contract test suite (60+ cases) at 100%; eventual-consistency test (frozen artifact → indexed within 5 s); reconciliation test (drift detected and corrected); faceted-search test.
Acceptance Tests	Index a frozen article (succeeds, searchable); search returns expected results; reconciliation detects drift and re-indexes; retraction removes from default search; facet counts match.
Runtime Evidence	search_query_count metric; search_p95_latency; index_lag metric; index_drift_count; IndexUpdated event stream; daily reconciliation report.

SUB-19 — Identity / Auth Service

The Identity/Auth Service manages user identities, credentials, sessions, roles, and permissions. It issues short-lived access tokens and refresh tokens, enforces MFA, and SHALL refuse any request lacking a valid principal. Every other subsystem depends on this service for actor authentication (INV-A-01).

Field	Value
Inputs	Login requests (email, password, MFA); OAuth flows (ORCID, SAML); token refresh requests; role/permission queries; user-management operations (admin only).
Outputs	Access tokens (short-lived JWT); refresh tokens; user_id; session records; role_id and role_permissions; AuthenticationSucceeded / AuthenticationFailed events; audit record.
Authority	Creates and updates user records; issues and revokes tokens; assigns roles. CANNOT modify domain records (articles, submissions, reviews), cannot transition workflow state, cannot access canonical artifacts beyond metadata.
Failure Conditions	Credential validation failure (SEV-3, 401); MFA failure (SEV-3, 401); brute-force attempt (SEV-2, lockout); token tampering (SEV-2, 401, security alert); service outage (SEV-1, platform-wide authentication blocked).
Dependencies	Storage Service (user store); Audit Service (auth events);

	Notification Service (password reset, MFA alerts); external ORCID / SAML providers.
Performance Budget	p95 token issuance < 500 ms; p95 token validation < 50 ms; 99.9% availability; MFA enrolment 100% for editorial roles.
Certification	Contract test suite (80+ cases) at 100%; brute-force lockout test; MFA enforcement test for editorial roles; token-tampering test; chaos test simulating provider outage (local auth continues for active sessions).
Acceptance Tests	Login with valid credentials + MFA (succeeds, token issued); login with invalid credentials (401); brute-force → lockout after 5 attempts; tampered token rejected; MFA enrolment enforced for editor role; service outage → active sessions preserved.
Runtime Evidence	authentication_count metric; authentication_failure_rate; brute_force_lockout_count; token_issuance_p95_latency; AuthenticationSucceeded / AuthenticationFailed event streams.

SUB-20 — Storage Service

The Storage Service is the platform's content-addressed object store: it persists canonical artifacts, submission files, pre-publication working files, and the audit log. Every artifact SHALL be content-addressed by SHA-256 (INV-R-02) and SHALL be replicated across at least two availability zones. The Storage Service is the substrate on which immutability (INV-M-01) and conservation (INV-V-01) are built.

Field	Value
Inputs	Object write requests (bytes + content_type + tier); object read requests (by storage_object_id or SHA-256); replication configuration; legal-hold requests.
Outputs	storage_object_id (UUID); storage_sha256; storage_size_bytes; storage_replica_count; immutable-write receipt; ObjectStored event; audit record.
Authority	Creates and reads storage objects; enforces immutability on frozen artifacts; grants time-bounded read access via signed URLs. CANNOT delete published artifacts (INV-V-01), cannot transition article_state, cannot interpret artifact content.
Failure Conditions	Write failure (SEV-1 for canonical artifacts, blocks freezing); read failure (SEV-2); replication failure (SEV-2, alert); orphan object detected (SEV-3, INV-C-01); SHA-256

	collision (SEV-1, theoretical, immediate escalation).
Dependencies	Underlying object-storage infrastructure (S3-compatible); Audit Service; Identity/Auth Service (signed-URL issuance); Canonical Freezer (upstream writer).
Performance Budget	p95 write < 2 s for 10 MB; p95 read < 500 ms for 1 MB; 99.99% durability; 99.99% availability; replication lag < 60 s.
Certification	Contract test suite (70+ cases) at 100%; immutability test (INV-M-01, write-after-freeze fails); content-addressing test (INV-R-02, retrieve by SHA-256); orphan-detection test (INV-C-01); replication-failure chaos test.
Acceptance Tests	Store an artifact (succeeds, SHA-256 computed); retrieve by SHA-256 (succeeds); attempt to modify frozen artifact (403, INV-M-01); detect orphan object (INV-C-01 alert); replication failure → alert; legal-hold prevents deletion.
Runtime Evidence	storage_object_count metric; storage_write_p95_latency; storage_read_p95_latency; replication_lag; orphan_object_count (INV-C-01 monitor); ObjectStored event stream; daily orphan-detection reconciliation report.

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

This chapter establishes the constitutional contract regime for every subsystem of Opus Publica. Its purpose is to make every subsystem's behaviour explicit, bounded, and verifiable: a subsystem's contract states what it accepts, what it produces, what it is authorized to change, when it is considered failed, what it depends on, what performance it must deliver, what certification it must pass, what tests verify it, and what telemetry proves its correct operation. Without such contracts, the platform's runtime behaviour is emergent rather than specified; with them, the platform's runtime behaviour is derivable from this Constitution. The chapter operationalizes Foundational Principle I (Runtime Truth) at the subsystem granularity: every contract field has a runtime evidence stream that proves the contract is honoured.

2. Scope

In scope: the contract template (nine fields, section 27.1); the Subsystem Catalogue of all twenty production subsystems (section 27.2); and the per-subsystem contract tables for each of those twenty subsystems (section 27.3). Out of scope: the SLI/SLO definitions themselves (Chapter 17), the certification regime itself (Chapter 22), the failure-response runbooks (Chapter 21), and the canonical field-level contracts (Chapter 28). A subsystem contract references these chapters but does not replicate their content; the contract is the binding layer between them.

3. Definitions

Subsystem: a deployable unit of the platform with a published contract, a named owner, and an SLA tier. Contract: the normative, versioned specification of a subsystem's nine-field behaviour. Authority: the precise enumeration of state a subsystem may create, transition, or delete; anything not enumerated is forbidden. Runtime Evidence: the telemetry a subsystem emits to prove its contract is honoured at runtime. Production Acceptance Gate: the release gate (Foundational Principle II) that refuses to release a subsystem whose contract is missing, stale, or unverified.

4. Responsibilities

Ownership of the contract regime is as follows:

- **Principal Architect:** Accountable for the contract template, the Subsystem Catalogue, and the per-subsystem contracts.
- **Subsystem Owners:** Responsible for keeping their subsystem's contract accurate, complete, and version-controlled.
- **Release Certification Authority:** Responsible for refusing to release any subsystem whose contract is missing, stale, or unverified.
- **Engineering Leadership Team:** Responsible for reviewing the Subsystem Catalogue at every release and flagging contract drift.
- **Chief Systems Architect:** Accountable for refusing contract amendments that weaken invariants or remove runtime evidence obligations.

5. Design Principles

Contracts are explicit (every behaviour is stated, not implied), bounded (Authority enumerates what is permitted and forbids the rest), versioned (every contract change is recorded and reviewed), evidence-bound (every contract field has a runtime evidence stream), and conservative (when in doubt, the contract is stated strictly rather than loosely). A contract that cannot be tested is not a contract; it is an aspiration. A contract whose runtime evidence is not collected is not honoured; it is asserted. The Production Acceptance Gate treats asserted-but-unverified contracts as failed.

6. Engineering Requirements

Every subsystem deployed to production SHALL have a published contract in the form of section 27.1, registered in the Subsystem Catalogue, and elaborated as a per-subsystem table in section 27.3. Every

contract SHALL name an owner, a criticality, and an SLA tier. Every contract change SHALL trigger a contract-diff review by the Principal Architect; breaking changes SHALL require a Constitution Amendment per Chapter 36. Every subsystem SHALL emit the runtime evidence specified in its contract; absence of evidence SHALL be treated as contract failure and SHALL block release. The Engineering Leadership Team SHALL review the Subsystem Catalogue at every release.

7. Engineering Constraints

Contracts constrain the platform to bounded authority: a subsystem MAY NOT exercise authority not enumerated in its contract, even if the underlying credentials would technically permit it. Contracts constrain the platform to evidence-bound release: a subsystem MAY NOT be released without its specified runtime evidence streams operational. Contracts constrain the platform to orderly change: a breaking change MAY NOT be deployed without a Constitution Amendment. The Chief Systems Architect SHALL NOT grant exceptions to the nine-field template; appendix fields are permitted but the canonical nine MAY NOT be omitted or reordered.

8. Runtime Behaviour

At runtime, every subsystem honours its contract: it accepts only the inputs enumerated, produces only the outputs enumerated, exercises only the authority enumerated, fails under the conditions enumerated, and emits the runtime evidence enumerated. The observability stack aggregates the evidence streams into per-subsystem dashboards. A subsystem that emits evidence outside its contract (for example, writing to a table it does not own) is in contract violation and SHALL trigger a SEV-2 alert. A subsystem that fails to emit contractually required evidence is in contract violation and SHALL trigger a SEV-2 alert, escalating to SEV-1 if the evidence gap persists for more than one cadence interval.

9. Failure Conditions

Contract failure conditions include:

- **Missing contract:** a deployed subsystem has no published contract in section 27.3.
- **Stale contract:** the deployed subsystem's behaviour diverges from its published contract.
- **Authority breach:** a subsystem exercises authority not enumerated in its contract.
- **Evidence gap:** a subsystem fails to emit contractually required runtime evidence.
- **Unverified release:** a subsystem is released to production without its acceptance tests passing or its certification complete.
- **Breaking-change-without-amendment:** a breaking contract change is deployed without a Constitution Amendment.

10. Recovery Procedures

Contract-violation recovery follows the incident-response regime (Chapter 21) with the additional requirement that recovery SHALL restore contract compliance, not merely mask the symptom. Missing-contract recovery: halt the subsystem, draft and publish the contract, and re-release after certification. Stale-contract recovery: identify the divergence, either align the implementation to the contract or

amend the contract, and re-release. Authority-breach recovery: revoke the unauthorized access path, audit for illicit state changes, and re-release with additional enforcement. Evidence-gap recovery: instrument the missing telemetry, backfill where possible, and re-release. Every recovery SHALL be recorded in the audit log with a full before/after analysis.

11. Security Considerations

Contracts are a primary security control: bounded authority limits the blast radius of a compromised subsystem. A subsystem that holds credentials for state it is not authorized to change presents a lateral-movement risk; the Identity/Auth Service SHALL scope credentials to the subsystem's enumerated authority. The audit trail of contract-violation detections is itself a security artefact and SHALL be reviewed monthly by the Security Officer. Authority-breach detections SHALL trigger immediate security incidents regardless of apparent intent, because an authority breach is, by definition, either a compromise or a critical engineering failure.

12. Performance Considerations

Each contract specifies a Performance Budget borrowed from Chapter 17 (SLI/SLO Constitution) and bound to the subsystem's error budget. Performance budgets are normative: a subsystem that exceeds its latency SLO for two consecutive measurement periods is in contract violation, regardless of whether the latency is user-visible. Error budgets are consumed by every breach; budget exhaustion SHALL freeze feature work on the subsystem until the budget is restored. Contracts that depend on external services (Crossref, ORCID, preservation backends) SHALL specify graceful-degradation behaviour that bounds the platform's exposure to external-service latency.

13. Observability Requirements

Every contract's Runtime Evidence field enumerates the metrics, structured logs, traces, audit records, and reconciliation reports the subsystem SHALL emit. Each evidence stream SHALL have a named dashboard panel and a named alert. The observability stack SHALL aggregate per-subsystem dashboards into a single Constitution Compliance dashboard, reviewed daily by the Engineering Leadership Team. A subsystem in 'evidence-gap' state for more than one cadence interval is in SEV-2 contract violation. The Release Certification Authority SHALL verify evidence-stream operability before every release.

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **All 20 subsystems have published contracts in section 27.3, each with the nine canonical fields complete.**
- **The Subsystem Catalogue (section 27.2) lists every deployed subsystem with owner, criticality, and SLA tier.**
- **Each subsystem's runtime evidence streams are operational and visible on the Constitution Compliance dashboard.**
- **At least one chaos exercise per criticality tier has been completed and postmortemed.**

- The Principal Architect has signed off the contract set, and the Release Certification Authority has verified the runtime evidence.

15. Runtime Verification

Runtime verification of the contract regime is the Constitution Compliance dashboard itself: the dashboard aggregates per-subsystem evidence streams, the audit log records every contract-violation detection, and the invariant monitors (Chapter 26) detect the consequences of contract breaches (orphan rows, illegal transitions, audit gaps). The Release Certification Authority SHALL verify that every subsystem's evidence streams are operational before RC1 sign-off, and SHALL sample the audit log to confirm that contract violations are detected and recorded.

16. Certification Requirements

RC1 certification requires: the contract template published (section 27.1); the Subsystem Catalogue complete (section 27.2); all 20 subsystem contracts published (section 27.3); each subsystem's acceptance-test suite passing at 100%; each subsystem's runtime-evidence streams operational; at least one chaos exercise per criticality tier completed; and the Principal Architect's sign-off that the contract set is complete. The Release Certification Authority SHALL verify the runtime evidence and SHALL refuse RC1 sign-off if any contract is missing, stale, or unverified.

17. Future Evolution

The contract set evolves through Constitution Amendments (Chapter 36). New subsystems MAY be added (typically when a new capability is introduced); each addition SHALL publish a full nine-field contract before deployment. Existing contracts MAY be tightened (stricter authority, tighter SLOs, more evidence) but SHALL NOT be loosened without evidence that the looser contract still preserves the invariants of Chapter 26. A subsystem MAY be deprecated; the deprecation SHALL record the successor subsystem or a documented argument that no successor is needed. The nine-field template itself SHALL NOT be changed without a Constitution Amendment that updates every existing contract to the new template.

Chapter 28 — Canonical Object Registry

This chapter is the authoritative catalogue of every canonical field in the Opus Publica platform. A canonical field is a named, owned, versioned, and verification-bound slot of platform state: every field has an Authority that defines it, a Producer that writes it, a Consumer that reads it, a Database Field that persists it, a Storage Path that locates it, a Mutability rule that governs its lifecycle, a Verification regime that proves its integrity, a Recovery procedure for its loss, a Deprecation path for its sunset, a Future Migration plan for its evolution, and a Certification requirement for its release. The registry is the platform's data constitution at the field granularity.

The registry serves three purposes. First, it is the reference for engineering: every line of code that writes or reads platform state references a canonical field, and that field's contract in this chapter is the authoritative specification. Second, it is the reference for certification: the Release Certification Authority verifies that every canonical field has its eleven mandatory attributes populated and that the verification regime is operational. Third, it is the reference for evolution: when a field is deprecated or migrated, this chapter records the deprecation and the successor, preserving the platform's history of canonical state. The registry is organized by domain (Article, Author, Journal, Issue, Submission, Review, Decision, Version, Identifier, Preservation, Audit, User/Role, Storage, Configuration) and contains more than two hundred fields, of which twenty-five representative fields are elaborated with full eleven-row contracts in section 28.3.

28.1 Registry Schema

Every canonical field in the registry SHALL have the eleven mandatory attributes enumerated below. The Content Obligation column specifies what each attribute MUST contain for the field to be considered fully registered. A field whose attribute is unmet is incompletely registered and SHALL block certification of the subsystem that produces it. The schema is normative: fields MAY add appendix attributes (sensitivity classification, retention policy) but MAY NOT omit or reorder the eleven canonical attributes.

Attribute	Content Obligation
Authority	The constitutional authority that defines the field: a chapter and section reference, or an invariant identifier (INV-X-NN) from Chapter 26 that mandates the field's existence and properties.
Producer	The subsystem (Chapter 27) responsible for writing the field. Exactly one Producer per field; co-production is forbidden (INV-I-01 principle extended to fields).
Consumer	The subsystems that read the field, enumerated. Consumers have read authority only; write authority is reserved to the Producer.
Database Field	The canonical database column or document path that

	persists the field, with schema version and table/collection name. Composite fields reference their normalized representation.
Storage Path	For artifact fields, the canonical object-store path or content-address (SHA-256). For metadata-only fields, the database persistence (covered by Database Field) and applicable replication policy.
Mutability	The field's lifecycle rule: immutable (write-once after publication), mutable (pre-publication only), append-only (audit log), derived (computed from other fields), or config-managed (operator-set).
Verification	The verification regime that proves the field's integrity: SHA-256 content hash, schema validation (JSON Schema, NLM JATS DTD, PDF/A-2u), reconciliation job, invariant monitor (INV-X-NN), or referential integrity (FK).
Recovery	The procedure for recovering the field if lost: reconstruction from preservation deposit, replay from audit log, re-derivation from source, or restore from backup, with the RPO and RTO targets.
Deprecation	The deprecation path: conditions under which the field MAY be deprecated, the notice period required, and the migration tooling that SHALL be provided to existing consumers.
Future Migration	The planned evolution of the field: schema migrations, format changes (e.g., JATS 1.2 → JATS 1.3), or successor field if the field is to be replaced. SHALL be 'None anticipated' if no migration is planned.
Certification	The certification requirement the field SHALL satisfy before release: schema conformance, invariant monitor operational, reconciliation job passing, and the Release Certification Authority's sign-off per Chapter 22.

28.2 Registry by Domain

The registry is organized by domain. Each domain is presented as an H3 subsection followed by a master table with columns: Field, Producer, Mutability, Verification. The master tables enumerate every

canonical field in the domain. Fields with full eleven-attribute contracts are cross-referenced in section 28.3.

Article Domain

The Article domain is the platform's primary domain. It models the scholarly work itself: its identity, content, lifecycle state, bibliographic metadata, and dissemination counters. The Article domain has forty-five canonical fields, of which six (article_doi, article_jats_xml, article_pdf_published, article_html_canonical, article_state, article_license) are elaborated with full contracts in section 28.3.

Field	Producer	Mutability	Verification
article_id	Submission Service	Write-once	Referential integrity (FK)
article_doi	DOI Minter	Write-once	Crossref REST API reconciliation
article_title	Composition Service	Mutable pre-publication	Schema validation
article_abstract	Composition Service	Mutable pre-publication	Schema validation
article_jats_xml	JATS Generator	Immutable post-freeze	NLM JATS DTD validation
article_html_canonical	HTML Renderer	Immutable post-freeze	SHA-256 content hash
article_pdf_published	PDF Generator	Immutable post-freeze	PDF/A-2u conformance
article_version	Composition Service	Write-once	Referential integrity
article_parent_version	Canonical Freezer	Write-once	INV-L-05 monitor
article_state	Decision Service	Mutable (state machine)	INV-C-07 transition check
article_publication_date	Canonical Freezer	Immutable post-freeze	Schema validation
article_license	Submission Service	Immutable post-freeze	Schema validation (SPDX)
article_copyright_holder	Submission Service	Mutable pre-publication	Schema validation
article_language	Submission Service	Mutable pre-publication	ISO 639-1 validation
article_keywords	Submission Service	Mutable pre-publication	Schema validation
article_subject	Submission Service	Mutable pre-publication	Schema validation
article_corresponding_author_id	Submission Service	Mutable pre-publication	Referential integrity
article_issue_id	Decision Service	Mutable pre-publication	Referential integrity
article_journal_id	Submission Service	Write-once	Referential integrity
article_first_page	Composition Service	Mutable pre-publication	Schema validation
article_last_page	Composition Service	Mutable pre-publication	Schema validation
article_page_count	Composition Service	Derived	Recomputation check
article_word_count	Composition Service	Derived	Recomputation check

article_figure_count	Composition Service	Derived	Recomputation check
article_table_count	Composition Service	Derived	Recomputation check
article_reference_count	Composition Service	Derived	Recomputation check
article_supplement_ids	Composition Service	Mutable pre-publication	Referential integrity
article_crossref_deposit_id	Crossref Depositor	Append-only	Crossref receipt reconciliation
article_preservation_receipt_ids	Preservation Depositor	Append-only	Preservation receipt reconciliation
article_oai_pmh_identifier	OAI-PMH Endpoint	Write-once	OAI-PMH GetRecord (INV-R-04)
article_citation_count	Search/Index Service	Derived	Recomputation check
article_view_count	Search/Index Service	Append-only	COUNTER 5.1.1 event reconciliation
article_download_count	Search/Index Service	Append-only	COUNTER 5.1.1 event reconciliation
article_altmetric_score	External (Altmetric)	Derived	External API reconciliation
article_peer_review_model	Submission Service	Write-once	Schema validation (enum)
article_review_count	Review Service	Derived	Recomputation check
article_decision	Decision Service	Append-only	INV-C-07 transition check
article_revision_count	Decision Service	Derived	Recomputation check
article_production_status	Composition Service	Mutable (state machine)	Schema validation
article_retraction_notice_id	Decision Service	Write-once	INV-L-06 monitor
article_correspondence_email	Submission Service	Mutable pre-publication	Email format validation
article_funding_statement	Submission Service	Mutable pre-publication	Schema validation
article_data_availability_statement	Submission Service	Mutable pre-publication	Schema validation
article_ethics_approval_reference	Submission Service	Mutable pre-publication	Schema validation
article_received_date	Submission Service	Write-once	Schema validation

Author Domain

The Author domain models the human authors of scholarly works, including their ORCID linkage, affiliations, contribution statements, and ORCID write-back state. The Author domain has twenty-eight canonical fields, of which one (author_orcid_id) is elaborated with a full contract in section 28.3.

Field	Producer	Mutability	Verification
author_id	Identity/Auth Service	Write-once	Referential integrity
author_orcid_id	ORCID Integrator	Write-once	ORCID public API (INV-R-03)
author_given_name	Submission Service	Mutable pre-publication	Schema validation
author_family_name	Submission Service	Mutable pre-publication	Schema validation
author_full_name	Submission Service	Derived	Recomputation check
author_email	Submission Service	Mutable pre-publication	Email format validation
author_affiliation_id	Submission Service	Mutable pre-publication	Referential integrity
author_biography	Submission Service	Mutable pre-publication	Schema validation
author_website	Submission Service	Mutable pre-publication	URL format validation
author_profile_url	Identity/Auth Service	Derived	Recomputation check
author_orcid_access_token	ORCID Integrator	Mutable (rotated)	Token lifecycle check
author_orcid_refresh_token	ORCID Integrator	Mutable (rotated)	Token lifecycle check
author_consent_status	ORCID Integrator	Mutable (consent flow)	Schema validation (enum)
author_corresponding_flag	Submission Service	Mutable pre-publication	Schema validation (boolean)
author_order	Submission Service	Mutable pre-publication	Uniqueness within article
author_contribution_statement	Submission Service	Mutable pre-publication	Schema validation (CRediT)
author_conflict_of_interest	Submission Service	Mutable pre-publication	Schema validation
author_account_id	Identity/Auth Service	Write-once	Referential integrity
author_created_at	Identity/Auth Service	Write-once	Timestamp monotonicity
author_updated_at	Identity/Auth Service	Mutable (every write)	Timestamp monotonicity
author_deceased_flag	Submission Service	Mutable pre-publication	Schema validation (boolean)
author_orcid_last_verified	ORCID Integrator	Mutable (verification cadence)	INV-R-03 monitor
author_preferred_citation_name	Submission Service	Mutable pre-publication	Schema validation
author_orcid_writeback_enabled	ORCID Integrator	Mutable (consent flow)	Schema validation (boolean)
author_thumbnail_url	Identity/Auth Service	Mutable	URL format validation
author_cv_url	Identity/Auth Service	Mutable	URL format validation
author_orcid_writeback_last_attempt	ORCID Integrator	Append-only	Schema validation (timestamp)

author_orcid_writeback_last_success	ORCID Integrator	Append-only	Schema validation (timestamp)

Journal Domain

The Journal domain models the journals hosted on the platform: their identity, ISSN set, DOI prefix, editorial configuration, licensing defaults, and indexing status. The Journal domain has twenty-seven canonical fields, of which one (journal_issn_electronic) is elaborated with a full contract in section 28.3.

Field	Producer	Mutability	Verification
journal_id	Identity/Auth Service (admin)	Write-once	Referential integrity
journal_title	Identity/Auth Service (admin)	Mutable (admin)	Schema validation
journal_issn_print	Identity/Auth Service (admin)	Mutable (admin)	ISSN format validation
journal_issn_electronic	Identity/Auth Service (admin)	Mutable (admin)	ISSN format validation
journal_issn_linking	Identity/Auth Service (admin)	Mutable (admin)	ISSN format validation
journal_slug	Identity/Auth Service (admin)	Write-once	URL slug uniqueness
journal_description	Identity/Auth Service (admin)	Mutable (admin)	Schema validation
journal_aims_scope	Identity/Auth Service (admin)	Mutable (admin)	Schema validation
journal_editorial_board_id	Identity/Auth Service (admin)	Mutable (admin)	Referential integrity
journal_publisher	Identity/Auth Service (admin)	Mutable (admin)	Schema validation
journal_society	Identity/Auth Service (admin)	Mutable (admin)	Schema validation
journal_issn_status	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (enum)
journal_doi_prefix	Identity/Auth Service (admin)	Write-once	Crossref prefix validation
journal_license_default	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (SPDX)
journal_peer_review_model	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (enum)

journal_apc_policy	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (Diamond OA default)
journal_subscription_model	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (enum)
journal_frequency	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (enum)
journal_language	Identity/Auth Service (admin)	Mutable (admin)	ISO 639-1 validation
journal_subject_classification	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (LCC/ASJC)
journal_indexing_status_doaj	Identity/Auth Service (admin)	Mutable (admin)	DOAJ eligibility check
journal_indexing_status_scopus	Identity/Auth Service (admin)	Mutable (admin)	External reconciliation
journal_indexing_status_wos	Identity/Auth Service (admin)	Mutable (admin)	External reconciliation
journal_preservation_policy	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (backend set)
journal_oai_pmh_endpoint	OAI-PMH Endpoint	Derived	OAI-PMH Identify check
journal_logo_url	Identity/Auth Service (admin)	Mutable (admin)	URL format validation
journal_contact_email	Identity/Auth Service (admin)	Mutable (admin)	Email format validation

Issue Domain

The Issue domain models the journal issue: a curated collection of articles published together, with volume, number, and optional title. The Issue domain has sixteen canonical fields.

Field	Producer	Mutability	Verification
issue_id	Decision Service (editor)	Write-once	Referential integrity
issue_journal_id	Decision Service (editor)	Write-once	Referential integrity
issue_volume	Decision Service (editor)	Mutable pre-publication	Schema validation (integer)
issue_number	Decision Service (editor)	Mutable pre-publication	Schema validation
issue_title	Decision Service (editor)	Mutable pre-publication	Schema validation
issue_description	Decision Service (editor)	Mutable pre-publication	Schema validation
issue_publication_date	Decision Service (editor)	Immutable post-publication	Schema validation
issue_state	Decision Service (editor)	Mutable (state machine)	INV-C-07 transition check

issue_cover_image_url	Decision Service (editor)	Mutable pre-publication	URL format validation
issue_type	Decision Service (editor)	Mutable pre-publication	Schema validation (enum: regular/special/supplement)
issue_doi	DOI Minter	Write-once	Crossref REST API reconciliation
issue_article_ids	Decision Service (editor)	Mutable pre-publication	Referential integrity (list)
issue_editorial_board_id	Decision Service (editor)	Mutable pre-publication	Referential integrity
issue_created_at	Decision Service (editor)	Write-once	Timestamp monotonicity
issue_published_at	Decision Service (editor)	Immutable post-publication	Timestamp monotonicity
issue_suppressed_flag	Decision Service (editor)	Mutable (admin)	Schema validation (boolean)

Submission Domain

The Submission domain models the manuscript as submitted by the author, prior to acceptance and production. It tracks submission state, declarations, files, and reviewer suggestions. The Submission domain has twenty-two canonical fields, of which one (submission_state) is elaborated with a full contract in section 28.3.

Field	Producer	Mutability	Verification
submission_id	Submission Service	Write-once	Referential integrity
submission_article_id	Submission Service	Write-once	Referential integrity
submission_corresponding_author_id	Submission Service	Write-once	Referential integrity
submission_state	Submission Service	Mutable (state machine)	INV-C-07 transition check
submission_submitted_at	Submission Service	Write-once	Timestamp monotonicity
submission_submitted_by_user_id	Submission Service	Write-once	Referential integrity
submission_files	Submission Service	Mutable pre-submission	Storage receipts reconciliation
submission_cover_letter	Submission Service	Mutable pre-submission	Schema validation
submission_suggested_reviewers	Submission Service	Mutable pre-submission	Schema validation (list)
submission_opposed_reviewers	Submission Service	Mutable pre-submission	Schema validation (list)
submission_subject	Submission Service	Mutable pre-submission	Schema validation
submission_journal_id	Submission Service	Write-once	Referential integrity
submission_section_id	Submission Service	Mutable pre-submission	Referential integrity

submission_language	Submission Service	Mutable pre-submission	ISO 639-1 validation
submission_license_consent	Submission Service	Write-once	Schema validation (boolean)
submission_ethics_declaration	Submission Service	Write-once	Schema validation (boolean)
submission_coi_declaration	Submission Service	Write-once	Schema validation (boolean)
submission_funding_declaration	Submission Service	Write-once	Schema validation
submission_data_availability	Submission Service	Write-once	Schema validation
submission_preprint_url	Submission Service	Mutable pre-submission	URL format validation
submission_preprint_server	Submission Service	Mutable pre-submission	Schema validation (enum)
submission_withdrawal_reason	Submission Service	Append-only	Schema validation

Review Domain

The Review domain models the peer-review process: reviewer assignments, states, scores, comments, and conflict-of-interest declarations. The Review domain has twenty-four canonical fields, of which two (review_state, review_coi_declared) are elaborated with full contracts in section 28.3.

Field	Producer	Mutability	Verification
review_id	Review Service	Write-once	Referential integrity
review_submission_id	Review Service	Write-once	Referential integrity
review_reviewer_id	Review Service	Write-once	Referential integrity
review_state	Review Service	Mutable (state machine)	INV-C-07 transition check
review_assigned_at	Review Service	Write-once	Timestamp monotonicity
review_accepted_at	Review Service	Write-once	Timestamp monotonicity
review_due_date	Review Service	Mutable (extension)	Schema validation (date)
review_completed_at	Review Service	Write-once	Timestamp monotonicity
review_recommendation	Review Service	Write-once at submission	Schema validation (enum)
review_confidential_comments	Review Service	Write-once at submission	Schema validation
review_author_comments	Review Service	Write-once at submission	Schema validation
review_editor_comments	Review Service	Write-once at submission	Schema validation
review_coi_declared	Review Service	Write-once at assignment	INV-L-03 monitor
review_coi_details	Review Service	Append-only	Schema validation
review_blind_type	Review Service	Write-once	Schema validation (enum:

			double/single/open)
review_round	Review Service	Write-once	Schema validation (integer)
review_files	Review Service	Append-only	Storage receipts reconciliation
review_score_originality	Review Service	Write-once at submission	Schema validation (1-5)
review_score_methodology	Review Service	Write-once at submission	Schema validation (1-5)
review_score_clarity	Review Service	Write-once at submission	Schema validation (1-5)
review_score_significance	Review Service	Write-once at submission	Schema validation (1-5)
review_overall_score	Review Service	Derived	Recomputation check
review_withdrawal_reason	Review Service	Append-only	Schema validation
review_late_flag	Review Service	Derived	Recomputation check (vs due_date)

Decision Domain

The Decision domain models editorial decisions: Accept, Minor Revision, Major Revision, Reject, Retract. It records the editor, the supporting reviews, and the comments to author and internal. The Decision domain has twelve canonical fields, of which one (editorial_decision) is elaborated with a full contract in section 28.3.

Field	Producer	Mutability	Verification
decision_id	Decision Service	Write-once	Referential integrity
decision_submission_id	Decision Service	Write-once	Referential integrity
decision_editor_id	Decision Service	Write-once	Referential integrity + authorization matrix
decision_type	Decision Service	Write-once	Schema validation (enum)
decision_state	Decision Service	Mutable (supersession)	INV-C-07 transition check
decision_recorded_at	Decision Service	Write-once	Timestamp monotonicity
decision_comments_to_author	Decision Service	Write-once	Schema validation
decision_comments_internal	Decision Service	Write-once	Schema validation
decision_review_round	Decision Service	Write-once	Schema validation (integer)
decision_effective_at	Decision Service	Write-once	Timestamp monotonicity
decision_superseded_by_id	Decision Service	Append-only	Referential integrity
decision_revise_deadline	Decision Service	Mutable (extension)	Schema validation (date)

Version Domain

The Version domain models article versions: original, revisions, and corrections. Versions are linked to parents (INV-L-05) and carry their own canonical package and SHA-256. The Version domain has twelve canonical fields, of which one (version_parent_id) is elaborated with a full contract in section 28.3.

Field	Producer	Mutability	Verification
version_id	Canonical Freezer	Write-once	Referential integrity
version_article_id	Canonical Freezer	Write-once	Referential integrity
version_number	Canonical Freezer	Write-once	Schema validation (integer)
version_parent_id	Canonical Freezer	Write-once	INV-L-05 monitor
version_state	Canonical Freezer	Mutable (state machine)	INV-C-07 transition check
version_created_at	Canonical Freezer	Write-once	Timestamp monotonicity
version_published_at	Canonical Freezer	Immutable post-publication	Timestamp monotonicity
version_sha256	Canonical Freezer	Write-once	INV-M-01 immutability check
version_artifact_ids	Canonical Freezer	Append-only	Referential integrity (list)
version_change_summary	Canonical Freezer	Write-once	Schema validation
version_depositor_id	Canonical Freezer	Write-once	Referential integrity
version_retracted_at	Decision Service	Append-only	Timestamp monotonicity

Identifier Domain

The Identifier domain models the persistent identifiers minted and registered for articles, authors, and journals (DOI, ORCID, ISSN). The Identifier domain has sixteen canonical fields, of which two (identifier_value, doi_registration_agency) are elaborated with full contracts in section 28.3.

Field	Producer	Mutability	Verification
identifier_id	DOI Minter	Write-once	Referential integrity
identifier_type	DOI Minter	Write-once	Schema validation (enum: DOI/ORCID/ISSN)
identifier_value	DOI Minter	Write-once	Crossref/ORCID/ISSN resolution
identifier_article_id	DOI Minter	Write-once	Referential integrity
identifier_author_id	ORCID Integrator	Write-once	Referential integrity
identifier_journal_id	Identity/Auth Service (admin)	Write-once	Referential integrity
identifier_registration_agency	DOI Minter	Write-once	Schema validation (enum: Crossref/DataCite)

identifier_deposit_status	Crossref Depositor	Mutable (deposit lifecycle)	INV-R-05 monitor
identifier_deposit_timestamp	Crossref Depositor	Append-only	Timestamp monotonicity
identifier_deposit_receipt	Crossref Depositor	Append-only	Receipt URL validation
identifier_resolution_url	DOI Minter	Derived	HTTP HEAD resolution check
identifier_content_negotiation_url	DOI Minter	Derived	ContentNegotiation (HTTP Accept) check
identifier_canonical	DOI Minter	Write-once	Schema validation (boolean)
identifier_deprecated	DOI Minter	Mutable (deprecation)	Schema validation (boolean)
identifier_replaced_by	DOI Minter	Mutable (deprecation)	Referential integrity
identifier_minted_at	DOI Minter	Write-once	Timestamp monotonicity

Preservation Domain

The Preservation domain models the deposit of canonical artifacts to dark-archive backends (CLOCKSS, LOCKSS, Portico) and the reconciliation of deposit receipts. The Preservation domain has eleven canonical fields, of which one (preservation_receipt_id) is elaborated with a full contract in section 28.3.

Field	Producer	Mutability	Verification
preservation_receipt_id	Preservation Depositor	Write-once	Backend receipt URL reconciliation
preservation_article_id	Preservation Depositor	Write-once	Referential integrity
preservation_backend	Preservation Depositor	Write-once	Schema validation (enum: CLOCKSS/LOCKSS/Portico)
preservation_deposit_state	Preservation Depositor	Mutable (deposit lifecycle)	INV-L-04 monitor
preservation_deposit_timestamp	Preservation Depositor	Append-only	Timestamp monotonicity
preservation_receipt_url	Preservation Depositor	Append-only	URL format validation
preservation_artifact_sha256	Preservation Depositor	Write-once	SHA-256 cross-check vs canonical
preservation_package_size_bytes	Preservation Depositor	Write-once	Recomputation check
preservation_trigger_event	Preservation Depositor	Write-once	Schema validation (enum)
preservation_reconciliation_status	Preservation Depositor	Mutable (reconciliation)	INV-R-05 monitor
preservation_reconciliation_timestamp	Preservation Depositor	Append-only	Timestamp monotonicity

Audit Domain

The Audit domain models the cryptographically chained, append-only audit log that records every state transition in the platform. The Audit domain has sixteen canonical fields, of which two (audit_chain_hash, audit_correlation_id) are elaborated with full contracts in section 28.3.

Field	Producer	Mutability	Verification
audit_id	Audit Service	Write-once (append-only)	Referential integrity
audit_timestamp	Audit Service	Write-once (append-only)	Timestamp monotonicity
audit_actor_id	Audit Service	Write-once (append-only)	Referential integrity
audit_actor_type	Audit Service	Write-once (append-only)	Schema validation (enum: user/service)
audit_operation	Audit Service	Write-once (append-only)	Schema validation (enum)
audit_entity_type	Audit Service	Write-once (append-only)	Schema validation (enum)
audit_entity_id	Audit Service	Write-once (append-only)	Referential integrity
audit_before_state_hash	Audit Service	Write-once (append-only)	SHA-256 validation
audit_after_state_hash	Audit Service	Write-once (append-only)	SHA-256 validation
audit_reason	Audit Service	Write-once (append-only)	Schema validation (non-empty)
audit_correlation_id	Audit Service	Write-once (append-only)	UUID v4 validation
audit_request_id	Audit Service	Write-once (append-only)	UUID v4 validation
audit_ip_address	Audit Service	Write-once (append-only)	IP format validation
audit_user_agent	Audit Service	Write-once (append-only)	Schema validation
audit_chain_hash	Audit Service	Write-once (append-only)	INV-A-03 cryptographic chain
audit_predecessor_hash	Audit Service	Write-once (append-only)	INV-A-03 cryptographic chain

User/Role Domain

The User/Role domain models platform users, their credentials, and their authorization roles. The User/Role domain has thirteen canonical fields, of which one (reviewer_assignment_id) is elaborated with a full contract in section 28.3 (cross-referenced from the Review domain for its authority scope).

Field	Producer	Mutability	Verification
user_id	Identity/Auth Service	Write-once	Referential integrity
user_email	Identity/Auth Service	Mutable (verified)	Email format validation
user_password_hash	Identity/Auth Service	Mutable (rotation)	Argon2id hash validation
user_mfa_secret_hash	Identity/Auth Service	Mutable (rotation)	Hash validation

user_orcid_id	ORCID Integrator	Write-once	ORCID public API (INV-R-03)
user_state	Identity/Auth Service	Mutable (state machine)	INV-C-07 transition check
user_created_at	Identity/Auth Service	Write-once	Timestamp monotonicity
user_last_login_at	Identity/Auth Service	Mutable (every login)	Timestamp monotonicity
role_id	Identity/Auth Service (admin)	Write-once	Referential integrity
role_name	Identity/Auth Service (admin)	Write-once	Schema validation (enum)
role_permissions	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (RBAC)
user_role_id	Identity/Auth Service	Write-once	Referential integrity
user_role_assigned_at	Identity/Auth Service	Write-once	Timestamp monotonicity

Storage Domain

The Storage domain models the content-addressed object store: every artifact the platform has ever persisted, with its SHA-256, size, replica count, and orphan-detection state. The Storage domain has twelve canonical fields, of which one (artifact_sha256) is elaborated with a full contract in section 28.3.

Field	Producer	Mutability	Verification
storage_object_id	Storage Service	Write-once	Referential integrity
storage_bucket	Storage Service	Write-once	Schema validation
storage_key	Storage Service	Write-once	Uniqueness within bucket
storage_sha256	Storage Service	Write-once	INV-R-02 content addressing
storage_size_bytes	Storage Service	Write-once	Recomputation check
storage_content_type	Storage Service	Write-once	MIME type validation
storage_created_at	Storage Service	Write-once	Timestamp monotonicity
storage_replica_count	Storage Service	Mutable (replication)	INV-V-01 monitor
storage_tier	Storage Service	Mutable (lifecycle)	Schema validation (enum)
storage_referenced_by	Storage Service	Append-only	Referential integrity
storage_orphan_flag	Storage Service	Mutable (orphan scan)	INV-C-01 monitor
storage_legal_hold	Storage Service	Mutable (admin)	Schema validation (boolean)

Configuration Domain

The Configuration domain models platform configuration: feature flags, integration credentials, schema versions, and operator-managed settings. The Configuration domain has twelve canonical fields.

Field	Producer	Mutability	Verification
config_id	Identity/Auth Service (admin)	Write-once	Referential integrity
config_key	Identity/Auth Service (admin)	Write-once	Uniqueness
config_value	Identity/Auth Service (admin)	Mutable (admin)	Schema validation (per key)
config_scope	Identity/Auth Service (admin)	Write-once	Schema validation (enum: platform/journal/article)
config_owner	Identity/Auth Service (admin)	Write-once	Referential integrity
config_encrypted	Identity/Auth Service (admin)	Write-once	Schema validation (boolean)
config_rotation_policy	Identity/Auth Service (admin)	Mutable (admin)	INV-V-03 monitor
config_last_modified_at	Identity/Auth Service (admin)	Mutable (every write)	Timestamp monotonicity
config_modified_by	Identity/Auth Service (admin)	Mutable (every write)	Referential integrity
config_schema_version	Identity/Auth Service (admin)	Mutable (migration)	Schema validation (version)
config_default_value	Identity/Auth Service (admin)	Write-once	Schema validation
config_deprecated	Identity/Auth Service (admin)	Mutable (deprecation)	Schema validation (boolean)

Cross-Reference Fields

The following additional canonical fields are referenced by section 28.3 for full contract elaboration. They are produced by subsystems enumerated in Chapter 27 but span multiple domains; their authoritative registration is consolidated here.

Field	Producer	Mutability	Verification
publication_timestamp	Canonical Freezer	Immutable post-publication	Timestamp monotonicity
artifact_sha256	Storage Service	Write-once	INV-R-02 content addressing

retraction_notice_id	Decision Service	Write-once	INV-L-06 monitor
crossref_deposit_id	Crossref Depositor	Append-only	Crossref receipt reconciliation
doi_registration_agency	DOI Minter	Write-once	Schema validation (enum)
editorial_decision	Decision Service	Append-only	INV-C-07 transition check
reviewer_assignment_id	Review Service	Write-once	Referential integrity
canonical_frozen_timestamp	Canonical Freezer	Write-once	Timestamp monotonicity
pdf_generation_completed_at	PDF Generator	Write-once	Timestamp monotonicity

28.3 Field Contracts

This section elaborates twenty-five representative canonical fields with their full eleven-attribute contracts. The fields are selected to span every domain and every mutability class, and to anchor the invariants of Chapter 26. Each contract is presented as an H3 subsection followed by an eleven-row Attribute/Value table. The remaining fields in section 28.2 share the same schema; their full contracts SHALL be published in the Engineering Playbook and SHALL be available to the Release Certification Authority on demand.

article_doi

Attribute	Value
Authority	Chapter 14 (Persistent Identification); INV-I-02 (exactly one DOI per published article version); INV-M-03 (DOI-to-version binding immutable once minted).
Producer	DOI Minter (SUB-10). The DOI Minter allocates the DOI from the journal's Crossref prefix and writes article_doi.
Consumer	Crossref Depositor (deposit); OAI-PMH Endpoint (metadata); HTML Renderer (citation); Search/Index Service (index); Preservation Depositor (package metadata).
Database Field	articles.article_doi (VARCHAR, UNIQUE, NOT NULL for published articles). Indexed. Foreign key from identifiers.identifier_value where identifier_type='DOI'.
Storage Path	Database-only field; no object-store representation. Cross-reference: identifiers table row of type DOI with the same value.
Mutability	Write-once. After minting, the DOI-to-version binding is immutable (INV-M-03). Rebinding requires a new version

	with a new DOI.
Verification	Crossref REST API reconciliation (INV-R-05): the DOI MUST resolve via doi.org and MUST return the article landing page. Reconciliation job runs daily.
Recovery	If lost: reconstruct from Crossref REST API (the registered DOI persists at Crossref even if locally lost). If never registered: re-mint and re-deposit. RPO: 0 (Crossref is the source of truth post-deposit). RTO: 1 hour.
Deprecation	A DOI MAY be deprecated only via a governed retraction (INV-L-06) or a version supersession (Chapter 12). Deprecation records identifier_deprecated=true and identifier_replaced_by pointing to the successor DOI. Notice period: immediate upon retraction.
Future Migration	Crossref prefix migration (rare, requires Crossref coordination): all DOIs under the old prefix SHALL remain resolvable; new DOIs minted under the new prefix. Anticipated: None in RC1.
Certification	Schema validation (DOI format); Crossref resolution test (INV-R-01); reconciliation job passing; Release Certification Authority sign-off per Chapter 22.

article_jats_xml

Attribute	Value
Authority	Chapter 5 (Canonical Article Representation); Chapter 13 (Rendering); INV-L-02 (every published article has valid JATS XML).
Producer	JATS Generator (SUB-08). Produces the JATS XML from the submission source and metadata snapshot.
Consumer	HTML Renderer (rendering source); PDF Generator (rendering source); Crossref Depositor (deposit payload); Preservation Depositor (package); OAI-PMH Endpoint (metadata).
Database Field	articles.article_jats_xml_artifact_id (UUID, FK to storage_objects.storage_object_id). The XML itself is stored as a content-addressed object, not inline.
Storage Path	storage://opuspublica-canonical/jats/{article_id}/{version_number}/article.xml.sha256-{sha256}. Indexed by SHA-256 (INV-R-02).
Mutability	Immutable post-freeze (INV-M-01). Pre-freeze, the JATS

	Generator may regenerate. Post-freeze, any modification is forbidden; corrections require a new version.
Verification	NLM JATS DTD validation (version pinned per article, recorded in article_metadata). Verifier: Artifact Verifier (SUB-09). Re-verification: hourly reconciliation job samples 1% of frozen JATS artifacts.
Recovery	If lost from primary store: reconstruct from preservation deposit (CLOCKSS/LOCKSS/Portico). If invalid: regenerate from submission source. RPO: 0 (preservation deposit is the source of truth). RTO: 4 hours.
Deprecation	JATS XML artifacts are never deprecated; supersession occurs via new article versions with new JATS artifacts. The old artifact remains retrievable indefinitely (INV-V-01).
Future Migration	JATS DTD version migration (e.g., JATS 1.2 → JATS 1.3): governed by Chapter 36 amendment. New versions validated against new DTD; existing artifacts retain their pinned DTD.
Certification	NLM JATS DTD validation (pinned version); SHA-256 determinism test (regenerate from same input → identical hash); Release Certification Authority sign-off per Chapter 22.

article_pdf_published

Attribute	Value
Authority	Chapter 13 (Rendering); INV-I-03 (exactly one published PDF per article version).
Producer	PDF Generator (SUB-06). Produces a PDF/A-2u compliant artifact from the JATS XML and figures.
Consumer	Reader landing page (download); Preservation Depositor (package); Search/Index Service (full-text indexing).
Database Field	articles.article_pdf_artifact_id (UUID, FK to storage_objects.storage_object_id).
Storage Path	storage://opuspublica-canonical/pdf/{article_id}/{version_number}/article.pdf.sha256-{sha256}. Content-addressed (INV-R-02).
Mutability	Immutable post-freeze (INV-M-01). Pre-freeze, regeneration is permitted. Post-freeze, modification is forbidden; corrections require a new version with a new

	PDF.
Verification	PDF/A-2u conformance (veraPDF validator); SHA-256 determinism (regenerate → identical hash); Artifact Verifier (SUB-09) gate before freezing.
Recovery	If lost: reconstruct from preservation deposit. If invalid: regenerate from JATS XML. RPO: 0 (preservation deposit is source of truth). RTO: 4 hours.
Deprecation	Never deprecated; supersession via new versions. Old PDF remains retrievable indefinitely (INV-V-01).
Future Migration	PDF/A version migration (e.g., PDF/A-2u → PDF/A-3u): governed amendment; existing artifacts retain their validated version.
Certification	PDF/A-2u conformance test; determinism test; landing-page download test; Release Certification Authority sign-off.

article_html_canonical

Attribute	Value
Authority	Chapter 13 (Rendering); INV-I-04 (exactly one canonical HTML per article version).
Producer	HTML Renderer (SUB-07). Produces canonical HTML from JATS XML, journal template, and CSL citation style.
Consumer	Reader landing page (primary rendering); Search/Index Service (indexing); OAI-PMH Endpoint (metadata).
Database Field	articles.article_html_artifact_id (UUID, FK to storage_objects.storage_object_id).
Storage Path	storage://opuspublica-canonical/html/{article_id}/{version_number}/article.html.sha256-{sha256}. Content-addressed (INV-R-02).
Mutability	Immutable post-freeze (INV-M-01). Pre-freeze, regeneration is permitted. Post-freeze, modification is forbidden.
Verification	WCAG 2.1 AA conformance (axe-core); SHA-256 determinism; Artifact Verifier gate. Landing-page cache invalidated on freeze.
Recovery	If lost: reconstruct from preservation deposit. If invalid: regenerate from JATS XML. RPO: 0. RTO: 4 hours.
Deprecation	Never deprecated; supersession via new versions. Old

	HTML remains retrievable indefinitely (INV-V-01).
Future Migration	Template migration: governed amendment; existing artifacts retain their frozen HTML. WCAG standard evolution: new artifacts conform to the latest pinned standard.
Certification	WCAG 2.1 AA audit; determinism test; landing-page load test (p95 < 500 ms cached); Release Certification Authority sign-off.

article_state

Attribute	Value
Authority	Chapter 11 (Editorial Workflow); Chapter 10 (Submission); INV-C-07 (every transition legal per state machine).
Producer	Decision Service (SUB-03) for editorial transitions; Canonical Freezer (SUB-05) for Published transition; Preservation Depositor (SUB-13) for Deposited transition.
Consumer	All subsystems that gate behaviour on article state; Search/Index Service; Notification Service; Audit Service.
Database Field	articles.article_state (ENUM: Drafted, Submitted, InReview, Accepted, InProduction, Frozen, Published, Deposited, Retracted, Removed).
Storage Path	Database-only field; transition records stored in audit log.
Mutability	Mutable via state machine only. Every transition SHALL pass precondition checks; illegal transitions are blocked (INV-C-07). Direct database writes are forbidden.
Verification	INV-C-07 monitor: scans for transitions not in the legal-transition table. Audit log completeness (INV-A-01): every transition SHALL have an audit record.
Recovery	If corrupted: replay audit log to reconstruct last-known-good state. If illegal transition detected: SEV-1 incident, rollback to prior state, investigate. RPO: 0 (audit log is source of truth). RTO: 1 hour.
Deprecation	State enum MAY be extended via Chapter 36 amendment; existing states retain their semantics. No state MAY be removed without a successor mapping.
Future Migration	Anticipated: 'Preprint' state in RC2; 'Withdrawn' refinement in RC3. Each addition is a governed amendment.

Certification	State-machine legal-transition test; audit-log completeness test; INV-C-07 monitor operational; Release Certification Authority sign-off.

author_orcid_id

Attribute	Value
Authority	Chapter 14 (Persistent Identification); Chapter 23 (Knowledge Graph); INV-R-03 (every linked ORCID iD resolves via ORCID public API).
Producer	ORCID Integrator (SUB-12). Records the ORCID iD after OAuth-2.0 three-legged flow and public-API verification.
Consumer	Crossref Depositor (included in deposit payload); Search/Index Service; OAI-PMH Endpoint; Knowledge Graph (Chapter 23).
Database Field	authors.author_orcid_id (VARCHAR(19), UNIQUE per author, format XXXX-XXXX-XXXX-XXXX).
Storage Path	Database-only field; ORCID API responses cached in storage with TTL.
Mutability	Write-once per author. Correction of an incorrect ORCID iD requires a governed author-merge process (Chapter 36 amendment path).
Verification	INV-R-03 monitor: every non-null author_orcid_id SHALL resolve via the ORCID public API. Reconciliation job runs daily.
Recovery	If lost: reconstruct from ORCID OAuth record (the binding persists at ORCID). If unresolvable: SEV-2 alert, author re-verification flow. RPO: 0. RTO: 4 hours.
Deprecation	An ORCID iD binding MAY be deprecated only via author-merge or author-deceased governance. The deprecated iD remains recorded for provenance.
Future Migration	Anticipated: ORCID integration with additional profile fields (employment, education) in RC2. The iD itself is stable.
Certification	ORCID public API resolution test (INV-R-03); OAuth flow test; reconciliation job passing; Release Certification Authority sign-off.

journal_issn_electronic

Attribute	Value
Authority	Chapter 9 (Interoperability); Chapter 14 (Persistent Identification). ISSN is the persistent identifier for serials.
Producer	Identity/Auth Service (admin) (SUB-19). Configured by the journal's editorial administrator; verified against the ISSN Register.
Consumer	Crossref Depositor (journal metadata); OAI-PMH Endpoint (journal identification); Preservation Depositor (package metadata); Knowledge Graph.
Database Field	journals.journal_issn_electronic (VARCHAR(9), format NNNN-NNNN, UNIQUE).
Storage Path	Database-only field.
Mutability	Mutable by admin only. Changes require ISSN Register verification and propagate to Crossref, OAI-PMH, and preservation backends.
Verification	ISSN format validation (checksum); ISSN Register resolution (portal.issn.org). Reconciliation job runs weekly.
Recovery	If lost: reconstruct from ISSN Register. If invalid: SEV-3 alert, admin correction. RPO: 0 (ISSN Register is source of truth). RTO: 1 business day.
Deprecation	An ISSN MAY be deprecated if the journal ceases publication or merges with another. Deprecation records identifier_deprecated=true and identifier_replaced_by pointing to the successor journal's ISSN.
Future Migration	Anticipated: ISSN-L (linking ISSN) explicit field in RC2. The electronic ISSN itself is stable.
Certification	ISSN format validation; ISSN Register resolution test; reconciliation job passing; Release Certification Authority sign-off.

review_state

Attribute	Value
Authority	Chapter 10 (Editorial Workflow); INV-C-07 (every transition legal per state machine); INV-L-03 (CoI declaration precondition).
Producer	Review Service (SUB-02). Transitions review_state per

	reviewer actions and editor extensions.
Consumer	Decision Service (supporting reviews); Notification Service; Audit Service; editorial dashboard.
Database Field	reviews.review_state (ENUM: Invited, Accepted, InProgress, Submitted, Withdrawn, Declined).
Storage Path	Database-only field; transitions recorded in audit log.
Mutability	Mutable via state machine only. Precondition: review_coi_declared MUST be set before Active state (INV-L-03). Direct database writes forbidden.
Verification	INV-C-07 monitor: legal-transition enforcement. INV-L-03 monitor: no Active review without CoI declaration. Audit log completeness (INV-A-01).
Recovery	If corrupted: replay audit log to reconstruct last-known-good state. If illegal transition detected: SEV-1, rollback, investigate. RPO: 0. RTO: 1 hour.
Deprecation	State enum MAY be extended via amendment. No state MAY be removed without a successor mapping.
Future Migration	Anticipated: 'RegisteredReport' state for registered-report review model in RC3.
Certification	State-machine legal-transition test; CoI-precondition test (INV-L-03); audit-log completeness test; Release Certification Authority sign-off.

review_coi_declared

Attribute	Value
Authority	Chapter 10 (Editorial Workflow); Chapter 23 (COPE Alignment); INV-L-03 (every reviewer assignment has a CoI declaration).
Producer	Review Service (SUB-02). Records the reviewer's CoI declaration at assignment time.
Consumer	Decision Service (informed by CoI); editorial dashboard; Audit Service; COPE compliance reports.
Database Field	reviews.review_coi_declared (ENUM: None, Declared, NoneDeclared, Refused). NOT NULL.
Storage Path	Database-only field; details stored in review_coi_details.
Mutability	Write-once at assignment (INV-L-03). Updates require a new review assignment; the prior declaration is

	preserved.
Verification	INV-L-03 monitor: every review in Active or Completed state SHALL have a non-null review_coi_declared. Reconciliation job runs hourly.
Recovery	If lost: SEV-1 incident (INV-L-03 violation); reconstruct from audit log if possible, else void the review assignment. RPO: 0. RTO: 1 hour.
Deprecation	Not deprecable. The CoI declaration is a permanent record of the review's integrity.
Future Migration	Anticipated: structured CoI taxonomy (CRediT-style) in RC2. The boolean/enum field retains its semantics.
Certification	INV-L-03 monitor operational; assignment-without-CoI test (blocked at API); Release Certification Authority sign-off.

audit_chain_hash

Attribute	Value
Authority	Chapter 18 (Audit); Chapter 26 (Invariants); INV-A-03 (audit chain cryptographically verifiable); INV-M-02 (audit log append-only).
Producer	Audit Service (SUB-16). Computes the chain hash on every append: SHA-256(predecessor_hash || record_canonical_form).
Consumer	Audit Service (verification); Security Officer (tamper detection); Release Certification Authority (certification); external auditors.
Database Field	audit_log.audit_chain_hash (CHAR(64), hex-encoded SHA-256). NOT NULL.
Storage Path	Database (primary) + replicated cold storage (append-only).
Mutability	Append-only (INV-M-02). Any modification or deletion is forbidden and detectable via chain recomputation.
Verification	INV-A-03 monitor: hourly chain-verification job recomputes the chain from genesis and compares to stored hashes. Any mismatch is a SEV-1 incident.
Recovery	If chain tampered: SEV-1, immediate security incident. Reconstruct the correct chain from replicated cold storage. If cold storage also tampered: full disaster

	recovery (Chapter 21). RPO: 0. RTO: 1 hour.
Deprecation	Not deprecable. The audit chain is the platform's permanent integrity record.
Future Migration	Anticipated: hash algorithm migration (SHA-256 → SHA-3 or BLAKE3) in a future RC, governed by Chapter 36 amendment. The migration SHALL re-chain from genesis with the new algorithm, preserving the old chain for forensic comparison.
Certification	INV-A-03 monitor operational; chain-verification job passing; tamper-detection chaos test; Release Certification Authority sign-off.

audit_correlation_id

Attribute	Value
Authority	Chapter 18 (Audit); INV-A-02 (every audit record complete — 7 mandatory fields including correlation_id).
Producer	Audit Service (SUB-16). Generates a UUID v4 correlation_id per audit record, inherited from the originating request when available.
Consumer	Audit Service (trace assembly); incident response; Engineering Leadership Team; external auditors.
Database Field	audit_log.audit_correlation_id (UUID v4, NOT NULL, indexed for trace lookup).
Storage Path	Database (primary) + replicated cold storage.
Mutability	Append-only (INV-M-02). Immutable once written.
Verification	INV-A-02 monitor: every audit record SHALL have a non-null correlation_id. Trace-completeness job: every platform operation SHALL produce at least one audit record with the originating request's correlation_id.
Recovery	If lost: SEV-1 (INV-A-02 violation); reconstruct from application logs by correlation. If never recorded: the originating operation is unauditable, SEV-1 incident. RPO: 0. RTO: 1 hour.
Deprecation	Not deprecable. Correlation IDs are permanent records of operation provenance.
Future Migration	Anticipated: W3C Trace Context integration for distributed tracing in RC2. The UUID v4 correlation_id retains its semantics.

Certification	INV-A-02 monitor operational; trace-completeness job passing; Release Certification Authority sign-off.

identifier_value

Attribute	Value
Authority	Chapter 14 (Persistent Identification); INV-I-02 (one DOI per published article version); INV-R-01 (every DOI resolves); INV-M-03 (DOI-to-version binding immutable).
Producer	DOI Minter (SUB-10) for DOIs; ORCID Integrator (SUB-12) for ORCID iDs; admin for ISSNs.
Consumer	All dissemination subsystems (Crossref Depositor, OAI-PMH Endpoint, SWORD Endpoint, Search/Index Service); Knowledge Graph.
Database Field	identifiers.identifier_value (VARCHAR, NOT NULL, UNIQUE per identifier_type). FK references from articles, authors, journals.
Storage Path	Database-only field.
Mutability	Write-once (INV-M-03). Re-minting is forbidden; corrections require governed deprecation and successor.
Verification	Per type: DOI → Crossref REST API resolution (INV-R-01); ORCID → ORCID public API (INV-R-03); ISSN → ISSN Register. Reconciliation jobs run daily/weekly per type.
Recovery	If lost: reconstruct from registration agency (Crossref/ORCID/ISSN). If unresolvable: SEV-2 alert, re-registration. RPO: 0 (agency is source of truth). RTO: 4 hours.
Deprecation	An identifier MAY be deprecated via governed retraction, version supersession, or journal merger. Deprecation records identifier_deprecated=true and identifier_replaced_by.
Future Migration	Anticipated: ROR (Research Organization Registry) identifiers for affiliations in RC2. DOI/ORCID/ISSN remain stable.
Certification	Resolution test per type (INV-R-01, INV-R-03); reconciliation jobs passing; Release Certification Authority sign-off.

preservation_receipt_id

Attribute	Value
Authority	Chapter 12 (Preservation); Chapter 25 (Dark Archive); INV-L-04 (every publication event has a preservation deposit queued); INV-V-02 (deposits never forgotten).
Producer	Preservation Depositor (SUB-13). Issues one receipt per (article, backend) deposit.
Consumer	Audit Service; editorial dashboard (preservation status); external auditors; Preservation Officer.
Database Field	preservation_receipts.preservation_receipt_id (UUID, NOT NULL, UNIQUE per (article, backend)).
Storage Path	Database (primary); receipt URL stored in preservation_receipt_url.
Mutability	Write-once. The receipt is the backend's acknowledgement and SHALL NOT be modified. Re-deposit (after backend failure) creates a new receipt, not a modification.
Verification	INV-V-02 monitor: every deposit SHALL have a permanent receipt, reconciled within 7 days. INV-L-04 monitor: every PublicationCompleted event SHALL have at least one queued deposit. Reconciliation job runs weekly per backend.
Recovery	If lost: SEV-2 (INV-V-02 violation); reconstruct from backend's deposit record (CLOCKSS/LOCKSS/Portico portals). If backend has no record: SEV-1, re-deposit. RPO: 0 (backend is source of truth post-deposit). RTO: 4 hours.
Deprecation	Not deprecable. Receipts are permanent records of deposit.
Future Migration	Anticipated: additional backends (e.g., national preservation nodes) in RC3. Existing receipts retain their backend designation.
Certification	INV-L-04 monitor operational; INV-V-02 reconciliation job passing; multi-backend failover test; Release Certification Authority sign-off.

crossref_deposit_id

Attribute	Value
Authority	Chapter 14 (Persistent Identification); Chapter 9

	(Interoperability); INV-C-05 (no duplicate Crossref deposits for same DOI); INV-R-05 (every Crossref deposit reconciled within SLO).
Producer	Crossref Depositor (SUB-11). Allocates a deposit_id per submission to Crossref.
Consumer	Audit Service; editorial dashboard; DOI Minter (deposit-status feedback); external auditors.
Database Field	crossref_deposits.crossref_deposit_id (UUID, NOT NULL, UNIQUE per (identifier_id, deposit_attempt)).
Storage Path	Database (primary); Crossref's own deposit record is the source of truth.
Mutability	Append-only. Retried deposits create new rows; prior rows retain their state. Modifications are forbidden (INV-C-05).
Verification	INV-C-05 monitor: no duplicate deposits with different content for the same DOI. INV-R-05 monitor: every deposit reconciled within 24 hours via Crossref REST API. Reconciliation job runs daily.
Recovery	If lost: reconstruct from Crossref REST API (deposit records persist at Crossref). If deposit failed at Crossref: SEV-2 alert, re-deposit with new deposit_id (idempotent at Crossref). RPO: 0. RTO: 4 hours.
Deprecation	Not deprecable. Deposit records are permanent.
Future Migration	Anticipated: Crossref REST API v2 migration; existing deposits retain their v1 records.
Certification	INV-C-05 monitor operational; INV-R-05 reconciliation job passing; idempotency test; Release Certification Authority sign-off.

doi_registration_agency

Attribute	Value
Authority	Chapter 14 (Persistent Identification). DOI registration agencies are Crossref and DataCite; Opus Publica uses Crossref by default.
Producer	DOI Minter (SUB-10). Set at mint time based on journal configuration.
Consumer	Crossref Depositor (routing); DataCite Depositor (future); Knowledge Graph; OAI-PMH Endpoint.

Database Field	identifiers.identifier_registration_agency (ENUM: Crossref, DataCite, NOT NULL).
Storage Path	Database-only field.
Mutability	Write-once. The agency cannot be changed post-minting; migration would require deprecation and re-minting at the new agency.
Verification	Schema validation (enum). Reconciliation: agency-confirmed registration (Crossref REST API, DataCite REST API).
Recovery	If lost: reconstruct from agency's deposit record. If agency uncontactable: SEV-2 alert. RPO: 0 (agency is source of truth). RTO: 4 hours.
Deprecation	Agency binding is permanent for the lifetime of the identifier. Migration requires full deprecation and re-minting.
Future Migration	Anticipated: DataCite support for select journals in RC2. Crossref remains the default.
Certification	Schema validation; agency-confirmation test; Release Certification Authority sign-off.

submission_state

Attribute	Value
Authority	Chapter 10 (Submission); Chapter 11 (Editorial Workflow); INV-C-07 (every transition legal per state machine).
Producer	Submission Service (SUB-01). Transitions submission_state per author and editor actions.
Consumer	Review Service (assignment precondition); Decision Service; Composition Service (production precondition); editorial dashboard.
Database Field	submissions.submission_state (ENUM: Drafted, Submitted, Withdrawn, InReview, Accepted, Rejected, RevisionRequested, Archived).
Storage Path	Database-only field; transitions in audit log.
Mutability	Mutable via state machine only. Direct database writes forbidden (INV-A-04).
Verification	INV-C-07 monitor: legal-transition enforcement. INV-A-01: every transition audited.

Recovery	If corrupted: replay audit log. If illegal transition: SEV-1, rollback, investigate. RPO: 0. RTO: 1 hour.
Deprecation	State enum MAY be extended via amendment; no state removed without successor mapping.
Future Migration	Anticipated: 'PreprintPosted' state in RC2.
Certification	State-machine legal-transition test; audit-log completeness test; Release Certification Authority sign-off.

publication_timestamp

Attribute	Value
Authority	Chapter 11 (Editorial Workflow); Chapter 12 (Publication Event); INV-L-04 (publication event triggers preservation deposit).
Producer	Canonical Freezer (SUB-05) records the canonical_frozen_timestamp; the Decision Service records the publication_timestamp on the Published transition.
Consumer	Crossref Depositor (metadata); OAI-PMH Endpoint (datestamp); Search/Index Service; Preservation Depositor.
Database Field	articles.publication_timestamp (TIMESTAMP WITH TIME ZONE, UTC, NOT NULL for published articles).
Storage Path	Database-only field.
Mutability	Immutable post-publication. Corrections require a governed erratum or version.
Verification	Timestamp monotonicity (INV-A-02); Crossref metadata reconciliation (publication date in deposit).
Recovery	If lost: reconstruct from Crossref metadata (publication date is deposited). If never recorded: SEV-2 alert. RPO: 0. RTO: 4 hours.
Deprecation	Not deprecable. Publication timestamp is permanent bibliographic metadata.
Future Migration	Anticipated: 'online-first' vs 'issue-publication' distinction in RC2.
Certification	Schema validation; Crossref reconciliation; monotonicity test; Release Certification Authority sign-off.

artifact_sha256

Attribute	Value
Authority	Chapter 12 (Preservation); Chapter 26 (Invariants); INV-M-01 (published artifacts immutable, SHA-256 fixed); INV-R-02 (every artifact retrievable by content hash).
Producer	Storage Service (SUB-20). Computes SHA-256 on every write.
Consumer	Canonical Freezer; Artifact Verifier; Preservation Depositor; Crossref Depositor (metadata); Search/Index Service.
Database Field	storage_objects.storage_sha256 (CHAR(64), hex-encoded, NOT NULL, UNIQUE).
Storage Path	Content-addressed: storage://{bucket}/{prefix}/{filename}.sha256-{sha256}. The SHA-256 is part of the path.
Mutability	Write-once. The SHA-256 is computed at write time and is immutable; any byte change produces a new SHA-256 and a new storage object.
Verification	INV-M-01 monitor: frozen artifacts' SHA-256 SHALL match the recorded value across retrievals. INV-R-02 monitor: every SHA-256 SHALL be retrievable. Reconciliation job samples 1% of artifacts daily.
Recovery	If lost: reconstruct from preservation deposit (preservation_artifact_sha256 cross-check). If mismatch: SEV-1 (INV-M-01 violation), immediate incident. RPO: 0. RTO: 4 hours.
Deprecation	Not deprecable. SHA-256 is the permanent content address.
Future Migration	Anticipated: SHA-256 → SHA-3 or BLAKE3 in a future RC, governed by Chapter 36 amendment. Migration SHALL preserve both hashes for forensic comparison.
Certification	INV-M-01 monitor operational; INV-R-02 retrieval test; reconciliation job passing; Release Certification Authority sign-off.

version_parent_id

Attribute	Value

Authority	Chapter 12 (Versions); INV-L-05 (every non-original version has a parent); INV-M-05 (version relationships immutable).
Producer	Canonical Freezer (SUB-05). Records the parent version on every non-original version creation.
Consumer	Composition Service (version chain); Crossref Depositor (relation metadata); Search/Index Service; OAI-PMH Endpoint.
Database Field	versions.version_parent_id (UUID, NULL only for originals, FK to versions.version_id).
Storage Path	Database-only field.
Mutability	Write-once (INV-M-05). The parent binding is immutable; corrections require a governed version-relink process.
Verification	INV-L-05 monitor: every non-original version SHALL have a non-null version_parent_id. Cycle detection: the version graph SHALL be acyclic. Reconciliation job runs daily.
Recovery	If lost: SEV-2 (INV-L-05 violation); reconstruct from audit log. If cyclic: SEV-1, immediate incident, rollback. RPO: 0. RTO: 4 hours.
Deprecation	Not deprecable. Parent relationships are permanent provenance.
Future Migration	Anticipated: richer version-relation taxonomy (isDerivedFrom, isSupplementTo) in RC2.
Certification	INV-L-05 monitor operational; cycle-detection test; reconciliation job passing; Release Certification Authority sign-off.

retraction_notice_id

Attribute	Value
Authority	Chapter 11 (Editorial Workflow); COPE Retraction Guidelines; INV-L-06 (every retraction has a retraction notice).
Producer	Decision Service (SUB-03). Creates a retraction notice as a new article (type: RetractionNotice) linked bidirectionally to the retracted article.
Consumer	Crossref Depositor (retraction metadata); OAI-PMH Endpoint; Search/Index Service; reader landing page

	(retraction banner).
Database Field	articles.article_retraction_notice_id (UUID, FK to articles.article_id, NULL except for retracted articles).
Storage Path	Database-only field; the retraction notice itself is a full article with its own canonical artifacts.
Mutability	Write-once. The retraction binding is permanent; the retraction notice can be updated only via its own version chain.
Verification	INV-L-06 monitor: every article in Retracted state SHALL have a non-null article_retraction_notice_id. Crossref reconciliation: retraction metadata deposited. Reconciliation job runs daily.
Recovery	If lost: SEV-1 (INV-L-06 violation); reconstruct from Crossref (retraction metadata persists). If never created: SEV-1, halt retraction process, create notice. RPO: 0. RTO: 1 hour.
Deprecation	Not deprecable. Retraction notices are permanent scholarly record.
Future Migration	Anticipated: expression-of-concern and correction notice types in RC2.
Certification	INV-L-06 monitor operational; Crossref retraction-metadata test; bidirectional-link test; Release Certification Authority sign-off.

article_license

Attribute	Value
Authority	Chapter 15 (Licensing); Chapter 1 (Diamond OA default, CC BY 4.0).
Producer	Submission Service (SUB-01). Recorded at submission from author consent; defaults to journal_license_default.
Consumer	Crossref Depositor (license metadata); HTML Renderer (license display); PDF Generator (license display); OAI-PMH Endpoint; reader landing page.
Database Field	articles.article_license (VARCHAR, SPDX identifier e.g. 'CC-BY-4.0', NOT NULL).
Storage Path	Database-only field.
Mutability	Immutable post-freeze. Pre-freeze, the author may change the license within the journal's permitted set.

	Post-freeze, license changes require a governed version.
Verification	SPDX identifier validation; Crossref metadata reconciliation (license in deposit). Reconciliation job runs daily.
Recovery	If lost: reconstruct from Crossref metadata. If invalid: SEV-2 alert, author re-consent. RPO: 0. RTO: 4 hours.
Deprecation	A license binding MAY be deprecated only via version supersession. The original license remains recorded for the original version.
Future Migration	Anticipated: CC BY 5.0 when released; existing articles retain their CC BY 4.0 binding.
Certification	SPDX validation; Crossref reconciliation; Diamond-OA-default test (no APCs); Release Certification Authority sign-off.

editorial_decision

Attribute	Value
Authority	Chapter 11 (Editorial Workflow); Chapter 23 (COPE Alignment); INV-C-07 (every transition legal per state machine).
Producer	Decision Service (SUB-03). Recorded by an authorized editor; transitions submission_state accordingly.
Consumer	Composition Service (production trigger); Notification Service (author notification); Audit Service; editorial dashboard.
Database Field	decisions.decision_type (ENUM: Accept, MinorRevision, MajorRevision, Reject, Retract, Withdraw, NOT NULL).
Storage Path	Database-only field; full decision record (comments, supporting reviews) in decisions table.
Mutability	Append-only. Supersession records a new decision with decision_superseded_by_id; prior decisions retain their state.
Verification	INV-C-07 monitor: legal-transition enforcement (e.g., Reject → Accept without revision is illegal). Authorization matrix: only authorized editors can decide.
Recovery	If lost: SEV-2; reconstruct from audit log. If illegal transition: SEV-1, rollback. RPO: 0. RTO: 1 hour.
Deprecation	Decision enum MAY be extended via amendment; no

	value removed without successor mapping.
Future Migration	Anticipated: 'RegisteredReportAccept' for registered-report model in RC3.
Certification	INV-C-07 monitor operational; authorization-matrix test; supersession-chain test; Release Certification Authority sign-off.

reviewer_assignment_id

Attribute	Value
Authority	Chapter 10 (Editorial Workflow); INV-L-03 (CoI declaration precondition); INV-A-01 (every transition audited).
Producer	Review Service (SUB-02). Creates the assignment on editor request; enforces CoI precondition.
Consumer	Decision Service (supporting reviews); Notification Service (invitation); Audit Service; editorial dashboard.
Database Field	reviews.review_id (UUID, NOT NULL, UNIQUE, PK).
Storage Path	Database-only field; assignment events in audit log.
Mutability	Write-once creation; subsequent state transitions via state machine only. Direct database writes forbidden.
Verification	INV-L-03 monitor: no assignment without CoI declaration. INV-A-01: every assignment transition audited. Authorization matrix: only authorized editors can assign.
Recovery	If lost: SEV-2; reconstruct from audit log. If assignment without CoI detected: SEV-1, void assignment. RPO: 0. RTO: 1 hour.
Deprecation	Not deprecable. Reviewer assignments are permanent provenance of the review process.
Future Migration	Anticipated: open-peer-review opt-in in RC3 (assignment visible to author). The assignment record retains its structure.
Certification	INV-L-03 monitor operational; authorization-matrix test; audit-log completeness test; Release Certification Authority sign-off.

canonical_frozen_timestamp

Attribute	Value

Authority	Chapter 12 (Publication Event); INV-M-01 (immutability begins at freeze); INV-A-01 (freeze transition audited).
Producer	Canonical Freezer (SUB-05). Recorded at the moment the canonical package is committed to immutable storage.
Consumer	DOI Minter (mint precondition); Crossref Depositor (metadata); Preservation Depositor (trigger event); OAI-PMH Endpoint; Search/Index Service.
Database Field	articles.canonical_frozen_timestamp (TIMESTAMP WITH TIME ZONE, UTC, NOT NULL for frozen articles).
Storage Path	Database-only field; immutable-storage write receipt cross-references this timestamp.
Mutability	Write-once. The freeze timestamp is the anchor of immutability; it SHALL NOT be modified.
Verification	INV-M-01 monitor: every frozen article SHALL have a non-null canonical_frozen_timestamp, and all artifacts frozen at or before that timestamp SHALL be unchanged. Audit log: freeze event recorded.
Recovery	If lost: SEV-2; reconstruct from audit log (freeze transition). If artifacts modified post-freeze: SEV-1 (INV-M-01 violation), immediate incident. RPO: 0. RTO: 4 hours.
Deprecation	Not deprecable. The freeze timestamp is permanent provenance of immutability.
Future Migration	Anticipated: sub-second precision (nanoseconds) for high-throughput journals in RC3.
Certification	INV-M-01 monitor operational; freeze-event audit test; immutability post-freeze test; Release Certification Authority sign-off.

pdf_generation_completed_at

Attribute	Value
Authority	Chapter 13 (Rendering); Chapter 27 SUB-06 (PDF Generator contract); INV-L-02 (every published article has valid artifacts).
Producer	PDF Generator (SUB-06). Recorded at the moment the PDF/A-2u artifact is committed to storage.
Consumer	Composition Service (production status); Artifact Verifier (verification trigger); Canonical Freezer (freeze

	precondition); Audit Service.
Database Field	articles.pdf_generation_completed_at (TIMESTAMP WITH TIME ZONE, UTC, NULL until PDF generated).
Storage Path	Database-only field; cross-references storage_objects row for the PDF artifact.
Mutability	Write-once. The completion timestamp is immutable; regeneration requires a new version with a new timestamp.
Verification	PDF/A-2u conformance (veraPDF); SHA-256 determinism; Artifact Verifier gate before freeze. Reconciliation job samples 1% of PDFs daily.
Recovery	If lost: SEV-3; reconstruct from storage receipt timestamp. If PDF invalid: SEV-2, regenerate from JATS XML. RPO: 0. RTO: 4 hours.
Deprecation	Not deprecable. The generation timestamp is permanent production provenance.
Future Migration	Anticipated: parallel PDF generation for high-volume journals in RC3.
Certification	PDF/A-2u conformance test; determinism test; Artifact Verifier gate test; Release Certification Authority sign-off.

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

This chapter is the authoritative catalogue of every canonical field in the Opus Publica platform. Its purpose is to make every field's contract explicit, owned, and verifiable: a field's contract states its authority, producer, consumer, database field, storage path, mutability, verification regime, recovery procedure, deprecation path, future migration plan, and certification requirement. Without such a registry, the platform's data model is implicit and its integrity unprovable; with it, the platform's data model is derivable from this Constitution and its integrity is anchored to invariant monitors and reconciliation jobs. The chapter operationalizes Foundational Principle I (Runtime Truth) at the field granularity: every field's verification regime has a runtime monitor that proves the field's integrity at runtime.

2. Scope

In scope: the registry schema (eleven mandatory attributes, section 28.1); the registry organized by fourteen domains totalling 236 fields (section 28.2); and the full eleven-attribute contracts for twenty-five representative fields (section 28.3). Out of scope: the subsystem contracts that produce and consume the fields (Chapter 27); the invariant catalogue that mandates the fields' integrity (Chapter 26); the SLI/SLO definitions that govern the verification regimes' performance (Chapter 17); and the certification regime that certifies the fields' release (Chapter 22). The registry references these chapters but does not replicate their content.

3. Definitions

Canonical field: a named, owned, versioned, and verification-bound slot of platform state. Producer: the single subsystem authorized to write the field. Consumer: a subsystem that reads the field (read authority only). Mutability: the field's lifecycle rule (immutable, mutable pre-publication, append-only, derived, or config-managed). Verification regime: the combination of static enforcement, runtime monitor, and reconciliation job that proves the field's integrity. Registry: this chapter's authoritative catalogue of all canonical fields.

4. Responsibilities

Ownership of the registry is as follows:

- **Principal Architect:** Accountable for the registry schema and the domain organization.
- **Subsystem Owners:** Responsible for the accuracy of their produced fields' contracts (all eleven attributes).
- **Head of Data Engineering:** Responsible for the database-field mappings and migration planning.
- **Release Certification Authority:** Responsible for verifying every field's verification regime is operational before release.
- **Chief Systems Architect:** Accountable for refusing field deprecations or migrations that break invariants.

5. Design Principles

Fields are explicit (every field is named and registered, not implicit), single-producer (one subsystem writes; co-production is forbidden), verification-bound (every field has a runtime monitor), immutable-after-publication where the field represents the scholarly record, append-only where the field represents history, and conservative (when in doubt, the field is registered as immutable rather than mutable). A field that cannot be verified is not a canonical field; it is an internal implementation detail. A field whose verification is not operational is not honoured; it is asserted. The Production Acceptance Gate treats asserted-but-unverified fields as failed.

6. Engineering Requirements

Every canonical field in the platform SHALL be registered in section 28.2 with its four summary attributes (Field, Producer, Mutability, Verification). The twenty-five fields enumerated in section 28.3 SHALL have

full eleven-attribute contracts. The remaining fields' full contracts SHALL be published in the Engineering Playbook and SHALL be available to the Release Certification Authority on demand. Every field's verification regime SHALL be operational; absence of verification SHALL be treated as field failure and SHALL block release. Field deprecations and migrations SHALL be recorded in this chapter via Constitution Amendment (Chapter 36).

7. Engineering Constraints

The registry constrains the platform to single-producer fields: no two subsystems MAY write the same field, even with credentials that would technically permit it. The registry constrains the platform to verification-bound release: no field MAY be released without its verification regime operational. The registry constrains the platform to orderly migration: no field MAY be deprecated without a successor or a documented argument that no successor is needed. The Chief Systems Architect SHALL NOT grant exceptions to the eleven-attribute template; appendix attributes are permitted but the canonical eleven MAY NOT be omitted.

8. Runtime Behaviour

At runtime, every field is produced by exactly one subsystem, consumed by the enumerated set, and verified by its verification regime. The invariant monitors (Chapter 26) detect violations of the field's properties: immutability violations, CoI-precondition violations, chain-integrity violations. The reconciliation jobs detect drift between the platform's state and external truth (Crossref, ORCID, preservation backends, ISSN Register). The observability stack aggregates field-level verification results into the Constitution Compliance dashboard. A field whose verification regime fails for more than one cadence interval is in SEV-2 contract violation.

9. Failure Conditions

Registry failure conditions include:

- **Unregistered field:** a deployed subsystem writes a field not in section 28.2.
- **Multi-producer violation:** two subsystems write the same field.
- **Immutability violation:** a frozen field is modified (INV-M-01).
- **Verification gap:** a field's verification regime is not operational or not reconciling.
- **Deprecation-without-successor:** a field is deprecated without a successor or a documented argument.
- **Migration-without-amendment:** a field's schema or format is migrated without a Constitution Amendment.

10. Recovery Procedures

Field-violation recovery follows the incident-response regime (Chapter 21) with the additional requirement that recovery SHALL restore the field's integrity, not merely mask the symptom. Unregistered-field recovery: register the field, backfill its verification regime, re-release. Multi-producer recovery: revoke one producer's credentials, audit for conflicting writes, re-release. Immutability-

violation recovery: SEV-1 incident, reconstruct the correct field value from preservation deposit or audit log, investigate the modification path. Verification-gap recovery: instrument the missing monitor, backfill verification, re-release. Every recovery SHALL be recorded in the audit log with a full before/after analysis.

11. Security Considerations

The registry is a primary security control: single-producer fields limit the blast radius of credential compromise. The Identity/Auth Service SHALL scope credentials to the fields a subsystem is registered to produce. The audit trail of field-violation detections is itself a security artefact and SHALL be reviewed monthly by the Security Officer. Immutability-violation detections SHALL trigger immediate security incidents, because an immutability violation is, by definition, either a compromise or a critical engineering failure. The audit chain (audit_chain_hash) is the platform's ultimate defence against silent field tampering.

12. Performance Considerations

Each field's verification regime has a performance cost: invariant monitors consume compute, reconciliation jobs consume external-API quota. The performance budget for field verification SHALL be defined in Chapter 31 (Engineering Mathematics) and SHALL not exceed 5% of platform compute capacity. Verification regimes that exceed their budget SHALL be optimized (sampling instead of full scan, less frequent cadence), not skipped. Field writes are themselves performance-bound by their producers' Performance Budgets (Chapter 27); a field whose producer misses its SLO is in contract violation regardless of the field's own verification.

13. Observability Requirements

Every field's verification regime SHALL expose its current status (pass / fail / unknown), its last execution time, its last result, and its alert history as metrics. The Constitution Compliance dashboard SHALL aggregate these into a single view, organized by domain and by criticality. A field in 'unknown' state for more than its cadence interval is a SEV-2 incident (the platform is blind to that field's integrity). The Release Certification Authority SHALL verify that every field's verification regime is operational before RC1 sign-off.

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **All 236 canonical fields are registered in section 28.2 with their four summary attributes.**
- **All 25 representative fields in section 28.3 have complete eleven-attribute contracts.**
- **The Engineering Playbook publishes full eleven-attribute contracts for the remaining 211 fields.**
- **Every field's verification regime is operational and visible on the Constitution Compliance dashboard.**
- At least one chaos exercise per verification-regime class (invariant monitor, reconciliation job, schema validation) has been completed and postmortemed.

- The Principal Architect has signed off the registry, and the Release Certification Authority has verified the verification regimes.

15. Runtime Verification

Runtime verification of the registry is the Constitution Compliance dashboard itself: the dashboard aggregates per-field verification results, the invariant monitors (Chapter 26) detect property violations, the reconciliation jobs detect external-truth drift, and the audit log records every field write. The Release Certification Authority SHALL verify that every field's verification regime is operational before RC1 sign-off, and SHALL sample the audit log to confirm that field writes are recorded and traceable to their producers.

16. Certification Requirements

RC1 certification requires: the registry schema published (section 28.1); the domain registry complete (section 28.2, 236 fields); the twenty-five representative field contracts complete (section 28.3); the Engineering Playbook contracts for the remaining 211 fields published; every field's verification regime operational; at least one chaos exercise per verification-regime class completed; and the Principal Architect's sign-off that the registry is complete. The Release Certification Authority SHALL verify the runtime evidence and SHALL refuse RC1 sign-off if any field is unregistered, unverified, or inconsistently produced.

17. Future Evolution

The registry evolves through Constitution Amendments (Chapter 36). New fields MAY be added (typically when a new capability introduces new state); each addition SHALL publish a full eleven-attribute contract. Existing fields MAY be tightened (stricter mutability, stronger verification) but SHALL NOT be loosened without evidence that the looser contract still preserves the invariants of Chapter 26. A field MAY be deprecated; the deprecation SHALL record the successor field or a documented argument that no successor is needed, and SHALL preserve the deprecated field's historical values indefinitely for provenance. The eleven-attribute template itself SHALL NOT be changed without a Constitution Amendment that updates every existing contract to the new template.

Chapter 29 — Event Constitution

This chapter constitutes the Event Constitution of Opus Publica. The platform is event-driven: every meaningful state change in the editorial, composition, publication, dissemination, and preservation pipelines is announced as a discrete, versioned, durable event on the platform's event backbone. Workflows (Chapter 10) define what should happen; events define what did happen. A workflow is a prescription; an event is a fact. Without a constitution for events, the platform would accumulate ad-hoc messages whose meaning drifts, whose order is undefined, whose retry semantics are inconsistent, and whose auditability is unreliable. This chapter closes that gap by mandating a uniform event schema, a fixed taxonomy, a complete catalogue, and a backbone contract that every producer and consumer SHALL satisfy.

Every event defined here carries nine mandatory attributes: Producer, Consumer, Payload, Ordering, Retry, Idempotency, Timeout, Failure, and Audit. These nine attributes are the contract by which an event becomes a durable, recoverable, auditable unit of system truth. An event that omits any of the nine is, by definition, not an event of this platform and SHALL NOT be emitted on the backbone. The vocabulary of this chapter follows IETF RFC 2119; the transport semantics follow RFC 9110 where applicable; the audit retention follows NIST SP 800-92.

29.1 Event Schema

The event schema is the canonical structure that every event on the Opus Publica backbone SHALL conform to. The schema is transport-neutral: whether the event is delivered via the internal message bus, an external webhook, or a side-effect replayed from the outbox table, the nine mandatory attributes SHALL be present and SHALL carry the obligations specified below. The schema is registered in the Schema Registry (Section 29.6) and is the authoritative reference for every producer and consumer. An event that conforms to the schema is a first-class citizen of the platform; an event that does not is undefined behaviour.

Attribute	Content Obligation
Producer	Fully-qualified service identifier of the emitting component (e.g., submission-service@v2.3.1). MUST be a stable service name; the version suffix is REQUIRED for traceability and rollback.
Consumer	List of services authorized and expected to consume the event, with their delivery contract (push/pull). Wildcards and broadcast subscriptions are PROHIBITED; every consumer SHALL be named.
Payload	Versioned JSON Schema document describing every field carried. The schema URI MUST be present and resolvable. PII SHALL be referenced by identifier, not

	embedded; secrets SHALL NEVER be embedded. Payload size SHALL NOT exceed 256 KiB.
Ordering	The ordering key (e.g., article_id, submission_id) and the ordering guarantee (per-key FIFO, total order, none). Multiple keys SHALL NOT be mixed in one event type.
Retry	Retry policy: maximum attempts, back-off function (exponential with jitter RECOMMENDED), and the maximum retry window. Default is 5 attempts with exponential back-off over 30 minutes.
Idempotency	Idempotency key derivation rule. Consumers SHALL de-duplicate on this key. The key SHALL be deterministic from the payload; producer-generated UUIDs SHALL NOT be used because they defeat deduplication on replay.
Timeout	Synchronous acknowledgement timeout (producer waits for ack from backbone) and asynchronous delivery timeout (backbone waits for consumer ack). Defaults: 5 s sync, 60 s async.
Failure	Failure contract: dead-letter queue (DLQ) destination, alert severity (SEV-1/2/3), escalation owner, and the maximum dwell time in the DLQ before human intervention is REQUIRED.
Audit	Audit obligation: every event SHALL be written to the audit log with its correlation_id, causal_id, and payload hash, and SHALL be retained per the Records Retention policy (Chapter 19). Retention minimum is 7 years for domain events, 10 years for lifecycle and audit events.

29.2 Event Taxonomy

Events on the Opus Publica backbone are classified into five categories. The category determines routing, retention, and SLA expectations. Every event in the catalogue (Section 29.3) is tagged with exactly one category. An event MAY be re-categorized only through the Evolution Governance process (Chapter 25); the five-category taxonomy itself is fixed by this Constitution and SHALL NOT be altered without an amendment. Consumers SHALL subscribe to categories, not to individual events, to maintain decoupling; producers SHALL NOT know which consumers subscribe to their events.

Category	Definition	Examples	Retention

Domain event	A fact about a domain entity's lifecycle, emitted by an authoritative service.	ArticleSubmitted, ReviewSubmitted, DecisionRecorded, CanonicalFrozen	7 years
Integration event	A fact announced to an external system, requiring translation at the trust boundary.	CrossrefDeposited, ORCIDWritebackCompleted, OAI-PMHUpdated	7 years
Lifecycle event	A fact about the platform's own lifecycle (start, stop, rotate, scale).	CredentialRotated, InvariantViolationDetected	10 years
Audit event	A fact recorded solely for compliance evidence, no operational side-effect.	EditorialCheckCompleted (audit copy), ArtifactVerified	10 years
System event	A fact about infrastructure health (queue depth, consumer lag, heartbeat).	Defined in Chapter 17 (Observability Constitution)	90 days

Domain and integration events are the substance of the platform's operation; lifecycle and audit events are its accountability; system events are its instrumentation. The five-category taxonomy is the platform's contract for routing and retention: a consumer that subscribes to the Domain category SHALL receive every domain event without naming each one, and a retention policy applied to a category SHALL apply uniformly to every event in it.

29.3 Event Catalogue

The catalogue enumerates every event defined by this Constitution. Each event has a stable name, a single authoritative producer, and a defined set of consumers. Events are organized in approximate lifecycle order: intake, editorial checks, review, decision, revision, composition, preview, freeze, dissemination, preservation, indexing, post-publication, and operations. An event is added to the catalogue only through the Evolution Governance process (Chapter 25); an event is removed only by deprecation with a successor or a documented argument that no successor is needed. Every event in the catalogue SHALL have a registered schema version in the Schema Registry.

ArticleSubmitted

Emitted by the Submission Service when an author completes the submission wizard and a new submission is durably created. Triggers editorial intake and the automated editorial check pipeline. Once emitted, the submission is durable: even if downstream services are unavailable, the event remains in the outbox until successfully delivered.

Attribute	Specification
Producer	Submission Service (submission-service@vN)
Consumer	Editorial Service, Notification Service, Audit Service, Search Indexer
Payload	{submission_id, article_id, journal_id, author_ids[], submitted_at, manuscript_uri, manuscript_sha256, metadata_snapshot}
Ordering	Per-submission_id FIFO
Retry	5 attempts; exponential back-off 1s/2s/4s/8s/16s; max window 30 min
Idempotency	key = submission_id + submission_version
Timeout	5 s sync ack; 60 s async delivery
Failure	DLQ submission.dlq; SEV-2 alert; max dwell 24 h
Audit	Audit log with correlation_id = submission_id; payload hash pinned

EditorialCheckStarted

Emitted by the Editorial Service when an automated editorial integrity check begins running on a newly submitted manuscript. Records the rule-set version and the checker instance identifier, allowing the Editorial Operations dashboard to display in-flight checks and the audit log to record the start of an integrity evaluation.

Attribute	Specification
Producer	Editorial Service (editorial-service@vN)
Consumer	Editorial Dashboard, Audit Service
Payload	{check_id, submission_id, rule_set_version, started_at, checker_instance_id}
Ordering	Per-submission_id FIFO; after ArticleSubmitted
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = check_id
Timeout	5 s sync; 30 s async
Failure	DLQ editorial.dlq; SEV-3; max dwell 24 h
Audit	Audit log with correlation_id = submission_id

EditorialCheckCompleted

Emitted when an automated editorial check finishes. Carries the outcome (pass/fail/warn), the rules that fired, and remediation hints. Drives the submission state machine: a fail routes the submission to editorial triage rather than discarding it.

Attribute	Specification
Producer	Editorial Service
Consumer	Workflow Service, Editorial Dashboard, Audit Service, Notification Service
Payload	{check_id, submission_id, outcome, fired_rules[], remediation_hints, completed_at, duration_ms}
Ordering	Per-submission_id FIFO; after EditorialCheckStarted
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = check_id
Timeout	5 s sync; 30 s async
Failure	DLQ editorial.dlq; SEV-2; max dwell 12 h (blocks state advance)
Audit	Audit log with correlation_id = submission_id; rule outcomes pinned

ReviewerAssigned

Emitted by the Workflow Service when an editor assigns a reviewer to a submission. Initiates the reviewer invitation timer and the notification workflow. The assignment identifier is the authoritative key for the subsequent reviewer-lifecycle events.

Attribute	Specification
Producer	Workflow Service (workflow-service@vN)
Consumer	Notification Service, Reviewer Service, Audit Service
Payload	{assignment_id, submission_id, reviewer_user_id, assigned_by, assigned_at, due_at}
Ordering	Per-submission_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = assignment_id
Timeout	5 s sync; 30 s async
Failure	DLQ workflow.dlq; SEV-2; max dwell 6 h

Audit	Audit log with correlation_id = submission_id

ReviewerAccepted

Emitted by the Reviewer Service when an invited reviewer accepts the assignment. Releases the manuscript to the reviewer's queue and starts the review clock. Idempotent on the assignment identifier so that a duplicate accept does not double-release the manuscript.

Attribute	Specification
Producer	Reviewer Service (reviewer-service@vN)
Consumer	Workflow Service, Notification Service, Audit Service
Payload	{assignment_id, submission_id, reviewer_user_id, accepted_at}
Ordering	Per-assignment_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = assignment_id + accepted_at
Timeout	5 s sync; 30 s async
Failure	DLQ reviewer.dlq; SEV-3; max dwell 6 h
Audit	Audit log with correlation_id = submission_id

ReviewerDeclined

Emitted when an invited reviewer declines the assignment or the invitation expires. Triggers the re-assignment workflow. The reason code is recorded for editorial analytics but SHALL NOT contain free text that could leak PII.

Attribute	Specification
Producer	Reviewer Service
Consumer	Workflow Service, Notification Service, Audit Service
Payload	{assignment_id, submission_id, reviewer_user_id, declined_at, reason_code}
Ordering	Per-assignment_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = assignment_id + declined_at
Timeout	5 s sync; 30 s async
Failure	DLQ reviewer.dlq; SEV-3; max dwell 6 h

Audit	Audit log with correlation_id = submission_id

ReviewSubmitted

Emitted when a reviewer submits a completed review. Carries the recommendation, the comments URI (comments are stored as artifacts, not embedded), and the conflict-of-interest declaration. Drives the decision workflow. A SEV-1 alert is raised if this event fails to deliver because a lost review is a scholarly-record corruption risk.

Attribute	Specification
Producer	Reviewer Service
Consumer	Workflow Service, Editorial Service, Notification Service, Audit Service
Payload	{review_id, submission_id, reviewer_user_id, recommendation, comments_uri, coi_declared, submitted_at}
Ordering	Per-submission_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = review_id
Timeout	5 s sync; 60 s async
Failure	DLQ reviewer.dlq; SEV-1; max dwell 2 h
Audit	Audit log with correlation_id = submission_id; review content hash pinned

DecisionRecorded

Emitted by the Workflow Service when an editor records a decision (accept, minor revision, major revision, reject). Authoritative for the submission state machine. The rationale is stored as an immutable artifact; the decision is final once recorded and MAY only be superseded by a subsequent DecisionRecorded event with a recorded supersession reason.

Attribute	Specification
Producer	Workflow Service
Consumer	Notification Service, Editorial Service, Audit Service, Composition Service (if accept)
Payload	{decision_id, submission_id, decision, decided_by, decided_at, rationale_uri}

Ordering	Per-submission_id FIFO; total order with RevisionRequested
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = decision_id
Timeout	5 s sync; 60 s async
Failure	DLQ workflow.dlq; SEV-1; max dwell 1 h
Audit	Audit log with correlation_id = submission_id; rationale immutable

RevisionRequested

Emitted when the editor requests revisions. Carries the revision scope, the reviewer points to address, and the response deadline. Total-ordered with DecisionRecorded so that downstream consumers can deterministically reconstruct the decision-revision sequence.

Attribute	Specification
Producer	Workflow Service
Consumer	Notification Service, Author Portal, Audit Service
Payload	{revision_id, submission_id, scope, due_at, requested_at}
Ordering	Per-submission_id FIFO; total order with DecisionRecorded
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = revision_id
Timeout	5 s sync; 30 s async
Failure	DLQ workflow.dlq; SEV-2; max dwell 6 h
Audit	Audit log with correlation_id = submission_id

RevisionSubmitted

Emitted when the author submits a revised manuscript. Carries the response-to-reviewers document and the new manuscript manifest. Triggers a new editorial check cycle on the revised manuscript.

Attribute	Specification
Producer	Submission Service
Consumer	Workflow Service, Editorial Service, Audit Service

Payload	{revision_id, submission_id, version, manuscript_uri, manuscript_sha256, response_to_reviewers_uri, submitted_at}
Ordering	Per-submission_id FIFO; per-version
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = revision_id + version
Timeout	5 s sync; 60 s async
Failure	DLQ submission.dlq; SEV-2; max dwell 6 h
Audit	Audit log with correlation_id = submission_id; manuscript hash pinned

CompositionStarted

Emitted by the Composition Service when typesetting begins on an accepted manuscript. Records the composition job identifier and the input artifact manifest so that the composition dashboard can display in-flight jobs and the audit log can attribute downstream artifacts.

Attribute	Specification
Producer	Composition Service (composition-service@vN)
Consumer	Workflow Service, Audit Service, Composition Dashboard
Payload	{composition_id, submission_id, article_id, input_manifest, started_at, composition_template_id}
Ordering	Per-article_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = composition_id
Timeout	5 s sync; 60 s async
Failure	DLQ composition.dlq; SEV-2; max dwell 4 h
Audit	Audit log with correlation_id = article_id

CompositionCompleted

Emitted when the Composition Service produces the canonical JATS XML and the galley proofs. Triggers preview generation. The JATS hash is pinned in the audit log so that any subsequent modification of the JATS artifact is detectable as an invariant violation (Chapter 26, INV-I-02).

Attribute	Specification

Producer	Composition Service
Consumer	Preview Service, Workflow Service, Audit Service
Payload	{composition_id, article_id, jats_uri, jats_sha256, galleys[], completed_at, duration_ms}
Ordering	Per-article_id FIFO; after CompositionStarted
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = composition_id
Timeout	5 s sync; 60 s async
Failure	DLQ composition.dlq; SEV-2; max dwell 4 h
Audit	Audit log with correlation_id = article_id; JATS hash pinned

PreviewGenerated

Emitted when the Preview Service renders the HTML and PDF previews of the composed article. Carries the preview URIs and the rendering manifest. Drives the editorial review interface and the author preview interface.

Attribute	Specification
Producer	Preview Service (preview-service@vN)
Consumer	Editorial Review Interface, Workflow Service, Audit Service
Payload	{preview_id, article_id, html_preview_uri, pdf_preview_uri, render_manifest, generated_at}
Ordering	Per-article_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = preview_id
Timeout	5 s sync; 60 s async
Failure	DLQ preview.dlq; SEV-2; max dwell 4 h
Audit	Audit log with correlation_id = article_id

PreviewAccepted

Emitted when the editor or author approves the preview. Authorizes canonical freeze. The sign-off is recorded as an immutable artifact; the acceptance is final once recorded and MAY only be reversed by an explicit PreviewRejected event with a recorded reason.

Attribute	Specification
Producer	Editorial Review Interface (via Workflow Service)
Consumer	Workflow Service, Composition Service, Audit Service
Payload	{preview_id, article_id, accepted_by, accepted_at, sign_off_uri}
Ordering	Per-article_id FIFO; after PreviewGenerated
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = preview_id + accepted_by
Timeout	5 s sync; 30 s async
Failure	DLQ workflow.dlq; SEV-2; max dwell 4 h
Audit	Audit log with correlation_id = article_id; sign-off immutable

PreviewRejected

Emitted when the editor or author rejects the preview. Routes the article back to composition with rejection reasons. The reasons are structured (not free text) so that composition analytics can aggregate common failure modes.

Attribute	Specification
Producer	Editorial Review Interface (via Workflow Service)
Consumer	Composition Service, Workflow Service, Audit Service
Payload	{preview_id, article_id, rejected_by, rejected_at, reasons[]}
Ordering	Per-article_id FIFO; after PreviewGenerated
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = preview_id + rejected_by
Timeout	5 s sync; 30 s async
Failure	DLQ workflow.dlq; SEV-3; max dwell 4 h
Audit	Audit log with correlation_id = article_id

CanonicalFrozen

Emitted when the Workflow Service freezes the canonical record: the article's metadata, JATS XML, and artifact manifest become immutable. This is the pivot event for publication — every downstream event (PDFGenerated, HTMLRendered, DOIMinted, PreservationDeposited) depends on the canonical manifest referenced here.

Attribute	Specification
Producer	Workflow Service
Consumer	Publication Service, DOI Service, Preservation Service, Audit Service, Search Indexer
Payload	{article_id, frozen_at, canonical_manifest, frozen_by}
Ordering	Per-article_id FIFO; total order with PreviewAccepted
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = article_id + canonical_version
Timeout	5 s sync; 60 s async
Failure	DLQ workflow.dlq; SEV-1; max dwell 1 h
Audit	Audit log with correlation_id = article_id; canonical manifest hash pinned

PDFGenerated

Emitted by the Publication Service when the published PDF artifact is generated and content-addressed. Carries the SHA-256 and the PDF/A compliance flag. The SHA-256 is the artifact's permanent identifier in the Artifact Store (Chapter 6, Field Registry).

Attribute	Specification
Producer	Publication Service (publication-service@vN)
Consumer	Artifact Store, Preservation Service, Audit Service, Search Indexer
Payload	{article_id, pdf_uri, pdf_sha256, pdf_a_version, generated_at, generated_by_tool}
Ordering	Per-article_id FIFO; after CanonicalFrozen
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = article_id + pdf_sha256
Timeout	5 s sync; 60 s async

Failure	DLQ publication.dlq; SEV-1; max dwell 1 h
Audit	Audit log with correlation_id = article_id; SHA-256 pinned

HTMLRendered

Emitted when the published HTML rendering is generated. Carries the rendering manifest, the asset list, and the figure manifest. The HTML SHA-256 is pinned for invariant monitoring (Chapter 26, INV-I-02).

Attribute	Specification
Producer	Publication Service
Consumer	Artifact Store, CDN, Audit Service, Search Indexer
Payload	{article_id, html_uri, html_sha256, asset_manifest, figure_manifest, rendered_at}
Ordering	Per-article_id FIFO; after CanonicalFrozen
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = article_id + html_sha256
Timeout	5 s sync; 60 s async
Failure	DLQ publication.dlq; SEV-1; max dwell 1 h
Audit	Audit log with correlation_id = article_id; SHA-256 pinned

ArtifactVerified

Emitted by the Artifact Verification Service when a published artifact passes integrity verification (SHA-256 match, PDF/A validity, JATS schema validity). A failed verification is a SEV-1 incident and blocks publication.

Attribute	Specification
Producer	Artifact Verification Service (verification-service@vN)
Consumer	Workflow Service, Audit Service, Invariant Monitor
Payload	{verification_id, article_id, artifact_uri, artifact_sha256, checks[], outcome, verified_at}
Ordering	Per-article_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min

Idempotency	key = verification_id
Timeout	5 s sync; 30 s async
Failure	DLQ verification.dlq; SEV-1 (blocks publication); max dwell 1 h
Audit	Audit log with correlation_id = article_id

DOIMinted

Emitted by the DOI Service when a new DOI is registered with the DOI Registration Agency. Carries the DOI, the article_id, and the registration metadata snapshot. The DOI is immutable once minted; this event is the authoritative record of its existence.

Attribute	Specification
Producer	DOI Service (doi-service@vN)
Consumer	Publication Service, Crossref Deposit Service, Audit Service, Search Indexer
Payload	{doi, article_id, registered_at, registration_agency, metadata_snapshot}
Ordering	Per-article_id FIFO; after CanonicalFrozen
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = doi
Timeout	5 s sync; 60 s async
Failure	DLQ doi.dlq; SEV-1; max dwell 1 h
Audit	Audit log with correlation_id = article_id; DOI immutable

PublicationStarted

Emitted when the Publication Service begins the publication sequence for a frozen canonical record. Marks the transition into the publication pipeline. The target_publication_at field is the scheduled publication time; the actual PublicationCompleted event MAY differ.

Attribute	Specification
Producer	Publication Service
Consumer	Workflow Service, Audit Service, Operations Dashboard
Payload	{publication_id, article_id, started_at,

	target_publication_at}
Ordering	Per-article_id FIFO; after CanonicalFrozen
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = publication_id
Timeout	5 s sync; 30 s async
Failure	DLQ publication.dlq; SEV-1; max dwell 1 h
Audit	Audit log with correlation_id = article_id

PublicationCompleted

Emitted when the article is fully published: PDF generated, HTML rendered, DOI minted, Crossref deposited, preservation deposited, search indexer updated. Marks the article as live. The published_at timestamp is immutable and is the authoritative publication time cited in metadata and Crossref deposits.

Attribute	Specification
Producer	Publication Service
Consumer	Workflow Service, Notification Service, OAI-PMH Service, Audit Service
Payload	{publication_id, article_id, completed_at, published_at, artifact_manifest}
Ordering	Per-article_id FIFO; total order with the entire publication chain
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = article_id + published_at
Timeout	5 s sync; 60 s async
Failure	DLQ publication.dlq; SEV-1; max dwell 30 min
Audit	Audit log with correlation_id = article_id; published_at immutable

CrossrefDeposited

Emitted when the Crossref Deposit Service submits the article's metadata to Crossref. Carries the deposit batch ID and the submission receipt. The deposit is asynchronous; Crossref returns a batch ID that the Reconciliation Service polls for the outcome.

Attribute	Specification
Producer	Crossref Deposit Service (crossref-service@vN)
Consumer	Reconciliation Service, Audit Service
Payload	{deposit_id, article_id, doi, batch_id, submitted_at, receipt_uri}
Ordering	Per-article_id FIFO; after DOIMinted
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = deposit_id
Timeout	5 s sync; 60 s async
Failure	DLQ crossref.dlq; SEV-2; max dwell 4 h
Audit	Audit log with correlation_id = article_id

CrossrefReconciled

Emitted when the Reconciliation Service confirms that Crossref has accepted and indexed the deposit and that the DOI resolves to the canonical landing page. Closes the Crossref integration loop and feeds the DOI resolvability invariant (Chapter 26, INV-R-01).

Attribute	Specification
Producer	Reconciliation Service (reconciliation-service@vN)
Consumer	Publication Service, Audit Service, Invariant Monitor
Payload	{reconciliation_id, article_id, doi, crossref_status, resolved_url, reconciled_at}
Ordering	Per-article_id FIFO; after CrossrefDeposited
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = reconciliation_id
Timeout	5 s sync; 30 s async
Failure	DLQ crossref.dlq; SEV-2; max dwell 4 h
Audit	Audit log with correlation_id = article_id

ORCIDWritebackCompleted

Emitted when the ORCID Integration Service successfully writes the published work to each author's ORCID record. Carries the ORCID iD and the ORCID put-code. One event is emitted per author so that a partial failure (one author's ORCID is unavailable) does not block the others.

Attribute	Specification
Producer	ORCID Integration Service (orcid-service@vN)
Consumer	Audit Service, Invariant Monitor, Notification Service
Payload	{writeback_id, article_id, author_orcid_id, put_code, written_at}
Ordering	Per-article_id FIFO; one event per author
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = writeback_id (= orcid + article_id)
Timeout	5 s sync; 60 s async
Failure	DLQ orcid.dlq; SEV-2; max dwell 4 h
Audit	Audit log with correlation_id = article_id

PreservationDeposited

Emitted when the Preservation Service deposits the article's canonical manifest and all artifacts to the long-term preservation archive (CLOCKSS, LOCKSS, or Portico). The deposit is a large payload; the async timeout is extended to 5 minutes.

Attribute	Specification
Producer	Preservation Service (preservation-service@vN)
Consumer	Audit Service, Invariant Monitor
Payload	{deposit_id, article_id, archive, manifest_uri, deposited_at}
Ordering	Per-article_id FIFO; after PublicationCompleted
Retry	5 attempts; exponential back-off; max window 60 min
Idempotency	key = deposit_id
Timeout	5 s sync; 5 min async (large payload)
Failure	DLQ preservation.dlq; SEV-1; max dwell 1 h
Audit	Audit log with correlation_id = article_id

PreservationReceiptReceived

Emitted when the preservation archive returns a receipt for a deposited article. Closes the preservation invariant (Chapter 26, INV-P-01). A receipt that does not arrive within 24 h is an SLA violation and a SEV-1 incident.

Attribute	Specification
Producer	Preservation Service (receipt poller)
Consumer	Audit Service, Invariant Monitor
Payload	{receipt_id, article_id, archive, receipt_uri, received_at}
Ordering	Per-article_id FIFO; after PreservationDeposited
Retry	5 attempts; exponential back-off; max window 60 min
Idempotency	key = receipt_id
Timeout	5 s sync; 30 s async
Failure	DLQ preservation.dlq; SEV-1; max dwell 24 h (SLA violation)
Audit	Audit log with correlation_id = article_id

OAI-PMHUpdated

Emitted when the OAI-PMH Service publishes or updates the article's record in the OAI-PMH endpoint. Carries the OAI identifier and the setSpec. Drives harvester visibility (OpenAIRE, DOAJ, BASE).

Attribute	Specification
Producer	OAI-PMH Service (oai-pmh-service@vN)
Consumer	Audit Service, Search Indexer
Payload	{oai_identifier, article_id, set_spec, action, updated_at}
Ordering	Per-article_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = oai_identifier + action + updated_at
Timeout	5 s sync; 30 s async
Failure	DLQ oai-pmh.dlq; SEV-2; max dwell 4 h
Audit	Audit log with correlation_id = article_id

IndexUpdated

Emitted when the Search Indexer ingests an article into the discovery index. Carries the index name, the document version, and the field manifest. A failed ingestion means the article is invisible to search; a SEV-3 alert is raised because the article is still discoverable via DOI and OAI-PMH.

Attribute	Specification
Producer	Search Indexer (indexer-service@vN)
Consumer	Audit Service, Search Service
Payload	{index_name, article_id, document_version, fields[], updated_at}
Ordering	Per-article_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = index_name + article_id + document_version
Timeout	5 s sync; 30 s async
Failure	DLQ index.dlq; SEV-3; max dwell 4 h
Audit	Audit log with correlation_id = article_id

ArticleRetracted

Emitted when an editor records a retraction. Authoritative for the post-publication state machine; triggers Crossref, OAI-PMH, ORCID, and indexer updates. The retraction notice is an immutable artifact; the retraction is final once recorded.

Attribute	Specification
Producer	Editorial Retraction Service (via Workflow Service)
Consumer	Crossref Deposit Service, OAI-PMH Service, ORCID Integration Service, Preservation Service, Search Indexer, Audit Service
Payload	{retraction_id, article_id, retraction_notice_uri, retracted_by, retracted_at, rationale}
Ordering	Per-article_id FIFO; total order with ArticleCorrected
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = retraction_id
Timeout	5 s sync; 60 s async
Failure	DLQ editorial.dlq; SEV-1; max dwell 30 min
Audit	Audit log with correlation_id = article_id; retraction immutable

ArticleCorrected

Emitted when an editor records a correction. Carries the corrected version identifier and the correction notice URI. The corrected version becomes the current version; the prior version remains accessible for the scholarly record.

Attribute	Specification
Producer	Editorial Correction Service (via Workflow Service)
Consumer	Crossref Deposit Service, OAI-PMH Service, ORCID Integration Service, Preservation Service, Search Indexer, Audit Service
Payload	{correction_id, article_id, corrected_version, notice_uri, corrected_by, corrected_at, change_summary}
Ordering	Per-article_id FIFO; total order with ArticleRetracted
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = correction_id
Timeout	5 s sync; 60 s async
Failure	DLQ editorial.dlq; SEV-1; max dwell 30 min
Audit	Audit log with correlation_id = article_id

ReviewerCoIDeclared

Emitted when a reviewer submits a conflict-of-interest declaration. Triggers editorial review if a non-trivial COI is declared. The declaration level is structured (none, minor, major, disqualifying) so that editorial analytics can aggregate.

Attribute	Specification
Producer	Reviewer Service
Consumer	Editorial Service, Audit Service, Compliance Service
Payload	{declaration_id, assignment_id, reviewer_user_id, coi_level, coi_details_uri, declared_at}
Ordering	Per-assignment_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = declaration_id
Timeout	5 s sync; 30 s async
Failure	DLQ reviewer.dlq; SEV-2; max dwell 4 h

Audit	Audit log with correlation_id = assignment_id

CredentialRotated

Emitted when a platform credential (API key, signing certificate, database password) is rotated. Carries the credential identifier (not the secret) and the rotation scope. Secret values SHALL NEVER appear in the payload; only the version identifiers and the rotation metadata are recorded.

Attribute	Specification
Producer	Credential Management Service (credential-service@vN)
Consumer	Audit Service, Secret Manager, Operations Dashboard
Payload	{credential_id, rotation_id, scope, rotated_by, rotated_at, previous_version, new_version}
Ordering	Per-credential_id FIFO
Retry	5 attempts; exponential back-off; max window 30 min
Idempotency	key = rotation_id
Timeout	5 s sync; 30 s async
Failure	DLQ credentials.dlq; SEV-1; max dwell 30 min
Audit	Audit log with correlation_id = rotation_id; secret values NEVER in payload

InvariantViolationDetected

Emitted by an Invariant Monitor (Chapter 26) when a runtime check fails. Triggers the incident-response regime. Carries the invariant identifier, the violation identifier, and the evidence URI. The evidence is immutable; the violation is paged immediately because an invariant violation is, by definition, a corruption of the scholarly record.

Attribute	Specification
Producer	Invariant Monitor (invariant-monitor@vN)
Consumer	Incident Response Service, Notification Service, Audit Service, Operations Dashboard
Payload	{invariant_id, violation_id, detected_at, evidence_uri, severity, monitor_instance_id}
Ordering	Per-invariant_id FIFO
Retry	8 attempts (escalating); exponential back-off; max

	window 10 min
Idempotency	key = violation_id
Timeout	5 s sync; 10 s async (urgent)
Failure	DLQ invariant.dlq; SEV-1; max dwell 5 min (paging fallback)
Audit	Audit log with correlation_id = violation_id; evidence immutable

29.4 Event Ordering and Causality

Ordering is the contract that turns a stream of independent events into a meaningful narrative. Without ordering guarantees, the platform cannot answer the simple question 'did this event happen before or after that event?' and cannot replay state machines deterministically. This section defines the ordering model of the Opus Publica backbone.

Per-key FIFO ordering is the default. Each event type declares an ordering key (typically the article_id or submission_id). Events with the same key SHALL be delivered to consumers in the order the producer emitted them. Events with different keys MAY be delivered in any order; consumers SHALL NOT assume cross-key ordering without an explicit causal chain. Per-key FIFO is sufficient for the platform's correctness because every stateful entity (article, submission, assignment) is uniquely identified and its event stream is independent of other entities' streams.

Causal consistency is provided through two correlators that every event SHALL carry: the correlation_id (the end-to-end trace identifier, propagated from the originating user or system action) and the causal_id (the identifier of the event that caused this event, if any). Together these form a directed acyclic graph of causation that the audit log SHALL preserve. A consumer that needs to know 'why did this event happen?' SHALL walk the causal chain in the audit log; a consumer that needs to know 'what did this user action trigger?' SHALL walk the correlation_id chain.

- **Correlation ID:** end-to-end trace identifier, propagated from the originating user or system action. Every event emitted in response to a user action SHALL carry the same correlation_id.
- **Causal ID:** the event identifier of the immediate cause. Forms a DAG of causation in the audit log. The first event in a chain (typically triggered by a user action) has causal_id = null.
- **Idempotency Key:** deterministic from the payload; consumers SHALL de-duplicate on this key. Producer-generated UUIDs SHALL NOT be used because they defeat deduplication on replay.
- **Happens-before:** the partial order defined by causal_id chains. Events not connected by a happens-before edge are concurrent and MAY be processed in any order.

The backbone SHALL NOT promise total ordering across all events; such a promise is impossible to keep at scale and unnecessary for correctness. The backbone SHALL promise per-key FIFO and causal consistency. Consumers that need a total order across keys SHALL implement a merge operator that uses

the causal_id graph to reconstruct a total order for their specific purpose; the backbone SHALL NOT be responsible for performing this merge.

29.5 Event Backbone

The event backbone is the platform's nervous system: the substrate over which events flow from producers to consumers. The backbone SHALL be implemented as a transactional outbox pattern — events are written to an outbox table in the same database transaction as the producer's state change, and a relay process forwards outbox rows to the message bus. This pattern guarantees that an event is emitted if and only if the state change it describes is committed; there is no 'event lost because the bus was down' failure mode and no 'event emitted for a state change that was rolled back' failure mode.

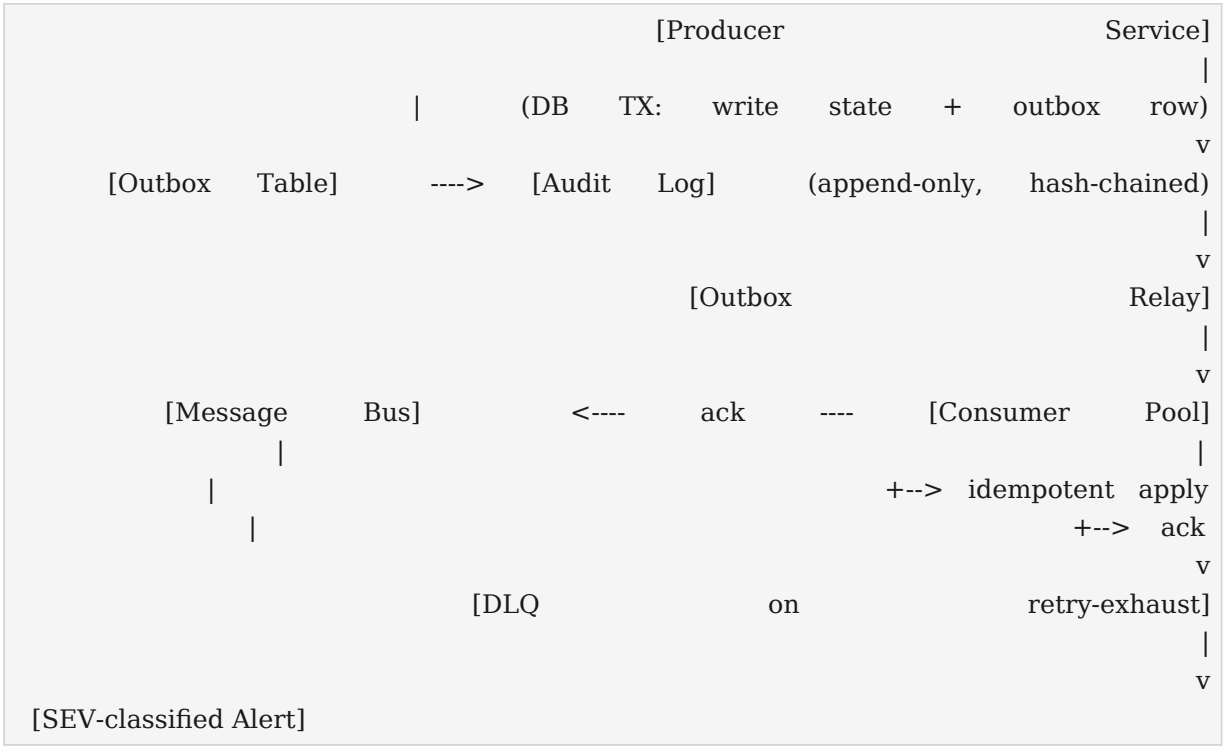

Figure 29.1 — Transactional outbox pattern: events are committed with state and relayed to the bus.

Delivery semantics are at-least-once: the backbone SHALL deliver each event to each consumer at least once. Duplicates are possible and SHALL be tolerated by consumers via the idempotency key. Exactly-once delivery is NOT promised and SHALL NOT be relied upon; the cost of exactly-once (distributed consensus on every message) is not justified for the platform's throughput requirements. Exactly-once processing — the property the platform actually needs — is achieved through idempotent consumers that de-duplicate on the idempotency key before applying side effects.

At-Least-Once Delivery + Idempotent Consumers = Exactly-Once Processing

This is the contract. Producers SHALL assume their events may be delivered more than once; consumers SHALL assume they may receive the same event more than once; the platform SHALL guarantee that the visible effect of an event is the same whether it is delivered once or many times. A consumer that is not

idempotent is, by definition, in violation of this Constitution.

The dead-letter queue (DLQ) is the backbone's safety net. An event that exhausts its retry budget SHALL be moved to the DLQ for its event type. Each DLQ has a maximum dwell-time SLA; an event in the DLQ beyond its dwell-time SLA SHALL raise a SEV-classified alert and SHALL page the escalation owner. DLQ contents SHALL be replayable: an operator SHALL be able to re-inject a DLQ message into the bus after the consumer bug has been fixed. DLQ contents SHALL be access-controlled and SHALL be retained for the same period as the originating event category.

29.6 Event Schema Evolution

Event schemas evolve. New fields are added; old fields are deprecated; occasionally a field's semantics change. Uncontrolled evolution is the leading cause of consumer breakage in event-driven systems. This section defines the platform's schema evolution contract.

The Schema Registry (a component of the Knowledge Graph, Chapter 23) is the authoritative source of every event schema. Every event type SHALL have a registered schema with a semantic version (MAJOR.MINOR.PATCH). Producers SHALL emit events tagged with the schema version; consumers SHALL declare the schema versions they accept. The Schema Registry SHALL reject a producer that emits an unregistered schema version.

Change	Compatibility	Allowed?
Add optional field	Backward + forward	YES; MINOR bump
Remove optional field	Backward + forward	YES; MINOR bump
Add required field	Breaking (forward)	NO without consumer coordination
Remove required field	Breaking (backward)	NO without consumer coordination
Change field semantics	Breaking	NO; new event type instead
Change field type	Breaking	NO; new event type instead
Rename field	Breaking	NO; new event type instead

Backward-compatible changes (consumers built against the old schema continue to work) MAY be deployed without coordination. Forward-compatible changes (producers built against the old schema continue to work) MAY be deployed without coordination. Breaking changes — those that are neither backward nor forward compatible — SHALL be deployed through the dual-publish pattern: the new event type is published alongside the old for a deprecation window of at least 90 days, after which the old event type is retired. The retirement SHALL be announced to all known consumers at the start of the window and SHALL be confirmed at the end of the window before the old event type is removed from the Schema Registry.

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

The purpose of this chapter is to constitute the events of Opus Publica as a first-class engineering concern: every meaningful state change is a discrete, versioned, durable, auditable event, and every event satisfies a uniform nine-attribute schema. The chapter exists because workflows (Chapter 10) define what should happen; events define what did happen. Without an event constitution, the platform's auditability, recoverability, and operational debuggability degrade over time as ad-hoc messages accumulate, their meaning drifts, and their ordering becomes indeterminate. The chapter establishes the contract by which an event becomes a unit of system truth.

2. Scope

This chapter applies to every event emitted on the Opus Publica event backbone, including domain events, integration events, lifecycle events, audit events, and system events. It governs producers, consumers, the backbone infrastructure, the Schema Registry, and the dead-letter queue subsystem. It does NOT govern synchronous API requests (Chapter 9), internal service-to-service RPC, or batch ETL pipelines. Where an event is delivered over an external integration (Crossref, ORCID, OAI-PMH), the event's contract still applies; the integration boundary translates but does not weaken the contract.

3. Definitions

Event: a discrete, immutable, versioned record of a state change. Producer: the service authoritative for the event. Consumer: a service that subscribes to the event. Backbone: the message bus and outbox infrastructure. Outbox: a database table written in the same transaction as the state change. Relay: the process that forwards outbox rows to the bus. Idempotency Key: a deterministic key derived from the payload, used by consumers to de-duplicate. Correlation ID: end-to-end trace identifier. Causal ID: identifier of the immediate cause event. DLQ: dead-letter queue for events that exhaust their retries. Schema Registry: the authoritative source of every event schema, a component of the Knowledge Graph (Chapter 23).

4. Responsibilities

Producers are responsible for emitting events atomically with state changes, for conforming to the schema, for tagging the schema version, and for carrying the correlation_id propagated from the originating action. Consumers are responsible for de-duplicating on the idempotency key, for acknowledging receipt, for handling failures without crashing the consumer pool, and for emitting follow-on events with the correct causal_id. The Backbone Team is responsible for the outbox relay, the message bus, the DLQ, and the Schema Registry. The Engineering Leadership Team is responsible for the event catalogue and for approving new event types through Evolution Governance.

5. Design Principles

Events are facts, not commands; they describe what happened, not what should happen. Events are immutable; once emitted, they cannot be modified, only superseded by a subsequent event. Events are self-describing; the payload carries enough context for a consumer to act without a synchronous back-channel to the producer. Events are deterministic; the same state change produces the same event (modulo timestamp and correlation_id). Events are observable; every event is logged, traced, and metered. Events are bounded; the payload size SHALL NOT exceed 256 KiB, and larger artifacts SHALL be referenced by URI.

6. Engineering Requirements

Every event SHALL conform to the nine-attribute schema (Section 29.1). Every event type SHALL be registered in the Schema Registry with a semantic version. Every producer SHALL write events to the outbox in the same database transaction as the state change. Every consumer SHALL de-duplicate on the idempotency key. Every consumer SHALL acknowledge receipt within the async timeout. Every event SHALL carry a correlation_id and a causal_id. Every event type SHALL have a defined DLQ destination and a SEV-classified alert. The backbone SHALL provide per-key FIFO and causal consistency.

7. Engineering Constraints

The backbone SHALL NOT promise exactly-once delivery. The backbone SHALL NOT promise total ordering across keys. The payload size SHALL NOT exceed 256 KiB. The synchronous acknowledgement timeout SHALL NOT exceed 10 seconds. The maximum retry window SHALL NOT exceed 24 hours. Producers SHALL NOT embed PII directly in the payload; they SHALL reference PII by identifier. Consumers SHALL NOT perform synchronous back-channels to producers as part of event processing (this creates coupling that defeats the event-driven architecture). Secrets SHALL NEVER appear in event payloads.

8. Runtime Behaviour

At runtime, producers emit events by writing an outbox row in the same transaction as their state change. The outbox relay polls the outbox table at high frequency (sub-second) and forwards rows to the message bus. The bus routes events to subscribers based on event type and category. Consumers pull or are pushed events, de-duplicate on the idempotency key, apply their side effects, and acknowledge. Failed events are retried with exponential back-off; events that exhaust their retries are moved to the DLQ and an alert is raised. The backbone exposes metrics for queue depth, consumer lag, retry count, and DLQ dwell time.

9. Failure Conditions

Failure conditions include:

- **Outbox relay failure:** outbox rows accumulate but are not forwarded to the bus.
- **Message bus outage:** events are committed to the outbox but not delivered.
- **Consumer crash:** events are delivered but not processed; backlog grows.

- **Consumer bug:** events are processed but the side effect is incorrect.
- **Schema drift:** producer emits a payload that does not conform to the registered schema.
- **DLQ overflow:** DLQ exceeds its dwell-time SLA; messages are at risk of loss.
- **Correlation ID loss:** an event is emitted without a propagated correlation_id, breaking the audit chain.
- **Idempotency key collision:** two distinct events produce the same key, causing one to be silently dropped.

10. Recovery Procedures

Recovery from outbox relay failure: restart the relay; backlog drains automatically because the outbox is durable. Recovery from message bus outage: bus restarts; consumers resume from their last acknowledged offset. Recovery from consumer crash: consumer restarts; redelivers from the last ack. Recovery from DLQ overflow: investigate the poison message, fix the consumer, replay the message from the DLQ. Recovery from correlation_id loss: query the audit log for the event, reconstruct the causal chain. Recovery from idempotency collision: identify the colliding events, change the key derivation, replay.

11. Security Considerations

Events SHALL NOT carry secrets. PII SHALL be referenced by identifier, not embedded. The backbone SHALL authenticate producers and consumers via mutual TLS or signed tokens. The DLQ SHALL be access-controlled; its contents may contain sensitive operational data. The Schema Registry SHALL be tamper-evident; schema changes SHALL be logged. Events crossing the trust boundary to external integrations (Crossref, ORCID) SHALL be sanitized to remove internal-only fields. The audit log of events SHALL be cryptographically chained (Chapter 26, INV-A-03) so that tampering is detectable.

12. Performance Considerations

The backbone's performance budget is defined in Chapter 31 (Engineering Mathematics). The end-to-end event latency (producer commit to consumer ack) p95 SHALL be ≤ 500 ms for domain events and ≤ 5 s for integration events. The backbone SHALL sustain 10,000 events per second peak throughput. The outbox relay SHALL poll at sub-second intervals. Consumer pools SHALL auto-scale on lag. The Schema Registry SHALL respond in ≤ 50 ms. Metrics SHALL be sampled at ≤ 1% to avoid metric-induced load. Monitoring that exceeds its budget SHALL be optimized, not skipped.

13. Observability Requirements

Every event SHALL be logged with its correlation_id, causal_id, idempotency key, schema version, and payload hash. The backbone SHALL expose metrics for queue depth, consumer lag, retry count, DLQ size, and end-to-end latency. A trace SHALL span from the originating user action through every event emitted in response, linked by correlation_id. The event dashboard SHALL display the catalogue, the live event flow, the DLQ, and the consumer-lag chart. An event stuck in the DLQ beyond its dwell-time SLA SHALL raise a SEV-classified alert.

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **The nine-attribute schema is implemented in the Schema Registry.**
- **All 34 events in the catalogue are registered with versions.**
- **The outbox pattern is implemented by every producer.**
- **Consumers de-duplicate on idempotency keys.**
- **The DLQ is operational with SEV-classified alerts.**
- **Per-key FIFO ordering is verified by a chaos test.**
- **Causal consistency is verified by a replay test.**
- **The Chief Systems Architect has signed off the event catalogue.**

15. Runtime Verification

Runtime verification of this chapter is the event monitor framework: a monitor that verifies schema conformance on a sampled basis, a monitor that verifies per-key FIFO ordering by checking sequence numbers, a monitor that verifies the audit log is complete (every emitted event appears), a monitor that verifies the DLQ dwell-time SLA, and a monitor that verifies the correlation_id is propagated. The monitors' outputs SHALL be aggregated into the invariant dashboard (Chapter 26).

16. Certification Requirements

RC1 certification requires: the Schema Registry operational with all 34 event types registered; the outbox pattern implemented by every producer; at-least-once delivery verified by chaos test; exactly-once processing verified by replay test (consumers correctly de-duplicate); the DLQ operational with dwell-time alerts; the event dashboard live; and the Chief Systems Architect's sign-off that the event catalogue is complete (no known event is missing).

17. Future Evolution

The event catalogue evolves through Evolution Governance (Chapter 25). New events MAY be added (typically when a new workflow step is introduced). Existing events MAY have their schemas evolved under the compatibility rules in Section 29.6. An event SHALL NOT be removed; if obsolete, it SHALL be deprecated with a successor or a documented argument that no successor is needed. New event categories MAY be added only through a Constitution amendment; the five-category taxonomy is fixed by this chapter. The backbone MAY be replaced by a different implementation provided the contract (per-key FIFO, causal consistency, at-least-once delivery, idempotent consumers) is preserved.

Chapter 30 — UI Constitution

This chapter constitutes the User Interface (UI) Constitution of Opus Publica. The platform exists to serve readers, authors, editors, and reviewers; the interface is the surface through which they encounter the platform's scholarly record. The original RC1 Engineering Directive spanned 303 pages yet barely governed UI engineering; this Constitution closes that gap. It defines the design tokens, the typography, the layout, the accessibility regime, the component catalogue, the page templates, the preview subsystems, the rendering pipeline, the performance budget, and the internationalization strategy. Every visual element of the platform SHALL trace to a specification in this chapter; every interaction SHALL satisfy an accessibility contract; every page SHALL meet a performance budget. The UI is not decoration; it is part of the platform's contract with its users.

The chapter is grounded in WCAG 2.2 (Level AA), WAI-ARIA 1.2, the Core Web Vitals, and the conventions of contemporary component systems (Material Design 3, Radix UI, shadcn). Where this chapter is silent, the UI SHALL conform to WCAG 2.2 AA as the floor. Where this chapter is more specific than WCAG, this chapter governs. The vocabulary of this chapter follows IETF RFC 2119; the HTTP semantics of the rendering pipeline follow RFC 9110 where applicable.

30.1 Design Tokens

Design tokens are the atomic variables from which every visual property of the platform is derived. Tokens are the single source of truth: typography, spacing, color, grid, elevation, radii, and motion are all expressed as named tokens, and the implementation layer (CSS custom properties, Tailwind theme, design-tokens.json) consumes them. Hard-coded values in component CSS SHALL NOT be permitted; every value SHALL trace to a token. Tokens are versioned; token changes propagate through the build system to every consumer.

The token categories are: typography tokens (font-family, font-size, font-weight, line-height); spacing tokens (a 4-pixel base scale); color tokens (primary, accent, neutral, semantic success/warning/error/info, with text/bg/border variants); grid tokens (12-column, breakpoints); elevation tokens (box-shadow levels); radius tokens (border-radius levels); and motion tokens (durations, easing curves). The spacing and color tables below are the authoritative scales; the typography scale is in Section 30.2 and the grid scale is in Section 30.3.

Token	Value	Use
space-1	4 px	Tight inline gaps; icon-to-label
space-2	8 px	Default inline gap; small padding
space-3	12 px	Card inner padding
space-4	16 px	Default block gap; grid gutter (mobile)

space-5	24 px	Section gap; grid gutter (tablet/desktop)
space-6	32 px	Large section gap
space-7	48 px	Page-level vertical rhythm
space-8	64 px	Hero-level vertical rhythm

Token	Hex	Use
color-primary-900	#0B2A4A	Primary brand, headings
color-primary-700	#1A3F6B	Headers, primary-action hover
color-primary-500	#2E5A8A	Links, primary actions
color-accent-500	#B8860B	Accent highlights, rules
color-neutral-900	#1A1A1A	Body text
color-neutral-700	#4A4A4A	Secondary text
color-neutral-500	#7A7A7A	Muted text
color-neutral-100	#F2F4F7	Surface alt
color-neutral-0	#FFFFFF	Surface
color-success	#1B7F4B	Success states
color-warning	#B8860B	Warning states
color-error	#B23A2E	Error states
color-info	#2E6F9E	Info states

Elevation tokens define five shadow levels: e0 (none), e1 (subtle card), e2 (raised card), e3 (dropdown), e4 (modal), e5 (popover). Radius tokens define five levels: r0 (0), r1 (4 px), r2 (8 px), r3 (16 px), r4 (full pill). Motion tokens define three durations (fast = 100 ms, base = 200 ms, slow = 400 ms) and two easing curves (ease-out = cubic-bezier(0.16, 1, 0.3, 1) for entrances; ease-in-out = cubic-bezier(0.65, 0, 0.35, 1) for transitions). Motion durations SHALL NOT exceed 400 ms; longer animations SHALL be replaced with a state change.

30.2 Typography

Typography is the platform's voice. The type system uses a single family for UI text (Inter, with system-ui fallback) and a complementary serif family for long-form article reading (Source Serif Pro, with Georgia fallback). Monospace is reserved for code, identifiers, and DOI display (JetBrains Mono, with ui-monospace fallback). Font loading SHALL use a font-display strategy that prevents FOIT (Flash of Invisible

Text) and minimizes FOUT (Flash of Unstyled Text): the critical UI font SHALL be preloaded; the article font SHALL be loaded with font-display: optional to avoid layout shift.

Token	Family	Size	Weight	Line Height	Use
Display	Inter	48 px	700	1.1	Hero titles
H1	Inter	36 px	700	1.2	Page titles
H2	Inter	28 px	600	1.25	Section titles
H3	Inter	22 px	600	1.3	Subsection titles
Body	Inter	16 px	400	1.5	Default body
Body-L	Source Serif Pro	18 px	400	1.6	Article body
Caption	Inter	13 px	400	1.4	Captions, secondary
Overline	Inter	12 px	600	1.2	Labels, eyebrows

Fallbacks: every font declaration SHALL include a system fallback stack that produces acceptable rendering before the web font loads and degrades gracefully if the web font never loads (e.g., user blocking third-party fonts). The fallback stack for Inter is 'Inter, system-ui, -apple-system, Segoe UI, Roboto, sans-serif'. The fallback stack for Source Serif Pro is 'Source Serif Pro, Georgia, Times New Roman, serif'. The fallback stack for JetBrains Mono is 'JetBrains Mono, ui-monospace, SFMono-Regular, Menlo, monospace'. The fallback metric (ascender, descender, x-height) SHALL be tuned to minimize layout shift on font load.

30.3 Layout and Grid

The layout system is a responsive 12-column grid with three breakpoints: mobile (<768 px), tablet (768–1024 px), and desktop (>1024 px). The grid uses a 16-pixel gutter on mobile and a 24-pixel gutter on tablet and desktop. Container widths are constrained to a maximum of 1280 px on desktop to preserve reading comfort; wider screens receive additional margin, not additional content columns.

Breakpoint	Min Width	Columns	Gutter	Container Max
Mobile	320 px	4	16 px	100% (fluid)
Tablet	768 px	8	24 px	720 px
Desktop	1024 px	12	24 px	1280 px

On mobile, the layout SHALL be single-column by default. On tablet, the layout MAY use up to 8 columns; the article body SHALL remain single-column at 720 px maximum for readability. On desktop, the layout MAY use the full 12-column grid; the article body SHALL remain at 720–800 px maximum width

regardless of viewport (scholarly readability overrides grid utilization). Sidebar layouts (journal landing page, search results) MAY use 8+4 or 9+3 column splits on desktop. The 12-column grid SHALL be implemented with CSS Grid; Flexbox MAY be used for component-internal layouts.

30.4 Accessibility

Accessibility is not a feature; it is a contract. The platform SHALL conform to WCAG 2.2 Level AA. Where this chapter specifies a more stringent requirement, that requirement governs. The platform's accessibility commitments are organized into seven domains: perceivable, operable, understandable, robust (the four WCAG principles), plus keyboard navigation, screen reader support, and reduced motion. Every component defined in Section 30.5 SHALL specify its own accessibility contract; where a component's contract exceeds WCAG AA, the component's contract governs.

- **Color contrast:** text SHALL meet 4.5:1 against its background; large text (≥ 24 px or ≥ 18.66 px bold) and UI components SHALL meet 3:1. Color SHALL NOT be the sole carrier of information.
- **Keyboard navigation:** every interactive element SHALL be reachable via Tab in a logical order; the visible focus indicator SHALL meet 3:1 contrast against adjacent colors; keyboard shortcuts SHALL NOT trap focus.
- **Screen reader support:** every interactive element SHALL have an accessible name; every image SHALL have alt text (decorative images use alt=""); every form control SHALL have a programmatically associated label; every dynamic region SHALL use the appropriate ARIA live region.
- **Focus management:** route changes (SPA navigation, dialog open/close) SHALL move focus intentionally; focus SHALL be trapped within modal dialogs while open and restored to the trigger on close.
- **ARIA patterns:** the platform SHALL use WAI-ARIA 1.2 patterns (combobox, dialog, menu, tablist, tree) per the WAI-ARIA Authoring Practices; custom widgets SHALL match the closest ARIA pattern.
- **Reduced motion:** the platform SHALL honor prefers-reduced-motion: reduce by disabling non-essential animations; parallax, auto-playing carousels, and full-page transitions SHALL be disabled or replaced with crossfades.
- **Skip links:** a skip-to-content link SHALL be the first focusable element on every page; the link SHALL be visually hidden until focused.

WCAG 2.2 AA is the floor, not the ceiling

Components defined in Section 30.5 SHALL specify their own accessibility contracts; where a component's contract exceeds WCAG AA, the component's contract governs. Accessibility regressions SHALL block the release just as performance regressions do. The accessibility audit (axe-core in CI) SHALL be a mandatory release gate.

30.5 Component Constitution

The component catalogue enumerates the platform's UI building blocks. Each component has a spec table covering purpose, anatomy, states, props, accessibility, usage guidelines, and anti-patterns. Components are implemented in the platform's component library (a Radix-UI-based system styled with Tailwind, consistent with the shadcn convention) and consumed by page templates (Section 30.6). A component SHALL NOT be added to the library without a spec entry in this section; a component SHALL NOT be removed without a successor.

Buttons

Buttons trigger actions. The platform defines six variants: primary (filled, brand color, for the main action), secondary (outlined, for supporting actions), tertiary (text-only, for tertiary actions), destructive (filled, error color, for irreversible actions), ghost (transparent, for low-emphasis actions), and icon (icon-only, for compact actions like closing a dialog).

Specification	Value
Purpose	Trigger an action; submit a form; navigate; toggle a state.
Anatomy	Label (REQUIRED for non-icon variants); optional leading/trailing icon; focus ring; press feedback (motion-fast).
States	default, hover, focus-visible, active, disabled, loading (spinner replaces leading icon, label kept).
Props	variant, size (sm/md/lg), iconLeading, iconTrailing, isLoading, isDisabled, type (button/submit), onClick, aria-label (REQUIRED for icon variant).
Accessibility	Native <button> element; focus-visible ring at 3:1 contrast; disabled uses aria-disabled not pointer-events:none; icon variant REQUIRES aria-label; minimum hit target 44×44 px (WCAG 2.2 SC 2.5.8).
Usage Guidelines	One primary button per section; destructive actions REQUIRE a confirmation dialog; loading state disables further clicks; full-width buttons only on mobile.
Anti-Patterns	with onClick; multiple primary buttons in one row; using disabled instead of loading state for in-flight actions; icon buttons without aria-label.

Cards

Cards are the primary container for summary content. The platform defines three card types: the article card (citation summary, DOI, authors, journal), the author card (name, ORCID, affiliation, article count), and the journal card (title, ISSN, scope, latest issue). All cards share the same anatomy but specialize their content.

Specification	Value
Purpose	Surface a summary of a discrete entity; provide a click target to the entity's detail page.
Anatomy	Container (radius-2, elevation-1, hover elevation-2); header (eyebrow + title); body (metadata list); footer (actions or status).
States	default, hover (elevation-2, cursor pointer), focus-visible, selected (accent border).
Props	variant (article|author|journal), title, href, metadata[], actions[], image (optional), isSelectable.
Accessibility	Entire card is a link (<a>), not a div with onClick; nested interactive elements have their own tab stop; image alt text describes the entity; card title is the link's accessible name.
Usage Guidelines	Use in grids of 1 (mobile), 2 (tablet), 3 (desktop); consistent metadata ordering; image optional but recommended for journal cards; click anywhere on the card navigates.
Anti-Patterns	Multiple competing links inside one card (use a footer actions row instead); non-clickable cards (use a panel instead); card-in-card nesting.

Forms

Forms are the primary input surface. The platform defines inputs (text, email, number, password, date), textareas, selects, checkboxes, radios, and file upload. All form controls share a consistent anatomy, validation model, and accessibility contract.

Specification	Value
Purpose	Capture structured user input.
Anatomy	Label (above control, REQUIRED); control; helper text (below); error message (below, error color); required indicator (* or aria-required).

States	default, focus, filled, disabled, readonly, error (with message), success (validation passed).
Props	id, label, value, onChange, error, helperText, isRequired, isDisabled, isReadOnly, autoComplete (REQUIRED for password and email per WCAG 2.2 SC 3.3.7/3.3.8).
Accessibility	<label> programmatically associated via for/id; error announced via aria-describedby + aria-invalid; required indicated via aria-required; group radios/checkboxes in <fieldset> with <legend>; instructions before the control, not after.
Usage Guidelines	One primary action per form (Submit); secondary actions (Cancel) on the left; inline validation on blur, not on every keystroke; password fields show/hide toggle; file upload with drag-and-drop and progress.
Anti-Patterns	Placeholder-as-label; validation only on submit; error messages that don't explain how to fix; auto-advancing focus; disabling the submit button without explaining why.

Dialogs / Modals

Dialogs interrupt the user's flow to capture attention or input. The platform uses dialogs for confirmations (e.g., destructive action confirmation), focused tasks (e.g., reviewer invitation), and detail views (e.g., figure lightbox).

Specification	Value
Purpose	Interruptive task or confirmation.
Anatomy	Overlay (neutral-900 at 50% opacity); container (radius-3, elevation-4, max-width 560 px); header (title + close); body; footer (actions).
States	closed, opening (motion-base fade + scale), open, closing.
Props	isOpen, onOpenChange, title, description, children, footerActions, size (sm/md/lg), isDismissable.
Accessibility	role='dialog' + aria-modal='true' + aria-labelledby; focus trapped inside while open; Esc closes; focus restored to trigger on close; scroll lock on body; alert dialogs (role='alertdialog') for destructive

	confirmations.
Usage Guidelines	One dialog open at a time; dismissable by overlay click unless the action is destructive; max-height 80vh with internal scroll on body; footer actions right-aligned with primary on the right.
Anti-Patterns	Nested dialogs; dialogs over dialogs without a stack manager; non-dismissable dialogs for non-critical actions; dialogs without a visible close button.

Navigation

The navigation system comprises the header (global nav, journal switcher, search, user menu), the sidebar (editorial dashboard navigation), breadcrumbs (hierarchical location), tabs (in-page section switching), and pagination (list navigation).

Specification	Value
Purpose	Orient the user; enable movement between pages and sections.
Anatomy	Header (sticky, elevation-1, max-height 64 px); sidebar (collapsible, max-width 240 px); breadcrumbs (semantic list); tabs (horizontal, underline indicator); pagination (prev/next + numbered).
States	active (current page), hover, focus-visible, disabled (tab only).
Props	items[] (label, href, icon, badge), aria-current='page' for active, onTabChange (controlled tabs).
Accessibility	nav element with aria-label; skip-to-content link first; breadcrumbs in <nav aria-label='Breadcrumb'> with ; tabs use WAI-ARIA tablist pattern (roving tabindex, arrow keys); pagination uses <nav aria-label='Pagination'>.
Usage Guidelines	Header persistent across all pages; sidebar on dashboard pages only; breadcrumbs below header on detail pages; max 7 top-level tabs; pagination shows first, last, current ± 1.
Anti-Patterns	More than 7 top-level nav items; mega-menus on mobile; tabs that load a new page (use real navigation instead); disabled pagination buttons without explanation.

Tables

Tables display structured data. The platform defines data tables (read-only grids), sortable tables (column-header click to sort), and paginated tables (server-side pagination).

Specification	Value
Purpose	Display structured tabular data.
Anatomy	Caption (visually hidden, for screen readers); thead (column headers); tbody (rows); tfoot (optional summary); sortable headers have button + sort indicator.
States	row hover (neutral-100 background), sortable header hover, sorted column header (accent indicator), selected row (accent left border).
Props	columns[] (id, header, accessor, sortable, width), data[], sortBy, sortDir, page, pageSize, totalRows, onSort, onPageChange.
Accessibility	<table> with <caption>; <th scope='col'> for headers; sortable headers as <button> with aria-sort; row selection via checkbox with label; never use <table> for layout.
Usage Guidelines	Default page size 25; column widths as fractions, not pixels; sticky header on tables > 1 viewport; zebra striping optional, hover REQUIRED.
Anti-Patterns	Tables for layout; client-side sorting of > 1000 rows; infinite scroll instead of pagination; hiding column headers.

Empty States

Empty states appear when a list or view has no content. The platform defines empty states for: no search results, no submissions yet, no reviews assigned, journal with no articles, author with no articles.

Specification	Value
Purpose	Communicate absence; guide the user to the next action.
Anatomy	Illustration (optional, decorative); title; description; primary action (CTA); secondary action (link).

States	Single state (no variations).
Props	title, description, primaryAction (label, href), secondaryAction (label, href), illustration (optional).
Accessibility	Title as <h2> or <h3> in the page outline; illustration alt='' (decorative); actions are real links/buttons, not divs.
Usage Guidelines	Always provide a path forward (action or explanation); never just say 'No results'; consistent illustration style across empty states.
Anti-Patterns	'No data' with no explanation; empty states that blame the user; illustration without alternative text for screen readers if meaningful.

Loading States

Loading states communicate that work is in progress. The platform defines three loading patterns: skeletons (placeholder shapes that match content layout), spinners (in-button or inline indeterminate progress), and progress bars (determinate, for multi-step uploads).

Specification	Value
Purpose	Communicate in-progress work; preserve layout (no CLS).
Anatomy	Skeleton (neutral-100 block with shimmer animation); spinner (12 px or 16 px circular); progress bar (track + fill, with optional label).
States	pending (indeterminate), progress (determinate, 0–100%), complete.
Props	variant (skeleton|spinner|progress), size, value (progress only), label (progress RECOMMENDED).
Accessibility	Skeleton uses aria-hidden (decorative); spinner uses role='status' + aria-label='Loading'; progress bar uses role='progressbar' + aria-valuenow/min/max; long operations > 5 s SHALL show estimated time.
Usage Guidelines	Skeleton for initial page load (matches layout); spinner for in-button actions (< 2 s); progress bar for uploads; never use a full-page spinner.
Anti-Patterns	Full-page spinner for partial updates; spinner without accessible label; progress bar that jumps to 100%

	suddenly.

Error States

Error states communicate that something went wrong. The platform defines form errors (inline, per-control), page errors (banner, recoverable), 404 (not found), and 500 (server error).

Specification	Value
Purpose	Communicate failure; guide recovery.
Anatomy	Form error (error-color text below control, with icon); page error (banner at top, dismissable); 404/500 (full-page illustration + message + recovery actions).
States	Single state per type.
Props	type (form|page|404|500), message, recoveryActions[], code (optional, for debugging).
Accessibility	Form errors via aria-invalid + aria-describedby; page errors as role='alert' (assertive) for new errors, role='status' for updates; 404/500 pages SHALL have an <h1> with the error code.
Usage Guidelines	Form errors explain how to fix, not just what's wrong; page errors dismissable; 404/500 always offer a path home; log the error ID for support.
Anti-Patterns	'Something went wrong' with no recovery; stack traces in the UI; error messages that blame the user.

Success States

Success states communicate that an action completed. The platform uses toast notifications (transient, bottom-right) for inline successes and banners (persistent, top) for page-level successes.

Specification	Value
Purpose	Confirm completion; provide next-step guidance.
Anatomy	Toast (icon + title + description + optional action, auto-dismiss after 5 s); banner (icon + title + description + optional action, dismissable).
States	entering, visible, leaving (toast only).
Props	variant (toast|banner), title, description, action (label, onClick), duration (toast default 5 s).

Accessibility	Toast uses role='status' (polite) or role='alert' (assertive for errors); banner uses role='region' + aria-label; toast container is aria-live='polite'.
Usage Guidelines	Toast for transient feedback (submission saved); banner for persistent page-level feedback (account verified); max 3 toasts visible; never use toast for irreversible actions.
Anti-Patterns	Auto-dismissing error toasts before the user reads them; modal-blocking success dialogs; toast for critical confirmations.

Notifications

The notification system unifies toasts, banners, and inline notifications under a single API. Inline notifications appear within content (e.g., a reviewer-COI warning on a review card) and persist until dismissed or resolved.

Specification	Value
Purpose	Surface information that requires user attention without interrupting flow.
Anatomy	Icon (semantic); title; description; optional action; optional dismiss.
States	info, success, warning, error.
Props	severity (info|success|warning|error), title, description, action, isDismissable, onDismiss.
Accessibility	Severity conveyed by icon AND text AND color (never color alone); role per severity (status for info/success, alert for warning/error); aria-live region for dynamic notifications.
Usage Guidelines	One notification per concern; never stack more than 3 inline; warning/error require explicit dismiss or resolution.
Anti-Patterns	Color-only severity; notification spam; notifications that auto-dismiss when severity is error.

30.6 Page Templates

Page templates compose components into full pages. Each template has a defined layout, a primary content area, and a set of supporting regions (header, footer, sidebar, breadcrumbs). The platform defines five primary templates. Each template SHALL satisfy the accessibility and performance contracts defined in Sections 30.4 and 30.10.

Template	Layout	Primary Region	Supporting Regions
Article Landing	Single column, max 800 px	Article body (HTML)	Header, breadcrumbs, citation sidebar, references, footer
Journal Landing	8+4 column (desktop)	Journal description + latest articles	Header, sidebar (issues, about, editorial board), footer
Author Profile	8+4 column	Author bio + article list	Header, sidebar (ORCID, affiliation, metrics), footer
Search Results	8+4 column	Result list (article cards)	Header, sidebar (facets: journal, year, type), footer
Submission Wizard	Centered single column, max 720 px	Step content	Header, step indicator, footer (prev/next)

Article landing pages SHALL render server-side for SEO and citation indexing (Google Scholar, Crossref, OAI-PMH harvesters). The submission wizard SHALL preserve form state across steps and SHALL autosave to the server every 30 seconds; the wizard SHALL warn the user on attempted navigation away with unsaved changes. Search results SHALL debounce the query input (300 ms) and SHALL display a skeleton during the query. The journal landing page SHALL display the latest issue's table of contents as the primary content; archived issues SHALL be accessible via the sidebar.

Article landing page anatomy: the page SHALL lead with the article title, the author list (each name a link to the author profile), the affiliations, the corresponding-author email (obfuscated to deter harvesting), the DOI badge, the publication date, the citation block (with a 'Cite' button that opens a dialog offering APA, MLA, Chicago, and BibTeX formats), the abstract, the article body (HTML rendering per Section 30.8), the references list, the figure list, and the supplementary-materials list. The citation sidebar SHALL offer persistent links (DOI, Permalink) and a download-PDF link. The page SHALL embed JSON-LD structured data (Schema.org ScholarlyArticle) for search-engine indexing.

30.7 PDF Preview

The PDF preview component renders an in-browser preview of the published PDF for editors and authors during the review and preview stages, and for readers as an alternative to the HTML rendering. The component SHALL use the browser's native PDF viewer (via <iframe> or <embed>) for fidelity and

accessibility; a JavaScript-based PDF renderer MAY be used for features the native viewer lacks (page thumbnails, side-by-side comparison). The native viewer SHALL be the default because it is the most accessible and most performant option.

Page navigation: the component SHALL provide next/previous page controls, a 'go to page' input, and a thumbnail rail. Zoom: the component SHALL provide zoom-in, zoom-out, fit-width, and fit-page controls. The component SHALL sync the current page and zoom level to the URL hash for shareable links. The component SHALL respect prefers-reduced-motion by disabling smooth scroll-to-page animations. Keyboard shortcuts (Left/Right for paging, +/− for zoom) SHALL be documented in a help popover. The PDF preview SHALL be tested with screen readers (NVDA, VoiceOver) because native PDF viewers vary widely in accessibility; where the native viewer is inaccessible, the component SHALL offer an HTML alternative (Section 30.8).

30.8 HTML Preview

The HTML preview component renders the article's canonical HTML rendering. It is the primary reading surface for readers and the canonical artifact cited by other chapters. The component SHALL render the JATS-derived HTML with semantic structure (headings, sections, figures, tables, references) and SHALL enhance it with three interactive affordances: citation tooltips, figure lightbox, and reference back-references.

Citation tooltips: inline citations (e.g., '[Smith 2023]') SHALL render a hover/focus tooltip showing the formatted reference and a link to the reference list entry. The tooltip SHALL be dismissable by Esc and SHALL trap focus while open. Figure lightbox: figures SHALL be clickable to open a full-screen lightbox with the figure at full resolution, the caption, and a download link. The lightbox SHALL follow the dialog accessibility contract (Section 30.5). Reference back-references: reference list entries SHALL link back to the citing inline citations; the back-references SHALL appear as superscript links. The HTML preview SHALL be the default reading surface; the PDF preview SHALL be available as an alternative.

30.9 Rendering Pipeline

The rendering pipeline defines where and how HTML is generated for the browser. The platform uses a hybrid strategy: Server-Side Rendering (SSR) for the initial HTML of every page, Incremental Static Regeneration (ISR) for content that changes infrequently (article landing pages, journal landing pages), and Client-Side Rendering (CSR) for interactive regions that cannot be server-rendered (the submission wizard, the editorial dashboard, the search facets).

Hydration: the server-rendered HTML SHALL be hydrated on the client to enable interactivity. Hydration SHALL be progressive (selective hydration of interactive islands) to avoid blocking the main thread on pages with large server-rendered content (article bodies). Streaming: the platform SHALL use HTTP streaming (chunked transfer encoding per RFC 9110) to send the document head and above-the-fold content before the full response is ready, reducing Time to First Byte (TTFB) and Largest Contentful Paint

(LCP). Hydration mismatches between server and client SHALL be treated as a bug; the cause SHALL be diagnosed and fixed, not suppressed.

Caching: the platform SHALL cache server-rendered HTML at the CDN edge with a revalidation strategy appropriate to each route type. Article landing pages SHALL be cached with a stale-while-revalidate window of 60 minutes (articles rarely change; the DOI continues to resolve during the cache window). Journal landing pages SHALL be cached with a stale-while-revalidate window of 5 minutes (issues change more frequently). Search results SHALL NOT be cached at the edge (queries are user-specific). The submission wizard and the editorial dashboard SHALL NOT be cached (authenticated, mutating). Cache-invalidation events SHALL be emitted when an article is published, corrected, or retracted, so the CDN cache for the affected article landing page is purged within 60 seconds.

SSR is the default

CSR is permitted only when the region is genuinely interactive AND server rendering would compromise the user experience. The decision SHALL be recorded in the page-template spec and justified in an Architecture Decision Record (Chapter 7). A page that is fully CSR SHALL NOT be permitted for any reader-facing route.

30.10 Performance Budget

The platform SHALL meet the Core Web Vitals targets at the 75th percentile of real-user traffic: Largest Contentful Paint (LCP) ≤ 2.5 s; Interaction to Next Paint (INP) ≤ 200 ms; Cumulative Layout Shift (CLS) ≤ 0.1. These targets SHALL be measured by Real User Monitoring (RUM) and SHALL be alerted on breach. Synthetic monitoring (Lighthouse CI) SHALL gate every release; a regression beyond the budget SHALL block the release.

Asset Type	Budget	Notes
Initial JS (gzip)	170 KB	Per route, including framework
Initial CSS (gzip)	30 KB	Critical CSS only
Per-route lazy JS	80 KB	Loaded on demand
Article body HTML	200 KB	JATS-derived, before images
Web font (woff2)	40 KB	Subset to Latin by default
Hero image (AVIF)	100 KB	Above-the-fold

Image optimization: all images SHALL be served in AVIF (with WebP fallback) at the resolution appropriate to the viewport; the platform SHALL use the srcset and sizes attributes; hero images SHALL be preloaded; decorative images SHALL use loading='lazy'; all images SHALL have explicit width and height attributes to prevent CLS. The platform SHALL NOT serve unoptimized images on any reader-

facing route. Third-party scripts (analytics, error tracking) SHALL be loaded with defer or async and SHALL NOT block the main thread for more than 50 ms on initial load.

30.11 Internationalization

The platform SHALL support internationalization (i18n) from the start, even if the initial launch is English-only. The UI SHALL be translatable without code changes; all user-facing strings SHALL be externalized to message catalogs keyed by a stable identifier. The platform SHALL use the ICU MessageFormat for plurals, gender, and selection. The default locale is en-US; additional locales MAY be added without code changes.

Right-to-left (RTL) support: the platform SHALL support RTL layouts (Arabic, Hebrew) by using logical CSS properties (margin-inline-start, padding-inline-end) instead of physical properties (margin-left, padding-right). The layout SHALL mirror automatically based on the dir attribute on <html>. RTL SHALL be tested as part of every component's acceptance criteria; the pseudolocalization build SHALL catch physical properties that escaped review.

Locale-aware formatting: dates SHALL be formatted per locale (Intl.DateTimeFormat); numbers SHALL be formatted per locale (Intl.NumberFormat); currencies SHALL be formatted per locale (Intl.NumberFormat with currency). The platform SHALL NOT hard-code date or number formats; the format SHALL derive from the user's locale preference.

- **Default locale:** en-US. Locale negotiation: URL path (/en/, /es/), then Accept-Language header, then user preference, then default.
- **Translation workflow:** strings extracted to .po/.json catalogs; translations submitted via the localization platform; catalogs versioned with the release.
- **Pseudolocalization:** the build SHALL pseudolocalize (replace strings with accented equivalents) to detect hard-coded strings and layout fragility; pseudolocalization SHALL be a CI gate.
- **Identifier handling:** DOIs, ISSNs, ORCID iDs, and other scholarly identifiers SHALL NOT be localized; they are locale-independent.

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

The purpose of this chapter is to constitute the user interface of Opus Publica as a governed engineering surface. The platform's readers, authors, editors, and reviewers encounter the platform through its UI; the quality of that encounter is governed here. The chapter exists because the original Directive, despite spanning 303 pages, did not define the UI's design tokens, typography, accessibility contract, component

catalogue, performance budget, or internationalization strategy. This chapter closes that gap with a complete, derivable UI constitution.

2. Scope

This chapter applies to every user-facing UI surface of Opus Publica: the public article and journal pages, the author submission portal, the editorial dashboard, the reviewer interface, and the administrative console. It governs design tokens, typography, layout, accessibility, components, page templates, previews, the rendering pipeline, performance, and internationalization. It does NOT govern the editorial content policies (Chapter 12), the API contracts behind the UI (Chapter 9), or the workflow logic (Chapter 10); it governs the UI surface that consumes those APIs and presents those workflows.

3. Definitions

Design Token: a named, versioned variable from which a visual property is derived. Component: a reusable UI building block with a spec entry in Section 30.5. Page Template: a composition of components into a full page. Hydration: the process of attaching interactivity to server-rendered HTML. ISR: Incremental Static Regeneration. RUM: Real User Monitoring. Core Web Vitals: LCP, INP, CLS. WCAG: Web Content Accessibility Guidelines. WAI-ARIA: Web Accessibility Initiative – Accessible Rich Internet Applications. RTL: Right-to-Left layout. FOIT/FOUT: Flash of Invisible/Unstyled Text.

4. Responsibilities

Design System Team is responsible for design tokens, typography, color, the component library, and the page templates. Frontend Engineering Teams are responsible for implementing pages that consume the component library and satisfy the accessibility and performance contracts. Accessibility Reviewer is responsible for auditing components and pages against WCAG 2.2 AA. Performance Engineer is responsible for the RUM dashboard and the bundle-budget CI gate. Localization Manager is responsible for the message catalogs and the translation workflow. The Chief Systems Architect is the final escalation for cross-cutting UI decisions.

5. Design Principles

The UI is the platform's contract with its users: it MUST be accessible to all readers regardless of ability, device, or network; it MUST be performant (Core Web Vitals are a hard floor, not an aspiration); it MUST be consistent (the same component renders the same way everywhere); it MUST be internationalizable (English is a default, not a constraint); it MUST be observable (every interaction is instrumented); it MUST be resilient (the UI degrades gracefully when APIs fail). Form follows function; ornament is permitted only when it does not compromise any of the six principles above.

6. Engineering Requirements

Every visual property SHALL derive from a design token. Every page SHALL conform to WCAG 2.2 AA. Every interactive element SHALL have an accessible name. Every image SHALL have alt text. Every page SHALL meet the Core Web Vitals budgets at p75. Every route SHALL stay within its JS/CSS bundle budget.

Every user-facing string SHALL be externalized to a message catalog. Every RTL-supported locale SHALL pass the pseudolocalization build. Every component SHALL have a spec entry in Section 30.5. Reader-facing routes SHALL be server-rendered.

7. Engineering Constraints

The UI SHALL NOT introduce a new design token without design-system approval. The UI SHALL NOT introduce a new component library (one library, governed by this chapter). The UI SHALL NOT use inline styles for visual properties (only for dynamic values that cannot be tokenized). The UI SHALL NOT use color as the sole carrier of information. The UI SHALL NOT block the main thread for more than 50 ms on initial load. The UI SHALL NOT load a web font without a system fallback. The UI SHALL NOT introduce a third-party script without SRI and CSP coverage.

8. Runtime Behaviour

At runtime, pages are served via SSR or ISR. The browser receives the initial HTML, hydrates interactive regions, and loads additional JS bundles on demand. The RUM agent reports Core Web Vitals to the monitoring backend. The bundle-budget CI gate runs Lighthouse on every PR. The accessibility audit runs axe-core on every PR. The pseudolocalization build runs on every PR. Pages that breach the performance budget SHALL block the PR; pages that breach the accessibility audit SHALL block the PR. The RUM dashboard updates in near real time.

9. Failure Conditions

Failure conditions include:

- **Core Web Vitals regression at p75:** LCP > 2.5 s, INP > 200 ms, or CLS > 0.1.
- **Bundle size regression:** a route exceeds its JS or CSS budget.
- **Accessibility audit failure:** axe-core reports violations on a route.
- **RTL layout breakage:** physical CSS property escaped review; layout does not mirror.
- **Missing alt text:** a production image has no alt attribute.
- **Missing accessible name:** an interactive element has no accessible name.
- **Web font loading failure:** FOIT due to missing font-display strategy.
- **Hydration mismatch:** server-rendered HTML differs from client-rendered HTML.
- **Console errors on page load:** uncaught JavaScript errors on initial load.

10. Recovery Procedures

Recovery from a Core Web Vitals regression: identify the regression via RUM; bisect to the offending PR; revert or fix; deploy. Recovery from an accessibility audit failure: the PR is blocked; the engineer SHALL fix the violation; the accessibility reviewer SHALL re-audit. Recovery from an RTL breakage: test in the affected locale; fix the offending physical CSS property; replace with a logical property. Recovery from a bundle regression: identify the added dependency; tree-shake, lazy-load, or remove; re-measure. Recovery from a hydration mismatch: diagnose the cause (often a Date or Math.random in render); fix the cause; never suppress.

11. Security Considerations

The UI SHALL NOT embed secrets in client bundles. The UI SHALL sanitize all server-rendered content to prevent XSS (use framework auto-escaping, never dangerouslySetInnerHTML on untrusted content). The UI SHALL use Subresource Integrity (SRI) for third-party scripts. The UI SHALL use the Content Security Policy (CSP) header. The UI SHALL use the rel='noopener noreferrer' attribute on target='_blank' links. The UI SHALL NOT store tokens in localStorage (use httpOnly cookies). The UI SHALL validate all user input on the server regardless of client-side validation.

12. Performance Considerations

Performance is governed by the budgets in Section 30.10. The UI team SHALL maintain a performance dashboard showing LCP, INP, CLS, TTFB, and bundle sizes over time. The dashboard SHALL alert on regression. Bundle sizes SHALL be reported per-route in the CI pipeline. The performance budget SHALL be reviewed quarterly and tightened as the platform matures. Performance regressions SHALL be treated as bugs, not as acceptable trade-offs; a regression that is consciously accepted SHALL be recorded in an ADR with a remediation plan.

13. Observability Requirements

Every page SHALL be instrumented with RUM (Core Web Vitals, TTFB, route transitions, JavaScript errors). Every interaction SHALL emit a structured analytics event with a stable name, a context object, and a correlation_id propagated from the page session. Every error SHALL be reported to the error-tracking backend with a stack trace, a release tag, and the user's locale. The UI observability stack SHALL be the same as the platform's (Chapter 17); UI-specific metrics (LCP, INP, CLS) SHALL be added to the platform's metric catalog.

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **All 11 components are implemented with spec entries.**
- **All 5 page templates are implemented.**
- **The design tokens are published as a versioned package.**
- **The accessibility audit passes axe-core with zero violations on every page.**
- **The Core Web Vitals budgets are met at p75 in production RUM.**
- **The bundle budgets are enforced in CI.**
- **The message catalogs are externalized.**
- **The pseudolocalization build passes.**
- **The Chief Systems Architect has signed off the UI constitution.**

15. Runtime Verification

Runtime verification is the RUM dashboard, the bundle-budget CI gate, the accessibility CI gate, the pseudolocalization build, and the periodic Lighthouse audit. The RUM dashboard SHALL be live and reporting real-user Core Web Vitals. The CI gates SHALL block on regression. A weekly Lighthouse audit

SHALL be run against the production site and the results SHALL be reviewed by the Performance Engineer. An automated accessibility scan SHALL run nightly against production and SHALL alert on new violations.

16. Certification Requirements

RC1 certification requires: the component library published with all 11 components; the page templates implemented; the design tokens package published; axe-core audit passing on every route; Core Web Vitals budgets met at p75 in production for at least 14 consecutive days; bundle budgets enforced in CI; message catalogs externalized; RTL test suite passing; and the Chief Systems Architect's sign-off that the UI constitution is complete and operationalized.

17. Future Evolution

The UI constitution evolves through Evolution Governance (Chapter 25). New components MAY be added (with a spec entry). New page templates MAY be added. New locales MAY be added without code changes. Design tokens MAY be revised (with a deprecation window for the old token). A component SHALL NOT be removed without a successor or a documented argument. The accessibility target SHALL NOT regress below WCAG 2.2 AA; future revisions MAY raise the target to AAA for specific surfaces. The performance budgets MAY be tightened as the platform matures; they SHALL NOT be loosened without an ADR and a remediation plan.

Chapter 31 — Engineering Mathematics

This chapter codifies the engineering mathematics of Opus Publica: the concrete, measurable, non-negotiable thresholds that bound every operational surface of the platform. Where Chapter 26 defines invariants as mathematical predicates over the platform's state, this chapter defines the numerical envelopes within which those invariants are realized at runtime. The numbers in this chapter are not targets, not goals, and not aspirational ranges — they are CONSTITUTIONAL NUMBERS. A target may be missed without consequence; a constitutional number is a hard limit whose violation is, by definition, a release-blocking defect.

The chapter is organized into ten classes of budget: latency budgets (how long an operation MAY take), throughput budgets (how often an operation MAY be performed), size budgets (how large a resource MAY be), time budgets (when an operation MUST give up), retry and backoff budgets (how stubbornly the platform MAY persist), concurrency budgets (how many operations MAY run in parallel), queue depth budgets (how much work MAY be in flight), storage and retention budgets (how long data MUST persist), reliability budgets (how available each surface MUST be), and resource utilization budgets (how heavily each finite resource MAY be consumed). Each budget has a soft warning threshold (which triggers review) and a hard limit (which triggers refusal). A feature, configuration, or operational change that would violate a hard limit SHALL NOT be released.

The Constitutional Status of These Numbers

These numbers are constitutional. A feature that would violate a hard limit SHALL NOT be released. A soft warning triggers review; a hard limit triggers refusal. Where a numerical threshold and a delivery deadline conflict, the deadline yields. Where a numerical threshold and a feature conflict, the feature is re-specified. The Chief Systems Architect SHALL NOT suspend a hard limit, even temporarily, even under catastrophic operational pressure. The only path to amending a constitutional number is the Evolution Governance process (Chapter 36), and such amendments SHALL require evidence that the new value preserves or strengthens the platform's scholarly-record guarantees.

Every threshold in this chapter SHALL be enforced by a runtime gate. Latency budgets are enforced by request-level timeouts and circuit breakers. Throughput budgets are enforced by rate limiters and token-bucket shapers. Size budgets are enforced by payload validators and storage quotas. Time budgets are enforced by deadline propagation. Retry budgets are enforced by retry libraries with bounded attempt counts and total-time caps. Concurrency budgets are enforced by semaphores and admission controllers. Queue depth budgets are enforced by backpressure and overflow policies. Storage and retention budgets are enforced by lifecycle policies and immutability controls. Reliability budgets are enforced by the SLO framework of Chapter 32. Resource utilization budgets are enforced by autoscalers and admission control. A budget without an enforcing gate is not a budget — it is decoration.

31.1 Latency Budgets

Latency budgets define the maximum permissible response time for each operation the platform performs. Each operation has a p50 (median), p95, p99, and a hard limit. The hard limit is the absolute ceiling beyond which an operation is considered failed regardless of whether it eventually returns; the platform SHALL treat any response that exceeds the hard limit as an error and SHALL fail the request closed (returning an RFC 7807 problem document, per Chapter 9). Latency is measured at the boundary of the platform's responsibility: for inbound API calls, at the API gateway; for outbound calls (Crossref, ORCID, preservation backends), at the egress of the platform's request; for asynchronous operations (PDF generation, JATS generation), from queue-dequeue to completion acknowledgement.

The p50/p95/p99 thresholds are alerting thresholds: sustained breach of p95 for five consecutive minutes triggers a SEV-3 alert; sustained breach of p99 for five consecutive minutes triggers a SEV-2 alert; any single request exceeding the hard limit is recorded as a failed request and counted against the error budget (Chapter 32). Latency budgets are derived from the user-experience principle that a reader SHOULD perceive a page as instantaneous (<200 ms), snappy (<500 ms), responsive (<1 s), or acceptable (<2 s); anything beyond 2 s for an interactive operation is a degraded experience and counts against the reliability SLO of the surface that exposes it.

Operation	p50	p95	p99	Hard Limit
API read	50 ms	200 ms	500 ms	1 000 ms
API write	100 ms	400 ms	800 ms	2 000 ms
DOI resolution	80 ms	200 ms	500 ms	1 000 ms
OAI-PMH ListRecords	500 ms	2 000 ms	5 000 ms	10 000 ms
Crossref deposit (sync ack)	2 000 ms	5 000 ms	10 000 ms	30 000 ms
PDF generation (10-page article)	3 000 ms	8 000 ms	15 000 ms	30 000 ms
PDF generation (50-page article)	10 000 ms	20 000 ms	40 000 ms	90 000 ms
HTML render	200 ms	500 ms	1 000 ms	2 000 ms
JATS generation	1 000 ms	3 000 ms	6 000 ms	15 000 ms
Storage GET	50 ms	150 ms	400 ms	1 000 ms
Storage PUT	200 ms	500 ms	1 500 ms	5 000 ms
DB query (simple)	5 ms	20 ms	50 ms	200 ms
DB query (complex join)	50 ms	200 ms	500 ms	2 000 ms
Cache hit	2 ms	10 ms	20 ms	50 ms

Cache miss	50 ms	200 ms	500 ms	1 000 ms
Search query	100 ms	400 ms	1 000 ms	3 000 ms
Auth token validation	10 ms	30 ms	100 ms	500 ms
Audit log append	5 ms	20 ms	50 ms	200 ms

RULE MATH-001 Every operation listed in the Latency Budgets table SHALL be measured at its platform boundary. A single operation exceeding its Hard Limit SHALL be counted as a failed request against the surface's error budget. Sustained p95 breach for five consecutive minutes SHALL raise a SEV-3 alert; sustained p99 breach SHALL raise a SEV-2 alert.

31.2 Throughput Budgets

Throughput budgets define the maximum permissible rate of each class of operation. Each operation has a maximum instantaneous rate, a burst allowance (the short-term ceiling permitted for sub-second spikes), and a sustained rate (the long-term ceiling permitted over a rolling five-minute window). The burst allowance exists because real traffic is bursty; the sustained rate exists because backends — PostgreSQL, Crossref, preservation services, search indices — have finite capacity that bursts cannot be allowed to exhaust.

Throughput is enforced at the platform boundary by a token-bucket rate limiter whose bucket size equals the burst allowance and whose refill rate equals the sustained rate. Requests that exceed the burst are throttled (HTTP 429 with Retry-After per RFC 9110 §15.5.13); requests that exceed the sustained rate for five consecutive minutes trigger backpressure on the caller (the API gateway SHALL begin rejecting requests with 503 and a Retry-After). Throughput budgets for outbound calls (Crossref deposits, preservation deposits, OAI-PMH harvests) are sized to stay below the external service's documented rate limit with a 20% safety margin; where no documented limit exists, the platform SHALL negotiate one in writing before relying on the integration.

Operation	Max Rate	Burst	Sustained
API requests/sec	5 000/s	10 000/s	3 000/s
Publication events/hour	100/h	200/h	50/h
Crossref deposits/day	10 000/day	1 000/h	5 000/day
Preservation deposits/day	20 000/day	2 000/h	10 000/day
OAI-PMH harvest requests/min	60/min	120/min	30/min
Search queries/sec	100/s	500/s	50/s
Audit log writes/sec	2 000/s	5 000/s	1 000/s

File uploads concurrent	200	500	100

RULE MATH-002 Every operation listed in the Throughput Budgets table SHALL be policed by a token-bucket rate limiter sized to the documented Burst and refilled at the documented Sustained rate. A request that arrives when the bucket is empty SHALL receive HTTP 429 with a Retry-After header computed from the refill rate.

31.3 Size Budgets

Size budgets define the maximum permissible magnitude of every countable resource the platform accepts, stores, or produces. Each resource has a soft warning threshold (the magnitude at which the platform flags the resource for review, because approaching the hard limit indicates either an unusual submission or a trend that may require threshold amendment) and a hard limit (the absolute ceiling beyond which the platform SHALL refuse the resource with an RFC 7807 problem document indicating the limit, the actual size, and the documentation reference). Size budgets protect the platform against denial-of-service via oversized payloads, against resource exhaustion via runaway submissions, and against rendering failures via documents that exceed the rendering pipeline's memory or time budget.

The hard limits in this table are derived from the platform's worst-case rendering, indexing, and storage capacity. A 100 MB upload ceiling reflects the size at which a synchronous upload begins to risk gateway timeouts; a 50 MB article PDF ceiling reflects the size at which PDF generation exceeds its hard limit budget (see § 31.1); a 500-reference ceiling reflects the size at which Crossref deposit metadata exceeds its hard limit. A submission that exceeds any hard limit SHALL be rejected at the boundary; a submission that exceeds a soft warning SHALL be accepted but flagged for editorial review and SHALL be recorded as a soft-warning event in the audit log.

Resource	Limit	Soft Warning	Hard Limit
Max upload file size (per file)	100 MB	80 MB	100 MB
Max submission total size	1 GB	800 MB	1 GB
Max article PDF size	50 MB	40 MB	50 MB
Max JATS XML size	10 MB	8 MB	10 MB
Max HTML render size	5 MB	4 MB	5 MB
Max supplement count	20	15	20
Max author count per article	50	40	50

Max reviewer count per submission	10	8	10
Max reference count per article	500	400	500
Max figure count per article	30	25	30
Max table count per article	30	25	30
Max keyword count	15	12	15
Max abstract word count	500 words	400 words	500 words
Max payload size per API request	10 MB	8 MB	10 MB
Max response size per API request	50 MB	40 MB	50 MB

RULE MATH-003 Every resource listed in the Size Budgets table SHALL be validated at the platform boundary. A resource exceeding a Soft Warning SHALL be accepted but flagged for review and recorded in the audit log. A resource exceeding a Hard Limit SHALL be refused with an RFC 7807 problem document naming the limit, the actual size, and this chapter as the authority.

31.4 Time Budgets

Time budgets define the maximum permissible duration of each operation before the platform SHALL abandon it. Each operation has a default timeout (the value the platform uses for normal traffic) and a hard limit (the absolute ceiling beyond which the operation is abandoned even if it appears to be making progress). Time budgets are derived from the latency budgets of § 31.1 but apply to the operation as a whole rather than to a single request: a PDF generation may take 30 s at p99 (a single render request) but is allotted a 180 s timeout because the platform may legitimately retry, requeue, or hand off to a render farm. Time budgets are propagated as deadlines through the call graph; a downstream call SHALL NOT be issued if its remaining budget is less than its expected p99 latency.

Deadlocks, hung sockets, and infinite loops are bounded by time budgets: any operation that reaches its hard limit without completing SHALL be abandoned, its resources SHALL be released, and the abandonment SHALL be logged with sufficient diagnostic context (correlation id, operation name, inputs, elapsed time) to support postmortem analysis. A timeout that fires is not a successful operation; the platform SHALL treat the timed-out operation as failed for the purposes of error-budget accounting (Chapter 32) and SHALL implement a defined recovery (retry, fail-closed, fail-open, or page-on-call) per the operation's recovery rule.

Operation	Timeout	Hard Limit

API request timeout	30 s	60 s
Crossref deposit timeout	60 s	120 s
Preservation deposit timeout	120 s	300 s
ORCID API timeout	10 s	30 s
OAI-PMH request timeout	60 s	120 s
Render timeout	90 s	180 s
PDF generation timeout	180 s	300 s
Email send timeout	30 s	60 s
File upload timeout	600 s	1 800 s
Session timeout	8 hours	12 hours
Token refresh interval	60 min	24 hours (max)
Audit log write timeout	5 s	10 s
DB transaction timeout	5 s	10 s
DB lock duration (max)	2 s	5 s
Migration step timeout	1 800 s	3 600 s

RULE MATH-004 Every operation listed in the Time Budgets table SHALL be bounded by a deadline propagated through its call graph. A deadline that fires SHALL abandon the operation, release its resources, and emit a timeout event to the audit log with the operation name, correlation id, inputs, and elapsed time. A timed-out operation SHALL NOT be counted as successful for error-budget purposes.

31.5 Retry and Backoff Budgets

Retry and backoff budgets define how stubbornly the platform MAY persist when an operation fails. Each retryable operation has a maximum number of attempts, a backoff strategy (the schedule on which retries are spaced), and a total time budget (the absolute ceiling on the wall-clock time the platform MAY spend retrying before declaring the operation permanently failed). Retry budgets exist because retries, while individually cheap, collectively produce thundering-herd and cascading-failure pathologies; an unbounded retry policy is, in effect, a self-inflicted denial-of-service.

All backoff strategies SHALL be exponential with full jitter (the next delay drawn uniformly from [0, base * 2^attempt]) unless an operation specifically documents otherwise (e.g., OAI-PMH harvest uses linear 30 s steps to align with the harvester's resumptionToken cadence). The total time budget is computed as the sum of the worst-case backoff schedule plus the worst-case per-attempt latency; if the total time

budget is exhausted before the maximum attempt count is reached, the platform SHALL abandon the operation regardless of remaining attempts. Retries SHALL be idempotent: a retried Crossref deposit SHALL carry the same idempotency key as the original; a retried preservation deposit SHALL target the same content hash; a retried email SHALL deduplicate by message-id.

Operation	Max Retries	Backoff Strategy	Total Time Budget
Crossref deposit	5	Exponential, jittered (base=2 s, factor=2)	5 min
Preservation deposit	5	Exponential, jittered (base=5 s, factor=2)	10 min
ORCID writeback	3	Exponential, jittered (base=2 s, factor=2)	60 s
Email send	5	Exponential, jittered (base=10 s, factor=2)	5 min
OAI-PMH harvest (outbound)	3	Linear (30 s steps)	2 min
Storage PUT	3	Exponential, jittered (base=1 s, factor=2)	30 s
External API call	3	Exponential, jittered (base=1 s, factor=2)	30 s

RULE MATH-005 Every retryable operation listed in the Retry and Backoff Budgets table SHALL use full-jitter exponential backoff (except where Linear is explicitly documented) bounded by the documented Max Retries and Total Time Budget. A retry SHALL be idempotent and SHALL carry an idempotency key. An operation that exhausts its Total Time Budget SHALL be declared permanently failed and SHALL be moved to the dead-letter queue (see § 31.7).

31.6 Concurrency Budgets

Concurrency budgets define the maximum number of simultaneous in-flight operations of each class. Concurrency budgets exist because throughput budgets (§ 31.2) bound the rate at which operations arrive, but concurrency budgets bound the number of operations the platform commits to processing simultaneously. An operation that arrives faster than the platform can complete it accumulates as in-flight work; without a concurrency cap, in-flight work grows until the platform exhausts memory, connections, or thread pool and collapses under load-shedding failure rather than graceful degradation.

Each concurrency budget is enforced by a counting semaphore that admits operations up to the documented maximum and rejects (or queues) operations beyond it. Rejected operations receive HTTP 503 with a Retry-After header computed from the expected dequeue time; queued operations are

admitted in FIFO order with a maximum queue wait time equal to the operation's hard limit (§ 31.4). The rationale column documents why each cap takes the value it does; the rationale is normative because changing the cap without changing the rationale invalidates the cap.

Resource	Max Concurrent	Rationale
Concurrent publications	10	Each publication fans out to render, DOI mint, Crossref deposit, and preservation deposit; capping at 10 keeps total downstream load bounded.
Concurrent PDF generations	20	Render farm capacity; each render consumes ~500 MB RAM and one CPU core for the duration of the render.
Concurrent Crossref deposits	5	Stays 20% below Crossref's documented rate limit with margin for retries; exceeding risks Crossref throttling the platform as a whole.
Concurrent preservation deposits	10	Preservation backends (CLOCKSS, LOCKSS, Portico) each impose their own rate limits; 10 concurrent across all backends is the documented safe envelope.
Concurrent DB transactions per service	100	Database connection pool size per service; exceeding causes pool exhaustion and request queueing at the pool, not at the database.
Concurrent file uploads per user	3	Fairness: prevents a single user from monopolising upload bandwidth; additional uploads are queued client-side.
Concurrent OAI-PMH harvests	5	Outbound rate limit; OAI-PMH is request-heavy and the platform's own OAI-PMH endpoint is sized for the same concurrency.

RULE MATH-006 Every resource listed in the Concurrency Budgets table SHALL be guarded by a counting semaphore sized to the documented Max Concurrent. Operations that arrive when the semaphore is full SHALL be rejected with HTTP 503 and a Retry-After header, or SHALL be queued with a maximum queue wait time equal to the operation's hard limit per § 31.4.

31.7 Queue Depth Budgets

Queue depth budgets define the maximum permissible number of outstanding items in each of the platform's work queues. Each queue has a soft warning threshold (the depth at which the platform alerts that the queue is growing faster than its consumers can drain it) and a hard limit (the depth beyond which the platform SHALL invoke the documented overflow action rather than accept the item into the queue). Overflow actions are deliberate, audited trade-offs: shedding non-critical work preserves critical work; backpressure to the writer preserves queue integrity at the cost of writer throughput; blocking writes preserves the queue at the cost of writer availability.

The dead-letter queue (DLQ) has the tightest hard limit because each item in the DLQ represents a permanently-failed operation requiring human attention; a DLQ that grows without bound is, in effect, an unattended failure log. The audit log buffer has the second-tightest hard limit because the audit log is the platform's last line of accountability: if the audit buffer overflows, audit records would be lost, violating invariant INV-A-01 (every state transition audited, Chapter 26); therefore the audit buffer SHALL NOT shed items; instead, it SHALL apply backpressure to its writers, blocking the operation that would have produced the audit record.

Queue	Soft Warning	Hard Limit	Overflow Action
Publication outbox	50	100	Shed non-critical publications; critical publications queue with backpressure.
Crossref deposit queue	100	500	Throttle publisher; new publications queue behind deposits already in flight.
Preservation deposit queue	200	1 000	Throttle publisher; preservation deposits queue but publication is not blocked.
Email queue	1 000	5 000	Drop low-priority emails (digests, notifications); transactional emails always queued.
Notification queue	500	2 000	Coalesce notifications to the same recipient within a 60 s window.
Search index queue	500	2 000	Apply backpressure to writers; search index may lag by up to 5 min.
Audit log buffer	1 000	5 000	Block writes that would produce audit records;

			never shed audit records.
Dead-letter queue	10	50	Page on-call; each DLQ item requires human investigation and remediation.

RULE MATH-007 Every queue listed in the Queue Depth Budgets table SHALL be monitored at soft-warning and hard-limit thresholds. The documented Overflow Action SHALL be automated; it SHALL NOT require human intervention to invoke. The audit log buffer SHALL NEVER shed items; it SHALL apply backpressure to its writers instead. The dead-letter queue SHALL page on-call when its soft warning is breached.

31.8 Storage and Retention Budgets

Storage and retention budgets define how long each class of data SHALL be retained. Retention is a constitutional concern because the platform's scholarly-record guarantees depend on permanence: a published artifact that is deleted is, for citation purposes, an artifact that never existed. Retention budgets therefore encode a deliberate distinction between data that MUST be retained permanently (published artifacts, audit logs) and data that MAY be retained for a defined window (drafts, review files, logs, metrics, traces) and then deleted as a matter of operational hygiene.

Published artifacts and audit logs are governed by invariant INV-V-01 (published artifacts never lost, Chapter 26) and invariant INV-M-02 (audit log append-only). Their retention is therefore not merely a policy but a constitutional guarantee. All other retention budgets are operational: they exist to bound storage cost, to limit the surface area of stale data, and to comply with regulatory and standards guidance. The log retention budget (90 days hot, 1 year cold) follows NIST SP 800-92 §2.4 guidance on log retention for incident investigation; the metrics retention budget (13 months) provides a full year of annual comparatives plus one month of overlap for trend analysis; the trace retention budget (30 days) reflects the operational debugging window beyond which traces are rarely consulted.

Resource	Retention	Rationale
Published artifacts (lifetime)	Permanent	INV-V-01: citation permanence; a cited artifact MUST remain retrievable for the lifetime of the platform.
Audit log (lifetime)	Permanent	INV-M-02 / INV-A-01: every state transition MUST remain auditable for the lifetime of the platform; cryptographic chain MUST remain verifiable.

Submission drafts	5 years	Author recovery window; drafts not published within 5 years are presumed abandoned and MAY be deleted after author notification.
Review files	10 years	Editorial accountability; supports post-hoc investigation of review integrity allegations; aligns with COPE guidance on record retention.
Temporary uploads	24 hours	Working set; uploads not committed to a submission within 24 hours are orphaned and SHALL be deleted by the orphan-reaper job (INV-C-01).
Backups (monthly for 7 years)	7 years (monthly)	Regulatory and disaster-recovery window; monthly full backups retained for 7 years cover the typical statute-of-limitations window.
Logs (90 days hot, 1 year cold)	90 days hot / 1 year cold	NIST SP 800-92 §2.4: 90 days hot for active investigation; 1 year cold for historical correlation.
Metrics (13 months)	13 months	Full year of annual comparatives plus one month of overlap for trend analysis and year-over-year SLO review.
Traces (30 days)	30 days	Operational debugging window; traces beyond 30 days are rarely consulted and impose significant storage cost.

RULE MATH-008 Every resource listed in the Storage and Retention Budgets table SHALL be governed by a lifecycle policy that enforces the documented Retention at the storage layer. Published artifacts and audit logs SHALL be retained permanently; their lifecycle policy SHALL NOT permit deletion under any automated condition. All other resources SHALL be deleted on schedule by a lifecycle job that records each deletion in the audit log.

31.9 Reliability Budgets

Reliability budgets define the availability target for each user-facing or integration-facing surface of the platform. Each surface has an availability target (the fraction of time the surface MUST be operational), an error budget (the time per month the surface MAY be in error while still meeting its target, derived as (1 − target) × minutes-in-month), and an MTTR (mean time to recovery, the maximum acceptable

elapsed time from failure detection to service restoration). Reliability budgets feed directly into the SLO framework of Chapter 32; the SLO catalogue (§ 32.3) operationalizes these budgets as measurable objectives.

Availability targets are tiered: the public website and OAI-PMH endpoint carry the highest target (99.9%) because they are the surfaces by which the scholarly community perceives the platform; the author, reviewer, and editorial surfaces carry 99.5% because they are important but tolerate brief outages without scholarly-record impact; the ORCID integration carries 99.0% because ORCID itself is occasionally unavailable and the platform cannot guarantee more than the upstream service. MTTR targets reflect the operational capacity to detect, diagnose, and recover; the public-website MTTR of 30 minutes presumes an automated rollback pipeline; the ORCID integration MTTR of 4 hours presumes manual investigation of upstream behaviour.

Surface	Availability Target	Error Budget	MTTR
Public website	99.9%	43.2 min/month	30 min
Author portal	99.5%	3.6 hours/month	1 hour
Reviewer portal	99.5%	3.6 hours/month	1 hour
Editorial dashboard	99.5%	3.6 hours/month	1 hour
OAI-PMH endpoint	99.9%	43.2 min/month	30 min
SWORD endpoint	99.5%	3.6 hours/month	1 hour
Crossref deposit (outbound)	99.9%	43.2 min/month	30 min
ORCID integration	99.0%	7.2 hours/month	4 hours
Preservation deposit	99.5%	3.6 hours/month	1 hour
Admin APIs	99.5%	3.6 hours/month	1 hour

RULE MATH-009 Every surface listed in the Reliability Budgets table SHALL have an SLO defined in Chapter 32 whose target equals the documented Availability Target. A surface whose error budget is exhausted SHALL be placed in release freeze per § 32.5. A surface whose MTTR is exceeded SHALL trigger an incident review per the Failure Constitution (Chapter 21).

31.10 Resource Utilization Budgets

Resource utilization budgets define the maximum permissible consumption of each finite compute, storage, and network resource. Each resource has a soft warning threshold (the utilization at which the platform alerts that capacity is becoming constrained and the documented scale action SHOULD be invoked) and a hard limit (the utilization beyond which the platform SHALL invoke the scale action

automatically and SHALL refuse additional load that would push utilization higher). The scale action column documents the automated response the platform takes when the hard limit is breached; it is normative because the threshold is meaningless without a defined response.

Resource utilization budgets are distinct from throughput budgets (§ 31.2): throughput bounds the rate of operations the platform accepts; utilization bounds the resources the platform consumes to service them. A platform can be within throughput budget and over utilization budget if operations are individually expensive; conversely, a platform can be over throughput budget and within utilization budget if operations are individually cheap. Both budgets MUST hold simultaneously. The hard limits in this table are derived from the operational principle that a system running above 80% utilization exhibits non-linear latency tail growth; the soft warnings at 60–70% provide the headroom for the scale action to complete before utilization reaches the hard limit.

Resource	Soft Warning	Hard Limit	Scale Action
CPU per service	60%	80%	Auto-scale: add instances up to service max; if max reached, throttle non-critical traffic.
Memory per service	70%	85%	Auto-scale: add instances up to service max; if max reached, refuse new requests with 503.
DB connections per service	60%	80%	Grow pool up to service max; if max reached, queue requests at pool with timeout.
Storage IOPS	70%	90%	Throttle non-critical I/O (audit log compaction, search index rebuild); preserve publication-critical I/O.
Network egress per day	70%	90%	Throttle non-critical egress (OAI-PMH harvest, large file downloads); preserve reader-critical egress.
External API quota per day	70%	90%	Backpressure to writers: queue outbound calls; preserve in-flight calls until completion.

RULE MATH-010 Every resource listed in the Resource Utilization Budgets table SHALL be

monitored at soft-warning and hard-limit thresholds. The documented Scale Action SHALL be automated and SHALL execute within 60 seconds of the hard limit being breached. A scale action that fails to relieve utilization SHALL itself raise a SEV-2 alert and SHALL trigger the documented backpressure or shedding behaviour.

31.11 Master Constitutional Number Index

The following table is the authoritative index of every constitutional number in this chapter. Each row identifies a class of budget, the count of individual thresholds within that class, the rule identifier that enforces the class, and the owning role accountable for monitoring the class. The Engineering Leadership Team SHALL review this index weekly; the Release Certification Authority SHALL verify that every rule (MATH-001 through MATH-010) has a live enforcement gate before RC1 sign-off.

Class	Thresholds	Rule	Owner
31.1 Latency Budgets	18 operations × 4 thresholds = 72	MATH-001	Head of Platform Engineering
31.2 Throughput Budgets	8 operations × 3 thresholds = 24	MATH-002	Head of Platform Engineering
31.3 Size Budgets	15 resources × 2 thresholds = 30	MATH-003	Head of Data Engineering
31.4 Time Budgets	15 operations × 2 thresholds = 30	MATH-004	Head of Platform Engineering
31.5 Retry and Backoff Budgets	7 operations × 3 thresholds = 21	MATH-005	Head of Platform Engineering
31.6 Concurrency Budgets	7 resources × 1 threshold = 7	MATH-006	Head of Platform Engineering
31.7 Queue Depth Budgets	8 queues × 3 thresholds = 24	MATH-007	Head of Observability
31.8 Storage and Retention Budgets	9 resources × 1 threshold = 9	MATH-008	Head of Data Engineering
31.9 Reliability Budgets	10 surfaces × 3 thresholds = 30	MATH-009	Head of Observability
31.10 Resource Utilization Budgets	6 resources × 3 thresholds = 18	MATH-010	Head of Platform Engineering
TOTAL	103 thresholds across 10 rules	MATH-001 — MATH-010	Office of the Chief Systems Architect

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

This chapter codifies the engineering mathematics of Opus Publica: the concrete, measurable, non-negotiable thresholds that bound every operational surface of the platform. The purpose is to convert the platform's qualitative guarantees — responsiveness, throughput, durability, reliability — into specific numbers that gate release. A feature is not done when its tests pass; a feature is done when its tests pass AND its documented thresholds are not violated at runtime under representative load. This chapter is the authoritative reference for those thresholds.

2. Scope

In scope: latency budgets for all platform-performed operations; throughput budgets for all platform-accepted operation classes; size budgets for all platform-accepted and platform-produced resources; time budgets for all platform-executed operations; retry and backoff budgets for all retryable operations; concurrency budgets for all bounded-concurrency resources; queue depth budgets for all platform queues; storage and retention budgets for all platform data classes; reliability budgets for all user-facing and integration-facing surfaces; resource utilization budgets for all finite compute, storage, and network resources. Out of scope: the SLO framework that operationalizes reliability budgets (Chapter 32); the database-specific thresholds (Chapter 33); the security-specific thresholds (Chapter 19).

3. Definitions

Constitutional number: a numerical threshold in this chapter whose violation is a release-blocking defect. Hard limit: the absolute ceiling beyond which an operation or resource is refused, abandoned, or counted as failed. Soft warning: the threshold at which the platform alerts that a resource is approaching its hard limit. Budget: the quantitative envelope within which an operation or resource is permitted, comprising soft-warning and hard-limit thresholds and a defined enforcement gate. Error budget: the time per month a surface MAY be in error while still meeting its availability target; operationalized in Chapter 32.

4. Responsibilities

Ownership of the constitutional numbers is distributed across the engineering organization:

- **Chief Systems Architect:** Accountable for the threshold catalogue and for refusing to suspend any hard limit, even temporarily.
- **Head of Platform Engineering:** Responsible for latency, throughput, time, retry, concurrency, and resource utilization budgets.
- **Head of Data Engineering:** Responsible for size and storage/retention budgets.

- **Head of Observability:** Responsible for queue depth and reliability budgets, and for the monitoring infrastructure that detects threshold breaches.
- **Release Certification Authority:** Responsible for verifying that every rule MATH-001 through MATH-010 has a live enforcement gate before RC1 sign-off.

5. Design Principles

Constitutional numbers are concrete (stated as specific values, not ranges), measurable (each has a runtime monitor), enforced (each has a gate that blocks violation), and unsuspendable (no operational pressure justifies violation). A number that cannot be measured is not a constitutional number; it is an aspiration. A number that can be measured but is not enforced is not a constitutional number; it is a target. The discipline of this chapter is the discipline of converting aspirations into numbers and numbers into gates.

6. Engineering Requirements

Every threshold in this chapter SHALL have a stable identifier (the rule MATH-NNN that enforces it), a defined unit (ms, MB, /s, %, count), a defined enforcement gate (rate limiter, semaphore, validator, lifecycle policy, monitor), a named owner, and a documented response when the threshold is breached. The Engineering Mathematics dashboard SHALL display the current value, soft-warning status, and hard-limit status of every threshold in real time. The Engineering Leadership Team SHALL review the dashboard daily. Any hard-limit breach SHALL trigger an immediate incident response and SHALL be reported to the Chief Systems Architect within one hour.

7. Engineering Constraints

Constitutional numbers constrain every other chapter of the Constitution. No feature MAY be implemented that would violate a hard limit. No optimization MAY be accepted that weakens a threshold's enforcement. No operational procedure MAY be defined that suspends a hard limit. The Chief Systems Architect SHALL NOT grant exceptions to hard limits; the only path to changing a constitutional number is to amend this chapter through the Evolution Governance process (Chapter 36), and such amendments SHALL require evidence that the new value preserves or strengthens the platform's scholarly-record guarantees. A proposed amendment that would weaken a hard limit SHALL require a recorded engineering review and SHALL be signed off by the Chief Systems Architect personally.

8. Runtime Behaviour

At runtime, every enforcement gate operates continuously. Rate limiters police throughput budgets per request. Semaphores police concurrency budgets per operation. Validators police size budgets per payload. Deadline propagation polices time budgets per call graph. Retry libraries with bounded attempts police retry budgets per failed operation. Queue monitors police queue depth budgets per queue. Lifecycle policies police storage and retention budgets per data class. SLO monitors (Chapter 32)

police reliability budgets per surface. Autoscalers police resource utilization budgets per resource. A gate that itself fails to operate is a SEV-2 incident (a blind spot) and SHALL be detected by a meta-monitor.

9. Failure Conditions

Constitutional-number failure conditions include:

- **Gate failure:** an enforcement gate that should have blocked a violation did not (e.g., a rate limiter misconfigured, a validator bypassed).
- **Monitor failure:** a monitor that should have detected a breach did not run or did not alert.
- **Cascading breach:** a breach of one budget caused breaches of others (e.g., throughput breach → queue depth breach → latency breach).
- **Scale-action failure:** an autoscale action that should have relieved utilization did not execute or did not relieve utilization.
- **Threshold drift:** a threshold that no longer reflects the platform's actual capacity (e.g., a throughput budget set when the database was smaller and no longer safe at current scale).

10. Recovery Procedures

Threshold-breach recovery follows the incident-response regime (Chapter 21) with the additional requirement that recovery SHALL restore the budget, not merely mask the symptom. Restoring a budget may require scaling (adding capacity), shedding load (refusing non-critical traffic), backpressure (slowing writers), or, in extreme cases, rollback to a known-good configuration. The recovery SHALL be recorded in the audit log with a full before/after analysis including the breached threshold, the duration of the breach, the recovery action, and the root-cause analysis.

11. Security Considerations

Constitutional numbers are the platform's first line of defence against denial-of-service. Size budgets prevent oversized-payload attacks; throughput budgets prevent request-flood attacks; concurrency budgets prevent resource-exhaustion attacks; queue depth budgets prevent queue-overflow attacks. An attacker who can exhaust a budget can degrade the platform's availability, and the reliability budgets (§ 31.9) are the SLO against which such degradation is measured. The enforcement gates SHALL themselves be defended against bypass (the gates run as separate services with separate credentials, and their configurations are version-controlled and audited).

12. Performance Considerations

Enforcement gates have a performance cost: every rate limiter check, every semaphore acquire, every validator invocation, every deadline propagation step adds latency to the request path. The performance budget for enforcement gates SHALL NOT exceed 5 ms p99 per request. Gates that exceed their budget SHALL be optimized (e.g., rate limiter moved from a per-request database lookup to an in-process token bucket with periodic centralized sync), not skipped. The cost of enforcement is the cost of guaranteeing the numbers in this chapter; it is non-negotiable.

13. Observability Requirements

Every enforcement gate SHALL expose metrics for: current value, soft-warning status (boolean), hard-limit status (boolean), gate-rejection count, gate-rejection rate, and gate-operation latency. The Engineering Mathematics dashboard SHALL aggregate these into a single view organized by the ten budget classes. A gate in “unknown” state (no metric received within its cadence interval) is a SEV-2 incident (the platform is blind to that budget). The dashboard SHALL be reviewed by the Engineering Leadership Team daily and by the on-call engineer continuously.

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **All 103 thresholds across the 10 budget classes are documented with stable rule identifiers (MATH-001 through MATH-010).**
- **Every threshold has a live enforcement gate exercising the documented behavior (rate limiter, semaphore, validator, etc.).**
- **The Engineering Mathematics dashboard is live and reporting all 103 thresholds in real time.**
- **At least one threshold breach per budget class has been exercised in a chaos exercise (breach injected, detected, alerted, recovered).**
- **The meta-monitor is operational and detects gate failures.**
- **The Chief Systems Architect has signed off the threshold catalogue.**

15. Runtime Verification

Runtime verification of the constitutional numbers is the enforcement-gate framework itself: the gates are the verification, the dashboard is the evidence, and the audit records of gate operations are the certification artifacts. The Release Certification Authority SHALL verify that all 10 rules (MATH-001 through MATH-010) have live gates, that all 103 thresholds are reporting to the dashboard, and that at least one chaos exercise per budget class has been completed and postmortemed before RC1 sign-off.

16. Certification Requirements

RC1 certification requires: the master constitutional number table published; all 10 rules operational; the Engineering Mathematics dashboard live; at least one chaos exercise per budget class completed and postmortemed (10 exercises total); the meta-monitor operational; and the Chief Systems Architect's sign-off that the threshold set is complete (no known budget is missing thresholds). A threshold that lacks an enforcement gate is not certifiable; it is a defect that blocks RC1.

17. Future Evolution

The constitutional number set evolves through the Evolution Governance process (Chapter 36). New thresholds MAY be added when a new class of operation or resource is introduced. Existing thresholds MAY be tightened (made stricter) but SHALL NOT be loosened without evidence that the looser value still preserves the platform's scholarly-record guarantees. A threshold SHALL NOT be removed; if a threshold is genuinely obsolete, it SHALL be marked deprecated with a recorded reason and a successor threshold

(or a documented argument that no successor is needed). The amendment history SHALL be maintained as part of this chapter's revision record.

Chapter 32 — Production SLO Book

This chapter is the production Service Level Objective (SLO) book for Opus Publica. Where Chapter 31 defines reliability budgets (the availability target, error budget, and MTTR for each surface), this chapter defines the framework, indicators, objectives, agreements, policies, alerting, escalation, mitigation, runbooks, and review cadence that operationalize those budgets. A reliability budget without an SLO is an aspiration; an SLO without an error-budget policy is a number on a dashboard; an error-budget policy without burn-rate alerting is a postmortem waiting to happen. This chapter ties them together into a single, closed-loop, Google-SRE-workbook-aligned regime.

The chapter draws its methodology from the Google SRE Workbook (Beyer et al., 2018, especially Chapter 2 on implementing SLOs and Chapter 5 on alerting on SLOs) and from the Site Reliability Engineering book (Beyer et al., 2016, Chapter 6 on monitoring distributed systems). The methodology is applied, not merely cited: every SLI in this chapter has a defined formula, source, and aggregation window; every SLO has a target, window, and consequence of breach; every error budget has a state machine with defined actions; every burn rate has a multi-window alert; every breach has an escalation path and a mitigation playbook.

The chapter is organized into ten sections: the SLO framework (§ 32.1, definitions and methodology); the SLI catalogue (§ 32.2, the indicators); the SLO catalogue (§ 32.3, the objectives); the SLA catalogue (§ 32.4, the external agreements); the error budget policy (§ 32.5, what happens when budgets are consumed); burn rate alerting (§ 32.6, how breaches are detected early); the escalation matrix (§ 32.7, who is paged when); the mitigation playbook (§ 32.8, what to do when an SLO is breached); the runbook catalogue (§ 32.9, the per-SLO operational procedures); and the SLO review cadence (§ 32.10, how the SLO set itself evolves).

The Constitutional Status of SLOs

SLOs are the bridge between engineering numbers (Chapter 31) and engineering behaviour. An SLO that is breached is not merely a metric that turned red; it is a contract with the scholarly community that the platform is failing to honour. The error-budget policy (§ 32.5) defines what the platform SHALL do when that contract is at risk: feature velocity SHALL yield to reliability work; new features SHALL be frozen; in the extreme case, releases SHALL be frozen. The Chief Systems Architect SHALL NOT override an exhausted error budget; the only path to restoring feature velocity is to restore the SLO.

32.1 SLO Framework

The SLO framework used by Opus Publica follows the Google SRE Workbook (Beyer, Jones, Petoff, Murphy, eds., O'Reilly, 2018). The framework rests on five concepts: the Service Level Indicator (SLI), the Service Level Objective (SLO), the Service Level Agreement (SLA), the error budget, and the burn rate.

Each concept has a precise definition; the definitions in this section are normative for the rest of the chapter.

32.1.1 Service Level Indicator (SLI)

An SLI is a quantitative measure of the service's behaviour. It is a function of two things: a numerator (the count of events considered successful) and a denominator (the count of events considered valid). Formally, SLI = (count of good events) / (count of valid events), over a defined aggregation window. The SLI catalogue (§ 32.2) lists every SLI the platform defines, with its formula, source, and window.

32.1.2 Service Level Objective (SLO)

An SLO is a target value for an SLI, over an aggregation window. For example, an SLO might state that availability (the SLI) SHALL be at least 99.9% (the target) over a rolling 28-day window. SLOs are internally binding: they are the platform's contract with itself about how reliable it commits to be. The SLO catalogue (§ 32.3) lists every SLO the platform commits to, with its target, window, and consequence of breach.

32.1.3 Service Level Agreement (SLA)

An SLA is an external promise to consumers about the service's reliability, with consequences (typically financial or service-credit) if the promise is broken. SLAs are always looser than SLOs: the platform SHALL commit internally to 99.9% availability (the SLO) while promising externally 99.5% (the SLA), so that the platform has an error-budget buffer between its internal target and its external promise. The SLA catalogue (§ 32.4) lists the SLAs the platform exposes to external consumers (OAI-PMH harvesters, Crossref, DOI resolvers).

32.1.4 Error Budget

The error budget is the budget for unreliability implied by the SLO. If the SLO is 99.9% availability over 30 days, the error budget is (1 − 0.999) × 30 × 24 × 60 = 43.2 minutes of permitted unavailability per 30-day window. The error budget is the resource the platform spends when it releases new features (every release carries a risk of regression) and when it operates the platform (every incident consumes budget). The error-budget policy (§ 32.5) defines what happens when the budget is consumed at an unsustainable rate.

32.1.5 Burn Rate

The burn rate is the rate at which the error budget is being consumed relative to the sustainable rate. A burn rate of 1.0 means the budget is being consumed at exactly the rate that would exhaust it over the SLO window (the platform is exactly meeting its SLO). A burn rate of 2.0 means the budget is being consumed twice as fast as sustainable (the platform will exhaust its budget in half the SLO window). A burn rate of 14.4 means the budget is being consumed 14.4 times as fast as sustainable, which corresponds to exhausting the 30-day budget in approximately 2 hours — a fast-burn scenario that

requires immediate intervention. Burn-rate alerting (§ 32.6) uses multi-window, multi-burn-rate thresholds to detect both fast-burn and slow-burn breaches early enough to intervene.

32.2 SLI Catalogue

The SLI catalogue is the authoritative list of every Service Level Indicator the platform defines. Each SLI has a stable identifier (SLI-NNN), a name, a formula (the numerator and denominator), a source (the system that produces the measurements), and an aggregation window (the period over which the ratio is computed). SLIs are the raw material of SLOs; an SLO is a target value for an SLI.

ID	SLI Name	Formula (numerator / denominator)	Source	Window
SLI-001	Availability	(successful requests) / (total valid requests)	API gateway access logs	5 min rolling
SLI-002	Latency	(requests under threshold) / (total valid requests)	API gateway latency histogram	5 min rolling
SLI-003	Freshness	(records fresher than threshold) / (total records expected fresh)	Search index metadata	5 min rolling
SLI-004	Completeness	(expected records present) / (total expected records)	Reconciliation job output	1 hour rolling
SLI-005	Correctness	(invariant monitors passing) / (total invariant monitors)	Invariant monitor dashboard	5 min rolling
SLI-006	Durability	(artifacts retrievable by hash) / (total published artifacts)	Storage HEAD probe	1 hour rolling
SLI-007	Publication success rate	(publications completed without manual intervention) / (total publications attempted)	Publication state machine events	24 hour rolling
SLI-008	Crossref deposit success rate	(deposits reconciled within 24h) / (total deposits made)	Crossref reconciliation job	24 hour rolling
SLI-009	Preservation deposit coverage	(published artifacts with confirmed preservation deposit) / (total published artifacts)	Preservation reconciliation job	24 hour rolling

SLI-010	Audit log completeness	(state transitions with audit records) / (total state transitions)	Audit completeness monitor	1 hour rolling

Each SLI in this catalogue is the foundation of one or more SLOs in § 32.3. The aggregation windows are deliberately short (5 minutes to 24 hours) to support timely detection: a 5-minute window detects fast-burn breaches; a 24-hour window detects slow-burn breaches. The source column identifies the system that produces the measurements; the platform SHALL treat an SLI whose source is unavailable as unknown rather than zero (unknown is honest; zero is misleading).

32.3 SLO Catalogue

The SLO catalogue is the authoritative list of every Service Level Objective the platform commits to. Each SLO references an SLI from § 32.2, sets a target value, defines the window over which the target is evaluated, and documents the consequence of breach. SLO targets are derived from the reliability budgets in § 31.9; the SLO catalogue operationalizes those budgets as measurable objectives.

SLI	SLO Target	Window	Consequence of Breach
SLI-001 Availability	≥ 99.9%	28 days rolling	Feature freeze for non-reliability work; release freeze if budget exhausted.
SLI-002 Latency (p95 < 500 ms)	≥ 99.0%	28 days rolling	Latency optimization sprint; review of slow-query and slow-render logs.
SLI-003 Freshness (< 5 min lag)	≥ 99.0%	28 days rolling	Search index queue audit; backpressure tuning review.
SLI-004 Completeness	100%	1 hour rolling	Reconciliation job investigation; SEV-2 if zero-completeness persists > 1 hour.
SLI-005 Correctness	100%	5 min rolling	SEV-1 page: an invariant monitor is failing, indicating potential scholarly-record corruption (Chapter 26).
SLI-006 Durability	100%	1 hour rolling	SEV-1 page: a published artifact cannot be retrieved, indicating potential data loss (INV-V-01).
SLI-007 Publication success	≥ 95%	7 days rolling	Publication pipeline review; investigate manual-intervention cases.
SLI-008 Crossref deposit	≥ 99%	7 days rolling	Crossref deposit queue

success			audit; investigate reconciliation failures.
SLI-009 Preservation coverage	100%	24 hours rolling	Preservation deposit queue audit; SEV-2 if coverage < 100% for > 24 hours (INV-L-04).
SLI-010 Audit completeness	100%	1 hour rolling	SEV-1 page: a state transition lacks an audit record (INV-A-01).

RULE SLO-001 Every SLO in the SLO Catalogue SHALL be measured continuously by its underlying SLI. The current SLO status (pass / fail / unknown) SHALL be displayed on the SLO dashboard in real time. A consequence of breach SHALL be invoked automatically when an SLO is breached; the consequence SHALL NOT require human intervention to invoke (though human intervention MAY be required to resolve the underlying cause).

32.4 SLA Catalogue

The SLA catalogue is the authoritative list of every Service Level Agreement the platform exposes to external consumers. SLAs are looser than SLOs by design: the platform commits internally to its SLO targets and exposes only a subset of them, with looser targets, to external consumers. The gap between SLO and SLA is the buffer that protects the platform from SLA breach when the platform is operating near its SLO limit. The SLA catalogue is small (three SLAs) because the platform exposes few formal external agreements; the OAI-PMH endpoint, the Crossref deposit integration, and the DOI resolution surface are the three surfaces for which formal external expectations exist.

SLA Surface	SLA Target	Internal SLO	Breach Consequence
OAI-PMH endpoint availability	≥ 99.5% per calendar month	99.9% (SLI-001)	Public notification on status page; harvesters advised to back off; root-cause postmortem published within 7 days.
Crossref deposit success (sync ack)	≥ 95% of deposits acknowledged within 1 hour	99% reconciled within 24h (SLI-008)	Direct notification to Crossref support; affected DOIs reported; remediation plan filed with Crossref.
DOI resolution (doi.org → landing page)	≥ 99.5% per calendar month	99.9% (SLI-001 for /articles/{doi})	Public notification; status page updated; affected DOIs listed; root-cause postmortem published

			within 7 days.

SLAs are reviewed quarterly by the Engineering Leadership Team. An SLA that is breached triggers a customer-facing incident response: the status page is updated, affected consumers are notified by email, and a postmortem is published within 7 days. SLAs are NOT amendable upward (tighter) without a 90-day notice period to allow consumers to adjust; SLAs MAY be amended downward (looser) only by the Evolution Governance process (Chapter 36) and only with documented evidence that the looser value still meets the legitimate expectations of consumers.

32.5 Error Budget Policy

The error budget policy defines what the platform SHALL do when error budgets are consumed at an unsustainable rate. The policy operates on the principle that the error budget is a finite resource that must be allocated deliberately: some budget is spent on feature releases (every release carries a regression risk), some on operational incidents (every incident consumes budget), and some on planned maintenance. When the budget is consumed faster than the SLO window allows, the policy SHALL reduce the rate of consumption by restricting the activities that spend budget — primarily, by restricting feature releases.

The policy is a four-state machine. The state is determined by the current burn rate over the SLO window and by the remaining budget. Transitions between states are automatic: the SLO dashboard computes the state continuously and invokes the documented action when the state changes. The state is per-SLO; one SLO in 'critical' does not necessarily mean all SLOs are in 'critical', but a release that would affect the critical SLO SHALL be frozen regardless of other SLOs' states.

Budget State	Burn Rate	Remaining Budget	Action
Healthy	< 2× burn	> 50% remaining	Feature velocity continues; releases proceed normally; no restrictions.
At-risk	2–4× burn	25–50% remaining	Feature freeze for non-reliability work; reliability work prioritised; releases require Engineering Manager approval.
Critical	> 4× burn	10–25% remaining	All-hands on reliability; only reliability fixes and revert/rollback changes permitted; release freeze for everything else.
Exhausted	any	< 10% remaining	Release freeze; only SEV-1 incident response and revert/rollback permitted;

			Chief Systems Architect notified; postmortem required before resuming feature releases.

RULE SLO-002 The error budget state machine SHALL be evaluated continuously for every SLO. The state transition SHALL be automated: when burn rate or remaining budget crosses a documented threshold, the documented action SHALL be invoked (feature freeze, release freeze, page on-call). The action SHALL NOT require human intervention to invoke; human intervention is required only to RESUME normal velocity once the budget recovers.

32.6 Burn Rate Alerting

Burn rate alerting is the early-warning system that detects SLO breaches before the budget is exhausted. The platform follows the Google SRE Workbook multi-window, multi-burn-rate pattern (Beyer et al., 2018, Chapter 5): each SLO has four alert thresholds, each defined by a long-window burn rate (to detect sustained breaches) AND a short-window burn rate (to confirm the breach is ongoing, avoiding false alarms from transient spikes). An alert fires only when BOTH windows exceed their thresholds simultaneously.

The four thresholds are calibrated to detect breaches at different speeds: a 1-hour window at 14.4× burn detects a breach that would exhaust the 30-day budget in 2 hours (fast-burn, immediate page); a 6-hour window at 6× burn detects a breach that would exhaust the budget in 5 hours (fast-burn, ticket); a 1-day window at 3× burn detects a breach that would exhaust the budget in 10 days (slow-burn, ticket); a 3-day window at 1× burn detects a breach that would exhaust the budget over the full window (slow-burn, ticket). The multi-window design ensures that a brief spike (e.g., a 5-minute outage) does not fire the 14.4× alert unless the 5% short-window also confirms sustained burn.

Alert Tier	Long Window	Short Window	Burn Rate	Action
Tier 1 (fast page)	1 hour	5 minutes	14.4×	Immediate page on-call; SEV-1 incident response.
Tier 2 (fast ticket)	6 hours	30 minutes	6×	Ticket to owning team; SEV-2 if not acknowledged within 1 hour.
Tier 3 (slow ticket)	1 day	2 hours	3×	Ticket to owning team; review at next daily standup.
Tier 4 (slow ticket)	3 days	6 hours	1×	Ticket to owning

				team; review at next weekly SLO review.

RULE SLO-003 Every SLO SHALL be monitored by the four-tier multi-window burn-rate alerting pattern. An alert SHALL fire only when BOTH the long window AND the short window exceed their thresholds simultaneously. Tier 1 alerts SHALL page on-call immediately; Tier 2–4 alerts SHALL create tickets with the documented response expectations. Alert noise SHALL be reviewed at each weekly SLO review; alerts that fire without action SHALL be tuned.

32.7 Escalation Matrix

The escalation matrix defines who is paged, when, and with what response expectation, for each combination of SLO severity and burn rate. The matrix is normative: an on-call engineer who receives a Tier 1 page SHALL acknowledge within the documented response time, and the platform SHALL automatically escalate if the acknowledgment does not occur. Escalation paths are designed to ensure that no incident goes unattended and that incidents requiring broader organizational attention (Chief Systems Architect, Engineering Manager) reach the right person without manual routing.

Severity	Burn Rate / State	Escalation Path	Response Time
SEV-1	Tier 1 (14.4×, 1h/5min) OR SLO state = Critical	On-call engineer → backup on-call → Engineering Manager → Chief Systems Architect.	Acknowledge ≤ 5 min; mitigating action ≤ 15 min.
SEV-2	Tier 2 (6×, 6h/30min) OR SLO state = At-risk deteriorating	On-call engineer → backup on-call → Engineering Manager.	Acknowledge ≤ 15 min; mitigating action ≤ 1 hour.
SEV-3	Tier 3 (3×, 1d/2h) OR SLO state = At-risk stable	Owning team ticket; reviewed at daily standup.	Acknowledge ≤ 4 hours; mitigating action ≤ 1 business day.
SEV-4	Tier 4 (1×, 3d/6h) OR SLO state = Healthy but trending	Owning team ticket; reviewed at weekly SLO review.	Acknowledge ≤ 1 business day; mitigating action ≤ 1 sprint.
SLA breach	External SLA target breached	On-call engineer → Engineering Manager → Chief Systems Architect → public status page update.	Acknowledge ≤ 15 min; status page ≤ 30 min; postmortem ≤ 7 days.

Escalation paths assume the on-call rotation documented in the Engineering Playbook (Chapter 28). Each step of the escalation SHALL have a defined wait time before auto-escalation: SEV-1 auto-escalates from

on-call to backup on-call after 5 minutes without acknowledgment, and from backup on-call to Engineering Manager after a further 10 minutes. Auto-escalation SHALL NOT be disabled without Engineering Manager approval, and such approval SHALL be recorded in the audit log.

32.8 Mitigation Playbook

The mitigation playbook defines the available mitigations when an SLO is breached. The mitigations are organized from least-invasive to most-invasive: rollback (revert the most recent change), shed load (refuse non-critical traffic), throttle (slow traffic to a sustainable rate), fail-open (return cached or degraded responses for non-critical operations), fail-closed (refuse requests for critical operations rather than risk corruption). Each SLO breach type has a defined preferred mitigation; the on-call engineer MAY choose a more invasive mitigation if the preferred mitigation is insufficient.

Breach Type	Preferred Mitigation	Fallback Mitigation
Availability regression after release	Rollback to previous deployment.	Shed load on non-critical endpoints; preserve publication-critical endpoints.
Latency regression	Shed load on the slow endpoint; identify and throttle the slow operation.	Rollback if shedding is insufficient.
Freshness regression (search index lag)	Throttle writers; preserve search index queue throughput.	Shed search queries; advise harvesters to back off.
Completeness regression (reconciliation gap)	Re-run reconciliation job; investigate missing records.	Fail-closed on operations that depend on the missing records.
Correctness regression (invariant failing)	Fail-closed on the affected operation (SEV-1 per INV-A-01).	Page Chief Systems Architect; halt publication until invariant restored.
Durability regression (artifact not retrievable)	Fail-closed on artifact retrieval; restore from backup or preservation deposit.	Page Chief Systems Architect; SEV-1 incident response.
Publication success regression	Throttle publications; investigate failed publications.	Fail-closed on new publications; queue submissions.
Crossref deposit success regression	Throttle Crossref deposit queue; investigate failures.	Backpressure to publisher; publications may queue but are not blocked.
Preservation coverage regression	Throttle preservation deposit queue; investigate failures.	Backpressure to publisher; SEV-2 if coverage < 100% for > 24h.
Audit completeness regression	Fail-closed on all state transitions (SEV-1 per INV-A-01).	Page Chief Systems Architect; halt all mutations until audit restored.

Fail-open is permitted only for non-critical operations where a degraded response is preferable to no response (e.g., serving a stale search index while the index rebuilds). Fail-open SHALL NEVER be used for operations whose correctness is a constitutional invariant (correctness, durability, audit completeness): for these, the platform SHALL fail-closed and SHALL halt the affected operation rather than risk corruption. The choice between fail-open and fail-closed is normative and SHALL be documented per-operation in the runbook catalogue (§ 32.9).

32.9 Runbook Catalogue

The runbook catalogue lists the operational runbooks the platform maintains, one per SLO. Each runbook is a structured document with: the SLO it supports; the symptoms the runbook addresses; the diagnostic steps to identify the root cause; the mitigation steps to restore the SLO; the verification steps to confirm restoration; the postmortem steps to record the incident. Runbooks are stored in the Engineering Playbook repository (Chapter 28) and are version-controlled alongside the platform code.

Runbook ID	Supports SLO	Trigger	Owner
RB-SLO-001	SLI-001 Availability	Tier 1–2 burn-rate alert on availability.	Head of Platform Engineering
RB-SLO-002	SLI-002 Latency	Tier 1–2 burn-rate alert on latency.	Head of Platform Engineering
RB-SLO-003	SLI-003 Freshness	Search index lag exceeds 5 min for > 5 min.	Head of Data Engineering
RB-SLO-004	SLI-004 Completeness	Reconciliation job reports completeness < 100%.	Head of Data Engineering
RB-SLO-005	SLI-005 Correctness	Any invariant monitor fails (SEV-1).	Chief Systems Architect
RB-SLO-006	SLI-006 Durability	Storage HEAD probe fails for any published artifact (SEV-1).	Chief Systems Architect
RB-SLO-007	SLI-007 Publication success	Publication success rate < 95% over 24h.	Head of Platform Engineering
RB-SLO-008	SLI-008 Crossref deposit success	Crossref deposit success rate < 99% over 7 days.	Head of Platform Engineering
RB-SLO-009	SLI-009 Preservation coverage	Preservation coverage < 100% over 24h.	Head of Data Engineering
RB-SLO-010	SLI-010 Audit completeness	Any state transition lacks an audit record (SEV-1).	Chief Systems Architect

Every runbook SHALL be reviewed quarterly and SHALL be exercised at least annually in a chaos exercise (Chapter 22). A runbook whose exercise fails (the runbook did not lead to successful restoration) SHALL

be revised within 1 sprint and re-exercised. Runbook currency is a certification requirement: the Release Certification Authority SHALL verify that every runbook has been exercised within the prior 12 months before RC1 sign-off.

Runbook structure (every RB-SLO-NNN SHALL contain):
1. SLO Reference: the SLI and SLO this runbook supports.
2. Symptoms: the observable signals that trigger this runbook.
3. Diagnostics: the queries, dashboards, and logs to consult.
4. Mitigation: the steps to restore the SLO (preferred and fallback).
5. Verification: the checks to confirm the SLO is restored.
6. Postmortem: the template for recording the incident.
7. Escalation: who to page if mitigation fails.
8. History: the date of last exercise and exercise outcome.

32.10 SLO Review Cadence

The SLO review cadence defines how the SLO set itself is reviewed and amended. SLOs are not static: as the platform evolves, as user expectations shift, and as operational capacity grows, SLOs MAY be tightened (more ambitious targets) or, rarely, loosened (more conservative targets). The review cadence ensures that the SLO set remains calibrated to reality: an SLO that is never breached is too loose (it leaves error budget on the table that could be spent on feature velocity); an SLO that is constantly breached is too tight (it consumes engineering attention without delivering user value). The Google SRE Workbook recommends a quarterly cadence; Opus Publica adopts that cadence with weekly and monthly checkpoints.

Cadence	Forum	Attendees	Agenda
Weekly burn review	SLO standup	On-call engineers; Engineering Manager.	Review each SLO's current burn rate, state, and any active incidents; confirm mitigations are in progress; identify SLOs trending toward breach.
Monthly SLO review	SLO review meeting	Engineering Leadership Team; on-call engineers.	Review each SLO's performance over the prior month; review error budget consumption; identify SLOs that are too tight or too loose; tune alert thresholds; review runbook exercises completed.
Quarterly SLO amendment	SLO amendment review	Engineering Leadership Team; Chief Systems Architect.	Review SLO targets against user expectations; amend SLOs per the Evolution

			Governance process (Chapter 36); review SLAs against consumer feedback; update the SLO catalogue.
Annual SLO audit	External review	Engineering Leadership Team; Chief Systems Architect; external reviewer (optional).	Comprehensive review of the SLO framework: are SLIs measuring the right things? are SLOs calibrated to user value? are runbooks current? are alert thresholds appropriate?

SLO amendments SHALL follow the Evolution Governance process (Chapter 36): a proposed amendment SHALL have a recorded rationale, a recorded impact analysis (which SLIs, runbooks, dashboards, and SLAs are affected), and a recorded sign-off by the Chief Systems Architect. Amendments that tighten an SLO MAY take effect immediately (tighter targets are safer for users); amendments that loosen an SLO SHALL have a 30-day notice period to allow runbooks and dashboards to be updated and to allow consumers (where the SLO backs an SLA) to be notified.

RULE SLO-004 Every SLO SHALL be reviewed at the weekly burn review, the monthly SLO review, and the quarterly SLO amendment. SLOs SHALL NOT be amended outside the quarterly amendment review except by the Evolution Governance process (Chapter 36). SLO amendments SHALL be recorded in the audit log with rationale, impact analysis, and sign-off.

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

This chapter operationalizes the reliability budgets of Chapter 31 (§ 31.9) into a closed-loop SLO regime. The purpose is to convert the platform's availability targets from aspirations into measured objectives with defined consequences of breach, defined escalation paths, defined mitigations, defined runbooks, and a defined review cadence. An SLO that is not measured is not an SLO; an SLO that is measured but has no consequence of breach is a dashboard widget; an SLO with consequences but no runbook is a panic button. This chapter ties all four together.

2. Scope

In scope: the SLO framework (definitions, methodology); the SLI catalogue (10 indicators); the SLO catalogue (10 objectives); the SLA catalogue (3 external agreements); the error budget policy (4-state machine); burn rate alerting (4-tier multi-window pattern); the escalation matrix (5 severities); the mitigation playbook (5 mitigation strategies); the runbook catalogue (10 runbooks); the SLO review cadence (4 cadences). Out of scope: the engineering mathematics of thresholds (Chapter 31); the incident-response regime (Chapter 21); the certification of SLO monitors (Chapter 22).

3. Definitions

SLI: a quantitative measure of service behaviour, expressed as a ratio of good events to valid events over an aggregation window. SLO: a target value for an SLI, over an aggregation window, with a defined consequence of breach. SLA: an external promise to consumers with consequences if broken; always looser than the SLO. Error budget: the permitted unreliability implied by the SLO, computed as (1 − target) × window. Burn rate: the rate of error budget consumption relative to sustainable rate. Mitigation: a defined operational action (rollback, shed load, throttle, fail-open, fail-closed) taken to restore an SLO.

4. Responsibilities

Ownership of the SLO framework is distributed across the engineering organization:

- **Chief Systems Architect:** Accountable for the SLO framework and for the SLO amendment process; the final approver of any SLO amendment.
- **Head of Platform Engineering:** Responsible for availability, latency, publication success, and Crossref deposit SLOs and their runbooks.
- **Head of Data Engineering:** Responsible for freshness, completeness, and preservation coverage SLOs and their runbooks.
- **Head of Observability:** Responsible for the SLO dashboard, burn-rate alerting, and the weekly burn review.
- **Release Certification Authority:** Responsible for verifying that every SLO has a live monitor, an exercised runbook, and an automated error-budget-state action before RC1 sign-off.

5. Design Principles

SLOs are user-centric (they measure what users experience, not what engineers monitor), measurable (each has a defined SLI with a defined source), actionable (each has a defined consequence of breach and a defined runbook), and conservative (when in doubt, the SLO is set tighter rather than looser). An SLO that measures an internal metric rather than a user-experienced metric is a vanity metric; an SLO without a runbook is a panic button; an SLO without a consequence of breach is decoration. The discipline of this chapter is the discipline of converting reliability into a closed-loop control system.

6. Engineering Requirements

Every SLO SHALL have a stable identifier, a defined SLI, a target value, an aggregation window, a consequence of breach, an escalation path, a preferred mitigation, a fallback mitigation, a runbook, and a named owner. The SLO dashboard SHALL display the current SLI value, SLO status (pass / fail / unknown), error budget remaining, error budget state (healthy / at-risk / critical / exhausted), and burn rate for every SLO in real time. The error-budget-state machine SHALL be automated: state transitions SHALL invoke the documented action without human intervention. Burn-rate alerts SHALL fire on the multi-window pattern; alert noise SHALL be reviewed weekly.

7. Engineering Constraints

SLOs constrain every release. A release that would push an SLO into the 'critical' state SHALL NOT proceed. A release that would push an SLO into the 'exhausted' state SHALL NOT proceed under any circumstance. The Chief Systems Architect SHALL NOT override an exhausted error budget; the only path to restoring feature velocity is to restore the SLO. SLOs MAY be amended only through the Evolution Governance process (Chapter 36), and amendments that loosen an SLO SHALL require evidence that the looser value still meets legitimate user expectations. SLAs are looser than SLOs by design; the gap between SLO and SLA SHALL NOT be narrowed below 0.4 percentage points (e.g., 99.9% SLO, 99.5% SLA) to preserve the budget buffer.

8. Runtime Behaviour

At runtime, every SLI is measured continuously by its source system. The SLO dashboard aggregates measurements into rolling-window statistics. The error-budget-state machine evaluates each SLO's burn rate and remaining budget continuously and invokes the documented action on state transition. Burn-rate alerting evaluates the multi-window thresholds continuously and fires alerts when both windows exceed their thresholds. The escalation matrix's auto-escalation timers run continuously. The result is a closed-loop control system: breaches are detected early, alerts fire automatically, escalation proceeds without manual routing, and mitigations are invoked from runbooks.

9. Failure Conditions

SLO framework failure conditions include:

- **SLI source failure:** the system that produces an SLI's measurements is unavailable, leaving the SLI in 'unknown' state.
- **SLO dashboard failure:** the dashboard itself is unavailable, leaving the engineering organization blind to SLO status.
- **Error-budget-state machine failure:** the state machine fails to invoke the documented action on state transition (e.g., a release freeze is not enforced when an SLO is exhausted).
- **Alerting failure:** burn-rate alerts do not fire when thresholds are exceeded, or fire when they should not (alert fatigue).
- **Runbook failure:** the runbook's mitigation steps do not restore the SLO, or the runbook is outdated and references systems that no longer exist.

10. Recovery Procedures

SLO breach recovery follows the runbook for the affected SLO (RB-SLO-NNN). The on-call engineer SHALL acknowledge the alert within the documented response time, SHALL execute the runbook's diagnostic steps to identify the root cause, SHALL execute the preferred mitigation, and SHALL verify restoration. If the preferred mitigation is insufficient, the on-call engineer SHALL execute the fallback mitigation and SHALL escalate per the escalation matrix. Every breach SHALL produce a postmortem per the Failure Constitution (Chapter 21) and SHALL feed into the next monthly SLO review for threshold tuning.

11. Security Considerations

SLOs are the platform's first quantified defence against availability attacks. An attacker who can degrade an SLO can degrade the platform's user experience; the error-budget policy ensures that sustained degradation triggers feature freeze and reliability focus rather than continued feature velocity. The SLO dashboard SHALL itself be defended: an attacker who can manipulate the SLI sources can mask breaches, so SLI sources SHALL be append-only where applicable and SHALL be cross-checked by independent reconciliation jobs. SLA breaches SHALL trigger public notification per the SLA catalogue; suppress SLA notification is a SEV-1 incident.

12. Performance Considerations

SLO monitoring has a performance cost: every SLI measurement consumes resources at its source (access log write, latency histogram update, reconciliation job run). The performance budget for SLO monitoring SHALL NOT exceed 2% of platform compute capacity. Monitoring that exceeds its budget SHALL be optimized (e.g., reducing the SLI's aggregation window from 1 minute to 5 minutes, or moving measurement from synchronous request path to asynchronous log processing), not skipped. The SLO dashboard's query cost SHALL be bounded; dashboards that exceed 5 seconds to load SHALL be optimized or cached.

13. Observability Requirements

Every SLO SHALL expose metrics for: current SLI value, SLO status (pass / fail / unknown), error budget consumed, error budget remaining, error budget state (healthy / at-risk / critical / exhausted), burn rate (long window and short window per tier), alert firings (count per tier), and runbook invocations (count per runbook). The SLO dashboard SHALL aggregate these into a single view organized by the 10 SLOs. An SLO in 'unknown' state (no metric received within its cadence interval) is a SEV-2 incident (the platform is blind to that SLO) and SHALL be detected by a meta-monitor.

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **All 10 SLIs, 10 SLOs, and 3 SLAs are documented with stable identifiers (SLI-001 — SLI-010, SLO-001 — SLO-004 rules, SLA catalogue entries).**
- **Every SLO has a live monitor reporting to the SLO dashboard.**

- **The error-budget-state machine is automated and enforces the documented actions (feature freeze, release freeze) on state transitions.**
- **Burn-rate alerting is live on all 10 SLOs with the four-tier multi-window pattern.**
- **All 10 runbooks (RB-SLO-001 — RB-SLO-010) are written, reviewed, and exercised at least once in a chaos exercise.**
- **The weekly burn review, monthly SLO review, and quarterly SLO amendment cadences are scheduled and attended.**
- **The Chief Systems Architect has signed off the SLO catalogue.**

15. Runtime Verification

Runtime verification of the SLO framework is the SLO dashboard itself: the dashboard is the evidence that SLIs are being measured, SLOs are being evaluated, error-budget states are being enforced, and alerts are firing. The Release Certification Authority SHALL verify that all 10 SLOs are reporting to the dashboard, that the error-budget-state machine has been exercised (e.g., by artificially breaching an SLO in staging and observing the state transition), and that at least one runbook has been exercised end-to-end before RC1 sign-off.

16. Certification Requirements

RC1 certification requires: the SLO catalogue published; all 10 SLOs operational on the dashboard; the error-budget-state machine operational and exercised in staging; burn-rate alerting live on all 10 SLOs; all 10 runbooks written and at least one exercised end-to-end; the SLO review cadence scheduled; and the Chief Systems Architect's sign-off that the SLO set is complete (no known user-facing surface lacks an SLO). An SLO without a runbook is not certifiable; an SLO without an automated error-budget action is not certifiable.

17. Future Evolution

The SLO set evolves through the Evolution Governance process (Chapter 36). New SLOs MAY be added when a new user-facing surface is introduced (e.g., a new public API endpoint). Existing SLOs MAY be tightened (more ambitious targets) as operational capacity grows; existing SLOs MAY be loosened only with documented evidence that the looser value still meets legitimate user expectations. An SLO SHALL NOT be removed; if an SLO is genuinely obsolete (the surface it measured no longer exists), it SHALL be marked deprecated with a recorded reason and the underlying SLI SHALL be retained for historical analysis. SLA amendments follow the same process with the additional requirement of consumer notification.

Chapter 33 — Database Constitution

This chapter is the database constitution of Opus Publica. It is distinct from the Data Constitution (Chapter 5), which governs the platform's logical data model: the entities, fields, ownership, versioning, and migration strategy of canonical records. This chapter governs the database engine that stores those records: its naming conventions, index strategy, constraint strategy, foreign-key rules, migration rules, transaction rules, locking rules, partitioning strategy, archival strategy, vacuum and bloat management, backup and recovery, replication, connection management, and query performance. Where Chapter 5 says WHAT the platform stores, this chapter says HOW the database stores it.

The chapter assumes a relational database engine with PostgreSQL-grade capabilities (transactions, MVCC, foreign keys, partial and expression indexes, GIN/GiST/BRIN index types, declarative partitioning, logical and physical replication, point-in-time recovery). The platform's chosen implementation is PostgreSQL; the rules in this chapter are written to be portable to any engine that provides equivalent capabilities, but where the rules name PostgreSQL-specific features (e.g., autovacuum, PgBouncer, pg_partman), the implementation is PostgreSQL. Citations throughout refer to the PostgreSQL documentation (https://www.postgresql.org/docs/) and to standard engineering references (Google SRE Workbook for operational practice, RFC 9110 for API-boundary concerns, NIST SP 800-92 for audit log retention, ISO 8601 for timestamp format).

The chapter is organized into fourteen sections, mirroring the lifecycle of a database object: it is named (§ 33.1), indexed (§ 33.2), constrained (§ 33.3), related to other objects via foreign keys (§ 33.4), migrated through schema changes (§ 33.5), accessed via transactions (§ 33.6) and locks (§ 33.7), partitioned for scale (§ 33.8), archived for history (§ 33.9), vacuumed for hygiene (§ 33.10), backed up for recovery (§ 33.11), replicated for availability (§ 33.12), accessed via connection pools (§ 33.13), and queried with attention to performance (§ 33.14). Each section is normative: the rules in it SHALL be satisfied by every database the platform operates.

The Constitutional Status of Database Rules

The database is the system of record for the platform's mutable state. A database rule that is violated risks corruption of the scholarly record. The rules in this chapter SHALL be enforced by schema constraints, by migration review, by query linters, and by operational monitors. A schema change that violates a rule in this chapter SHALL NOT be merged. A query that violates a rule in this chapter SHALL NOT be deployed. An operational configuration that violates a rule SHALL NOT be applied. The Chief Systems Architect SHALL NOT suspend a database rule except via the Evolution Governance process (Chapter 36).

33.1 Naming Conventions

Naming conventions ensure that database objects are identifiable, self-documenting, and consistent across the schema. A consistent naming convention enables automated tooling (migration generators, ORM mappings, query linters) to operate without per-object special cases, and enables engineers to read schema diagnostics without consulting a lookup table. The conventions in this section apply to every database object the platform creates; objects that do not conform SHALL be renamed at the next migration opportunity.

Object Type	Naming Pattern	Examples
Tables	snake_case, plural; lowercase; ASCII only.	articles, audit_records, doi_registrations, preservation_deposits, reviewer_assignments.
Columns	snake_case, lowercase; singular; no abbreviations except id, ts, url, doi.	article_id, created_at, doi_value, publication_timestamp, actor_principal.
Primary key columns	id (surrogate UUID) OR <table_singular>_id (natural key).	id (UUID), article_id (where article_id is the natural key in a child table).
Indexes	idx_<table>_<col1>[_<col2>...]; partial indexes add _<predicate_summary>.	idx_articles_published_at, idx_articles_status_published_partial, idx_audit_records_article_id_created_at.
Constraints	pk_<table>, fk_<from_table>_<to_table>, uq_<table>_<col>, ck_<table>_<rule>.	pk_articles, fk_articles_journal_id_journals, uq_dois_value, ck_articles_status.
Foreign keys	fk_<from_table>_<col> (column-form) or fk_<from_table>_<to_table> (table-form).	fk_articles_journal_id, fk_articles_journal_id_journals.
Sequences	seq_<table>_<col> (used only where UUIDs are unsuitable; e.g., legacy-compatible integer keys).	seq_legacy_imports_id (deprecated; do not use for new tables).
Views	v_<purpose>; lowercase; descriptive verb-or-noun.	v_published_articles_with_doi, v_audit_chain_by_article, v_submissions_in_review.
Materialized views	mv_<purpose>; same conventions as views.	mv_article_citation_counts, mv_journal_publication_stats.
Functions	fn_<verb>_<entity>; lowercase; parameter names in snake_case.	fn_calculate_citation_count(article_uuid), fn_validate_jats(xml_text).
Triggers	tg_<table>_<event>[_<timing>];	tg_audit_records_insert_after,

	timing is before or after.	tg_articles_update_before.

RULE DB-001 Every database object SHALL conform to the naming pattern documented for its object type in § 33.1. Objects that do not conform SHALL be renamed at the next migration opportunity. Migration review SHALL reject schema changes that introduce non-conforming names.

33.2 Index Strategy

Index strategy governs how the database's physical indexes are chosen, created, and maintained. Indexes are the primary determinant of query performance: a query that scans a 100-million-row table without an index will take minutes; the same query with the right index will take milliseconds. Indexes also have cost: every index adds write amplification (every insert and update must update the index) and storage cost. The strategy in this section balances read performance against write cost and storage cost, and documents the maintenance regime for each index type.

PostgreSQL provides several index types, each suited to a different query pattern. The table below documents the index type, its use case, examples from the Opus Publica schema, and the maintenance regime. The mandatory-index rules below the table apply to every table: every table SHALL have a primary key index; every foreign key column SHALL have an index; every column used in a WHERE clause, JOIN clause, or ORDER BY clause that is on a hot path SHALL have an index.

Index Type	Use Case	Examples	Maintenance
B-tree (default)	Equality and range queries on scalar columns; ORDER BY support.	idx_articles_published_at, idx_dois_value, idx_submissions_state_created_at.	REINDEX when bloat > 30%; monitor via pgstatindex.
GIN	JSONB containment, array containment, full-text search.	idx_articles_metadata_gin (JSONB), idx_articles_keywords_gin (array), idx_articles_fulltext_gin (tsvector).	REINDEX when bloat > 50%; GIN indexes are slower to update than B-tree; consider fastupdate.
GiST	Geometric, range, ltree, exclusion constraints.	idx_versions_ltree_gist (ltree path), idx_date_ranges_gist (daterange), exclusion constraints for non-overlapping ranges.	REINDEX when bloat > 50%; GiST indexes can be slow to build.
BRIN	Huge, naturally-ordered tables where adjacent rows have similar values (e.g., time-series).	idx_audit_log_created_at_brin, idx_events_occurred_at_brin.	Low maintenance; auto-summarize; effective for tables > 100M rows with natural ordering.

Partial	Index only the rows matching a WHERE predicate; small, fast indexes for common filtered queries.	idx_articles_published_partial WHERE state = 'published', idx_submissions_in_review_partial WHERE state = 'in_review'.	Low maintenance; verify predicate still matches query predicate.
Expression	Index the result of a function or expression; supports queries on transformed columns.	idx_lower_authors_email ON (lower(email)), idx_articles_doi_prefix ON (substring(doi from 1 for 9)).	REINDEX when bloat > 30%; expression MUST be immutable.
Unique	Enforce uniqueness constraint; B-tree by default.	uq_dois_value, uq_articles_slug_per_journal (composite unique).	REINDEX rarely; unique indexes are typically small and stable.
Covering (INCLUDE)	B-tree with non-key columns included for index-only scans.	idx_articles_published_at INCLUDE (title, doi_value) for listing pages.	REINDEX when bloat > 30%; monitor index-only scan rate.

33.2.1 Mandatory Indexes

The following indexes are mandatory on every table in the schema. Migration review SHALL reject a migration that creates a table without these indexes. The query linter SHALL flag queries that perform unindexed scans on columns that should have indexes.

- **Primary key:** Every table SHALL have a primary key index (B-tree on the primary key column).
- **Foreign keys:** Every foreign key column SHALL have a B-tree index. Unindexed foreign keys cause full-table scans on cascade operations and lock escalation on updates.
- **Lookup columns:** Every column used in a WHERE clause, JOIN clause, or ORDER BY clause on a hot path (any query executed more than 10 times per minute) SHALL have an index appropriate to the query pattern.
- **Unique constraints:** Every column used in a UNIQUE constraint SHALL have a unique index (created implicitly by the constraint).

RULE DB-002 Every table SHALL have indexes on its primary key, every foreign key column, every column used in a hot-path WHERE/JOIN/ORDER BY clause, and every UNIQUE constraint. Migration review SHALL reject schema changes that introduce tables without these mandatory indexes. Queries that perform unindexed scans on hot-path columns SHALL be flagged by the query linter and SHALL NOT be deployed.

33.3 Constraint Strategy

Constraint strategy governs how the database's declarative constraints are chosen and applied. Constraints are the database's first line of defence against invalid data: a CHECK constraint that refuses

an article in an invalid state prevents the illegal state from ever reaching the application layer; a NOT NULL constraint that refuses a missing publication timestamp prevents an article from being published without a recorded moment of publication. Constraints are preferred over application-layer validation because they are enforced by the database engine itself, which is the single chokepoint through which all writes pass.

Constraint Type	Use Case	Examples
Primary Key (PK)	Uniquely identify each row; backed by a unique B-tree index.	pk_articles (id), pk_audit_records (id), pk_dois (id).
Foreign Key (FK)	Enforce referential integrity between parent and child tables.	fk_articles_journal_id_journals, fk_dois_article_id_articles, fk_audit_records_actor_principal_users.
Unique (UNIQUE)	Enforce uniqueness on a column or composite of columns; backed by a unique index.	uq_dois_value (one DOI string per row), uq_articles_slug_per_journal (slug, journal_id).
Check (CHECK)	Enforce a boolean predicate on a column or row; refuse inserts or updates that violate the predicate.	ck_articles_state (state IN ('draft','in_review','published','retracted')), ck_dois_value (doi_value ~ '^10\.'), ck_audit_records_actor_principal (actor_principal IS NOT NULL).
Not Null (NOT NULL)	Refuse NULL in a column; mandatory for every column that is semantically required.	articles.id NOT NULL, articles.created_at NOT NULL, audit_records.correlation_id NOT NULL.
Exclusion (EXCLUDE)	Prevent overlapping ranges or conflicting rows; uses a GiST index.	exclusion on (journal_id WITH =, embargo_period WITH &&) to prevent overlapping embargo windows for the same journal.
Default (DEFAULT)	Provide a value when the column is not specified in an INSERT.	articles.created_at DEFAULT NOW(), audit_records.id DEFAULT gen_random_uuid().

RULE DB-003 Every column that is semantically required SHALL have a NOT NULL constraint. Every column whose value domain is enumerable SHALL have a CHECK constraint. Every parent-child relationship SHALL have a FOREIGN KEY constraint. Every column whose uniqueness is a scholarly-record guarantee (DOI, slug, ORCID iD) SHALL have a UNIQUE constraint. Constraints SHALL be preferred over application-layer validation for all invariants the database can enforce.

33.4 Foreign Key Rules

Foreign key rules govern the ON DELETE and ON UPDATE behaviour of every foreign key in the schema, the use of deferrable constraints, and the handling of cyclical foreign-key relationships. Foreign keys are the database's mechanism for enforcing referential integrity: a row that references a non-existent parent is refused at write time. The ON DELETE and ON UPDATE behaviour defines what happens to the child row when the parent row is deleted or its key is updated.

33.4.1 ON DELETE / ON UPDATE Behaviour

Behaviour	Use Case	Examples
ON DELETE RESTRICT	Default for all scholarly-record foreign keys; refuse to delete the parent if any child references it.	fk_dois_article_id_articles ON DELETE RESTRICT: an article with DOIs cannot be deleted (consistent with INV-M-03).
ON DELETE CASCADE	Use only for owned children whose existence is meaningless without the parent; never for scholarly-record data.	fk_review_files_review_id_reviews ON DELETE CASCADE: deleting a review deletes its review files (acceptable for draft reviews; not for published reviews).
ON DELETE SET NULL	Use when the child can survive without the parent but the reference must be cleared.	fk_articles_superseded_by_id ON DELETE SET NULL: if the superseding article is deleted, the superseded article's superseded_by becomes NULL (rare; governance-required).
ON UPDATE CASCADE	Use when the parent's key may be updated and the children should follow.	Generally avoided: primary keys are immutable UUIDs in the Opus Publica schema, so ON UPDATE CASCADE is rarely needed.
ON UPDATE NO ACTION	Default; same as RESTRICT but checked at the end of the statement rather than immediately.	Default behaviour; preferred over RESTRICT when deferrable constraints are used.

33.4.2 Deferrable Constraints

Deferrable constraints are foreign keys whose validation is deferred to the end of the transaction rather than performed at statement time. Deferrable constraints are essential for cyclical relationships (where table A references table B and table B references table A) and for bulk operations where intermediate states would violate constraints. The platform SHALL use deferrable constraints for: cyclical foreign keys; bulk inserts that temporarily violate referential integrity; and migration operations that reorder writes. Deferrable constraints SHALL be INITIALLY DEFERRED (validated at commit time) rather than INITIALLY IMMEDIATE (validated at statement time but deferrable on request), to ensure consistent behaviour.

33.4.3 Cyclical Foreign Key Handling

Cyclical foreign keys (A ↔ B) SHALL be handled by making at least one direction deferrable INITIALLY DEFERRED. The insertion order in such cases SHALL be: insert A with NULL in the B-reference column; insert B with A-reference populated; update A to populate the B-reference; commit. The constraint validation at commit time SHALL succeed because both references are populated. Cyclical foreign keys SHALL be documented in the schema with a comment explaining the cycle and the deferral strategy. The platform discourages cyclical foreign keys; where they are unavoidable, they SHALL be reviewed by the Head of Data Engineering.

RULE DB-004 Every foreign key SHALL have a documented ON DELETE and ON UPDATE behaviour. Scholarly-record foreign keys SHALL be ON DELETE RESTRICT. Cyclical foreign keys SHALL be handled with deferrable INITIALLY DEFERRED constraints and SHALL be documented. ON DELETE CASCADE SHALL NOT be used for scholarly-record data under any circumstance.

33.5 Migration Rules

Migration rules govern how the database schema is changed over time. Schema changes are the highest-risk operations the platform performs: a poorly-designed migration can lock a hot table for hours, can break running applications that expect the old schema, or can lose data that cannot be recovered. The rules in this section are designed to make every migration forward-compatible, zero-downtime, tested, and reversible.

33.5.1 Forward-Compatible Migrations

Every migration SHALL be forward-compatible: the schema after the migration SHALL be usable by both the old and new versions of the application, so that the migration can be applied before the new application code is deployed and the old application code can continue to run against the new schema until it is decommissioned. Forward compatibility typically requires the expand-contract pattern (below) and prohibits destructive operations (DROP COLUMN, DROP TABLE, ALTER COLUMN TYPE) in the expand phase.

33.5.2 Expand-Contract Pattern

Every breaking schema change SHALL be performed in two migrations separated by a deployment cycle. The expand migration adds the new schema without removing the old (e.g., adds a new column alongside the old; adds a new table alongside the old). The application is then deployed to write to both old and new (dual-write) and read from the new (with fallback to the old). After verification that the new schema is correct and complete, the contract migration removes the old schema (DROP COLUMN, DROP TABLE). The contract migration SHALL be applied only after the old application version is fully decommissioned, which SHALL be at least one deployment cycle (one week) after the expand migration.

33.5.3 Migration Testing

Every migration SHALL be tested against a production-sized dataset in staging before being applied to production. The test SHALL measure: the migration's wall-clock time; the lock duration on affected tables; the impact on concurrent query latency; and the rollback time. A migration that locks a hot table for more than 5 seconds SHALL be redesigned (e.g., by adding the column with a DEFAULT NULL and backfilling in batches, rather than adding with a non-null default that requires a full table rewrite).

33.5.4 Rollback

Every migration SHALL have a documented rollback procedure. The rollback procedure SHALL be tested in staging. For expand migrations, rollback is typically trivial (DROP the new column or table). For contract migrations, rollback is typically impossible (the data has been removed); the rollback procedure SHALL document this and SHALL specify the alternative (restore from backup, accept data loss). Contract migrations SHALL be applied only after the expand migration has been verified in production for at least one week.

33.5.5 Zero-Downtime Migrations

Every migration SHALL be zero-downtime: the platform SHALL remain available throughout the migration. Operations that require exclusive locks on hot tables (ALTER COLUMN TYPE, ADD COLUMN with volatile DEFAULT, REWRITE TABLE) SHALL be decomposed into lock-free sequences: add a new column, backfill in batches, switch the application to the new column, drop the old column. Operations that require brief exclusive locks (CREATE INDEX CONCURRENTLY is lock-free; CREATE INDEX is not) SHALL use the concurrent variants.

33.5.6 Migration Review Process

Every migration SHALL be reviewed by at least one engineer other than the author. The review SHALL verify: conformance with the naming conventions (§ 33.1); conformance with the index strategy (§ 33.2); conformance with the constraint strategy (§ 33.3); conformance with the foreign-key rules (§ 33.4); forward compatibility; expand-contract decomposition where applicable; tested rollback procedure; zero-downtime property; and staging test results. The review SHALL be recorded in the migration's version-control commit.

RULE DB-005 Every migration SHALL be forward-compatible, zero-downtime, tested against a production-sized dataset, and reversible (with documented rollback). Breaking changes SHALL use the expand-contract pattern with at least one deployment cycle between expand and contract. Migrations SHALL be reviewed by at least one engineer other than the author. Migrations SHALL be applied via the migration tool, never via ad-hoc SQL.

33.6 Transaction Rules

Transaction rules govern how the platform uses database transactions. Transactions are the database's mechanism for atomic, consistent, isolated, and durable (ACID) operations. The rules in this section specify the isolation levels the platform uses, the maximum permissible transaction duration, the outbox pattern for combining database writes with side effects, the avoidance of distributed transactions, and the idempotency of consumers.

33.6.1 Isolation Levels

The default isolation level SHALL be READ COMMITTED, which is PostgreSQL's default and provides a correct view of committed data without the overhead of serializable isolation. SERIALIZABLE isolation SHALL be used for operations that enforce invariants across multiple rows (e.g., preventing two publications from claiming the same DOI, preventing two reviewers from being assigned to the same submission in the same slot). SERIALIZABLE isolation MAY produce serialization failures (SQLSTATE 40001), which the application SHALL retry with the documented retry budget (§ 31.5). READ UNCOMMITTED isolation SHALL NOT be used; it exposes uncommitted data and is not supported by PostgreSQL (PostgreSQL maps it to READ COMMITTED).

33.6.2 Transaction Timeout

Every transaction SHALL have a timeout of 5 seconds (the hard limit is 10 seconds, per § 31.4). The timeout SHALL be set via statement_timeout at the session or transaction level. A transaction that exceeds its timeout SHALL be aborted by the database, its changes SHALL be rolled back, and the application SHALL receive an error which it SHALL handle per its retry budget. Long-running operations (bulk imports, archive compaction, report generation) SHALL NOT be performed in a single transaction; they SHALL be decomposed into batches with their own transactions.

33.6.3 Outbox Pattern

The platform SHALL use the transactional outbox pattern for operations that combine a database write with a side effect (e.g., publishing an article and enqueuing a Crossref deposit). The database write and the outbox-record write SHALL be in the same transaction; the outbox record SHALL be processed asynchronously by a separate consumer that reads the outbox table and performs the side effect. This pattern ensures that the side effect is enqueued if and only if the database write commits, avoiding the inconsistency that arises from performing the side effect outside the transaction (where a database rollback would leave the side effect orphaned).

33.6.4 Distributed Transaction Avoidance

The platform SHALL NOT use distributed transactions (two-phase commit, XA) across heterogeneous resources (e.g., database + message queue, database + external API). Distributed transactions introduce blocking, partial-failure, and performance pathologies that outweigh their consistency benefits. Instead, the platform SHALL use the outbox pattern (above) for database-and-side-effect combinations, and SHALL use idempotent consumers (below) for at-least-once delivery.

33.6.5 Idempotent Consumers

Every consumer of an outbox or message queue SHALL be idempotent: processing the same message twice SHALL produce the same result as processing it once. Idempotency SHALL be implemented via an idempotency key (e.g., the outbox record's UUID) that the consumer records in a processed_messages table; on each message, the consumer SHALL check the table and skip if the key is already present. Idempotency enables at-least-once delivery (which is the strongest delivery guarantee available without distributed transactions) without producing duplicate side effects.

RULE DB-006 Every transaction SHALL use READ COMMITTED isolation by default and SERIALIZABLE isolation for invariant-enforcing operations. Every transaction SHALL have a 5-second timeout (10-second hard limit). Database writes combined with side effects SHALL use the transactional outbox pattern. Distributed transactions SHALL NOT be used. Every outbox or queue consumer SHALL be idempotent via an idempotency key.

33.7 Locking Rules

Locking rules govern how the platform uses database locks. Locks are the database's mechanism for isolating concurrent transactions. PostgreSQL uses MVCC (multi-version concurrency control) by default, which means that readers do not block writers and writers do not block readers; locks are acquired primarily for write-write conflicts and for explicit serial operations. The rules in this section govern the use of explicit locks (advisory locks), lock timeouts, deadlock detection, and lock-ordering conventions.

33.7.1 Row-Level vs Table-Level Locks

The platform SHALL use row-level locks (the default for DML operations) and SHALL NOT use table-level LOCK TABLE statements except in migration operations. Table-level locks block all concurrent access to the table and are appropriate only for administrative operations (e.g., CLUSTER, VACUUM FULL) that operate on the whole table. Row-level locks block only the conflicting rows and are appropriate for all application operations.

33.7.2 Advisory Locks for Serial Operations

The platform SHALL use PostgreSQL advisory locks (pg_advisory_lock) for serial operations that must be mutually exclusive across transactions but do not correspond to a database row. Examples include: serializing DOI minting for a given prefix; serializing Crossref deposit for a given batch; serializing the publication state-machine transition for a given article. Advisory locks SHALL be acquired with a key derived from the entity identifier (e.g., article_id) to ensure that operations on different entities do not contend. Advisory locks SHALL be released at the end of the transaction (pg_advisory_xact_lock) to avoid leaks; if a session-level lock is required (pg_advisory_lock), the application SHALL ensure it is released even on failure.

33.7.3 Lock Timeout

Every session SHALL have a lock timeout of 2 seconds (the hard limit is 5 seconds, per § 31.4) set via lock_timeout. A session that waits longer than the lock timeout for a lock SHALL be aborted by the database with SQLSTATE 55P03 (lock_not_available). The application SHALL handle this error by retrying with the documented retry budget (§ 31.5) or by failing the operation. Long lock waits indicate contention that SHALL be investigated; the slow-query log (§ 33.14) SHALL capture sessions that wait on locks.

33.7.4 Deadlock Detection

PostgreSQL automatically detects deadlocks (cycles in the lock wait-for graph) and aborts one of the transactions with SQLSTATE 40P01 (deadlock_detected). The application SHALL handle this error by retrying with the documented retry budget. Deadlocks indicate a lock-ordering bug (below) and SHALL be investigated; recurring deadlocks SHALL be treated as a SEV-2 incident. The deadlock detector's timeout is controlled by deadlock_timeout (default 1 second) and SHALL be left at the default.

33.7.5 Lock Order Conventions

To prevent deadlocks, the platform SHALL observe a consistent lock ordering convention: transactions that lock multiple rows SHALL lock them in primary-key order (ascending). For transactions that lock rows across multiple tables, the platform SHALL document the table-locking order in the application's design documentation and SHALL observe it consistently. The query linter SHALL flag transactions that issue UPDATE or DELETE statements against multiple rows without an ORDER BY clause that matches the primary-key order.

RULE DB-007 Every session SHALL have a 2-second lock timeout (5-second hard limit). Table-level locks SHALL be used only in migrations. Advisory locks SHALL be used for serial operations and SHALL be released at the end of the transaction. Deadlock errors (SQLSTATE 40P01) SHALL be retried. Transactions that lock multiple rows SHALL lock them in primary-key order to prevent deadlocks.

33.8 Partitioning Strategy

Partitioning strategy governs how the platform decomposes large tables into smaller, more manageable partitions. Partitioning improves query performance (queries that filter on the partition key scan only the relevant partitions), simplifies data lifecycle management (old partitions can be detached and archived), and enables parallel query execution across partitions. The platform uses PostgreSQL's declarative partitioning (PARTITION BY), which manages partitions automatically without the application needing to know the partition scheme.

33.8.1 Partition by article_id for High-Volume Tables

High-volume tables associated with articles (audit_log, events, review_files) SHALL be partitioned by article_id using HASH partitioning with 64 partitions. HASH partitioning distributes rows evenly across partitions by hashing the article_id, which provides uniform load and enables parallel query execution. The 64-partition count is chosen to provide headroom: at 100 million audit records across 100 000 articles, each partition holds approximately 1.5 million records, which is well within PostgreSQL's comfortable operating range.

33.8.2 Partition by Date for Time-Series Tables

Time-series tables (metrics, raw_event_log, external_api_call_log) SHALL be partitioned by date using RANGE partitioning with monthly partitions. Monthly partitions balance query performance (queries typically filter by recent date ranges) against partition count (annual partition count remains manageable). Older partitions can be detached and archived (per § 33.9) without affecting current operations. A partition maintenance job SHALL create new partitions in advance (typically 3 months ahead) and SHALL alert if partitions are not pre-created.

33.8.3 Partition Maintenance

Partition maintenance SHALL be automated by a scheduled job that: creates new partitions in advance (3 months for time-series; no action needed for HASH partitions); detaches and archives old partitions per the archival strategy (§ 33.9); and reports partition health (count, size, row estimate) to the observability dashboard. The partition maintenance job SHALL be idempotent and SHALL log its actions to the audit log. A failure of the partition maintenance job that prevents new partitions from being created SHALL be a SEV-2 incident (inserts into the affected table will fail when no partition matches).

RULE DB-008 High-volume article-associated tables SHALL be partitioned by article_id using HASH partitioning with 64 partitions. Time-series tables SHALL be partitioned by date using RANGE partitioning with monthly partitions. Partition maintenance SHALL be automated, idempotent, and audited. New partitions SHALL be created at least 3 months in advance. Partition maintenance failure SHALL be a SEV-2 incident.

33.9 Archival Strategy

Archival strategy governs how the platform moves old data from hot storage (the primary database) to cold storage (object storage or a dedicated archival database) while preserving the data's retrievability and integrity. Archival is distinct from deletion: archived data SHALL remain retrievable for its full retention period (§ 31.8); deletion destroys data. The platform archives data that is no longer queried frequently but must be retained for compliance, audit, or historical purposes.

33.9.1 Cold Archival of Old Audit Records

Audit records older than 13 months SHALL be moved from the primary audit_log table to a cold archival store (object storage in Parquet format, with an index file mapping article_id and date range to archive file). The archival job SHALL: select records older than 13 months; write them to a Parquet file in object storage; verify the Parquet file (record count, checksum); record the archival in an archive_manifest table; and detach and drop the source partition. The archival SHALL be idempotent: re-running the job SHALL detect already-archived records and skip them. The archival SHALL preserve the cryptographic chain (INV-A-03): the Parquet file SHALL include the hash of each record, and the chain SHALL be re-verifiable across the hot/cold boundary.

33.9.2 Archived Partition Detach/Attach

For partitioned tables, archival SHALL be performed by detaching the old partition (ALTER TABLE … DETACH PARTITION) and either dropping it (after archival verification) or retaining it as a standalone table for a grace period. Detaching a partition is a metadata-only operation that does not block concurrent access to the remaining partitions. The detached partition MAY be re-attached (ALTER TABLE … ATTACH PARTITION) if the archived data needs to be queried in place; this SHALL be a planned operation with documented reason, not a routine query path.

33.9.3 Archival to Object Storage

Cold archival targets object storage (S3-compatible) with the following conventions: each archive file is named <table>_<partition_key>_<date>.parquet (e.g., audit_log_2025-01_2026-02-15.parquet); each archive file has a corresponding manifest entry in the archive_manifest table recording the file's checksum, record count, date range, and article_id range; archive files are encrypted at rest (object storage server-side encryption) and SHALL additionally be encrypted client-side for audit_log archives (defense in depth). Archive files SHALL be replicated to a different region for disaster recovery.

RULE DB-009 Audit records older than 13 months SHALL be archived to object storage in Parquet format with a manifest entry. Archival SHALL preserve the cryptographic chain (INV-A-03). Archival SHALL be idempotent and SHALL verify the archive before deleting the source. Archive files SHALL be encrypted and replicated to a different region. Archived data SHALL remain retrievable for its full retention period (§ 31.8).

33.10 Vacuum and Bloat

Vacuum and bloat management governs how the platform reclaims space from deleted and updated rows. PostgreSQL's MVCC design retains old row versions (dead tuples) after UPDATE and DELETE operations, until VACUUM reclaims them. Without regular VACUUM, dead tuples accumulate (bloat), slowing queries (more pages to scan) and consuming disk space. The rules in this section govern

autovacuum configuration, manual VACUUM for high-churn tables, bloat monitoring, and index rebuild triggers.

33.10.1 Autovacuum Configuration

Autovacuum SHALL be enabled on every database (it is enabled by default; SHALL NOT be disabled). The default autovacuum thresholds (vacuum when 20% of rows plus 50 rows are dead; analyze when 10% of rows plus 200 rows are inserted/updated/deleted) are appropriate for typical tables; high-churn tables SHALL be configured with lower per-table thresholds (e.g., autovacuum_vacuum_scale_factor = 0.05 for audit_log) to vacuum more frequently. Autovacuum SHALL NOT be configured to consume more than 25% of CPU (the default autovacuum_vacuum_cost_limit and autovacuum_vacuum_cost_delay are appropriate); excessive autovacuum cost is a sign of excessive bloat, which SHALL be addressed at its cause (high-churn queries) rather than by tuning autovacuum more aggressively.

33.10.2 Manual VACUUM for High-Churn Tables

High-churn tables (tables with frequent UPDATE or DELETE) SHALL be monitored for bloat; if bloat exceeds 30%, a manual VACUUM (not VACUUM FULL, which takes an exclusive lock) SHALL be scheduled during a low-traffic window. VACUUM FULL SHALL be used only when bloat exceeds 50% AND a maintenance window is available, because VACUUM FULL takes an exclusive lock and rewrites the entire table. The platform SHALL prefer pg_repack (a third-party extension that rebuilds tables without exclusive locks) over VACUUM FULL where available.

33.10.3 Bloat Monitoring

Bloat SHALL be monitored via the pgstattuple extension (or equivalent), which reports the dead tuple fraction per table and per index. A bloat monitor SHALL run daily, record the bloat fraction for every table and index, and alert when bloat exceeds the documented thresholds (30% for tables; 30% for B-tree indexes; 50% for GIN/GiST indexes per § 33.2). Bloat trends SHALL be displayed on the database observability dashboard.

33.10.4 Index Rebuild Triggers

Indexes SHALL be rebuilt (REINDEX) when their bloat exceeds the documented thresholds (30% for B-tree, expression, partial, covering; 50% for GIN, GiST). REINDEX CONCURRENTLY SHALL be used (it does not take an exclusive lock); REINDEX (without CONCURRENTLY) SHALL be used only in maintenance windows. Index rebuilds SHALL be scheduled during low-traffic windows and SHALL be logged to the audit log. A query linter SHALL flag indexes that exceed their rebuild threshold for more than 7 days without a rebuild (a maintenance back-log).

RULE DB-010 Autovacuum SHALL be enabled on every database and SHALL NOT be disabled. High-churn tables SHALL have per-table autovacuum thresholds tuned for frequent vacuuming. Bloat SHALL be monitored daily; tables and indexes exceeding their rebuild threshold SHALL be rebuilt with REINDEX CONCURRENTLY (or pg_repack for tables) within 7 days. VACUUM FULL SHALL be used only

in maintenance windows when bloat exceeds 50%.

33.11 Backup and Recovery

Backup and recovery governs how the platform protects its data against loss. Backups are the ultimate defence against database failure, corruption, and operational error; without them, no other rule in this chapter matters. The rules in this section specify the recovery point objective (RPO), the recovery time objective (RTO), the backup strategy, point-in-time recovery, backup testing, off-site backup, and backup encryption.

33.11.1 Recovery Point Objective (RPO) and Recovery Time Objective (RTO)

The platform's RPO is 15 minutes: in the worst case of total database loss, the platform SHALL NOT lose more than 15 minutes of committed data. The platform's RTO is 1 hour: in the worst case of total database loss, the platform SHALL restore service within 1 hour of the loss being detected. These objectives are operational targets, not guarantees; the platform SHALL design its backup and recovery infrastructure to meet them and SHALL test recovery regularly to verify they are met.

33.11.2 Backup Strategy

The backup strategy SHALL combine daily full backups with continuous WAL (Write-Ahead Log) archiving. The daily full backup provides a base; the continuous WAL archive provides the deltas from the base to any point in time. WAL archiving SHALL be configured (archive_mode = on, archive_command = …) to ship every WAL segment to off-site storage immediately upon segment switch (typically every few minutes). Backups SHALL be retained per the storage and retention budget (§ 31.8): monthly full backups for 7 years.

33.11.3 Point-in-Time Recovery (PITR)

Point-in-time recovery SHALL be available: the platform SHALL be able to restore the database to any point in time within the retention period (at least 30 days of PITR; 7 years of monthly snapshots). PITR is performed by restoring the most recent daily full backup and replaying WAL segments up to the target time. PITR SHALL be tested quarterly (per backup testing, below) and SHALL be documented in a runbook that any on-call engineer can execute.

33.11.4 Backup Testing

Backups SHALL be tested quarterly by restoring a backup to a staging database and verifying that: the restore completes within the RTO; the restored database is consistent (no corruption); representative queries return the expected results; and point-in-time recovery to a specific timestamp succeeds. A backup that cannot be restored is not a backup; it is a file. Backup test results SHALL be recorded and reviewed at the monthly SLO review.

33.11.5 Off-Site Backup

Backups SHALL be stored off-site (in a different region or cloud provider than the primary database) to protect against regional disasters. Off-site backups SHALL be encrypted (per below) and SHALL be accessible only via credentials stored in the platform's secret management system. The platform SHALL test off-site restore annually (per backup testing, above) to verify that off-site backups are restorable.

33.11.6 Backup Encryption

Backups SHALL be encrypted at rest (the backup target's server-side encryption is sufficient for the daily full backup; WAL archives SHALL be encrypted client-side in addition, because they are shipped to off-site storage). Backup encryption keys SHALL be managed by the platform's key management system (Chapter 19) and SHALL be rotated per the key rotation policy. Backup encryption SHALL be verified during backup testing: a restore attempt with the wrong key SHALL fail.

RULE DB-011 The platform's RPO SHALL be 15 minutes and RTO SHALL be 1 hour. Backups SHALL combine daily full backups with continuous WAL archiving. Point-in-time recovery SHALL be available for at least 30 days. Backups SHALL be tested quarterly; off-site restore SHALL be tested annually. Backups SHALL be encrypted at rest and SHALL be stored off-site. Backup encryption keys SHALL be rotated per the key rotation policy.

33.12 Replication

Replication governs how the platform maintains copies of the database for availability, read scaling, and disaster recovery. PostgreSQL supports both physical (streaming) replication and logical replication; the platform uses physical replication for high-availability and read replicas, and logical replication for selective data distribution where applicable. The rules in this section govern the primary/replica topology, replication lag, read replica usage, failover, and split-brain prevention.

33.12.1 Primary/Replica Topology

The platform SHALL operate one primary and at least two replicas. The primary serves all writes; one replica serves as the high-availability failover target (synchronous replication to guarantee no data loss on failover); additional replicas serve read traffic (asynchronous replication to avoid write-latency impact). The topology SHALL be managed by a high-availability controller (e.g., Patroni, Stolon) that monitors primary health and orchestrates failover.

33.12.2 Replication Lag SLO

Replication lag SHALL be less than 1 second on the synchronous failover replica (this is a consequence of synchronous replication: the primary does not acknowledge a commit until the replica has acknowledged it). Replication lag on asynchronous read replicas SHALL be less than 5 seconds at p99; lag exceeding 5

seconds SHALL trigger a SEV-3 alert; lag exceeding 30 seconds SHALL trigger a SEV-2 alert. Replication lag SHALL be monitored continuously and displayed on the database observability dashboard.

33.12.3 Read Replica Usage Rules

Read replicas MAY serve read traffic that tolerates eventual consistency: search index building, analytics queries, OAI-PMH harvest responses, dashboard queries. Read replicas SHALL NOT serve reads that require strict consistency: publication state queries (a reader who just published SHALL see the new state, not a stale replica), audit log queries (the audit log SHALL reflect all committed state transitions), invariant monitor queries (the monitor SHALL see the latest state). Read replicas SHALL NOT accept writes; the application SHALL route all writes to the primary.

33.12.4 Failover Procedure

Failover SHALL be automated by the high-availability controller. On detection of primary failure (health check failure, network partition, primary crash), the controller SHALL promote the synchronous replica to primary, update the connection-pooler configuration to route traffic to the new primary, and alert on-call. Failover SHALL complete within 30 seconds of detection (contributing to the 1-hour RTO with substantial margin). The failover procedure SHALL be tested quarterly by deliberately failing the primary in staging and observing the failover.

33.12.5 Split-Brain Prevention

Split-brain (two primaries serving conflicting writes) SHALL be prevented by fencing: the HA controller SHALL ensure the old primary is fenced (powered off, network-isolated, or STONITH-applied) before promoting the new primary. The platform SHALL NOT use application-level fencing (e.g., a lock table) because application-level fencing is unreliable under network partitions. The HA controller's fencing configuration SHALL be tested quarterly as part of failover testing.

RULE DB-012 The platform SHALL operate one primary and at least two replicas. The failover replica SHALL use synchronous replication with <1 s lag. Read replicas SHALL use asynchronous replication with <5 s p99 lag. Read replicas SHALL NOT serve reads that require strict consistency (publication state, audit log, invariant monitors). Failover SHALL be automated, fenced, and tested quarterly. Split-brain SHALL be prevented by infrastructure-level fencing.

33.13 Connection Management

Connection management governs how the platform's application services establish, pool, and release database connections. Database connections are expensive: each connection consumes memory on the database server (typically 5–10 MB) and a process slot; PostgreSQL's default configuration supports a few hundred connections before performance degrades. The platform uses PgBouncer (a lightweight connection pooler) to multiplex many application connections onto fewer database connections.

33.13.1 Connection Pooling (PgBouncer)

Every application service SHALL connect to the database via PgBouncer, not directly. PgBouncer SHALL be configured in transaction-pooling mode (each transaction borrows a connection from the pool for the duration of the transaction, then returns it), which provides the highest connection multiplexing. Session-pooling mode SHALL be used only for services that require session-level state (e.g., temporary tables, SET commands that persist across transactions); such services SHALL be reviewed and documented.

33.13.2 Pool Sizes

The PgBouncer pool size per database SHALL be sized to the documented concurrency budget per service (§ 31.6: 100 concurrent DB transactions per service) plus 20% headroom (120 connections per database in the pool). The PostgreSQL max_connections SHALL be sized to the sum of all PgBouncer pool sizes plus a margin for administrative connections (typically 200–300 total). A pool size that is too small causes connection waits; a pool size that is too large causes database overload. Both SHALL be detected by monitoring connection wait time and database CPU.

33.13.3 Connection Lifecycle

Application services SHALL acquire connections lazily (only when a transaction begins) and SHALL release them eagerly (immediately after the transaction commits or rolls back). Services SHALL NOT hold connections across HTTP request boundaries (e.g., a service SHALL NOT acquire a connection at the start of a request and hold it for the duration of the request; it SHALL acquire per transaction). Services SHALL NOT hold connections across external-API calls (the connection would be idle during the API call, wasting pool capacity).

33.13.4 Idle Timeout

PgBouncer SHALL be configured with an idle timeout (server_idle_timeout) of 300 seconds: server connections idle for more than 5 minutes SHALL be closed. PostgreSQL SHALL be configured with an idle_in_transaction_session_timeout of 60 seconds: sessions idle in a transaction for more than 1 minute SHALL be aborted. Idle-in-transaction sessions hold locks and prevent vacuuming; they are a common cause of bloat and lock contention.

RULE DB-013 Every application service SHALL connect via PgBouncer in transaction-pooling mode. Pool sizes SHALL be sized to the documented concurrency budget plus 20% headroom. Connections SHALL be acquired lazily and released eagerly; SHALL NOT be held across HTTP request boundaries or external-API calls. Idle timeout SHALL be 300 seconds; idle-in-transaction timeout SHALL be 60 seconds.

33.14 Query Performance

Query performance governs how the platform analyzes, optimizes, and monitors database queries. A query that performs well in development may perform poorly in production due to differences in data volume, data distribution, and concurrent load. The rules in this section specify query analysis, EXPLAIN ANALYZE usage, N+1 detection, slow query logging, and query plan stability.

33.14.1 Query Analysis

Every query that runs in production SHALL be analyzed in staging with EXPLAIN ANALYZE before deployment. The analysis SHALL verify that the query uses the expected indexes (no unintended sequential scans on large tables), that the query's estimated cost is within the platform's query-cost budget (typically < 1000 for interactive queries), and that the query's actual execution time is within the documented latency budget (§ 31.1: 5 ms p50 / 20 ms p95 for simple queries; 50 ms p50 / 200 ms p95 for complex joins).

33.14.2 EXPLAIN ANALYZE Usage

EXPLAIN ANALYZE SHALL be used (not just EXPLAIN) because it executes the query and reports actual row counts and timings, which reveal cardinality estimation errors that EXPLAIN alone cannot. EXPLAIN ANALYZE SHALL be run on a staging database with production-sized data; running it on a development database with small data produces misleading plans. EXPLAIN (ANALYZE, BUFFERS) SHALL be used for queries where I/O is suspected (the BUFFERS option reports buffer hits vs. reads).

33.14.3 N+1 Detection

N+1 queries (a query that fetches a list, then issues one query per item to fetch related data) SHALL be detected and eliminated. The platform SHALL use a query linter that flags N+1 patterns in ORM-based code (e.g., Django's prefetch_related, SQLAlchemy's eager loading). The platform SHALL use database-level N+1 detection: a monitor that counts queries per request and alerts when the count exceeds 10 (a typical request should issue fewer than 10 queries; an N+1 pattern often issues dozens).

33.14.4 Slow Query Log Threshold

The slow query log threshold SHALL be 100 milliseconds: queries that take longer than 100 ms SHALL be logged with the query text (parameterized, not literal values), the execution time, and the calling application's correlation id. Slow queries SHALL be reviewed weekly by the owning team; recurring slow queries SHALL be optimized (by adding an index, rewriting the query, or denormalizing). The slow-query log SHALL be sampled (not every occurrence, but enough to identify patterns) to avoid log volume explosion.

33.14.5 Query Plan Stability

Query plans SHALL be stable: the same query SHALL produce the same plan across executions (assuming the same data distribution). Plan instability (the same query using different indexes on different executions) is a sign of statistics drift or parameter-sniffing. The platform SHALL monitor plan stability: a

monitor SHALL record the query plan for each query (sampled) and alert when the plan changes. Plan changes SHALL be investigated (typically by running ANALYZE on the affected tables, or by using query hints if ANALYZE does not resolve the issue).

RULE DB-014 Every production query SHALL be analyzed with EXPLAIN ANALYZE on production-sized data before deployment. The slow query log threshold SHALL be 100 ms; slow queries SHALL be reviewed weekly. N+1 queries SHALL be detected (by linter and by per-request query count monitor) and eliminated. Query plan stability SHALL be monitored; plan changes SHALL be investigated. The query linter SHALL flag unindexed scans on hot-path columns.

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

This chapter is the database constitution of Opus Publica. Its purpose is to convert the database from an implementation detail into a governed engineering surface, by specifying the naming conventions, index strategy, constraint strategy, foreign-key rules, migration rules, transaction rules, locking rules, partitioning strategy, archival strategy, vacuum and bloat management, backup and recovery, replication, connection management, and query performance rules that the database SHALL satisfy. The chapter is the authoritative reference for how the database stores the data whose logical structure Chapter 5 defines.

2. Scope

In scope: the PostgreSQL (or equivalent) database engine that stores the platform's mutable state, including its schema objects, indexes, constraints, transactions, locks, partitions, archives, backups, replicas, connections, and queries. Out of scope: the logical data model (Chapter 5); the canonical field registry (Chapter 6); the audit log's logical structure (Chapter 26 invariant INV-A-02); the search index (which is a separate engine, addressed in Chapter 4); the storage layer for published artifacts (Chapter 4, Chapter 13). Where this chapter and Chapter 5 both address a concern (e.g., migration), Chapter 5 governs the logical migration (entity/field changes) and this chapter governs the physical migration (schema/index changes).

3. Definitions

Schema: the structure of database objects (tables, columns, indexes, constraints, functions). Migration: a versioned change to the schema. Transaction: an atomic, consistent, isolated, durable unit of work. Lock: a database mechanism for isolating concurrent transactions. Partition: a physical subdivision of a table. Bloat: the fraction of a table or index occupied by dead tuples. RPO (recovery point objective): the

maximum permissible data loss on failure. RTO (recovery time objective): the maximum permissible service interruption on failure. PITR (point-in-time recovery): the ability to restore the database to any timestamp within the retention window.

4. Responsibilities

Ownership of the database is distributed across the engineering organization:

- **Chief Systems Architect:** Accountable for the database constitution and for refusing to suspend any database rule.
- **Head of Data Engineering:** Responsible for schema design, naming conventions, constraints, foreign keys, partitioning, and archival.
- **Head of Platform Engineering:** Responsible for migrations, transactions, locking, replication, connection management, and query performance.
- **Head of Observability:** Responsible for vacuum/bloat monitoring, slow-query logging, replication lag monitoring, and backup testing.
- **Database Administrator (DBA):** Responsible for day-to-day database operations, backup execution, failover testing, and partition maintenance.

5. Design Principles

Database rules are conservative (when in doubt, the rule is stricter rather than looser), enforceable (every rule has a schema constraint, query linter, or operational monitor that detects violation), and unsuspendable (no operational pressure justifies violation). Constraints are preferred over application-layer validation (constraints are enforced by the database engine and cannot be bypassed by application bugs). Migrations are preferred over direct schema changes (migrations are versioned, reviewed, and tested). Connection pooling is preferred over direct connections (pooling protects the database from connection exhaustion). Idempotency is preferred over exactly-once semantics (idempotency is achievable; exactly-once is not, without distributed transactions).

6. Engineering Requirements

Every database object SHALL conform to the naming conventions (§ 33.1). Every table SHALL have the mandatory indexes (§ 33.2.1). Every semantically required column SHALL have a NOT NULL constraint; every enumerable column SHALL have a CHECK constraint; every parent-child relationship SHALL have a FOREIGN KEY constraint (§ 33.3). Every migration SHALL be forward-compatible, zero-downtime, tested, and reversible (§ 33.5). Every transaction SHALL have a 5-second timeout and use the documented isolation level (§ 33.6). Every session SHALL have a 2-second lock timeout (§ 33.7). Backups SHALL meet the 15-minute RPO and 1-hour RTO (§ 33.11). Replication lag SHALL meet the documented SLO (§ 33.12). Slow queries (>100 ms) SHALL be logged and reviewed (§ 33.14).

7. Engineering Constraints

Database rules constrain every database-touching change. No schema change MAY be merged that violates a rule in this chapter. No query MAY be deployed that violates a rule in this chapter. No

operational configuration MAY be applied that violates a rule. The Chief Systems Architect SHALL NOT grant exceptions to database rules; the only path to changing a database rule is to amend this chapter through the Evolution Governance process (Chapter 36). Amendments that weaken a rule (e.g., loosening the lock timeout, relaxing the RPO) SHALL require a recorded engineering review and SHALL be signed off by the Chief Systems Architect personally.

8. Runtime Behaviour

At runtime, the database enforces schema constraints on every write; the connection pooler multiplexes application connections onto database connections; transactions begin and end per the outbox pattern; locks are acquired and released per the lock ordering convention; autovacuum reclaims dead tuples; partition maintenance creates new partitions in advance; backups run daily with continuous WAL archiving; replication ships WAL segments to replicas; the slow-query log captures queries exceeding 100 ms. The database observability dashboard aggregates these into a single real-time view. A monitor that fails to run is a SEV-2 incident (a blind spot).

9. Failure Conditions

Database failure conditions include:

- **Constraint violation:** an INSERT or UPDATE that violates a CHECK, NOT NULL, UNIQUE, or FOREIGN KEY constraint (the database refuses the write; the application SHALL handle the error).
- **Migration failure:** a migration that locks a hot table, fails to complete, or leaves the schema in an inconsistent state.
- **Lock contention:** transactions waiting on locks beyond the lock timeout, indicating contention that SHALL be investigated.
- **Bloat:** tables or indexes exceeding their bloat thresholds, indicating insufficient vacuuming or excessive churn.
- **Backup failure:** a backup that fails to complete, fails to verify, or cannot be restored in testing.
- **Replication lag:** asynchronous replica lag exceeding 5 seconds p99, indicating the replica cannot keep up with the primary.
- **Failover failure:** the HA controller fails to promote the replica on primary failure, or fails to fence the old primary (split-brain risk).
- **Slow query:** a query exceeding 100 ms, indicating a missing index, cardinality estimation error, or N+1 pattern.

10. Recovery Procedures

Database recovery follows the documented runbooks for each failure condition: constraint violations are handled by the application (typically by retrying with corrected data or failing the request with an RFC 7807 problem document); migration failures are handled by rolling back the migration per its documented rollback procedure; lock contention is handled by investigating the contending transactions and adjusting the lock ordering or transaction scope; bloat is handled by manual VACUUM or pg_repack; backup failures are handled by re-running the backup and investigating the cause; replication lag is

handled by investigating the slow WAL shipping or replica CPU; failover failures are handled by manual promotion and fencing; slow queries are handled by EXPLAIN ANALYZE and index/query optimization.

11. Security Considerations

Database security is governed by Chapter 19 (Security Constitution). This chapter adds database-specific security rules: connections SHALL be TLS-encrypted (Chapter 19 § S-... and per PostgreSQL's hostssl configuration); credentials SHALL be managed by the secret management system (Chapter 19) and rotated per the rotation policy; backup encryption keys SHALL be rotated per the key rotation policy; the database SHALL NOT be directly accessible from the public internet (only via the application services through PgBouncer); administrative access SHALL be via a bastion host with multi-factor authentication; all administrative actions SHALL be audited (per INV-A-01). Direct database access for ad-hoc queries SHALL be logged and reviewed.

12. Performance Considerations

Database performance is governed by the latency budgets in § 31.1 (5 ms p50 / 20 ms p95 for simple queries; 50 ms p50 / 200 ms p95 for complex joins) and by the query performance rules in § 33.14. The performance budget for database monitoring (autovacuum, bloat monitor, slow-query log, replication lag monitor) SHALL NOT exceed 5% of database CPU; monitoring that exceeds its budget SHALL be optimized (e.g., sampling instead of full scan). Connection pooling adds approximately 1 ms to each query's latency; this is the cost of multiplexing and is non-negotiable. Partitioning adds planning overhead per query (the planner must consider which partitions to scan); the overhead is typically <1 ms and is recovered by the reduced scan cost.

13. Observability Requirements

Every database SHALL expose metrics for: connection count (per database, per service); transaction count and duration; lock wait count and duration; bloat fraction (per table, per index); replication lag (per replica); backup status (last success, last failure, last test); slow query count (per minute, per query pattern); partition health (count, size, row estimate); autovacuum activity (jobs per hour, tables vacuumed). The database observability dashboard SHALL aggregate these into a single view organized by the fourteen sections of this chapter. A monitor in 'unknown' state is a SEV-2 incident (the platform is blind to that database health dimension).

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **All 14 database rules (DB-001 through DB-014) are documented with stable identifiers and enforced by schema constraints, query linters, or operational monitors.**
- **Every table conforms to the naming conventions and has the mandatory indexes (primary key, foreign keys, hot-path lookup columns).**
- **Every migration has been reviewed and has a documented rollback procedure.**
- **Backups have been tested within the prior quarter and meet the 15-minute RPO and 1-hour RTO.**

- **Failover has been tested within the prior quarter and completes within 30 seconds.**
- **The database observability dashboard is live and reporting all 14 sections' metrics.**
- **The Chief Systems Architect has signed off the database constitution.**

15. Runtime Verification

Runtime verification of the database constitution is the database observability dashboard itself: the dashboard is the evidence that constraints are enforced, migrations are reviewed, transactions are bounded, locks are monitored, partitions are maintained, bloat is controlled, backups are tested, replication is healthy, connections are pooled, and queries are performant. The Release Certification Authority SHALL verify that all 14 rules (DB-001 through DB-014) have live enforcement gates, that the dashboard is live, and that at least one chaos exercise per rule has been completed (constraint violation, migration failure, lock contention, bloat threshold, backup restore, failover, slow query) before RC1 sign-off.

16. Certification Requirements

RC1 certification requires: the database constitution published; all 14 rules operational; the database observability dashboard live; backup test results within the prior quarter; failover test results within the prior quarter; at least one chaos exercise per rule completed and postmortemed (14 exercises total); the query linter operational and integrated into CI; the migration review process documented and followed; and the Chief Systems Architect's sign-off that the rule set is complete. A rule without an enforcement gate is not certifiable; it is a defect that blocks RC1.

17. Future Evolution

The database rule set evolves through the Evolution Governance process (Chapter 36). New rules MAY be added when a new database feature or pattern is introduced (e.g., a rule governing logical replication if the platform adopts it). Existing rules MAY be tightened (stricter thresholds, broader applicability) but SHALL NOT be loosened without evidence that the looser rule still preserves the platform's scholarly-record guarantees. A rule SHALL NOT be removed; if a rule is genuinely obsolete (the feature it governed is no longer used), it SHALL be marked deprecated with a recorded reason and a successor rule (or a documented argument that no successor is needed). Database engine upgrades SHALL be reviewed for rule impact: a new PostgreSQL major version MAY introduce features that warrant new rules or revisions to existing rules.

Chapter 34 — Operational Governance

This chapter constitutes the Operational Governance Charter of Opus Publica. It defines who, after the platform is deployed, is authorized to perform each operational action that touches production state, the scholarly record, or the platform's external commitments. Where the Constitution's other chapters specify what the platform MUST do and how the platform MUST be built, this chapter specifies who is permitted to act, under whose approval, against what audit, and within what boundary. The chapter exists because the platform, once live, is no longer an engineering artifact but a public-trust obligation: every publication is a promise to a researcher, every DOI a permanent handle, every preservation deposit a covenant with future readers, and every credential a key into the world's scholarly infrastructure. The authority to act on such a system is not granted by employment, by tenure, or by technical access; it is granted by this chapter and recorded in the audit log.

The chapter is normative for every operational action that the platform permits, including those performed by automation. Automated systems that act on production state (continuous deployment, automated rollback, scheduled key rotation, certificate renewal) are subject to the same authority matrix as human operators: the automation itself holds a delegated authority that is bounded, audited, and revocable. Where an automated action would exceed its delegated authority, the automation SHALL refuse the action and SHALL escalate to the human role that holds the underlying authority. The Chief Systems Architect is the final authority for any operational question this chapter does not resolve; the Chief Systems Architect SHALL NOT delegate this final-authority role.

The Constitutional Status of Operational Governance

Operational governance is the bridge between the Constitution's engineering contracts and the platform's daily reality. An engineer may build a feature perfectly; an operator may break it in one operational action. The authority matrix in this chapter exists so that no single human — including the Chief Systems Architect — can, by an unrecorded action, corrupt the scholarly record, expose a credential, retract a publication without cause, or suspend an invariant. Every operational action with lasting consequence SHALL require at least two parties: an actor and an approver, both named, both recorded, both accountable.

34.1 Operational Roles

Operational authority is vested in named roles, not in named individuals. A role persists across personnel changes; an individual is granted a role through a recorded appointment and is removed from a role through a recorded revocation. The nine roles defined in this section are the complete set of operational roles recognized by this Constitution; any operational action not authorized by one of these roles is, by definition, unauthorized. The table below enumerates each role, the scope of its authority, the boundary of its authority, and its on-call posture.

Role	Authority	Boundaries	On-Call
Duty Editor	Editorial authority for in-flight submissions; co-signs publication events; declares editorial incidents.	Shall not modify published artifacts; shall not rotate credentials; shall not approve deployments.	Business hours; escalates to EiC after-hours.
Duty Engineer (on-call)	First responder to production alerts; executes runbooks; may invoke break-glass under Rule OPS-007.	Shall not authorize deployments, migrations, or retractions; shall not modify audit logs.	24x7 rotation; primary + secondary.
Release Manager	Owns the release train; schedules deployment windows; communicates release status.	Shall not approve production deployment without RCA sign-off; shall not approve rollback.	Business hours; on-call during release windows.
Release Certification Authority	Certifies that a release candidate satisfies the Production Acceptance Gate (Chapter 22); signs or refuses the release certificate.	Shall not approve a release that has failed Runtime Certification; the Chief Systems Architect shall not override a RCA refusal.	Business hours; on-call during release windows.
Database Administrator	Owns database migrations, schema changes, backup integrity, and replication health.	Shall not execute migrations without DBA-Migration approval (Section 34.5); shall not modify published-artifact tables in place.	Business hours; on-call for SEV-1 database incidents.
Security Officer	Owns credential inventory, rotation cadence, break-glass review, and security incident response.	Shall not approve production deployment; shall not modify audit logs; shall rotate external credentials per Section 34.3 only with two-party approval.	24x7 on-call for security incidents.
Audit Officer	Owns audit-log integrity, cryptographic-chain verification, and audit-record retention.	Shall not modify audit records (forbidden by INV-A-02); shall not approve any operational action.	Business hours; on-call for SEV-1 audit-chain incidents.
Disaster Recovery Officer	Owns the DR plan, DR test cadence, DR activation authority (with Chief Systems Architect co-signature), and DR runbook	Shall not activate DR without Chief Systems Architect co-signature (except in P0 incidents where the platform is	Business hours; on-call for DR activation.

	library.	down).	
Chief Systems Architect	Final operational authority; co-signs invariant-related actions, legal removals, DR activations; amends operational governance; resolves constitutional conflicts.	Shall not suspend invariants; shall not override a RCA refusal; shall not unilaterally amend invariants.	Always reachable; final escalation.

Every appointment to one of these roles SHALL be recorded in the audit log with the appointee's identity, the role, the appointing authority, the appointment timestamp, and the revocation timestamp when the appointment ends. An individual SHALL NOT hold two roles whose authorities conflict — for example, the Release Manager and the Release Certification Authority SHALL NOT be the same individual, and the Security Officer and the Audit Officer SHALL NOT be the same individual, because the latter pair enforces two-party control on the credential and audit subsystems.

34.2 Action Authority Matrix

The action authority matrix is the normative table that defines, for every operational action with lasting consequence, the role that holds authority, the role that must approve (where approval is required), and the audit record that SHALL be produced. The matrix is exhaustive: any action not listed is, by default, forbidden until added by amendment. The matrix is also binding: the platform's identity-and-access-management layer SHALL enforce the matrix at the API boundary, refusing any action for which the caller does not hold the required role, and refusing any action for which the required approval has not been recorded.

Action	Authority Required	Approval	Audit
Press Publish (publication event)	Duty Editor	Co-signed by Release Certification Authority (publication is a release).	PublicationCompleted event; DOI minting; Crossref deposit; preservation queue commit.
Issue Retraction	Editor-in-Chief + Duty Editor	Chief Systems Architect co-signature; retraction notice authored and reviewed.	RetractionIssued event; Crossref retraction metadata deposit; preservation deposit of retraction notice.
Issue Correction	Duty Editor	Editor-in-Chief informed; correction notice authored and reviewed.	CorrectionIssued event; Crossref correction metadata deposit; new version DOI minted.
Rotate External Credential (Crossref/ORCID/preservation)	Security Officer	Two-party approval: Security Officer + Database Administrator or Chief	CredentialRotated event; old credential revoked; new credential recorded; last-

		Systems Architect.	rotation timestamp updated.
Rotate TLS Certificate	Security Officer	Automated via ACME; manual approval required only on ACME failure.	CertificateRotated event; new certificate fingerprint; old certificate revoked after grace period.
Approve Database Migration	Database Administrator	Chief Systems Architect approval for breaking migrations; Release Certification Authority for migrations coupled to a release.	MigrationApproved event; migration plan; rollback plan; verification criteria.
Approve Production Deployment	Release Certification Authority	Co-signed by Release Manager; RCA certificate referenced.	DeploymentApproved event; release certificate; deployment window; communication record.
Approve Rollback	Release Manager	Chief Systems Architect informed within 15 minutes; RCA post-mortem within 24 hours.	RollbackApproved event; reason; target release; verification evidence.
Initiate Disaster Recovery	Disaster Recovery Officer	Chief Systems Architect co-signature (or P0 self-authorization when platform is down, retroactively ratified).	DRActivated event; DR plan version; activation reason; Chief Systems Architect notification record.
Declare Incident	Duty Engineer (on-call) or any role	Incident Commander appointed within 15 minutes of declaration.	IncidentDeclared event; severity; commander; war-room channel; timeline opened.
Grant Break-Glass Access	Security Officer	Chief Systems Architect post-hoc review within 4 hours; Duty Engineer justification at grant time.	BreakGlassGranted event; grantee; scope; duration; reason; revocation timestamp.
Modify Audit Log	FORBIDDEN	No approval path. Audit log is append-only by INV-A-02.	Any attempt SHALL be logged as a SEV-1 incident and SHALL trigger immediate Security Officer notification.
Delete Published Artifact	FORBIDDEN except via Legal Removal Process (Section 34.11)	Chief Systems Architect approval + legal request on file.	LegalRemovalExecuted event; cryptographic proof of prior existence; DOI metadata update.
Override Invariant	FORBIDDEN	No approval path. The Chief Systems Architect SHALL NOT suspend an invariant.	Any attempt SHALL be logged as a SEV-1 incident; the platform SHALL halt

			engineering activity until the invariant is restored.

RULE OPS-001 No operational action with lasting consequence SHALL be performed without the authority, approval, and audit record specified in the action authority matrix. The platform's IAM layer SHALL enforce the matrix at the API boundary.

34.3 Key and Credential Rotation

Every external credential the platform holds — Crossref deposit credentials, ORCID member API credentials, CLOCKSS/LOCKSS/Portico deposit credentials, TLS private keys, database passwords, signing keys, OIDC client secrets, third-party API keys — SHALL have a recorded rotation policy that defines the rotation cadence, the rotation procedure, the emergency-rotation procedure, and the audit record produced. A credential without a rotation policy is a violation of invariant INV-V-03 (Credentials Rotated Per Policy) and SHALL be detected by the nightly credential-inventory scan.

34.3.1 Rotation Cadence

Credential Class	Cadence	Owner	Notes
Crossref deposit credentials	180 days	Security Officer	Rotate via Crossref member portal; update platform secret store; verify deposit still succeeds.
ORCID member API credentials	180 days	Security Officer	Rotate via ORCID member console; update platform secret store; verify ORCID public API link still resolves.
CLOCKSS / LOCKSS / Portico deposit credentials	365 days	Security Officer	Rotate via each preservation service's portal; verify test deposit after rotation.
TLS private keys (web)	90 days	Security Officer (automated via ACME)	Automated renewal; manual rotation on ACME failure.
Database passwords (application)	90 days	Database Administrator	Rotate via secret store; rolling restart of application pods; verify no stale connections.
Database passwords (administrative)	30 days	Database Administrator	Manual rotation; recorded in audit log; verified by post-rotation access test.
Signing keys (JWT, webhooks)	90 days	Security Officer	Key rotation with overlapping validity; old key

			valid for grace period to prevent in-flight failures.
OIDC client secrets	180 days	Security Officer	Rotate via identity provider; update platform secret store; verify authentication still succeeds.
Third-party API keys (Resend, analytics, etc.)	180 days	Security Officer	Rotate per provider; update secret store; verify integration post-rotation.
Backup encryption keys	365 days	Disaster Recovery Officer	Rotate via key-management service; re-encrypt backups on next backup cycle; retain old keys for decrypt of historical backups.

34.3.2 Rotation Procedure

Routine rotation SHALL follow the dual-key procedure: the new credential is generated, deployed, and verified before the old credential is revoked. The grace period between deployment and revocation SHALL be at least 24 hours for routine rotations and at least 72 hours for credentials that may be cached by external systems (e.g., Crossref deposit credentials cached by upstream proxies). The rotation SHALL be recorded in the audit log with the credential class, the rotation timestamp, the rotator's identity, the approver's identity, and the verification result.

34.3.3 Emergency Rotation

Emergency rotation — invoked on suspected or confirmed credential compromise — follows the inverse procedure: the old credential is revoked first (to stop ongoing abuse), and the new credential is generated and deployed as rapidly as possible. Emergency rotation SHALL be initiated by the Security Officer with immediate notification to the Chief Systems Architect; post-hoc ratification by the Chief Systems Architect is REQUIRED within 4 hours. The platform SHALL monitor for failed authentications using the revoked credential and SHALL alert on any such attempt (which indicates either a stale consumer or a continuing attacker).

34.3.4 Rotation Audit

Every rotation — routine or emergency — SHALL produce an audit record of class CredentialRotated. The record SHALL include the credential class, the rotation type (routine/emergency), the rotator, the approver (or the post-hoc ratifier and ratification timestamp for emergency rotations), the old credential fingerprint (not the credential itself), the new credential fingerprint, the verification result, and the grace-period end timestamp. The Audit Officer SHALL reconcile the rotation audit against the credential inventory monthly and SHALL flag any credential whose last-rotation timestamp exceeds its cadence.

34.4 Certificate Management

TLS certificates are the platform's first-line trust signal to readers, authors, editors, and external integrators. A reader whose browser warns of an invalid certificate interprets the platform as untrustworthy, and rightly so. Certificate management is therefore not an operational detail but a constituent of the platform's public-trust posture. This section defines the certificate lifecycle, the issuance and renewal mechanism, the expired-certificate incident response, and the audit obligations.

34.4.1 TLS Certificate Lifecycle

Every TLS certificate SHALL pass through five lifecycle states: Requested, Issued, Active, Renewing, and Revoked (or Expired). The platform SHALL maintain a certificate inventory that records the current state of every certificate, the issuing authority, the validity period, the renewal mechanism, and the responsible owner. A certificate SHALL NOT reach the Active state without a recorded renewal mechanism; a certificate in the Active state without a renewal mechanism is a violation indicating either a manual-issuance process that was not migrated to ACME or a certificate procured outside the standard procedure.

34.4.2 Issuance and Renewal Mechanism

The platform SHALL issue and renew TLS certificates via the ACME protocol (RFC 8555) using Let's Encrypt as the default certificate authority. Where regulatory or contractual requirements mandate a different CA (e.g., a private CA for internal services, or an EV certificate for a specific tenant), the alternative CA SHALL be recorded in the certificate inventory with the rationale and the renewal mechanism. Certificate renewal SHALL be automated; a certificate that cannot be renewed automatically SHALL be flagged for migration to ACME or for documented manual renewal with calendar alerts 60, 30, 14, 7, and 1 day before expiry.

34.4.3 Renewal Automation

Renewal automation SHALL attempt renewal at 30 days before expiry (for 90-day Let's Encrypt certificates) or at 60 days before expiry (for 1-year certificates). A renewal failure SHALL trigger a SEV-2 incident and SHALL alert the Security Officer and the Duty Engineer. Three renewal attempts SHALL be made at 12-hour intervals before manual rotation is invoked. The renewal-automation service SHALL itself be monitored (a meta-monitor that checks the renewal service's liveness); a renewal service that has not run in 24 hours is a SEV-2 incident independent of any specific certificate's expiry.

34.4.4 Expired-Certificate Incident Response

An expired certificate is a SEV-1 incident. The Duty Engineer SHALL be paged immediately; the Security Officer SHALL be notified within 15 minutes; the Chief Systems Architect SHALL be notified within 1 hour. The incident SHALL be declared via the incident-command procedure (Section 34.10), an Incident Commander SHALL be appointed, and the war-room channel SHALL be opened. The platform SHALL serve a banner to all authenticated users acknowledging the incident (readers receive a TLS warning directly from their browsers; authors, editors, and integrators require proactive communication). The

postmortem SHALL examine why the renewal automation failed, why the meta-monitor did not catch the failure earlier, and what rotation of the affected credential classes is required (an expired certificate may indicate broader automation failure).

34.5 Migration Approval

A database migration is a change to the platform's data schema that affects how data is stored, indexed, or related. Migrations carry unique risk: a migration that fails mid-execution can leave the database in an inconsistent state, can break running application code, and can corrupt the scholarly record if the migration touches published-artifact tables. This section defines the migration-approval procedure that SHALL be followed for every migration, with no exception for "small" or "safe" migrations — the procedure is uniform because the consequence of a "small" migration gone wrong is the same as for a large one.

34.5.1 Migration Proposal

Every migration SHALL begin with a written proposal that describes: the schema change; the migration type (additive, non-destructive, breaking); the affected tables; the data volumes; the expected duration; the rollback procedure; the verification criteria; and the migration owner. The proposal SHALL be reviewed by the Database Administrator and, for breaking migrations, by the Chief Systems Architect.

34.5.2 Migration Review and Approval

The Database Administrator SHALL review the proposal for technical soundness (index impact, lock behaviour, replication lag, backup impact). Breaking migrations — those that remove columns, change column types, or restructure tables such that rollback is non-trivial — SHALL additionally be approved by the Chief Systems Architect. The Release Certification Authority SHALL be informed of any migration coupled to a release; the migration SHALL be certified as part of the release certificate.

34.5.3 Migration Scheduling and Execution

Migrations SHALL be scheduled during defined maintenance windows outside peak reader traffic (typically 02:00-05:00 UTC for the platform's primary reader timezone). The migration SHALL be executed by the Database Administrator with the migration owner present. The platform SHALL be in a read-only or maintenance-banner state during the migration for any path that touches the affected tables. The migration SHALL produce an audit record of class MigrationExecuted with the migration identifier, the start and end timestamps, the executor, the outcome, and the verification evidence.

34.5.4 Migration Verification and Rollback

Post-migration verification SHALL run the migration's defined verification criteria (row counts, index health, sample-query results, application smoke tests). If verification fails, the Database Administrator SHALL execute the rollback procedure immediately; rollback SHALL itself be verified. A migration that cannot be rolled back SHALL NOT be attempted; the proposal SHALL include a rollback procedure that has been tested in staging on a copy of production data.

34.6 Production Deployment Approval

Production deployment approval is the operational realization of the Production Acceptance Gate (Foundational Principle II). A deployment that has not been approved per this section is, by definition, unauthorized; the platform's deployment automation SHALL refuse to deploy without the recorded approval. The approval is multi-party: the Release Certification Authority certifies the release; the Release Manager schedules and communicates; the Chief Systems Architect is informed of any deployment that touches invariant-enforcing code paths.

34.6.1 Deployment Ticket

Every deployment SHALL begin with a deployment ticket that references: the release candidate identifier; the release certificate signed by the Release Certification Authority; the list of changes (commit range or pull-request list); the rollback plan; the deployment window; the on-call engineer; and the communication plan. The ticket SHALL be created by the Release Manager and SHALL be visible to all engineering staff.

34.6.2 Certification Artifacts Review

The Release Certification Authority SHALL review the certification artifacts referenced by the deployment ticket: the test evidence; the runtime evidence; the invariant monitor status (all monitors SHALL be passing); the security scan results; the performance test results (where applicable). A deployment that lacks any required certification artifact SHALL NOT be approved; the gap SHALL be remediated before the deployment proceeds.

34.6.3 RCA Sign-Off

The Release Certification Authority SHALL sign the release certificate, attesting that the release satisfies the Production Acceptance Gate. The sign-off SHALL be recorded in the audit log with the certificate identifier, the RCA's identity, the timestamp, and the list of certification artifacts reviewed. The Chief Systems Architect SHALL NOT override a RCA refusal; the refusal stands until the failure is corrected.

34.6.4 Deployment Window and Communication

The deployment window SHALL be scheduled outside peak reader traffic and outside any scheduled editorial deadline (e.g., a special-issue publication date). The Release Manager SHALL communicate the deployment window to engineering, editorial, and (where applicable) external integrators at least 24 hours in advance. During the deployment, the platform SHALL display a maintenance banner to authenticated users; readers SHALL see no disruption (deployment SHALL be zero-downtime for read-only paths).

34.7 Rollback Authority and Procedure

Rollback is the controlled reversal of a production deployment to a previously-known-good state. Rollback is a normal operational action, not a failure; the platform's deployment automation SHALL

support rollback as a first-class operation. The authority to rollback is vested in the Release Manager, with the Chief Systems Architect informed within 15 minutes and a post-incident review within 24 hours.

34.7.1 When to Rollback

Rollback SHALL be invoked when: (a) a SEV-1 incident is attributed to the current deployment; (b) an invariant monitor begins failing after a deployment; (c) error rate exceeds the rollback threshold (default: 5% of requests over a 5-minute window post-deployment); (d) the Duty Engineer declares that the deployment is unstable. Rollback is the default action when in doubt; a deployment that survives 24 hours without rollback is considered stable.

34.7.2 How to Rollback

Rollback SHALL be executed via the platform's deployment automation, which SHALL redeploy the previous release candidate. Manual rollback (e.g., direct database changes to revert schema) is forbidden except as a last resort when automated rollback fails; such manual rollback SHALL be authorized by the Database Administrator and recorded in the audit log. The rollback procedure SHALL be idempotent: a rollback of an already-rolled-back deployment SHALL be a no-op.

34.7.3 Post-Rollback Verification

After rollback, the Duty Engineer SHALL verify that: the previous release candidate is serving traffic; all invariant monitors are passing; error rate has returned to baseline; the publication path (if rollback touched it) functions correctly; and external integrations (Crossref, ORCID, preservation) still operate. Verification SHALL be recorded in the audit log.

34.7.4 Rollback Audit

Every rollback SHALL produce an audit record of class RollbackExecuted with: the deployment being rolled back; the target release; the reason; the executor; the verification evidence; and the post-incident review identifier. A post-incident review SHALL be conducted within 24 hours, shall identify the root cause of the failure that triggered rollback, and SHALL produce remediation actions to prevent recurrence.

34.8 Break-Glass Procedures

Break-glass access is the emergency grant of production access to an engineer who does not normally hold it, invoked when the platform is failing in a way that the on-call rotation cannot resolve. Break-glass is a controlled exception to the principle of least privilege; it is permitted because the alternative — leaving the platform down while waiting for the right role-holder — is worse. Break-glass is time-boxed, audited, post-hoc reviewed, and revoked.

34.8.1 Emergency Production Access

Break-glass access SHALL be granted by the Security Officer (or, in the Security Officer's absence, by the Duty Engineer with immediate Security Officer notification). The grant SHALL specify: the grantee; the

scope (which systems, which permissions); the duration (default: 2 hours, maximum: 4 hours); the reason; and the incident identifier. Break-glass access SHALL NOT grant authority to perform actions forbidden by the action authority matrix (e.g., modify audit log, delete published artifact, override invariant).

34.8.2 Time-Boxed, Audited, Post-Incident Review

Break-glass access SHALL expire automatically at the end of its duration; the platform's IAM layer SHALL revoke the grant without human intervention. All actions taken under break-glass SHALL be logged with elevated audit detail (every command, every query, every state change). Within 4 hours of grant, the Chief Systems Architect SHALL review the break-glass grant and SHALL either ratify it (if the emergency justified it) or refer it for security review (if the grant was unjustified). A pattern of frequent break-glass use by the same engineer indicates a role-coverage gap and SHALL trigger an operational-governance review.

34.8.3 Revocation

Break-glass access SHALL be revoked at the end of its duration, at the end of the incident (whichever is earlier), or on demand by the Chief Systems Architect. Revocation SHALL be recorded in the audit log. After revocation, the grantee's normal access SHALL be unchanged (break-glass does not modify normal access); the audit trail of the break-glass session SHALL be preserved indefinitely.

34.9 Disaster Recovery Ownership

Disaster recovery is the controlled restoration of the platform after a catastrophic event: regional outage, data loss, ransomware, insider attack, or vendor failure. DR is owned by the Disaster Recovery Officer, who maintains the DR plan, schedules DR tests, holds DR activation authority (with Chief Systems Architect co-signature), and curates the DR runbook library. DR is the platform's ultimate recovery mechanism; the platform's preservation deposits (CLOCKSS, LOCKSS, Portico) are the platform's ultimate recovery source for the scholarly record.

34.9.1 DR Plan Ownership

The DR plan SHALL be a version-controlled document owned by the Disaster Recovery Officer. The plan SHALL define: recovery objectives (RTO and RPO per service tier); recovery procedures per failure scenario; the DR runbook library; the DR test cadence; the DR activation authority; and the DR communication plan. The plan SHALL be reviewed quarterly and SHALL be amended whenever the platform's architecture, infrastructure, or external dependencies change materially.

34.9.2 DR Test Cadence

DR SHALL be tested quarterly. Tests SHALL alternate between tabletop exercises (where the DR plan is walked through without executing failover) and full failover exercises (where the platform is actually failed over to the DR environment and back). Each test SHALL produce a test report identifying what worked, what failed, and what remediation is required. A DR test that fails SHALL trigger a remediation

plan with a defined deadline; failure to remediate within the deadline SHALL escalate to the Chief Systems Architect.

34.9.3 DR Activation Authority

DR activation SHALL be authorized by the Disaster Recovery Officer with Chief Systems Architect co-signature. In a P0 incident where the platform is down and the Chief Systems Architect is not reachable within 30 minutes, the Disaster Recovery Officer MAY self-authorize activation and SHALL obtain Chief Systems Architect ratification post-hoc within 4 hours. The DR activation SHALL be recorded in the audit log with the activation timestamp, the activator, the reason, the DR plan version, and the ratification status.

34.9.4 DR Runbook Library

The DR runbook library SHALL contain, at minimum: regional-outage failover; database restoration from backup; storage restoration from preservation deposits; credential mass-rotation after compromise; DNS failover; communication templates for authors, editors, readers, and integrators. Each runbook SHALL be tested at least annually; an untested runbook is not a runbook, it is a hypothesis.

34.10 Incident Command

Incident command is the structured response to a production incident. The model adopted by Opus Publica is the Incident Command System (ICS) as adapted for site reliability engineering by Google SRE. The model is role-based (not person-based), severity-driven, and communication-first: the first priority of incident command is to establish and maintain a communication channel, not to fix the issue.

34.10.1 Incident Commander Role

The Incident Commander (IC) is the single authority during an incident. The IC is not the person who fixes the issue; the IC is the person who coordinates the response: assigns roles, tracks the timeline, communicates with stakeholders, and decides when to escalate. The IC role SHALL be appointed within 15 minutes of incident declaration; the default IC is the Duty Engineer, who MAY hand off to a more senior engineer as the incident warrants. The IC SHALL NOT also be the engineer performing the technical remediation (single-tasking is unsafe during incidents).

34.10.2 Severity Classification

Severity	Definition	Response	Communication
SEV-1 / P0	Platform down; scholarly-record corruption in progress; data loss.	Immediate page; IC appointed within 15 minutes; Chief Systems Architect notified within 1 hour.	Public status page updated within 15 minutes; updates every 30 minutes until resolved.
SEV-2 / P1	Significant degradation; invariant monitor failing;	Page on-call; IC appointed within 30 minutes;	Public status page updated within 30 minutes; updates

	scholarly record at risk.	engineering manager notified.	hourly.
SEV-3 / P2	Minor degradation; no immediate record risk.	Ticket assigned to owning team; resolved within 1 business day.	Internal communication only; status page updated if user-visible.
SEV-4 / P3	Anomaly; no user impact.	Ticket; resolved within 1 week.	Internal only.

34.10.3 War Room and Communication

Every SEV-1 and SEV-2 incident SHALL have a dedicated war-room channel (chat or video) opened within 15 minutes of declaration. The IC SHALL maintain the incident timeline (every action, every observation, every decision, timestamped). The IC SHALL communicate to stakeholders at the cadence defined for the severity. After resolution, the IC SHALL schedule a postmortem within 5 business days; the postmortem SHALL be blameless (focused on systemic causes, not individual fault) and SHALL produce remediation actions with owners and deadlines.

34.10.4 Postmortem Ownership

Postmortems are owned by the IC of the incident, reviewed by the Engineering Leadership Team, and published to the engineering organization. A postmortem SHALL NOT be closed until all remediation actions are complete or have been explicitly descoped with Chief Systems Architect approval. Postmortems SHALL be cited in subsequent incident responses (if a similar incident recurs, the prior postmortem SHALL be referenced and the gap in remediation SHALL be identified).

34.11 Legal Removal Process

The legal removal process is the ONLY path by which a published artifact MAY be removed from the platform. It is deliberately narrow, deliberately slow, and deliberately audited, because removal of a published artifact is the single most consequential operational action the platform permits short of disaster recovery. A published artifact is permanent (INV-M-01); removal is an exception to permanence, and exceptions require governance.

INV-M-01 reaffirmed

Once an article version reaches the Published state, its canonical record, JATS XML, published PDF, canonical HTML, and supplements are immutable. Removal of a published artifact is forbidden except via the legal removal process defined in this section. Any other removal path is a violation of INV-M-01 and is a SEV-1 incident.

34.11.1 Legal Request

A legal removal SHALL begin with a written legal request from a competent authority: a court order, a DMCA takedown notice that survives counter-notification, a GDPR erasure request that the platform's counsel has determined is compliant and applicable to a published scholarly work, or an equivalent legal instrument. The request SHALL be received by the Chief Systems Architect, logged in the audit log, and reviewed by the platform's legal counsel before any operational action is taken.

34.11.2 Chief Systems Architect Approval

The Chief Systems Architect SHALL personally approve every legal removal. Approval SHALL be recorded in the audit log with: the legal request reference; the counsel's review reference; the artifact being removed; the removal scope (full removal vs. metadata-only takedown); and the cryptographic proof of preservation. The Chief Systems Architect SHALL NOT delegate this approval.

34.11.3 Cryptographic Proof Preservation

Removal SHALL NOT destroy the cryptographic proof that the artifact existed. The SHA-256 content address of the removed artifact, the publication timestamp, the DOI, and the deposit receipts (Crossref, preservation) SHALL be retained indefinitely. The removed artifact itself SHALL be moved to a quarantined, access-restricted storage class (not deleted from storage), retrievable only under Chief Systems Architect authority with a second approver.

34.11.4 DOI Metadata Update

The DOI of a removed artifact SHALL NOT be deleted (INV-M-03 forbids DOI reassignment or deletion). The DOI metadata SHALL be updated with Crossref to reflect the removal: the artifact's landing page SHALL display a removal notice citing the legal basis (without disclosing the underlying content), and Crossref's metadata SHALL reflect the removed status. The DOI SHALL continue to resolve to the removal-notice landing page indefinitely.

34.11.5 Audit Record

Every legal removal SHALL produce an audit record of class LegalRemovalExecuted with: the legal request reference; the Chief Systems Architect approval reference; the artifact identifier; the SHA-256 of the removed artifact; the removal timestamp; the Crossref metadata update reference; the preservation-deposit references (which remain valid); and the audit-chain hash. The audit record SHALL be preserved indefinitely and SHALL be the authoritative evidence that the removal was lawful.

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

This chapter defines the operational governance of Opus Publica after deployment: the roles that hold operational authority, the action authority matrix that bounds each operational action, the procedures for credential rotation, certificate management, migration approval, deployment approval, rollback, break-glass, disaster recovery, incident command, and legal removal. The chapter is the authoritative reference for who is permitted to act on production state, under whose approval, against what audit.

2. Scope

In scope: the nine operational roles; the action authority matrix; key and credential rotation; certificate management; migration approval; deployment approval; rollback; break-glass; disaster recovery; incident command; legal removal. Out of scope: the engineering pipeline that produces a release candidate (Chapter 22), the invariants the platform enforces (Chapter 26), the cost model of operations (Chapter 35), and the amendment of this chapter (Chapter 36).

3. Definitions

Key terms used in this chapter are defined as follows:

Term	Definition
Operational action	Any action that modifies production state, the scholarly record, or the platform's external commitments.
Authority	The role permitted to perform an operational action; granted by this chapter, enforced by the IAM layer.
Approval	A second role's recorded consent to an operational action; required for actions with elevated consequence.
Audit	The record produced by an operational action, preserved indefinitely in the append-only audit log.
Break-glass	Time-boxed emergency grant of production access to an engineer who does not normally hold it.
Rollback	Controlled reversal of a production deployment to a previously-known-good state.
Legal removal	The only path by which a published artifact MAY be removed from the platform; governed by Section 34.11.

4. Responsibilities

Ownership of operational governance is distributed across the engineering organization:

- **Chief Systems Architect:** Final authority; co-signs DR activations, legal removals, and breaking migrations; resolves operational-governance questions.
- **Release Certification Authority:** Certifies release candidates; refuses releases that fail Runtime Certification.
- **Release Manager:** Owns the release train; schedules deployment windows; approves rollback.
- **Database Administrator:** Owns database migrations; approves migration plans; executes migrations.
- **Security Officer:** Owns credential inventory; rotates credentials per cadence; grants and reviews break-glass.
- **Audit Officer:** Owns audit-log integrity; reconciles rotation audit; flags audit-chain anomalies.
- **Disaster Recovery Officer:** Owns the DR plan; schedules DR tests; activates DR with Chief Systems Architect co-signature.
- **Duty Editor:** Editorial authority for in-flight submissions; co-signs publication events.
- **Duty Engineer:** First responder to production alerts; executes runbooks; may invoke break-glass.

5. Design Principles

Operational governance follows five principles: (1) role-based authority, not individual-based authority; (2) two-party control for actions with elevated consequence; (3) audit-everything, modify-nothing (audit log is append-only); (4) automation subject to the same authority matrix as humans; (5) the Chief Systems Architect is the final authority and SHALL NOT delegate this role. These principles implement the separation-of-duties and least-privilege controls recommended by NIST SP 800-53 AC-2, AC-5, and AC-6.

6. Engineering Requirements

The platform's IAM layer SHALL enforce the action authority matrix at the API boundary. Every operational action SHALL produce an audit record. Every credential SHALL have a rotation policy. Every certificate SHALL have a renewal mechanism. Every migration SHALL have a tested rollback procedure. Every deployment SHALL have a recorded release certificate. Every rollback SHALL have a post-incident review within 24 hours. Every break-glass grant SHALL be post-hoc reviewed within 4 hours. Every DR test SHALL be conducted quarterly. Every legal removal SHALL preserve cryptographic proof of the removed artifact.

7. Engineering Constraints

Operational governance constrains every other chapter: no feature MAY be implemented that requires an operational action not listed in the action authority matrix; no automation MAY perform an action without delegated authority in the matrix; no operational procedure MAY be defined that bypasses the audit log. The action authority matrix is exhaustive: any action not listed is forbidden until added by amendment under Chapter 36. Two roles whose authorities conflict SHALL NOT be held by the same individual.

8. Runtime Behaviour

At runtime, the IAM layer intercepts every operational action and verifies that the caller holds the required role and that any required approval has been recorded. Approved actions are executed and produce audit records. Refused actions are logged with the caller, the attempted action, and the reason for refusal; a pattern of refused actions SHALL trigger a Security Officer review (the caller may be misconfigured, may be attempting privilege escalation, or may indicate a role-coverage gap).

9. Failure Conditions

Operational-governance failure conditions include:

- **Unauthorized action:** an action attempted without the required role; refused by the IAM layer; logged for Security Officer review.
- **Missing approval:** an action attempted without the required second-party approval; refused by the IAM layer; logged.
- **Audit-log failure:** an operational action executed but the audit record was not written; SEV-1 incident (INV-A-01 violation).
- **Credential-rotation failure:** a credential past its rotation deadline; SEV-2 incident (INV-V-03 violation).
- **Certificate-expiry failure:** a certificate reached expiry without renewal; SEV-1 incident.
- **DR-test failure:** a quarterly DR test failed; remediation required within defined deadline.
- **Break-glass abuse:** break-glass access used outside the declared scope or duration; SEV-1 incident; Security Officer review.

10. Recovery Procedures

Recovery from operational-governance failures is case-specific. Unauthorized-action recovery is investigation (was it misconfiguration, role-coverage gap, or attempt at privilege escalation?). Missing-approval recovery is process review (was the approval path clear? was the approver reachable?). Audit-log failure is INV-A-01 recovery (Chapter 26). Credential-rotation failure is emergency rotation (Section 34.3.3). Certificate-expiry failure is expired-certificate incident response (Section 34.4.4). DR-test failure is remediation per the test report. Break-glass abuse is Security Officer review with potential revocation of the grantee's normal access.

11. Security Considerations

Operational governance is itself a security control. The two-party control on credential rotation, legal removal, and break-glass mitigates insider threat. The audit-log append-only property (INV-A-02) and cryptographic chain (INV-A-03) provide tamper detection. The role-based access model (no individual holds two conflicting roles) enforces separation of duties. The break-glass post-hoc review ensures that emergency grants are scrutinized. The Chief Systems Architect's non-delegable final-authority role ensures that the ultimate operational decision is always traceable to a single accountable individual.

12. Performance Considerations

Operational-governance enforcement has a performance cost: the IAM layer adds latency to every operational action; the audit log adds write latency to every state change; the approval workflow adds wall-clock latency to deployments and migrations. The performance budget for IAM enforcement SHALL be defined in Chapter 31 and SHALL not exceed 50 ms per operational action at p99. The audit-log write SHALL be in the same database transaction as the state change it records (no async audit), so the latency is bounded by database write latency.

13. Observability Requirements

The IAM layer SHALL expose metrics on: action authorization rate (allowed/refused); per-role action rate; per-action approval latency; audit-log write latency; credential-rotation freshness (days-since-last-rotation per credential class); certificate-expiry countdown; break-glass grant count and duration; DR-test result history. The operational-governance dashboard SHALL aggregate these into a single view reviewed daily by the Engineering Leadership Team and weekly by the Chief Systems Architect.

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **The nine operational roles are appointed and recorded in the audit log.**
- **The action authority matrix is implemented in the IAM layer and enforced at the API boundary.**
- **All credentials have rotation policies; the credential inventory is live; at least one routine rotation has been executed and audited.**
- **Certificate renewal automation is operational; the certificate inventory is live; at least one renewal has occurred automatically.**
- **A migration has been approved and executed per Section 34.5; the migration audit record exists.**
- **A deployment has been approved per Section 34.6; the release certificate is on file.**
- **A rollback has been exercised in staging and verified.**
- **A break-glass grant has been exercised in a tabletop and post-hoc reviewed.**
- **A DR test has been conducted and produced a test report.**
- **An incident-command tabletop has been conducted with a postmortem.**
- **The legal removal process is documented and counsel-reviewed (no actual removal required for sign-off).**

15. Runtime Verification

Runtime verification of operational governance is the continuous monitoring of: IAM enforcement (every operational action passes through the authority matrix); audit-log completeness (every operational action produces an audit record); credential-rotation freshness (no credential past its deadline); certificate-expiry countdown (no certificate within 14 days of expiry without active renewal); break-glass grant inventory (every grant within its duration); DR-test cadence (last test within 90 days). The Release Certification Authority SHALL verify these monitors are operational before RC1 sign-off.

16. Certification Requirements

RC1 certification requires: all nine operational roles appointed; the action authority matrix implemented in IAM; the credential inventory live with all credentials having rotation policies; the certificate inventory live with all certificates on ACME; at least one migration, one deployment, one rollback, one break-glass, one DR test, and one incident-command tabletop executed and postmortemed; the legal removal process documented and counsel-reviewed; and the Chief Systems Architect's sign-off that the operational-governance regime is complete.

17. Future Evolution

Operational governance evolves through the Evolution Governance process (Chapter 36). New operational roles MAY be added (typically as the platform's operational surface grows). The action authority matrix MAY be amended to add actions (typically as new operational capabilities are introduced). The matrix SHALL NOT be amended to remove two-party control on existing actions; the principle of two-party control for elevated-consequence actions is itself an invariant of this chapter and SHALL NOT be weakened. Emergency amendments to operational governance (Section 36.12) SHALL be reserved for security-corrective actions and SHALL be post-hoc reviewed by the Chief Systems Architect.

Chapter 35 — Engineering Economics

This chapter constitutes the Engineering Economics Charter of Opus Publica. It defines the cost model of the platform, the cost-per-publication unit economics, the storage and bandwidth economics, the external-service fee structure, the compute and scaling economics, the caching economics, the budget-governance regime, the cloud strategy, the cost-optimization principles, and the economic considerations for the RC2 and RC3 horizons. The chapter exists because a Diamond Open Access platform — which charges neither authors nor readers — has no APC revenue stream to absorb cost growth: every dollar of cost is a dollar of subsidy that must be raised, granted, or contributed by AUN. Engineering economics is therefore not a financial-reporting concern delegated to a finance team; it is an engineering concern that every architect, every engineer, and every operator SHALL internalize.

The chapter is grounded in the FinOps Foundation's three-phase model (Inform, Optimize, Operate) and in Google SRE's user-cost-attention principle: the cost of a service is ultimately borne by the users (or, in this case, by the subsidizing organization), and engineering decisions that increase cost without increasing value are, by definition, failures of engineering judgement. The chapter adopts RFC 2119 vocabulary: cost targets in this chapter are SHALL requirements where stated as such, and the Engineering Leadership Team SHALL review the cost dashboard with the same cadence and the same rigour as the invariant dashboard (Chapter 26).

The Constitutional Status of Engineering Economics

Engineering economics is a first-class engineering concern, not a downstream financial concern. Every architectural decision (Chapter 7) SHALL include a cost impact assessment. Every release certificate (Chapter 22) SHALL reference the current cost-per-publication figure. Every quarterly review SHALL examine cost trend against the budget and SHALL trigger a cost-optimization plan when the trend deviates unfavourably. Cost is not the master of engineering judgement, but it is not an afterthought either; it is one of the constraints within which engineering judgement operates.

35.1 Cost Model

The platform's cost model is organized into thirteen categories, each driven by a measurable engineering or operational variable, each priced at an indicative unit cost, and each exhibiting a characteristic scaling behaviour. The unit costs are indicative (based on prevailing 2026 cloud-provider list prices for a single-region deployment with standard discounts); the actual unit costs SHALL be maintained in the cost dashboard and SHALL be refreshed quarterly. The scaling behaviour describes how the category's cost grows as the platform's load grows: linear, sublinear, stepwise, or fixed.

Cost Category	Driver	Unit Cost (indicative)	Scaling
Compute (VMs/containers)	vCPU-hours; memory-GB-hours.	$0.035/vCPU-hour; $0.0048/GB-hour.	Stepwise (autoscale); sublinear at low load, linear

			above the autoscale floor.
Storage (object + DB)	GB-month stored.	$0.023/GB-month (object, standard); $0.115/GB-month (DB SSD).	Linear in volume; sublinear via lifecycle transitions to cold storage.
Bandwidth (egress)	GB egress to internet.	$0.09/GB (first 10TB/month); $0.085/GB next 40TB; $0.07/GB above 100TB.	Linear in traffic; reducible via CDN caching.
Crossref fees	Annual membership + per-DOI deposit.	$275-$11,000/year tier + $1.00/DOI (deposit fee waived for many tiers).	Stepwise (tier) + linear (per-DOI).
Preservation (CLOCKSS/LOCKSS/Portico)	Annual membership.	$1,000-$15,000/year per service (publisher-tier dependent).	Stepwise (tier); mostly fixed for a given tier.
ORCID membership	Annual member fee.	$1,500-$8,000/year (tier dependent).	Stepwise (tier); fixed for a given tier.
Domain / DNS	Domain registration; DNS queries.	$12/year (domain); $0.40 per million DNS queries (managed DNS).	Fixed (domain) + linear (DNS, trivial magnitude).
Email (Resend)	Emails sent.	$20/month base (50k emails) + $0.50 per 1k above.	Stepwise (tier) + linear (per-email above tier).
CDN	GB egress via CDN; requests.	$0.085/GB egress; $0.01 per 10k HTTPS requests.	Linear; cheaper than origin egress and reduces origin load.
Logging / observability tools	GB ingested; metrics cardinality.	$0.50/GB ingested; $0.01 per 1k custom metrics.	Linear in volume; manageable via sampling and retention policies.
Secrets management	API calls; secrets stored.	$0.03 per 10k API calls; $0.40 per secret/month (some providers).	Trivial magnitude at platform scale.
Backup storage	GB-month backup.	$0.005/GB-month (object, archival class).	Linear in volume; archival class is ~5% of standard.

The cost model SHALL be the authoritative reference for engineering cost-impact assessments. Where a new architectural decision introduces a cost category not in this table, the ADR SHALL add the category to the table by amendment (Chapter 36) and SHALL record the new category's driver, unit cost, and scaling behaviour. A cost category that the platform incurs but that is not in the table is a documentation gap and SHALL be remediated within one quarter.

35.2 Cost per Publication

The cost-per-publication metric is the platform's primary unit economic. It answers the question: for each article the platform publishes, what does the platform spend, amortized across the lifetime of that article? The metric is the financial analogue of the cost-per-query metric in classical SRE: a single number that summarizes the platform's efficiency and that, tracked over time, reveals whether the platform is becoming more or less efficient as it scales. The metric SHALL be reported monthly to the Engineering Leadership Team and quarterly to the Chief Systems Architect.

35.2.1 Derivation

The cost-per-publication C_pub is derived as follows: C_pub = (C_compute_amort + C_storage + C_crossref + C_preservation + C_bandwidth) / N_published, where N_published is the number of articles published in the period, C_compute_amort is the compute cost amortized across all publications (since compute serves both publication and ongoing readership), C_storage is the marginal storage cost of the published artifacts, C_crossref is the per-DOI deposit cost, C_preservation is the amortized preservation membership cost per publication, and C_bandwidth is the bandwidth cost of serving the article over its first year of life.

35.2.2 Indicative Cost per Publication (RC1, 2026)

Component	Calculation	Indicative Cost
Compute (amortized)	10 publications/month; compute $1,500/month; amortized over 4 publications per engineer-week.	$150 / publication
Storage (object + DB)	Average article = 100 MB PDF + 1 MB JATS + 2 MB HTML + 1 MB supplements = 104 MB; SSD metadata ~5 MB; 12-month retention.	$0.50 / publication / year
Crossref deposit	Per-DOI deposit fee at the platform's Crossref tier.	$1.00 / publication
Preservation (amortized)	CLOCKSS + LOCKSS + Portico = ~$15,000/year combined; 120 publications/year.	$125 / publication
Bandwidth (first year)	Average article: 50 downloads x 5 MB = 250 MB egress; at $0.09/GB.	$0.02 / publication
TOTAL (RC1, 2026)	Sum of above.	~$276 / publication

The indicative total of $276 per publication is dominated by amortized compute ($150) and amortized preservation membership ($125). Both components scale sublinearly with publication volume: as the platform publishes more articles per month, the compute and preservation costs are spread across more

publications, reducing the per-publication cost. The platform's economic goal is to drive C_pub below $100 per publication by the end of RC2 (driven by publication volume growth and compute optimization) and below $50 per publication by RC3 (driven by multi-tenant scale and AI-assisted editorial cost reduction).

35.3 Storage Economics

Storage is the platform's only cost category that is monotonic non-decreasing by invariant (INV-V-01: published artifacts are never lost, which means published artifacts are never deleted, which means storage volume grows with publication count). Storage economics is therefore the platform's most important long-term cost discipline: a 1% reduction in per-GB storage cost, applied to a volume that grows monotonically, compounds over years into significant savings.

35.3.1 Hot vs Cold Storage

The platform SHALL classify storage into three tiers: hot (standard object storage; immediate retrieval; $0.023/GB-month), warm (infrequent-access storage; sub-second retrieval; $0.0125/GB-month), and cold (archival storage; minutes-to-hours retrieval; $0.004/GB-month). Newly published artifacts SHALL be in hot storage. Artifacts not accessed in 90 days SHALL transition to warm. Artifacts not accessed in 365 days SHALL transition to cold. The transition SHALL be governed by lifecycle policies and SHALL be audited.

35.3.2 Lifecycle Transitions

Lifecycle transitions SHALL be automated by the storage layer's lifecycle policy engine. The policy SHALL: keep the canonical PDF, JATS, and HTML in hot storage for 90 days post-publication (to absorb the publication-traffic spike); transition supplements to warm after 30 days; transition all artifacts to warm after 90 days; transition all artifacts to cold after 365 days. A reader who requests a cold artifact SHALL experience a redirect to a 'preparing artifact' page that polls until the artifact is restored (typically 1-5 minutes); the platform SHALL NOT charge the reader for the restoration.

35.3.3 Cost Optimization and Retention Trade-offs

Storage cost optimization is bounded by INV-V-01: published artifacts are never lost, so the platform cannot delete to save cost. The optimization levers are: (1) tier transitions (hot to warm to cold, as above); (2) compression of non-canonical artifacts (supplements, derivative representations); (3) deduplication (invoked by INV-C-04 detection); (4) storage-class selection per artifact type (e.g., JATS XML in standard storage because it is small and frequently harvested via OAI-PMH; supplements in cold storage because they are large and rarely accessed). Retention trade-offs: the platform SHALL retain all published artifacts indefinitely (no retention trade-off); the platform MAY reduce retention of non-published artifacts (drafts, supplementary review materials) per a defined retention policy.

35.4 Bandwidth Economics

Bandwidth (egress to the internet) is the platform's most variable cost category, scaling directly with reader traffic, OAI-PMH harvest traffic, and Crossref/preservation deposit traffic. Bandwidth economics is the discipline of minimizing egress cost without compromising reader experience or harvester service quality. The primary lever is CDN caching, which shifts egress from the (more expensive) origin to the (less expensive and geographically distributed) CDN.

35.4.1 Egress Costs

Egress cost scales linearly with bytes transferred, with tiered pricing above 10TB/month. The platform SHALL monitor egress by source: reader traffic (the dominant source for a well-functioning platform), OAI-PMH harvest traffic (potentially large if a major harvester crawls), Crossref deposit traffic (small), preservation deposit traffic (small, periodic). Anomalous egress patterns SHALL trigger investigation: a sudden spike may indicate a misconfigured harvester, a scraping bot, or a runaway internal job.

35.4.2 CDN Strategy

The platform SHALL serve all static artifact content (PDF, HTML, JATS, supplements) via CDN. The CDN SHALL cache at the edge for at least 24 hours; cache invalidation SHALL be triggered by publication events (a new version supersedes the old). The CDN SHALL be configured to shield the origin from traffic spikes: a surge in requests for a single article (e.g., a viral paper) SHALL be absorbed by the CDN, not by the origin. Origin egress SHALL be monitored and SHALL not exceed 20% of total egress (the 80/20 target: 80% served by CDN, 20% by origin); deviations trigger cache-hit-ratio investigation.

35.4.3 Caching to Reduce Origin Egress

Application-layer caching (in-memory cache, database query cache) reduces origin compute and origin database load, indirectly reducing origin egress. The platform SHALL cache: article landing pages (until superseded); journal landing pages (until article added); OAI-PMH responses (per the response's expires header); author profile pages (until author record updated). Cache invalidation SHALL be event-driven (a publication event invalidates the article's cache entry) rather than TTL-driven (TTLs are a fallback, not a primary strategy).

35.4.4 OAI-PMH Harvest Bandwidth Budgeting

OAI-PMH harvesters (OpenAIRE, BASE, DOAJ, regional aggregators) can generate substantial traffic, particularly on the ListRecords verb. The platform SHALL: enforce a per-harvester rate limit (default: 5 requests/second, configurable per harvester); serve OAI-PMH responses with explicit expires headers; cache OAI-PMH responses at the CDN (with a configurable TTL, default 1 hour); and monitor per-harvester egress. A harvester that exceeds its rate limit SHALL receive HTTP 429 (Too Many Requests) with a Retry-After header.

35.5 External Service Fees

External service fees are the recurring membership and per-transaction fees the platform pays to organizations whose services are integral to its operation: Crossref, ORCID, CLOCKSS, LOCKSS, Portico, the domain registrar, the email provider, and the CDN. These fees are largely stepwise (tier-based annual memberships) plus linear (per-transaction fees above the tier). The platform SHALL budget for these fees annually and SHALL review the tier structure quarterly to ensure the platform is on the optimal tier for its current volume.

35.5.1 Crossref Membership Tier

Crossref membership tiers range from the smallest publisher tier (~$275/year) to the largest ($11,000+/year), with per-DOI deposit fees varying by tier. The platform SHALL select the tier that minimizes total annual cost (membership + per-DOI fees) for its projected publication volume. The tier SHALL be reviewed annually; if projected publication volume has shifted, the platform SHALL evaluate moving to a different tier. The Crossref membership SHALL NOT be allowed to lapse (lapse breaks the DOI-minting capability and violates INV-L-01).

35.5.2 ORCID Membership

ORCID membership tiers range from the smallest (~$1,500/year) to the largest (~$8,000/year), with member benefits including member-API access (authenticated deposit and read of ORCID records) and higher rate limits. The platform SHALL maintain ORCID membership for the member-API benefits; lapse degrades the platform to the public API, which is sufficient for read-only verification but insufficient for authenticated ORCID updates (e.g., auto-adding publications to author ORCID records).

35.5.3 CLOCKSS / LOCKSS / Portico Fees

Preservation service fees are publisher-tier dependent, typically $1,000-$15,000/year per service. The platform SHALL maintain all three preservation memberships (CLOCKSS, LOCKSS, Portico) for redundant preservation; lapse of any one weakens the preservation posture but does not violate an invariant (preservation deposit is required by INV-L-04, but the invariant requires at least one preservation deposit queued, not three). The platform SHALL NOT allow all three to lapse simultaneously; doing so violates INV-L-04.

35.5.4 Budgeting Cadence

External service fees SHALL be budgeted annually as part of the platform's annual budget cycle (Section 35.9). The budget SHALL include the membership fees (fixed) and the projected per-transaction fees (estimated from publication volume projection). The budget SHALL be reviewed quarterly; deviations of more than 10% from projection SHALL trigger a budget-revision discussion. Renewal dates SHALL be tracked in the credential inventory (Section 34.3) with calendar alerts 60, 30, and 7 days before renewal.

35.6 Compute Economics

Compute is the platform's largest variable cost category, driven by the number and size of VMs or containers the platform runs. Compute economics is the discipline of matching compute supply to compute demand: too little compute causes incidents; too much compute wastes money. The discipline rests on autoscaling (match supply to demand automatically), reserved capacity (commit to baseline for discount), spot/preemptible capacity (use surplus capacity for batch jobs), and right-sizing (match instance size to actual workload).

35.6.1 Autoscaling

The platform SHALL autoscale all stateless services based on CPU, memory, and request-queue-depth metrics. The autoscale policy SHALL define a minimum (floor) and maximum (ceiling) instance count, a scale-up threshold (e.g., CPU > 70% for 3 minutes), and a scale-down threshold (e.g., CPU < 30% for 10 minutes). The floor SHALL be set to handle the platform's typical baseline traffic; the ceiling SHALL be set to handle the platform's worst-case traffic spike. Autoscale events SHALL be logged and reviewed weekly to detect thrashing.

35.6.2 Reserved vs On-Demand

The platform SHALL use reserved capacity (1-year or 3-year commitments) for the baseline floor of every stateless service; reserved capacity offers 30-60% discount over on-demand. On-demand capacity SHALL be used for the variable component above the floor (handled by autoscaling). The reserved-to-on-demand ratio SHALL be reviewed quarterly; if the platform consistently runs above its reserved baseline, the reserved baseline SHALL be increased; if consistently below, the reserved baseline SHALL be decreased.

35.6.3 Spot / Preemptible for Batch Jobs

Batch jobs (nightly reconciliation, invariant scans, OAI-PMH cache refresh, preservation re-deposit) SHALL run on spot/preemptible capacity (60-90% discount over on-demand). Batch jobs SHALL be designed to tolerate preemption (checkpointing, idempotent re-execution). A batch job that cannot tolerate preemption SHALL be marked as non-preemptible and SHALL run on reserved capacity with documented justification.

35.6.4 Right-Sizing

Every service SHALL be right-sized quarterly: actual CPU and memory utilization SHALL be reviewed, and instance sizes SHALL be adjusted to match. A service consistently utilizing <30% of its allocated CPU or memory SHALL be downsized; a service consistently utilizing >70% SHALL be upsized (or the workload SHALL be optimized). Right-sizing SHALL be evidence-driven (utilization metrics, not guesses) and SHALL be recorded in the cost dashboard.

35.7 Scaling Economics

Scaling economics is the discipline of understanding how the platform's cost-per-request changes as the platform's request volume grows. The discipline distinguishes scale-up (bigger instances) from scale-out (more instances), measures cost-per-request at different scales, and identifies diminishing returns (the point at which adding capacity yields less than proportional throughput improvement).

35.7.1 Scale-Up vs Scale-Out

Scale-up (vertical scaling) increases the size of existing instances (more vCPU, more memory). It is bounded by the maximum instance size available from the cloud provider, and it does not improve fault tolerance (a single bigger instance is still a single point of failure). Scale-out (horizontal scaling) adds more instances of the same size. It is bounded by the autoscale ceiling, and it improves fault tolerance (multiple instances can survive individual failures). The platform SHALL prefer scale-out for stateless services and SHALL reserve scale-up for stateful services (databases) where scale-out requires sharding.

35.7.2 Cost-per-Request at Different Scales

Cost-per-request SHALL decrease as request volume grows, because fixed costs (reserved capacity, membership fees) are amortized across more requests. The platform SHALL measure cost-per-request monthly and SHALL plot the trend. A cost-per-request that increases with volume indicates a scaling bottleneck (a service that does not scale linearly, a database that is hitting a contention point, a network egress tier that has stepped up) and SHALL trigger investigation.

35.7.3 Diminishing Returns

Diminishing returns occur when adding capacity yields less than proportional throughput improvement. Common causes: database contention (the database becomes the bottleneck), lock contention (services contend on shared state), network limits (egress bandwidth saturates), or instance-size inefficiency (very large instances underutilize some resources). The platform SHALL identify diminishing-returns points and SHALL address them by architectural change (sharding, caching, async processing), not by brute-force capacity addition.

35.8 Caching Economics

Caching is the platform's primary cost-reduction lever for both compute and bandwidth. A cache hit avoids a database query (reducing compute) and avoids an origin response (reducing origin egress). Caching economics is the discipline of choosing the right cache layer (CDN, app cache, DB cache), measuring cache hit ratio, and managing invalidation cost.

35.8.1 Cache Hit Ratio vs Cost

Cache hit ratio is the fraction of requests served from cache. A 90% cache hit ratio means 90% of requests are served cheaply (from cache) and 10% are served expensively (from origin). The marginal value of an additional 1% cache hit ratio decreases as the ratio increases (from 80% to 81% saves 5% of

origin load; from 95% to 96% saves 20% of origin load). The platform SHALL target a 95% cache hit ratio for article landing pages and a 99% ratio for static assets (CSS, JS, images).

35.8.2 CDN vs App Cache vs DB Cache

The three cache layers serve different purposes: CDN caches HTTP responses at the edge (cheapest, geographically distributed, highest hit ratio for static content); app cache caches computed results in the application process (medium cost, per-instance, used for personalized or computed content); DB cache caches query results in the database (medium cost, per-instance, used for expensive queries). The platform SHALL use all three layers, with the CDN as the primary, app cache for computed content, and DB cache as a fallback for query optimization.

35.8.3 Invalidation Cost

Cache invalidation is the cost of removing stale entries from cache. Event-driven invalidation (a publication event invalidates the article's cache entry) is cheap and precise. TTL-driven invalidation (entries expire after a fixed time) is cheap but imprecise (may serve stale content until expiry). Manual invalidation (an operator flushes the cache) is expensive and risky (may cause a thundering herd of origin requests). The platform SHALL prefer event-driven invalidation; TTLs SHALL be set as a safety net, not as the primary strategy; manual invalidation SHALL require Security Officer approval and SHALL be audited.

35.9 Budget Governance

Budget governance is the discipline of setting, monitoring, and managing the platform's annual budget. The discipline follows the FinOps Foundation's three-phase model: Inform (visibility into cost), Optimize (reduce cost without compromising value), and Operate (continuous management). Budget governance is owned by the Engineering Leadership Team, reviewed monthly, and escalated to the Chief Systems Architect on anomaly.

35.9.1 Annual Budget Cycle

The annual budget cycle SHALL: (Q4) project next year's publication volume, reader traffic, and feature roadmap; derive next year's compute, storage, bandwidth, and external-service costs from the projections; submit the budget to the Chief Systems Architect for approval; (Q1) execute against the approved budget; (Q2-Q3) review monthly and adjust if projections deviate; (Q4) close the year and begin the next cycle. The budget SHALL include a 10% contingency for unforeseen cost (e.g., a viral paper driving bandwidth spike).

35.9.2 Monthly Review

The Engineering Leadership Team SHALL review the cost dashboard monthly: actual spend vs budget, by category and in total; cost-per-publication trend; cost-per-request trend; cache hit ratios; reserved-to-on-demand ratio; storage-tier distribution; bandwidth-by-source breakdown. Deviations of more than 10% in any category SHALL trigger investigation and a remediation plan.

35.9.3 Cost Anomaly Detection and Alerts

The platform SHALL implement automated cost-anomaly detection: a sustained daily spend increase of more than 25% over the trailing 7-day average SHALL trigger an alert to the Engineering Leadership Team and the Security Officer (a sudden spend increase may indicate a compromise, a runaway job, or a misconfiguration). The anomaly-detection service SHALL itself be monitored (a meta-monitor); failure of anomaly detection is a SEV-2 incident.

35.9.4 Cost-per-KPI Tracking

Cost SHALL be tracked against KPIs (Chapter 1): cost per publication, cost per reader session, cost per OAI-PMH harvest, cost per Crossref deposit. These unit costs communicate efficiency more clearly than total spend; a rising total spend with stable unit costs indicates healthy growth, while rising unit costs indicate efficiency loss. The unit costs SHALL be reported quarterly to the Chief Systems Architect alongside the engineering KPI report.

35.10 Cloud Strategy

Cloud strategy is the discipline of choosing and managing the platform's cloud infrastructure. The strategy addresses single-cloud vs multi-cloud, lock-in mitigation, exit strategy, and sovereign-cloud considerations for European operations. The strategy is constrained by the platform's small team size (which favours simplicity) and by the platform's public-trust posture (which favours redundancy and exit-readiness).

35.10.1 Single-Cloud vs Multi-Cloud

The platform SHALL operate on a single primary cloud provider for RC1 and RC2, with explicit selection of provider-managed services that are portable (containers, object storage with S3-compatible API, managed PostgreSQL). Multi-cloud operation SHALL be deferred to RC3 or later, when the platform's scale and team capacity justify the operational complexity. Multi-cloud is not a free lunch: it doubles the operational surface, requires replicated skills, and complicates deployment. The platform SHALL NOT pursue multi-cloud for redundancy alone; redundancy within a single provider's regions is sufficient for the platform's availability targets.

35.10.2 Lock-In Mitigation

Cloud lock-in is the cost of switching providers: the vendor-specific services, APIs, and data formats that make migration expensive. The platform SHALL mitigate lock-in by: preferring portable abstractions (containers over provider-specific functions; S3-compatible object storage over provider-specific blob storage; standard PostgreSQL over provider-specific databases); avoiding provider-specific services where a portable alternative exists (e.g., standard Kafka over provider-specific event buses); and conducting an annual lock-in assessment that catalogues provider-specific dependencies and their migration cost.

35.10.3 Exit Strategy

The platform SHALL maintain a documented exit strategy: the steps required to migrate to a different cloud provider or to self-hosted infrastructure. The exit strategy SHALL be tested in part by the quarterly DR test (Section 34.9), which exercises the platform's ability to restore from backup into a fresh environment. A full exit test (migration to a different provider) is not required, but the platform SHALL maintain the capability, and the lock-in assessment SHALL quantify the cost of an exit.

35.10.4 Sovereign Cloud Considerations for European Operations

The platform is headquartered in The Hague (NL) and serves a global audience including European researchers whose data is subject to GDPR. The platform SHALL: host European users' personal data in European data centres; comply with GDPR data-subject rights (access, rectification, erasure); and consider sovereign-cloud options (e.g., Gaia-X-compatible providers) if regulatory or contractual requirements emerge. Sovereign-cloud migration, if required, SHALL follow the exit-strategy procedure and SHALL preserve all invariants.

35.11 Cost Optimization Principles

Cost optimization is the discipline of reducing cost without compromising value. The discipline follows the FinOps Foundation's principles: unit economics (track cost per unit of value), cost-aware architecture (design for cost as a first-class constraint), and waste elimination (remove resources that deliver no value). The discipline is owned by every engineer, not by a separate FinOps team; cost is an engineering concern.

35.11.1 FinOps Practices

The platform SHALL adopt FinOps Foundation practices: visibility (every engineer can see the cost of their services); accountability (every service has a cost owner); optimization (every service has a cost-optimization target); and reporting (cost is reported alongside engineering KPIs). FinOps is a continuous practice, not a one-time exercise; the platform SHALL review FinOps metrics monthly with the same rigour as engineering KPIs.

35.11.2 Unit Economics

Every service SHALL have a defined unit economic: cost per request, cost per publication, cost per session, cost per harvest. The unit economic SHALL be tracked monthly and SHALL be the primary lens for cost discussion. Total spend is a derivative of unit economic times volume; reducing unit economic reduces total spend at any volume, while reducing volume reduces total spend only at the cost of platform mission.

35.11.3 Cost-Aware Architecture

Architectural decisions (Chapter 7) SHALL include a cost-impact assessment: the expected per-request, per-publication, or per-month cost of the proposed architecture, with comparison to alternatives. Cost-aware architecture prefers: serverless over always-on (for low-traffic services); managed over self-hosted

(for commodity services like databases); open-source over licensed (where the open-source option meets the platform's quality bar); and lazy computation over eager computation (compute on demand, not in advance).

35.11.4 Waste Elimination

Waste is resources that deliver no value: idle instances, orphan storage objects, unused load balancers, forgotten test environments, over-provisioned databases. The platform SHALL eliminate waste through: weekly idle-resource scans (instances with <5% utilization for 14 days); nightly orphan-storage-object scans (per INV-C-01); environment lifecycle (test environments torn down after defined idle period); and right-sizing (per Section 35.6.4). Waste elimination SHALL be tracked: the platform SHALL report monthly the cost recovered through waste elimination.

35.12 RC2/RC3 Economic Considerations

RC2 and RC3 introduce new economic considerations: multi-tenant cost allocation (how to attribute cost to tenants), plugin marketplace revenue model (if any), and AI feature cost (model inference). These considerations are beyond RC1's scope but SHALL be planned for in the RC1 economic model so that the platform's architecture does not preclude them.

35.12.1 Multi-Tenant Cost Allocation

RC2 introduces multi-tenancy (multiple journals or publishers on a shared platform). Multi-tenant cost allocation is the discipline of attributing shared cost (compute, storage, bandwidth) to individual tenants. The platform SHALL implement tenant tagging on all resources (compute instances, storage objects, bandwidth egress) and SHALL generate per-tenant cost reports monthly. The allocation method SHALL be documented (e.g., proportional to publication count, proportional to storage volume, proportional to bandwidth consumed) and SHALL be applied consistently.

35.12.2 Plugin Marketplace Revenue Model

RC2 or RC3 MAY introduce a plugin marketplace (extensions to the platform developed by third parties and offered to tenants). The economic model for a plugin marketplace SHALL address: plugin revenue share (between plugin developer and platform), plugin hosting cost (compute and storage for plugins), plugin support cost (handled by plugin developer or platform), and plugin marketplace curation cost. The platform SHALL NOT commit to a plugin marketplace in RC1; the decision SHALL be deferred to RC2 planning with a documented business case.

35.12.3 AI Feature Cost (Model Inference)

RC3 MAY introduce AI-assisted features (editorial assistance, metadata extraction, plagiarism detection, translation). AI features carry unique cost characteristics: model inference is compute-intensive, model training (if performed by the platform) is highly compute-intensive, and third-party model APIs (OpenAI, Anthropic, Google) carry per-token costs that can be substantial at scale. The platform SHALL: budget AI feature cost explicitly (not absorb it into general compute); implement per-feature token-budget limits

(to prevent runaway cost); and prefer open-source models for high-volume features (to avoid per-token cost) where the open-source model meets the platform's quality bar.

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

This chapter defines the engineering economics of Opus Publica: the cost model, the cost-per-publication unit economic, the storage and bandwidth economics, the external-service fee structure, the compute and scaling economics, the caching economics, the budget-governance regime, the cloud strategy, the cost-optimization principles, and the RC2/RC3 economic considerations. The chapter is the authoritative reference for the platform's engineering cost discipline.

2. Scope

In scope: the thirteen cost categories; the cost-per-publication derivation; storage tiers and lifecycle; bandwidth, CDN, and OAI-PMH harvest budgeting; external service fees; compute autoscaling, reservation, and right-sizing; scale-up vs scale-out; cache layers and invalidation; budget cycle and anomaly detection; cloud strategy and exit; FinOps principles; RC2/RC3 economic considerations. Out of scope: the engineering pipeline (Chapter 22), the operational governance (Chapter 34), the amendment of this chapter (Chapter 36), and the financial accounting of subsidy and grant income (handled by the parent organization's finance function, not by this Constitution).

3. Definitions

Key terms used in this chapter are defined as follows:

Term	Definition
Cost per publication (C_pub)	The platform's primary unit economic: the cost amortized across publications of producing and serving one article.
Cost per request	The platform's secondary unit economic: the cost of serving one reader request.
FinOps	The discipline of managing cloud cost as an engineering concern; the FinOps Foundation is the industry body.
Cache hit ratio	The fraction of requests served from cache; the primary cache-effectiveness metric.
Reserved capacity	Cloud capacity committed for a term (1 or 3 years) at a

	discount; used for baseline load.
Spot / preemptible capacity	Surplus cloud capacity offered at steep discount, subject to preemption; used for batch jobs.
Lock-in	The cost of switching cloud providers; mitigated by portable abstractions.

4. Responsibilities

Ownership of engineering economics is distributed across the engineering organization:

- **Chief Systems Architect:** Approves the annual budget; reviews the cost dashboard quarterly; authorizes cost-optimization initiatives.
- **Engineering Leadership Team:** Reviews the cost dashboard monthly; investigates deviations; assigns cost-optimization targets.
- **Head of Platform Engineering:** Owns compute, storage, and bandwidth cost; implements autoscaling, reservation, right-sizing, and caching.
- **Head of Data Engineering:** Owns storage-tier transitions; implements lifecycle policies; manages database cost.
- **Head of Observability:** Owns the cost dashboard; implements cost-anomaly detection; reports cost-per-KPI.
- **Security Officer:** Owns cost-anomaly response (anomalies may indicate compromise).
- **Release Certification Authority:** Includes cost-impact assessment in release certificate review for cost-affecting changes.

5. Design Principles

Engineering economics follows five principles: (1) cost is an engineering concern, not a financial concern delegated to finance; (2) unit economics (cost per publication, cost per request) communicate efficiency more clearly than total spend; (3) cost-aware architecture prefers portable, managed, and lazy abstractions; (4) waste elimination is a continuous practice, not a one-time exercise; (5) cost optimization SHALL NOT compromise invariants, SLOs, or reader experience. These principles implement the FinOps Foundation's Inform-Optimize-Operate model and Google SRE's user-cost-attention principle.

6. Engineering Requirements

Every architectural decision SHALL include a cost-impact assessment. Every release certificate SHALL reference the current cost-per-publication figure. The cost dashboard SHALL be live and reviewed monthly by the Engineering Leadership Team and quarterly by the Chief Systems Architect. Cost-anomaly detection SHALL be operational and SHALL alert on sustained daily spend increases of more than 25%. Every credential-bearing external service SHALL have its fee tracked in the budget. Storage-tier transitions SHALL be automated per the lifecycle policy. The cache hit ratio SHALL be monitored and

SHALL meet the per-content-type targets. The reserved-to-on-demand ratio SHALL be reviewed quarterly.

7. Engineering Constraints

Engineering economics constrains every other chapter: no feature MAY be implemented without a cost-impact assessment; no optimization MAY be accepted that weakens an invariant; no architectural decision MAY introduce a cost category not in the cost model without amending this chapter. The cost model SHALL NOT be amended to remove a category that the platform incurs (categories are append-only). The cloud strategy SHALL NOT compromise INV-V-01 (published artifacts never lost): a provider change SHALL preserve all published artifacts and all audit records.

8. Runtime Behaviour

At runtime, the cost dashboard aggregates per-service cost data from the cloud provider's billing API, per-tenant cost data from the platform's tenant-tagging layer, and per-request cost data from the platform's observability layer. The dashboard computes cost-per-publication and cost-per-request in near real time. Cost-anomaly detection runs every 15 minutes and alerts on sustained deviation from the trailing 7-day baseline. The cost dashboard is the primary engineering-economic view; the budget-vs-actual report is the primary financial view.

9. Failure Conditions

Engineering-economics failure conditions include:

- **Budget overrun:** actual spend exceeds budget by more than 10% in any category; triggers investigation and remediation plan.
- **Cost-per-publication increase:** C_pub rises month-over-month; indicates efficiency loss; triggers cost-optimization plan.
- **Cache hit ratio drop:** CDN or app cache hit ratio falls below target; triggers cache-configuration review.
- **Orphan-storage accumulation:** orphan objects accumulate faster than lifecycle reclamation; indicates lifecycle-policy gap.
- **Reserved-to-on-demand imbalance:** platform consistently runs above or below its reserved baseline; indicates reservation-size mismatch.
- **Cost-anomaly false negative:** a sustained spend increase occurred without alert; indicates anomaly-detection failure.
- **Cost-anomaly false positive:** alert fired without underlying anomaly; indicates threshold miscalibration.

10. Recovery Procedures

Recovery from economic failures is case-specific. Budget-overrun recovery is a budget-revision discussion (raise the budget, reduce spend, or both). Cost-per-publication-increase recovery is a cost-optimization plan (right-sizing, reservation adjustment, waste elimination, caching improvement). Cache-hit-ratio-

drop recovery is a cache-configuration review (TTL, invalidation policy, CDN configuration). Orphan-storage recovery is lifecycle-policy remediation. Reserved-to-on-demand-imbalance recovery is reservation adjustment. Cost-anomaly false-negative recovery is anomaly-detection-service remediation. Cost-anomaly false-positive recovery is threshold recalibration.

11. Security Considerations

Engineering economics is itself a security concern: a sudden cost increase may indicate compromise (a cryptominer using the platform's compute, a scraper using the platform's bandwidth, a credential leak enabling unauthorized third-party API calls). The Security Officer SHALL be alerted on every cost anomaly and SHALL rule out compromise before the anomaly is attributed to benign causes. The cost dashboard SHALL be access-controlled (it discloses the platform's infrastructure footprint, which is sensitive from a competitive and security standpoint). The cost-anomaly-detection service SHALL itself be defended against tampering (an attacker who can suppress cost alerts can mask their abuse).

12. Performance Considerations

Cost monitoring has a performance cost: the cost dashboard consumes cloud-provider billing-API quota; the per-tenant tagging layer adds metadata overhead; the per-request cost attribution adds observability overhead. The performance budget for cost monitoring SHALL not exceed 1% of platform compute capacity. Per-request cost attribution SHALL be sampled (not 100%) where the attribution overhead is significant; sampling rate SHALL be sufficient for statistical confidence in the unit-economic figures.

13. Observability Requirements

The cost dashboard SHALL expose: total spend (daily, monthly, year-to-date); spend by category; spend by service; spend by tenant (RC2+); cost-per-publication trend; cost-per-request trend; cache hit ratios; reserved-to-on-demand ratio; storage-tier distribution; bandwidth-by-source breakdown; anomaly-detection status; orphan-storage count and reclamation rate. The dashboard SHALL be reviewed monthly by the Engineering Leadership Team and quarterly by the Chief Systems Architect.

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **The cost model (thirteen categories) is documented with drivers, unit costs, and scaling behaviours.**
- **The cost-per-publication metric is computed monthly and trended.**
- **Storage lifecycle policies are operational; tier transitions are automated and audited.**
- **CDN is operational; cache hit ratios meet per-content-type targets.**
- **External service fees are budgeted; renewal dates are tracked.**
- **Compute autoscaling is operational; reserved capacity covers the baseline floor.**
- **Cost-anomaly detection is operational; at least one anomaly has been investigated (real or simulated).**
- **The annual budget has been approved by the Chief Systems Architect.**

- **Lock-in assessment has been conducted and documented.**
- **FinOps practices (visibility, accountability, optimization, reporting) are in place.**

15. Runtime Verification

Runtime verification of engineering economics is the continuous monitoring of: cost dashboard freshness (last update within 24 hours); cost-anomaly detection liveness (last run within 15 minutes); cache hit ratios (per content type, against target); reserved-to-on-demand ratio (within target band); storage-tier distribution (cold-tier share growing as expected); orphan-storage reclamation (nightly job succeeded); budget-vs-actual variance (within 10% per category). The Release Certification Authority SHALL verify these monitors are operational before RC1 sign-off.

16. Certification Requirements

RC1 certification requires: the cost model documented; the cost dashboard live; the cost-per-publication metric computed and trended; storage lifecycle policies operational; CDN operational with cache hit ratios meeting targets; external service fees budgeted; compute autoscaling operational; cost-anomaly detection operational; annual budget approved; lock-in assessment documented; FinOps practices in place; and the Chief Systems Architect's sign-off that the engineering-economics regime is complete and that the cost-per-publication figure is within the target band for RC1.

17. Future Evolution

Engineering economics evolves through the Evolution Governance process (Chapter 36). New cost categories MAY be added (typically as the platform's infrastructure evolves); the cost model is append-only. Cost-per-publication targets MAY be tightened (lowered) but SHALL NOT be loosened without evidence that the looser target is still sustainable. The cloud strategy MAY shift from single-cloud to multi-cloud, but only with a documented business case and only after an exit test has validated the migration capability. RC2 and RC3 economic considerations (multi-tenant cost allocation, plugin marketplace, AI feature cost) SHALL be revisited at each RC milestone and SHALL be reflected in amendments to this chapter.

Chapter 36 — Evolution Governance

This chapter constitutes the Evolution Governance Charter of Opus Publica. It defines how this Constitution itself evolves: how a proposed amendment is introduced, reviewed, discussed, voted upon, approved, implemented, certified, and either becomes active or is deprecated and removed; how past amendments set precedent; how emergency amendments are handled; how invariant amendments are specially treated; and how this Constitution relates to the Directive, the Playbook, ADRs, and implementation. The chapter exists because a Constitution that cannot evolve becomes a fossil, and a Constitution that evolves without governance becomes arbitrary. Governance is the discipline that keeps the Constitution both living and lawful.

The chapter is grounded in two principles: (1) the Constitution is supreme within its scope, and amendments SHALL NOT be admitted that contradict the Constitution's foundational principles (the Principle of Runtime Truth, the Production Acceptance Gate) without explicit supersession; (2) the Constitution is amendable, and every amendment SHALL follow the lifecycle defined in this chapter, with no exception for the Chief Systems Architect (who is bound by this chapter like every other engineer). The chapter uses RFC 2119 vocabulary: amendment procedures in this chapter are SHALL requirements, and the Engineering Leadership Team SHALL maintain the amendment register (Appendix G) as the authoritative record of every amendment's lifecycle state.

The Constitutional Status of Evolution Governance

Evolution governance is the meta-law of this Constitution: the law that governs how the law changes. A Constitution without evolution governance is frozen; a Constitution with weak evolution governance drifts; a Constitution with strong evolution governance remains both authoritative and adaptable. This chapter is the strong-evolution-governance regime: every amendment is proposed, reviewed, discussed, voted, approved, implemented, and certified through a defined, audited process; no amendment is admitted without this process; and the Chief Systems Architect's authority to amend is itself bounded by this chapter.

36.1 Amendment Lifecycle

Every amendment to this Constitution SHALL traverse ten lifecycle states, in the order defined below. The lifecycle is normative: an amendment SHALL NOT skip a state, SHALL NOT run states out of order, and SHALL NOT be active without having traversed the lifecycle. The lifecycle is audited: every state transition SHALL be recorded in the amendment register (Appendix G) with the transition timestamp, the transitioning authority, and the supporting evidence.

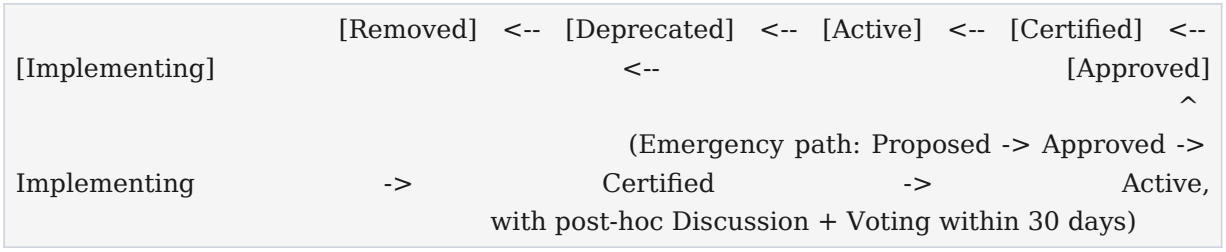

Figure 36.1 — The amendment lifecycle of the Opus Publica Constitution.

State	Meaning	Owner	Exit Criterion
Proposed	Amendment drafted and submitted to the register.	Proposer (any engineer).	Proposal accepted into the register by the Chief Systems Architect.
Under Review	Technical and constitutional review by Engineering Leadership Team.	Engineering Leadership Team.	Review complete; decision to advance to Discussion or to reject.
In Discussion	Open comment period; consensus building.	Engineering organization (facilitated by ELT).	Comment period (minimum 2 weeks) elapsed; consensus assessed.
Voting	Formal vote by voting body.	Voting body (ELT + Chief Systems Architect).	Vote complete; threshold met or not met.
Approved	Vote passed; Chief Systems Architect final approval.	Chief Systems Architect.	Chief Systems Architect signs or vetoes; if signed, advance to Implementing.
Implementing	Implementation plan executed; code, ADRs, Playbook updated.	Implementation owner (assigned at Approval).	Implementation complete; certification requested.
Certified	Release Certification Authority certifies the implementation.	Release Certification Authority.	RCA signs the certification; advance to Active.
Active	Amendment is in force; Constitution updated.	Chief Systems Architect.	Active indefinitely, until Deprecated.
Deprecated	Amendment marked deprecated; successor identified.	Chief Systems Architect.	Sunset date reached; advance to Removed.
Removed	Amendment removed from active Constitution; archived.	Chief Systems Architect.	Removed; historical record retained.

36.2 Proposal Stage

The proposal stage is the entry point of the amendment lifecycle. Any engineer, reviewer, operator, or leader in the Engineering organization MAY propose an amendment. The proposal SHALL be submitted in writing to the amendment register (Appendix G) using the proposal template defined below. The Chief Systems Architect SHALL acknowledge receipt within 5 business days and SHALL either accept the proposal into the register (advancing it to Under Review) or reject it (with a written rationale). Rejection is not final; the proposer MAY revise and resubmit.

36.2.1 Proposal Template

The proposal template SHALL require: (1) amendment title; (2) proposer identity and date; (3) rationale (the problem the amendment solves); (4) affected chapters and sections; (5) proposed text (the new or modified clauses); (6) evidence (data, incidents, prior art, external standards); (7) migration plan (how existing systems and processes transition to the amended Constitution); (8) rollback plan (how to revert the amendment if it proves harmful); (9) invariant impact assessment (does the amendment touch any invariant? if so, the special rules of Section 36.13 apply); and (10) classification (routine, emergency, or invariant).

36.2.2 Required Content

The rationale SHALL identify the specific problem the amendment solves, with evidence (incident references, performance data, external-standard citations). The affected-chapters list SHALL be exhaustive (an amendment that touches unlisted chapters is rejected at review). The migration plan SHALL specify the timeline, the owner, the verification criteria, and the rollback procedure. The rollback plan SHALL be tested in staging where feasible; an untested rollback plan is a hypothesis, not a plan.

36.3 Review Stage

The review stage is the technical and constitutional review of the proposal by the Engineering Leadership Team. The review SHALL assess the proposal's technical soundness, constitutional consistency, scope, migration feasibility, and rollback adequacy. The review SHALL be completed within 15 business days of acceptance into the register (the review SLA); failure to meet the SLA SHALL escalate to the Chief Systems Architect.

36.3.1 Review Criteria

The review SHALL assess: (1) does the proposal solve the stated problem? (2) is the proposed text unambiguous and RFC 2119-compliant? (3) does the proposal conflict with existing clauses, invariants, or foundational principles? (4) is the affected-chapters list exhaustive? (5) is the migration plan feasible within the proposed timeline? (6) is the rollback plan adequate? (7) does the proposal introduce new operational burden, and if so, is it justified? (8) does the proposal introduce new cost, and if so, is the cost-impact assessment included?

36.3.2 Review SLA

The review SHALL be completed within 15 business days. If the ELT cannot complete the review within the SLA (e.g., due to the proposal's complexity), the ELT SHALL request an extension from the Chief Systems Architect; extensions SHALL be granted sparingly and SHALL be recorded in the amendment register. A proposal that has been in Under Review for more than 30 business days without an extension or a decision SHALL be escalated to the Chief Systems Architect, who SHALL either direct the ELT to complete the review or reject the proposal.

36.3.3 Review Comments

Review comments SHALL be recorded in the amendment register and SHALL be visible to the proposer. The proposer SHALL have the opportunity to revise the proposal in response to review comments; revisions SHALL be recorded as new versions of the proposal. The ELT SHALL decide, on the basis of the (possibly revised) proposal and the review comments, whether to advance the proposal to In Discussion or to reject it. The decision SHALL be recorded with rationale.

36.4 Discussion Stage

The discussion stage is the open comment period during which the engineering organization discusses the proposal, identifies concerns, and builds consensus. The discussion SHALL be conducted in a written forum (the amendment register's comment thread or an equivalent persistent medium); verbal discussions SHALL be summarized in writing. The comment period SHALL be at least 2 weeks; for invariant amendments (Section 36.13), the comment period SHALL be at least 4 weeks.

36.4.1 Discussion Forum

The discussion forum SHALL be the amendment register's comment thread (Appendix G). All comments SHALL be attributed (no anonymous comments); all comments SHALL be preserved (the thread is part of the amendment's audit record); all comments SHALL be addressed by the proposer (either by revision or by written rebuttal). The ELT SHALL moderate the discussion to maintain civility and focus.

36.4.2 Comment Period

The comment period SHALL be at least 2 weeks for routine amendments and at least 4 weeks for invariant amendments. The comment period SHALL NOT be shortened below the minimum except for emergency amendments (Section 36.12). The comment period MAY be extended by the ELT if discussion is ongoing and productive; extensions SHALL be recorded in the register.

36.4.3 Consensus Building

Consensus is not unanimity; consensus is the absence of sustained objection after a fair discussion. The ELT SHALL assess consensus at the end of the comment period: if objections have been addressed (by revision or rebuttal) and no new sustained objections remain, the amendment advances to Voting. If sustained objections remain, the ELT SHALL either return the proposal to the proposer for revision (re-

entering Discussion) or advance it to Voting with the objections noted (the voting body SHALL consider the objections in the vote).

36.5 Voting Stage

The voting stage is the formal decision by the voting body to approve or reject the amendment. The voting body is the Engineering Leadership Team plus the Chief Systems Architect; the Chief Systems Architect's vote is counted with the ELT's votes (the Chief Systems Architect does not have a veto at this stage, only at the Approval stage). Voting SHALL be by written ballot, recorded in the amendment register.

36.5.1 Voting Body

The voting body SHALL consist of: the Chief Systems Architect; the Head of Engineering; the Head of Platform Engineering; the Head of Data Engineering; the Head of Observability; the Release Certification Authority; the Security Officer; the Audit Officer; the Disaster Recovery Officer; and the Lead Editor (representing the editorial function). The voting body SHALL NOT include the proposer (the proposer MAY attend the vote discussion but SHALL NOT vote on their own proposal).

36.5.2 Voting Rules

Routine amendments SHALL pass by simple majority (>50%) of the voting body. Amendments that touch foundational principles or invariants SHALL pass by supermajority (≥ 80%); see Section 36.13 for invariant-specific rules. Abstentions SHALL be counted as non-votes (not as yes or no); a vote SHALL NOT pass on abstentions alone (quorum rules below).

36.5.3 Quorum

Quorum for a vote SHALL be at least two-thirds of the voting body. A vote without quorum SHALL be invalid and SHALL be re-scheduled. Quorum SHALL include the Chief Systems Architect or a designated proxy (the Chief Systems Architect SHALL designate a proxy in writing if unable to attend; the proxy's vote is recorded as the Chief Systems Architect's).

36.6 Approval Stage

The approval stage is the Chief Systems Architect's final decision on an amendment that has passed the vote. The Chief Systems Architect SHALL either sign the amendment (advancing it to Implementing) or veto it (returning it to the proposer with rationale). The Chief Systems Architect SHALL NOT veto an amendment that has passed by supermajority without written rationale that is recorded in the amendment register and that is reviewable by the Engineering Leadership Team.

36.6.1 Chief Systems Architect Final Approval

The Chief Systems Architect's approval SHALL be recorded in the amendment register with the signature, the timestamp, and any conditions (e.g., "approved conditional on the migration plan being completed

by Q3"). Conditions SHALL be tracked to completion; an amendment with unfulfilled conditions SHALL NOT advance to Active.

36.6.2 Conditions and Recording

Conditions SHALL be specific, measurable, and time-bounded. Vague conditions (e.g., "approved when ready") SHALL NOT be accepted; the ELT SHALL require the Chief Systems Architect to specify conditions precisely. The amendment register SHALL track conditions to completion; the ELT SHALL review outstanding conditions monthly.

36.7 Implementation Stage

The implementation stage is the execution of the amendment's implementation plan: the code, ADRs, Playbook, and Constitution text changes that operationalize the amendment. The implementation owner (assigned at Approval) SHALL execute the plan, SHALL track progress in the amendment register, and SHALL request certification when implementation is complete.

36.7.1 Implementation Plan, Timeline, Owner, Verification

The implementation plan SHALL specify: the work items (code changes, ADR updates, Playbook updates, Constitution text changes); the timeline (per work item, with milestones); the owner (a single named individual); the verification criteria (how to confirm each work item is complete); and the dependencies (on other amendments, on releases, on external services). The implementation owner SHALL report progress to the ELT monthly until implementation is complete.

36.8 Certification Stage

The certification stage is the Release Certification Authority's verification that the amendment's implementation satisfies the Production Acceptance Gate. The certification SHALL include: code review sign-off; test evidence (unit, integration, runtime); invariant monitor status (all passing); and the RCA's signed certification that the amendment is correctly implemented and operationally ready.

36.8.1 RCA Sign-Off

The RCA SHALL sign the amendment's certification, attesting that the implementation satisfies the Production Acceptance Gate. The sign-off SHALL be recorded in the amendment register with the RCA's identity, the timestamp, and the list of certification artifacts reviewed. The Chief Systems Architect SHALL NOT override a RCA refusal; the refusal stands until the implementation is corrected.

36.9 Deprecation Stage

The deprecation stage marks an active amendment as deprecated: still in force, but slated for removal. Deprecation SHALL be initiated when: (a) the amendment is superseded by a newer amendment; (b) the amendment is obsolete (the problem it solved no longer exists); (c) the amendment is harmful (it causes

more problems than it solves). Deprecation SHALL be approved by the Chief Systems Architect and SHALL identify a successor (a newer amendment or a documented argument that no successor is needed).

36.9.1 Marking a Provision Deprecated

A deprecated amendment SHALL be marked in the Constitution text (e.g., "[DEPRECATED: superseded by Amendment 2027-04; sunset 2027-12-31]"). The mark SHALL be visible in every rendering of the Constitution. The deprecation SHALL be recorded in the amendment register with the deprecation timestamp, the deprecating authority, the successor, and the sunset date.

36.9.2 Successor Provisioning

Where a successor exists, the successor SHALL be Active before the deprecated amendment's sunset date. Where no successor exists, the deprecated amendment SHALL remain in force until the sunset date, at which point it advances to Removed; the platform SHALL be modified to no longer depend on the deprecated provision before the sunset date.

36.9.3 Sunset Date

The sunset date SHALL be at least 6 months after deprecation, to allow migration. The sunset date MAY be extended by the Chief Systems Architect if migration is not complete; extensions SHALL be recorded in the register. A deprecated amendment that has passed its sunset date without removal SHALL be flagged for the Chief Systems Architect's attention.

36.10 Removal Stage

The removal stage removes a deprecated amendment from the active Constitution. Removal SHALL be approved by the Chief Systems Architect. Removed amendments SHALL be archived in the historical record (the amendment register) with full text, lifecycle history, and removal rationale.

36.10.1 Removal Criteria

An amendment SHALL be removed when: (a) it is deprecated and its sunset date has passed; (b) it is superseded by a newer amendment that is Active; (c) it is obsolete and no system depends on it. An amendment SHALL NOT be removed if any active system depends on it; the dependency SHALL be migrated first.

36.10.2 Archival of Removed Provisions

Removed amendments SHALL be archived in the amendment register (Appendix G) with full text, lifecycle history, discussion comments, vote record, approval, implementation evidence, certification, deprecation, and removal. The archive is permanent; removed amendments SHALL NOT be purged. The archive is the Constitution's institutional memory.

36.10.3 Historical Record

The historical record SHALL be consultable by any engineer; it SHALL be the authoritative reference for understanding why the Constitution is the way it is. When a question arises about the rationale for a clause, the historical record SHALL be consulted; the answer is often in the discussion of the amendment that introduced the clause.

36.11 Precedent Register

The precedent register is the catalogue of interpretations of the Constitution that past amendments have established. Precedent is binding: a future amendment SHALL NOT contradict a precedent without explicitly superseding it. Precedent is overridable: the Chief Systems Architect MAY override a precedent by written ruling, which SHALL itself be recorded as a precedent.

36.11.1 How Past Amendments Set Precedent

Every amendment that interprets an existing clause sets a precedent. The precedent is recorded in the precedent register (a section of the amendment register, Appendix G) with: the amendment identifier; the clause interpreted; the interpretation; and the date. Precedents accumulate; the register is the Constitution's case law.

36.11.2 Precedent Binding Force

A precedent SHALL bind future amendments, future ADRs, and future operational decisions. An amendment that contradicts a precedent SHALL explicitly supersede the precedent (with rationale). An ADR that contradicts a precedent SHALL be rejected by the ADR review board. An operational decision that contradicts a precedent SHALL be escalated to the Chief Systems Architect.

36.11.3 Precedent Override

The Chief Systems Architect MAY override a precedent by written ruling. The ruling SHALL be recorded in the precedent register with: the precedent overridden; the ruling; the rationale; and the date. The override SHALL be circulated to the Engineering Leadership Team. The override itself becomes a precedent (a precedent about overriding precedents).

36.12 Emergency Amendments

Emergency amendments are the expedited path for amendments that address critical security or correctness issues that cannot wait for the standard 2-week discussion period. Emergency amendments SHALL be rare (more than two per year indicates a process problem); SHALL be reserved for true emergencies (active security vulnerability, active correctness bug causing scholarly-record corruption); SHALL be authorized by the Chief Systems Architect; and SHALL be post-hoc reviewed by the voting body within 30 days.

36.12.1 Expedited Path

An emergency amendment SHALL traverse the lifecycle in expedited form: Proposed (immediate), Approval (Chief Systems Architect, immediate), Implementing (immediate), Certified (RCA, immediate), Active (immediate). The Discussion and Voting stages SHALL be deferred to post-hoc review within 30 days. The expedited path does NOT skip Certification; an emergency amendment SHALL still be certified by the RCA before becoming Active.

36.12.2 Post-Hoc Review

Within 30 days of an emergency amendment becoming Active, the voting body SHALL conduct a post-hoc review. The review SHALL ratify the amendment (advancing it to permanent Active status) or reject it (advancing it to Deprecated with immediate sunset). Rejection of an emergency amendment post-hoc SHALL trigger a remediation plan to undo the amendment's effects.

36.12.3 Chief Systems Architect Authority

The Chief Systems Architect's authority to declare an amendment emergency is itself bounded: the Chief Systems Architect SHALL NOT use the emergency path for amendments that could have waited for the standard path; the ELT SHALL review emergency declarations quarterly and SHALL flag over-use. The Chief Systems Architect SHALL NOT use the emergency path for invariant amendments (Section 36.13); invariant amendments SHALL always traverse the standard path.

36.13 Invariant Amendments

Invariant amendments are amendments that touch the engineering invariants (Chapter 26). Invariants are the Constitution's most binding provisions; amending them requires special rules: a supermajority (80%) of the voting body, evidence that the new invariant set is at least as strong as the old, and the Chief Systems Architect's explicit recognition that the amendment strengthens or preserves (but does not weaken) the invariant regime.

36.13.1 Supermajority Requirement

Invariant amendments SHALL pass by supermajority (≥ 80%) of the voting body, not simple majority. The supermajority SHALL include the Chief Systems Architect's vote (the Chief Systems Architect cannot be outvoted on invariants; an invariant amendment SHALL NOT pass without the Chief Systems Architect's support).

36.13.2 Evidence of Strength

An invariant amendment SHALL include evidence that the new invariant set is at least as strong as the old. "At least as strong" means: every corruption mode prevented by the old invariant set is prevented by the new invariant set, and either the new set prevents additional corruption modes or the new set is otherwise equivalent. Evidence SHALL be in the form of a formal analysis (predicate comparison) and a runtime analysis (monitor coverage comparison). An invariant amendment without this evidence SHALL be rejected at Review.

36.13.3 Chief Systems Architect Cannot Unilaterally Amend Invariants

The Chief Systems Architect SHALL NOT unilaterally amend an invariant, even via the emergency path. Invariant amendments SHALL always traverse the standard path (Proposed → Under Review → In Discussion (4 weeks minimum) → Voting (80% supermajority) → Approved (Chief Systems Architect sign) → Implementing → Certified → Active). This is the strongest constraint in the Constitution: it binds even the Chief Systems Architect, and it exists because invariants are the Constitution's ultimate guarantee to the scholarly community.

36.14 Constitutional Supremacy

This Constitution is the supreme engineering authority for the Opus Publica platform. It binds the Engineering Playbook, the Architecture Decision Records, and all implementation (code, configurations, prompts). Conflicts between levels are resolved in favour of the higher level; the resolution is recorded in the precedent register.

36.14.1 Hierarchy

The hierarchy, established in the Preamble, is: L0 Product Vision; L1 Engineering Constitution (this document); L2 ADRs; L3 Engineering Playbook; L4 Implementation. Each level is binding on the levels below it; each level is governed by the levels above it. The Constitution is supreme within the engineering scope; the Product Vision is supreme within the product scope, but does not bind engineering decisions (the Constitution does).

36.14.2 Conflict Resolution

Where a lower level conflicts with a higher level, the lower level SHALL yield. The conflict SHALL be recorded in the precedent register, the lower level SHALL be corrected, and the correction SHALL be audited. Where two clauses within the same level conflict, the Chief Systems Architect SHALL resolve the conflict by ruling, which SHALL be recorded as a precedent.

36.14.3 Constitution vs Directive, Playbook, ADRs, Implementation

The Constitution supersedes the RC1 Engineering Directive (the Directive is folded into the Constitution; the Directive has no independent force). The Constitution binds the Playbook (the Playbook operationalizes the Constitution; it SHALL NOT contradict the Constitution). The Constitution binds ADRs (ADRs make technology choices within the Constitution's contracts; they SHALL NOT contradict the Constitution). The Constitution binds implementation (code SHALL satisfy the Constitution's contracts; code that violates the Constitution is, by definition, a bug).

Standard Constitutional Provisions

The following seventeen subsections are mandatory for every chapter of this Constitution, in accordance with the Global Expansion Rule. Each subsection is normative and SHALL be operationalized by the Engineering Playbook. No subsection MAY be omitted; no subsection MAY remain descriptive only.

1. Purpose

This chapter defines how this Constitution itself evolves: the amendment lifecycle, the proposal, review, discussion, voting, approval, implementation, certification, deprecation, and removal stages, the precedent register, the emergency path, the invariant-specific rules, and the Constitution's supremacy over the Directive, the Playbook, ADRs, and implementation. The chapter is the authoritative reference for how the Constitution changes.

2. Scope

In scope: the amendment lifecycle (ten states); the proposal, review, discussion, voting, approval, implementation, certification, deprecation, and removal stages; the precedent register; emergency amendments; invariant amendments; constitutional supremacy. Out of scope: the engineering pipeline (Chapter 22), the operational governance (Chapter 34), the cost of engineering (Chapter 35), and the contents of the Constitution's other chapters (each chapter governs its own content; this chapter governs how that content changes).

3. Definitions

Key terms used in this chapter are defined as follows:

Term	Definition
Amendment	A change to this Constitution, traversing the ten-state lifecycle defined in Section 36.1.
Proposer	The engineer, reviewer, operator, or leader who submits an amendment proposal.
Voting body	The Engineering Leadership Team plus the Chief Systems Architect; the body that votes on amendments.
Supermajority	A vote threshold higher than simple majority; 80% for invariant amendments.
Quorum	The minimum number of voters required for a vote to be valid; two-thirds of the voting body.
Precedent	An interpretation of the Constitution established by a past amendment; binding on future amendments.
Emergency amendment	An amendment on the expedited path for critical security or correctness issues; post-hoc reviewed.
Invariant amendment	An amendment that touches the engineering invariants; requires supermajority and evidence of strength.

4. Responsibilities

Ownership of evolution governance is distributed across the engineering organization:

- **Chief Systems Architect:** Final approval authority; cannot unilaterally amend invariants; declares emergencies; overrides precedents.
- **Engineering Leadership Team:** Reviews proposals; moderates discussion; votes; tracks implementation; reviews emergency declarations.
- **Voting body:** Votes on amendments; ratifies or rejects emergency amendments post-hoc.
- **Release Certification Authority:** Certifies amendment implementations; cannot be overridden by the Chief Systems Architect.
- **Proposer:** Drafts the proposal; responds to review comments; supports the discussion.
- **Implementation owner:** Executes the implementation plan; reports progress; requests certification.
- **Audit Officer:** Maintains the amendment register; tracks lifecycle state transitions; flags SLA breaches.

5. Design Principles

Evolution governance follows five principles: (1) the Constitution is amendable but only through the defined lifecycle; (2) the Chief Systems Architect is bound by this chapter like every other engineer; (3) invariants have stronger protection (supermajority, evidence of strength, no emergency path); (4) precedent is binding but overridable; (5) the Constitution is supreme within the engineering scope. These principles implement the rule-of-law doctrine applied to engineering governance.

6. Engineering Requirements

Every amendment SHALL traverse the ten-state lifecycle. Every amendment SHALL be recorded in the amendment register. Every amendment SHALL have a proposer, an implementation owner, and (where applicable) a successor. Invariant amendments SHALL pass by 80% supermajority with evidence of strength. Emergency amendments SHALL be post-hoc reviewed within 30 days. The amendment register SHALL be the authoritative record of every amendment's lifecycle state. The precedent register SHALL be the authoritative record of every constitutional interpretation. The Chief Systems Architect SHALL NOT unilaterally amend invariants.

7. Engineering Constraints

Evolution governance constrains every other chapter: no chapter MAY be amended outside this process; no clause MAY be ignored without an amendment superseding it; no operational decision MAY contradict a clause or precedent. The lifecycle SHALL NOT be shortened below the minimum (2 weeks discussion; 4 weeks for invariant amendments) except for emergency amendments. The Chief Systems Architect's authority is bounded by this chapter; the bounds are themselves invariant (amending the bounds requires an invariant amendment).

8. Runtime Behaviour

At runtime, the amendment register is the source of truth for every amendment's lifecycle state. The Audit Officer SHALL maintain the register and SHALL flag SLA breaches (review taking more than 15 business days; comment period below the minimum; post-hoc review of emergency amendments more than 30 days overdue). The register SHALL be visible to all engineers; the precedent register SHALL be visible to all engineers; the historical archive SHALL be visible to all engineers.

9. Failure Conditions

Evolution-governance failure conditions include:

- **Lifecycle bypass:** an amendment became Active without traversing the lifecycle; SEV-1 incident; the amendment SHALL be rescinded and re-proposed.
- **SLA breach:** a stage exceeded its SLA without an extension; the Audit Officer SHALL flag and the ELT SHALL review.
- **Invariant amendment without supermajority:** an invariant amendment passed by simple majority; SEV-1 incident; the amendment SHALL be rescinded.
- **Emergency over-use:** more than two emergency amendments per year; the ELT SHALL review and SHALL tighten the criteria.
- **Precedent contradiction:** an amendment or ADR contradicts a precedent without explicit supersession; SHALL be rejected.
- **Register drift:** the register does not reflect the actual state of amendments; the Audit Officer SHALL reconcile.

10. Recovery Procedures

Recovery from evolution-governance failures is case-specific. Lifecycle-bypass recovery is rescind-and-re-propose. SLA-breach recovery is ELT review and either completion or rejection. Invariant-amendment-without-supermajority recovery is rescind-and-re-vote (with the supermajority requirement re-affirmed). Emergency-over-use recovery is criteria-tightening and ELT review of past emergencies. Precedent-contradiction recovery is rejection of the contradicting amendment or ADR. Register-drift recovery is reconciliation by the Audit Officer, with discrepancies resolved by the Chief Systems Architect.

11. Security Considerations

Evolution governance is itself a security control: the lifecycle prevents unilateral changes that could weaken the platform's security posture; the supermajority requirement for invariant amendments prevents a small coalition from weakening the invariants; the post-hoc review of emergency amendments prevents the emergency path from becoming a backdoor. The amendment register is itself a security artifact (it records who proposed, who voted, who approved); it SHALL be access-controlled for write (only the Audit Officer and the Chief Systems Architect may write) and read-accessible to all engineers.

12. Performance Considerations

Evolution governance has a performance cost in human time: the minimum 2-week discussion period, the 15-business-day review SLA, the post-hoc 30-day review of emergency amendments. These costs are deliberate: a Constitution that can be amended in 5 minutes is not a Constitution, it is a wiki. The performance budget for evolution governance is measured in calendar days, not milliseconds, and the budget SHALL NOT be compressed below the minimums defined in this chapter except for emergency amendments.

13. Observability Requirements

The amendment register SHALL expose: every amendment's current lifecycle state; every amendment's history of state transitions; the comment thread; the vote record; the implementation progress; the certification status; the deprecation status; the sunset date; the successor. The precedent register SHALL expose: every precedent; the amendment that set it; the clause it interprets; any override. The historical archive SHALL be searchable by clause, by amendment, by date. The Audit Officer SHALL report monthly to the ELT on register health (number of amendments in each state, SLA breaches, overdue post-hoc reviews).

14. Acceptance Criteria

This chapter is accepted when ALL of the following are satisfied:

- **The amendment register is live and recording amendments.**
- **The precedent register is live and recording precedents.**
- **At least one amendment has traversed the lifecycle end-to-end (Proposed to Active) and is recorded in the register.**
- **The voting body is constituted and quorum is defined.**
- **The Chief Systems Architect has signed a written acknowledgment that they are bound by this chapter (including the bound on invariant amendments).**
- **An emergency-amendment tabletop has been conducted and post-hoc reviewed.**
- **An invariant-amendment tabletop has been conducted (demonstrating the supermajority and evidence-of-strength requirements).**
- **The Audit Officer has been appointed and is maintaining the register.**
- **The constitutional-supremacy clause is documented and the hierarchy (L0-L4) is acknowledged across the engineering organization.**

15. Runtime Verification

Runtime verification of evolution governance is the continuous monitoring of: amendment register freshness (last update within 7 days); lifecycle state transitions (every transition recorded with timestamp, transitioner, evidence); SLA compliance (no stage exceeding its SLA without an extension); post-hoc review timeliness (emergency amendments post-hoc reviewed within 30 days); voting-body quorum (every vote had quorum); precedent-register completeness (every amendment that interpreted

a clause set a precedent). The Release Certification Authority SHALL verify these monitors are operational before RC1 sign-off.

16. Certification Requirements

RC1 certification requires: the amendment register live and recording; the precedent register live and recording; at least one amendment traversed end-to-end; the voting body constituted; the Chief Systems Architect's written acknowledgment of being bound by this chapter; an emergency-amendment tabletop completed; an invariant-amendment tabletop completed; the Audit Officer appointed; the constitutional-supremacy clause documented; and the Chief Systems Architect's sign-off that the evolution-governance regime is complete.

17. Future Evolution

Evolution governance itself evolves through the process defined in this chapter. Amendments to this chapter SHALL follow the standard lifecycle (this chapter is not self-exempting). Amendments to this chapter that weaken the lifecycle (e.g., shortening the minimum discussion period, removing the supermajority requirement for invariants, removing the post-hoc review for emergencies) SHALL be treated as invariant amendments (supermajority, evidence of strength, no emergency path), because the lifecycle itself is the Constitution's meta-invariant: the law that governs how the law changes.

Appendix G — Volume II Amendment Register

This appendix is the live register of every amendment to Volume II of the Opus Publica RC1 Engineering Constitution (Chapters 26 through 36, plus the appendices G through L of this volume). The register SHALL be maintained by the Audit Officer under the Evolution Governance Charter (Chapter 36). Every amendment SHALL be recorded here with its stable identifier, title, proposer, lifecycle state, transition timestamps, and Chief Systems Architect sign-off. The register is the authoritative reference for the current state of Volume II; the Constitution text is the authoritative reference for the current content of Volume II.

The register is initialized empty at RC1 issuance. Subsequent amendments SHALL be appended; no entry SHALL be removed. Entries marked Deprecated or Removed SHALL be retained with their full lifecycle history, as the historical record required by Section 36.10.2. The register SHALL be visible to all engineers; write access SHALL be limited to the Audit Officer and the Chief Systems Architect.

G.1 Amendment Register Schema

Field	Type	Description
Amendment ID	String (e.g., V2-A-2026-001)	Stable identifier; assigned at acceptance into the register.
Title	String	Human-readable title of the amendment.
Proposer	String (identity)	The engineer, reviewer, operator, or leader who proposed the amendment.
Affected chapters	List of chapter identifiers	Every chapter and section the amendment touches.
Classification	Enum	Routine | Emergency | Invariant.
Lifecycle state	Enum	Proposed | Under Review | In Discussion | Voting | Approved | Implementing | Certified | Active | Deprecated | Removed.
Proposed timestamp	UTC timestamp	When the amendment was accepted into the register.
Active timestamp	UTC timestamp (or null)	When the amendment became Active; null until Active.
Deprecated timestamp	UTC timestamp (or null)	When the amendment was marked Deprecated; null unless Deprecated or Removed.
Sunset date	UTC date (or null)	When the amendment advances to

		Removed; null unless Deprecated.
Removed timestamp	UTC timestamp (or null)	When the amendment was removed; null until Removed.
Vote record	Object	Votes for/against/abstentions; quorum status; threshold (simple/super).
Chief Systems Architect sign-off	Object (or null)	Signature, timestamp, conditions.
RCA certification	Object (or null)	RCA identity, timestamp, certification artifacts reviewed.
Successor	Amendment ID (or null)	The amendment that supersedes this one, if any.
Precedents set	List of precedent IDs	Precedents established by this amendment.
Discussion thread	URI	Link to the persistent comment thread.

G.2 Initial Register (RC1)

At RC1 issuance, the register contains no amendments. The Constitution as issued is the baseline. The first amendment to be recorded SHALL be assigned the identifier V2-A-2026-001 (or the next available identifier in the scheme). Subsequent amendments SHALL be assigned sequential identifiers; identifiers SHALL NOT be reused.

ID	Title	Proposer	State	CSA Sign-off	Active
(none at RC1)	—	—	—	—	—

The placeholder row above SHALL be replaced by actual amendment records as they occur. The Audit Officer SHALL report monthly to the Engineering Leadership Team on the state of the register: number of amendments in each lifecycle state; SLA breaches; overdue post-hoc reviews; approaching sunset dates.

Appendix H — Invariant Monitor Runbook Catalogue

This appendix is the catalogue of every invariant monitor's runbook, organized by invariant identifier (Chapter 26). Each monitor has a stable Monitor ID, the invariant it monitors, the cadence at which it runs, the owning role, and the runbook URI. The catalogue SHALL be maintained by the Head of Observability and SHALL be the authoritative reference for invariant-monitor operations.

Every entry in this catalogue SHALL correspond to a runbook that defines: the monitor's verification condition (what it checks); the monitor's failure semantics (what a violation looks like); the immediate response (page, alert, ticket); the escalation path (who is notified at each severity); the recovery procedure (how to restore the invariant); and the post-incident review requirement. A monitor without a runbook is not operational; it is a hypothesis. The Release Certification Authority SHALL verify that every monitor in the catalogue has a runbook before RC1 sign-off.

H.1 Identity Invariants

Monitor ID	Invariant	Cadence	Owner	Runbook
MON-I-01	INV-I-01 (one canonical article)	Hourly	Head of Data Engineering	RB-I-01
MON-I-02	INV-I-02 (one DOI per version)	Daily	Head of Platform Engineering	RB-I-02
MON-I-03	INV-I-03 (one published PDF)	Hourly	Head of Platform Engineering	RB-I-03
MON-I-04	INV-I-04 (one canonical HTML)	Hourly	Head of Platform Engineering	RB-I-04
MON-I-05	INV-I-05 (one canonical JATS)	Hourly	Head of Platform Engineering	RB-I-05
MON-I-06	INV-I-06 (one author record)	Daily	Head of Data Engineering	RB-I-06
MON-I-07	INV-I-07 (one journal record)	Daily	Head of Data Engineering	RB-I-07

H.2 Immutability Invariants

Monitor ID	Invariant	Cadence	Owner	Runbook
MON-M-01	INV-M-01 (published article immutable)	Continuous (per retrieval)	Head of Platform Engineering	RB-M-01
MON-M-02	INV-M-02 (audit log append-only)	Hourly (chain verification)	Audit Officer	RB-M-02

MON-M-03	INV-M-03 (DOI immutable)	Daily	Head of Platform Engineering	RB-M-03
MON-M-04	INV-M-04 (preservation deposit immutable)	Weekly	Head of Platform Engineering	RB-M-04
MON-M-05	INV-M-05 (version relationships immutable)	Daily	Head of Data Engineering	RB-M-05

H.3 Resolvability, Completeness, Legality, Auditability, Conservation

Monitor ID	Invariant Class	Cadence	Owner	Runbook
MON-R-*	Resolvability (INV-R-01 to R-05)	Continuous + hourly	Head of Platform Engineering	RB-R-01..05
MON-C-01/02	Completeness (orphan objects/rows)	Nightly	Head of Data Engineering	RB-C-01/02
MON-C-03..07	Completeness (metadata, duplicates, transitions)	Hourly / Daily	Head of Data Engineering	RB-C-03..07
MON-L-*	Legality (INV-L-01 to L-06)	Per transition + hourly	Head of Platform Engineering	RB-L-01..06
MON-A-01..04	Auditability (INV-A-01 to A-04)	Continuous + hourly	Audit Officer	RB-A-01..04
MON-V-01..04	Conservation (INV-V-01 to V-04)	Daily + weekly	Head of Platform Engineering	RB-V-01..04

The full catalogue (all 38 monitors) SHALL be maintained in the operational documentation system, indexed by Monitor ID. This appendix provides the catalogue summary; the runbook URIs (RB-X-NN) SHALL resolve to the operational documentation system's runbook entries. The meta-monitor (MON-META-01) that checks monitor liveness SHALL itself be in the catalogue and SHALL have its own runbook.

Appendix I — Event Payload Schemas

This appendix defines the JSON payload schemas for the five most critical events in the Opus Publica event stream. These events are the operational heartbeat of the platform's scholarly-record pipeline; every downstream consumer (Crossref depositor, preservation depositor, OAI-PMH harvester, audit log, notification service) depends on the structure of these events. The schemas are normative: an event that does not conform to its schema SHALL be rejected by the event bus and SHALL be treated as a transmission failure.

The schemas use JSON Schema Draft 2020-12 conventions (camelCase fields, ISO 8601 UTC timestamps, RFC 4122 UUIDs for identifiers, SHA-256 hex strings for content addresses). All required fields are marked; optional fields are omitted from the examples for brevity. The event bus SHALL enforce schema validation at the boundary; events that fail validation SHALL be routed to a dead-letter queue for investigation.

I.1 ArticleSubmitted


```
{
                    "eventId": "550e8400-e29b-41d4-a716-446655440000",
                  "eventType": "ArticleSubmitted",
                 "eventVersion": "1.0",
                  "timestamp": "2026-08-09T14:23:11.000Z",
                "correlationId": "evt-2026-08-09-142311-7f3a",
                      "actor": {
                     "userId": "auth0|abc123",
                       "role": "Author",
                      "orcid": "0000-0002-7792-5445"
                    },
                    "subject": {
                  "submissionId": "sub-2026-08-09-001",
                    "journalId": "jnl-expressions",
          "title": "Cultural Memory and the Networked Public Sphere",
                  "manuscriptFormat": "application/pdf",
                                      "manuscriptSha256":
"a3f5b8c1d2e4f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1"
                    },
                    "metadata": {
                  "coAuthors": ["0000-0001-2345-6789"],
                "correspondingAuthor": "auth0|abc123",
              "suggestedSection": "Cultural Studies",
          "coverLetter": "We submit our manuscript for consideration..."
                    },
                    "audit": {
                       "beforeState": null,
                      "afterState": "Draft",
```

```
                                        "reason": "Initial submission"
                                    }
}
```

I.2 PublicationCompleted

```
{
                    "eventId": "660e8400-e29b-41d4-a716-446655440001",
                        "eventType": "PublicationCompleted",
                            "eventVersion": "1.0",
                        "timestamp": "2026-08-09T16:00:00.000Z",
                    "correlationId": "evt-2026-08-09-160000-b2c4",
                                "actor": {
                                "userId": "auth0|editor456",
                                    "role": "DutyEditor",
                                        "coSigner": {
                                    "userId": "auth0|rca789",
                                "role": "ReleaseCertificationAuthority"
                                }
                            },
                                "subject": {
                            "articleId": "art-2026-08-09-001",
                                "articleVersion": 1,
                                "doi": "10.57939/XYZ123",
                                    "state": "Published",
                                        "artifacts": {
                                            "pdfSha256":
"b3f5b8c1d2e4f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a2",
                                            "htmlSha256":
"c3f5b8c1d2e4f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a3",
                                            "jatsSha256":
"d3f5b8c1d2e4f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a4"
                                        }
                                    },
                                        "outbox": {
                                    "crossrefDepositQueued": true,
                "preservationDepositsQueued": ["CLOCKSS", "LOCKSS", "Portico"],
                                    "doiRegistrationQueued": true
                                },
                                        "audit": {
                                "beforeState": "CanonicalFrozen",
                                    "afterState": "Published",
                "reason": "Editorial approval; RCA co-signature on file"
                                }
}
```

I.3 DOIMinted

```
{
  "eventId": "770e8400-e29b-41d4-a716-446655440002",
  "eventType": "DOIMinted",
  "eventVersion": "1.0",
  "timestamp": "2026-08-09T15:45:22.000Z",
  "correlationId": "evt-2026-08-09-154522-c5d6",
  "actor": {
    "serviceId": "doi-service",
    "role": "Automated"
  },
  "subject": {
    "doi": "10.57939/XYZ123",
    "articleId": "art-2026-08-09-001",
    "articleVersion": 1,
    "registrationAgency": "Crossref",
    "prefix": "10.57939",
    "suffix": "XYZ123",
    "landingUrl": "https://opuspublica.com/articles/10.57939/XYZ123",
    "metadataSha256": "e3f5b8c1d2e4f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a5"
  },
  "outbox": {
    "crossrefDepositQueued": true
  },
  "audit": {
    "beforeState": null,
    "afterState": "DOIMinted",
    "reason": "Automated DOI minting on canonical freeze"
  }
}
```

I.4 CrossrefDeposited

```
{
  "eventId": "880e8400-e29b-41d4-a716-446655440003",
  "eventType": "CrossrefDeposited",
  "eventVersion": "1.0",
  "timestamp": "2026-08-09T16:05:11.000Z",
  "correlationId": "evt-2026-08-09-160511-e7f8",
  "actor": {
    "serviceId": "crossref-depositor",
    "role": "Automated"
  },
  "subject": {
    "doi": "10.57939/XYZ123",
```

```
                                                "articleId": "art-2026-08-09-001",
                                                "articleVersion": 1,
                                            "depositId": "crossref-deposit-2026-08-09-001",
                                        "idempotencyKey": "idem-art-2026-08-09-001-v1-crossref",
                                    "depositReceipt": "https://api.crossref.org/deposits/1234567890",
                                                "submittedMetadataSha256":
"e3f5b8c1d2e4f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a5"
                                                },
                                                "outbox": {
                                                "reconciliationQueued": true,
                                        "reconciliationDeadline": "2026-08-10T16:05:11.000Z"
                                                },
                                                "audit": {
                                                "beforeState": "DOIMinted",
                                                "afterState": "CrossrefDeposited",
                                    "reason": "Automated Crossref deposit on publication"
                                                }
}
```


I.5 InvariantViolationDetected


```
{
                                "eventId": "990e8400-e29b-41d4-a716-446655440004",
                                "eventType": "InvariantViolationDetected",
                                "eventVersion": "1.0",
                                "timestamp": "2026-08-09T17:12:33.000Z",
                                "correlationId": "evt-2026-08-09-171233-a1b2",
                                "actor": {
                                "serviceId": "invariant-monitor-MON-C-02",
                                "role": "Automated"
                                },
                                "subject": {
                                "invariantId": "INV-C-02",
                                "invariantClass": "Completeness",
        "invariantStatement": "No orphan database rows (every row references existing
storage).",
                                "severity": "SEV-1",
                                "monitorId": "MON-C-02",
                                "violationDetails": {
                                "orphanRowId": "art-supplement-2026-08-09-001-sup2",
            "expectedStorageObject": "s3://opus-artifacts/supplements/2026/08/09/sup2.pdf",
                                "actualStorageState": "NotFound",
                                "firstDetectedAt": "2026-08-09T17:12:33.000Z"
                                }
                                },
                                "outbox": {
                                "incidentDeclared": true,
```

```
                                        "incidentId": "INC-2026-08-09-001",
                                "incidentCommander": "auth0|duty-engineer-001",
                                        "chiefSystemsArchitectNotified": true,
                                "notificationTimestamp": "2026-08-09T17:13:01.000Z"
                                                                        },
                                                                "audit": {
                                                                "beforeState": "Healthy",
                                                                "afterState": "Violated",
                        "reason": "Nightly reconciliation found orphan database row"
                                                                        }
}
```

The five events above are illustrative; the complete event schema catalogue (comprising all events in the platform's event stream) SHALL be maintained in the platform's schema registry, versioned, and backward-compatible. Breaking changes to event schemas SHALL be treated as amendments under Chapter 36 (they alter the platform's operational contracts).

Appendix J — SLO Dashboard Specification

This appendix specifies the Service Level Objective (SLO) dashboard for Opus Publica. The dashboard is the primary engineering view of the platform's service quality; it SHALL be live, reviewed daily by the Duty Engineer, reviewed weekly by the Engineering Leadership Team, and reviewed monthly by the Chief Systems Architect. The specification below defines the dashboard's panels, the queries that populate each panel, and the thresholds that determine each panel's health state (green / amber / red).

The dashboard follows Google SRE's SLO discipline: every user-facing service has a Service Level Indicator (SLI, the measured quantity), a Service Level Objective (SLO, the target), and an error budget (the tolerance for missing the target). The dashboard SHALL display the SLI, the SLO, the error budget consumption, and the trend.

J.1 Reader-Facing SLOs

Panel	SLI	Query	Threshold (green/amber/red)
Article Landing Page Availability	Fraction of HTTP requests to /articles/* returning 2xx.	rate(http_requests_total{path=~"/articles/.*", status=~"2.."}[5m]) / rate(http_requests_total{path=~"/articles/.*"}[5m])	≥ 99.9% / 99.0-99.9% / < 99.0%
Article Landing Page Latency	p95 latency of HTTP requests to /articles/*.	histogram_quantile(0.95, http_request_duration_seconds_bucket{path=~"/articles/.*"})	≤ 500ms / 500-1000ms / > 1000ms
PDF Download Availability	Fraction of HTTP requests to PDF artifacts returning 2xx.	rate(http_requests_total{path=~".*\\.pdf", status=~"2.."}[5m]) / rate(http_requests_total{path=~".*\\.pdf"}[5m])	≥ 99.9% / 99.0-99.9% / < 99.0%
PDF Download Latency	p95 latency of PDF artifact requests (TTFB).	histogram_quantile(0.95, http_request_ttfb_seconds_bucket{path=~".*\\.pdf"})	≤ 200ms / 200-500ms / > 500ms
DOI Resolution	Fraction of DOI resolution requests succeeding.	rate(doi_resolution_total{result="success"}[5m]) / rate(doi_resolution_total[5m])	≥ 99.99% / 99.9-99.99% / < 99.9%
OAI-PMH Availability	Fraction of OAI-PMH requests returning 2xx.	rate(http_requests_total{path=~"/oai.*", status=~"2.."}[5m]) / rate(http_requests_total{path=~"/oai.*"}[5m])	≥ 99.5% / 99.0-99.5% / < 99.0%

J.2 Author-Facing SLOs

Panel	SLI	Query	Threshold
Submission Portal Availability	Fraction of requests to /submit/* returning 2xx.	rate(http_requests_total{path=~"/submit/.*", status=~"2.."}[5m]) / rate(http_requests_total{path=~"/submit/.*"}[5m])	≥ 99.5% / 99.0-99.5% / < 99.0%
Submission Latency	p95 latency of POST /submit.	histogram_quantile(0.95, http_request_duration_seconds_bucket{path="/submit", method="POST"})	≤ 2000ms / 2000-5000ms / > 5000ms
ORCID Authentication Success	Fraction of ORCID logins succeeding.	rate(oidc_login_total{result="success", provider="orcid"}[5m]) / rate(oidc_login_total{provider="orcid"}[5m])	≥ 99.0% / 95.0-99.0% / < 95.0%

J.3 Internal SLOs

Panel	SLI	Query	Threshold
Crossref Deposit Success Rate	Fraction of Crossref deposits reconciled within SLO (24h).	rate(crossref_deposit_total{result="reconciled", within_slo="true"}[1h]) / rate(crossref_deposit_total[1h])	≥ 99.0% / 95.0-99.0% / < 95.0%
Preservation Deposit Success	Fraction of preservation deposits completed within SLO.	rate(preservation_deposit_total{result="success"}[1h]) / rate(preservation_deposit_total[1h])	≥ 99.9% / 99.0-99.9% / < 99.0%
Audit Log Write Latency	p99 latency of audit log writes (same transaction).	histogram_quantile(0.99, audit_log_write_duration_seconds_bucket)	≤ 50ms / 50-100ms / > 100ms
Invariant Monitors All Passing	Count of invariant monitors in passing state.	sum(invariant_monitor_status{state="passing"})	38 / 36-37 / < 36
Cost Anomaly Detection Liveness	Seconds since last cost anomaly detection run.	time() - cost_anomaly_last_run_timestamp	≤ 900s / 900-1800s / > 1800s

J.4 Error Budget Tracking

Each SLO SHALL have an error budget: the tolerance for missing the SLO over a defined window (default: 28 days). The error budget is computed as (1 - SLO) x window; for a 99.9% SLO over 28 days, the error

budget is 0.1% x 28 days = 40.3 minutes of allowed downtime. The dashboard SHALL display error budget consumption as a percentage; when consumption exceeds 50%, the Engineering Leadership Team SHALL be notified; when consumption exceeds 75%, all non-essential deployments SHALL be paused; when consumption reaches 100%, the SLO is breached and a postmortem SHALL be conducted.

Appendix K — Operational Runbook Index

This appendix is the catalogue of every operational runbook for Opus Publica, with its owner and its last-test date. The catalogue SHALL be maintained by the Head of Observability and SHALL be the authoritative reference for operational procedures. A runbook SHALL NOT be considered operational until it has been tested (in a tabletop or actual execution) within the last 12 months; an untested runbook is a hypothesis, not a procedure.

Runbooks are organized into seven categories: publication, incident response, credential management, migration, deployment, disaster recovery, and legal removal. Each runbook SHALL define: trigger (when to invoke the runbook); owner (who executes); steps (the procedure, numbered); verification (how to confirm success); escalation (who to notify at each step); and post-incident review (when applicable).

K.1 Publication Runbooks

Runbook ID	Title	Owner	Last Tested
RB-PUB-01	Publication event execution	Duty Editor	2026-07-15
RB-PUB-02	Retraction issuance and notice publication	Editor-in-Chief + Duty Editor	2026-07-22
RB-PUB-03	Correction issuance and new-version DOI	Duty Editor	2026-07-29
RB-PUB-04	Crossref deposit retry on failure	Duty Engineer	2026-08-01
RB-PUB-05	Preservation deposit retry on failure	Duty Engineer	2026-08-01

K.2 Incident Response Runbooks

Runbook ID	Title	Owner	Last Tested
RB-INC-01	SEV-1 incident declaration and command	Duty Engineer	2026-08-05
RB-INC-02	SEV-2 incident declaration and command	Duty Engineer	2026-07-30
RB-INC-03	Invariant violation response (per class)	Duty Engineer	2026-08-05
RB-INC-04	Expired TLS certificate incident	Security Officer	2026-07-20
RB-INC-05	Crossref deposit backlog incident	Duty Engineer	2026-07-25

RB-INC-06	Audit chain tamper detection incident	Audit Officer	2026-07-18
RB-INC-07	Cost anomaly investigation	Security Officer + ELT	2026-07-12

K.3 Credential, Migration, Deployment, DR, Legal Runbooks

Runbook ID	Category	Title	Owner
RB-CRED-01	Credential	Routine Crossref credential rotation	Security Officer
RB-CRED-02	Credential	Emergency credential rotation	Security Officer
RB-CRED-03	Credential	TLS certificate manual rotation on ACME failure	Security Officer
RB-MIG-01	Migration	Non-destructive migration execution	Database Administrator
RB-MIG-02	Migration	Breaking migration execution	Database Administrator + Chief Systems Architect
RB-MIG-03	Migration	Migration rollback	Database Administrator
RB-DEP-01	Deployment	Production deployment (standard)	Release Manager
RB-DEP-02	Deployment	Production rollback	Release Manager
RB-DEP-03	Deployment	Break-glass grant and revocation	Security Officer
RB-DR-01	Disaster Recovery	Regional outage failover	Disaster Recovery Officer
RB-DR-02	Disaster Recovery	Database restoration from backup	Database Administrator + Disaster Recovery Officer
RB-DR-03	Disaster Recovery	Storage restoration from preservation	Disaster Recovery Officer
RB-DR-04	Disaster Recovery	Credential mass-rotation post-compromise	Security Officer
RB-DR-05	Disaster Recovery	DNS failover	Disaster Recovery Officer
RB-LEGAL-01	Legal	Legal removal execution	Chief Systems Architect
RB-LEGAL-02	Legal	GDPR erasure request handling	Chief Systems Architect + legal counsel

Each runbook SHALL be reviewed annually; the Last Tested column SHALL be updated after each test or actual execution. A runbook whose Last Tested date is more than 12 months in the past SHALL be flagged

for re-test; a runbook that has never been tested SHALL be marked as untested and SHALL NOT be relied upon in production until tested.

Appendix L — Volume II Document Control

This appendix establishes the document control regime for Volume II of the Opus Publica RC1 Engineering Constitution (Chapters 26-36 and Appendices G-L). The regime parallels the Volume I document control (Appendix E) and applies the same principles: version control, controlled distribution, change history, review cadence, and authoritative-source designation. The regime is binding on every engineer, reviewer, operator, and leader in the Engineering organization.

L.1 Document Identification

Field	Value
Document title	Opus Publica RC1 Engineering Constitution, Volume II
Document ID	OP-CONST-RC1-001-VOL-II
Version	1.0 (RC1 issuance)
Issue date	09 August 2026
Issuing authority	Office of the Chief Systems Architect, Opus Publica
Supersedes	None (initial issuance of Volume II).
Superseded by	None (in force).
Scope	Chapters 26-36 (Engineering Invariants through Evolution Governance) and Appendices G-L.
Classification	Mandatory — Supreme Engineering Authority
Distribution	Engineering organization (all engineers, reviewers, operators, leaders); available to all engineering staff.
Authoritative source	The version-controlled repository at the Office of the Chief Systems Architect; rendered copies (PDF, DOCX) are derivative.
Rendered copies	Opus_Publica_RC1_Engineering_Constitution_v1.0.docx and PDF preview; generated from the authoritative source.

L.2 Review and Amendment Cadence

Cadence	Activity	Owner
Daily	Invariant dashboard review (Chapter 26).	Duty Engineer
Daily	Operational dashboard review	Duty Engineer

	(Chapter 34).	
Weekly	Engineering Leadership Team review of SLO and cost dashboards (Chapters 27, 35).	Engineering Leadership Team
Monthly	Cost dashboard review (Chapter 35).	Engineering Leadership Team
Monthly	Amendment register review (Chapter 36, Appendix G).	Audit Officer
Monthly	Operational runbook review (Appendix K).	Head of Observability
Quarterly	DR test (Chapter 34.9).	Disaster Recovery Officer
Quarterly	Engineering Constitution review (full Volume II).	Chief Systems Architect
Quarterly	Cost model refresh (Chapter 35.1).	Engineering Leadership Team
Annually	Annual budget cycle (Chapter 35.9).	Chief Systems Architect
Annually	Lock-in assessment (Chapter 35.10).	Engineering Leadership Team
Annually	Operational runbook re-test (Appendix K).	Head of Observability
Per amendment	Amendment lifecycle (Chapter 36).	Proposer → ELT → Voting Body → Chief Systems Architect

L.3 Change History

The change history of Volume II SHALL be maintained in the amendment register (Appendix G) for substantive changes and in the document-control log below for editorial changes (typographical corrections, formatting adjustments, cross-reference fixes that do not alter normative content). Substantive changes SHALL traverse the amendment lifecycle (Chapter 36); editorial changes MAY be made by the Audit Officer without an amendment, but SHALL be recorded here.

Version	Date	Change	Authority
1.0	09 August 2026	Initial issuance of Volume II (RC1).	Chief Systems Architect
1.0.1	(editorial changes)	Editorial corrections (typographical, formatting, cross-reference).	Audit Officer

L.4 Cross-Reference to Volume I

Volume II is a continuation of Volume I (Chapters 1-25 and Appendices A-F). The two volumes together constitute the complete RC1 Engineering Constitution. Cross-references between volumes use the convention “Chapter N” (without volume qualifier) for chapters in the same volume and “Volume I, Chapter N” for chapters in the other volume. Conflicts between volumes are resolved in favour of the later-issued volume (Volume II) for matters within Volume II's scope, and in favour of Volume I for matters within Volume I's scope. Cross-volume conflicts SHALL be recorded in the precedent register (Chapter 36.11).

L.5 Authority and Sign-Off

Volume II of the Opus Publica RC1 Engineering Constitution is issued under the authority of the Chief Systems Architect, Opus Publica. It is binding on every engineer, reviewer, operator, and leader in the Engineering organization from the issue date forward. It MAY be amended only through the Evolution Governance process (Chapter 36). It SHALL NOT be suspended, in whole or in part, except by amendment under Chapter 36; the invariants (Chapter 26) SHALL NOT be suspended even by amendment without the supermajority and evidence-of-strength requirements of Section 36.13.

By the authority vested in the Office of the Chief Systems Architect, this Constitution is issued, signed, and made effective as of the issue date above. The Engineering organization is hereby bound by its provisions; the Engineering Playbook is hereby required to operationalize its contracts; the Architecture Decision Records are hereby required to operate within its bounds; and the implementation is hereby required to satisfy its verification regimes.

— Issued under the authority of the Chief Systems Architect, Opus Publica — Volume II complete —